

LogiCORE IP Tri-Mode Ethernet MAC v4.5

User Guide

UG138 March 1, 2011



Xilinx is providing this product documentation, hereinafter "Information," to you "AS IS" with no warranty of any kind, express or implied. Xilinx makes no representation that the Information, or any particular implementation thereof, is free from any claims of infringement. You are responsible for obtaining any rights you may require for any implementation based on the Information. All specifications are subject to change without notice.

XILINX EXPRESSLY DISCLAIMS ANY WARRANTY WHATSOEVER WITH RESPECT TO THE ADEQUACY OF THE INFORMATION OR ANY IMPLEMENTATION BASED THEREON, INCLUDING BUT NOT LIMITED TO ANY WARRANTIES OR REPRESENTATIONS THAT THIS IMPLEMENTATION IS FREE FROM CLAIMS OF INFRINGEMENT AND ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE.

Except as stated herein, none of the Information may be copied, reproduced, distributed, republished, downloaded, displayed, posted, or transmitted in any form or by any means including, but not limited to, electronic, mechanical, photocopying, recording, or otherwise, without the prior written consent of Xilinx.

© Copyright 2004-2011. Xilinx, Inc. XILINX, the Xilinx logo, Virtex, Spartan, ISE and other designated brands included herein are trademarks of Xilinx in the United States and other countries. The PowerPC name and logo are registered trademarks of IBM Corp. and used under license. All other trademarks are the property of their respective owners.

Tri-Mode Ethernet MAC Revision History

This table shows the revision history for this document.

Date	Version	Revision
9/30/04	1.1	Initial Xilinx release.
4/28/05	2.0	Updated to version 2.1 of the core, Xilinx tools 7.1i, support for the Spartan®-3E device.
1/18/06	2.1	Updated to version 2.2 of the core, release date, and Xilinx tools 8.1i.
7/13/06	3.1	Updated to version 3.1 of the core; Xilinx tools 8.2i.
9/21/06	3.2	Updated to version 3.2 of the core, added support for the Spartan-3A device.
2/15/07	3.3	Updated to version 3.3 of the core; Xilinx tools 9.1i.
8/8/07	3.4	Updated to version 3.4 of the core; Xilinx tools 9.2i.
3/24/08	3.5	Updated to version 3.5 of the core; Xilinx tools 10.1.
4/24/09	3.6	Updated to version 4.1 of the core; Xilinx tools 11.1.
6/24/09	3.7	Updated to version 4.2 of the core; Xilinx tools 11.2. Added Virtex-6 CXT support.
09/16/09	3.8	Updated to version 4.3 of the core; Xilinx tools 11.3; Added licensing support for 10/100 Mb/s only core. Added Virtex-6 HXT and Virtex-6 -1L support.
12/02/09	3.8.1	Documentation corrections to pages 43, 75, and 104.
04/19/10	3.9	Updated to version 4.4 of the core; Xilinx tools 12.1.
03/01/11	4.0	Updated to version 4.5 of the core; Xilinx tools 13.1.

Table of Contents

Preface: About This Guide

Guide Contents	23
Additional Resources	24
Conventions	25
Typographical	25
Online Document	26
List of Acronyms	26

Chapter 1: Introduction

About the Core	29
System Requirements	29
Windows	29
Linux	29
Software	29
.....	30
Recommended Design Experience	30
Additional Core Resources	30
Related Xilinx Ethernet Products and Services	30
Specifications	30
Technical Support	30
Feedback	31
Tri-Mode Ethernet MAC Core	31
Document	31

Chapter 2: Licensing the Core

Before you Begin	33
License Options	33
Simulation Only	33
Full System Hardware Evaluation	34
Full	34
Obtaining your License Key	34
Simulation License	34
Full System Hardware Evaluation License	34
Obtaining a Full License	34
Installing your License File	35

Chapter 3: Ethernet Overview

Typical Ethernet Application Overview	37
Ethernet Switch or Router	37
Ethernet Communications Port for an Embedded Processor	38
Ethernet Protocol Overview	39
Ethernet Sublayer Architecture	39
MAC and MAC CONTROL Sublayer	39
Physical Sublayers PCS, PMA, and PMD	40
Ethernet Data Format	40
Preamble	41
Start of Frame Delimiter	41
MAC Address Fields	41
Length/Type	42
Data	42
Pad	42
FCS	43
Frame Transmission and Interframe Gap	43
Half-Duplex Frame Transmission	43
Full-Duplex Frame Transmission	44

Chapter 4: Generating the Core

GUI Interface	45
Component Name	46
Physical Interface	46
MAC Speed	46
Clock Enables	46
Half Duplex	47
Management Interface	47
Address Filter	47
Number of Address Table Entries	47
Parameter Values in the XCO File	48
Output Generation	49

Chapter 5: Tri-Mode Ethernet MAC Overview

Key Features	51
Architecture Description	52
Core Components	52
Transmit Engine	52
Receive Engine	53
Flow Control	53
Address Filter	53
Optional Management Interface and MDIO	53
GMII/MII Block	54
Core Interfaces	54
Client Side Interface Signals	54
Transmitter Interface	54
Receiver Interface	56
Flow Control Interface	56
Management Interface Signals	57
Configuration Vector Signals	57

Address Filter Signals	58
Clock, Speed Indication, and Reset Signals	58
Physical Interface Signals	59
GMII/MII	59
MDIO Signals	60

Chapter 6: Designing with the Core

General Design Guidelines	61
Design Steps	61
Using the Example Design as a Starting Point	61
Implementing the Tri-Mode Ethernet MAC in Your Application	63
Keep it Registered	63
Recognize Timing Critical Signals	63
Use Supported Design Flows	63
Make Only Allowed Modifications	64

Chapter 7: Client Interface

Introduction to the Timing Diagrams	65
Receiving Inbound Frames	66
Normal Frame Reception	66
emacclientrxgoodframe and emacclientrxbadframe Timing	67
Frame Reception with Errors	67
Client-Supplied FCS Passing	68
VLAN Tagged Frames	68
Maximum Permitted Frame Length	69
Length/Type Field Error Checks	69
Enabled	69
Disabled	70
Address Filter	70
Receiver Statistics Vector	71
Transmitting Outbound Frames	73
Normal Frame Transmission	73
Padding	75
Client-Supplied FCS Passing	75
Client Underrun	76
Back-to-Back Transfers	76
VLAN Tagged Frames	77
Maximum Permitted Frame Length	78
Frame Collisions: Half-Duplex Operation Only	78
Interframe Gap Adjustment: Full-Duplex Mode Only	79
Transmitter Statistics Vector	80

Chapter 8: Flow Control

Overview of Flow Control	83
Flow Control Requirement	83
Flow Control Basics	84
Pause Control Frames	84
Flow Control Operation of the TEMAC	85
Transmitting a Pause Control Frame	85
Core-Initiated Pause Request	85
Client-Initiated Pause Request	86
Receiving a Pause Control Frame	86
Core-Initiated Response to a Pause Request	86
Client-Initiated Response to a Pause Request	87
Flow Control Implementation Example	87
Method	87
Operation	88

Chapter 9: Configuration and Status

Using the Optional Management Interface	89
hostclk	90
Configuration Register Map	90
Configuration Register Definition	91
Receiver Configuration	91
Transmitter Configuration	93
Flow Control Configuration	94
MDIO Configuration	94
TEMAC Operational Speed Configuration	95
Address Filter Configuration	95
Using the Management Interface	98
Accessing Configuration	98
MDIO Interface	101
Introduction to MDIO	101
Connecting the TEMAC to an MDIO bus	104
Accessing PHY Configuration Registers, via MDIO using the Management Interface	106
Accessing Configuration without the Management Interface	108
TEMAC Configuration Settings	112
Half-Duplex Configuration Settings	112
Half-Duplex and Flow Control Configuration Settings	112
MAC Address Settings	112

Chapter 10: Physical Interface for the 10 Mb/s and 100 Mb/s Only Ethernet MAC IP Core

MII Transmitter Interface	113
MII Receiver Interface	114
Multiple Core Instances with the MII	114

Chapter 11: Physical Interfaces for 1 Gb/s Only Ethernet MAC IP Core

Gigabit Media Independent Interface (GMII)	117
GMII Transmitter Interface.....	117
GMII Receive Interface	119
Virtex-6 Devices	119
Spartan-6 Devices	121
Virtex-5 Devices	122
Virtex-4, Spartan-3, Spartan-3E, and Spartan-3A Devices.....	123
Clock Sharing across Multiple Cores with GMII for 1 Gb/s Operation	125
Clock Resource Sharing in Virtex-6 Devices.....	125
Clock Resource Sharing in Spartan-6 Devices.....	127
Clock Resource Sharing in Virtex-5 Devices.....	128
Clock Resource Sharing in Virtex-4, Spartan-3, Spartan-3E, Spartan-3A and Spartan-3A DSP Devices	129
Reduced Gigabit Media Independent Interface (RGMII)	130
Virtex-6 Devices	130
Transmitter Logic	130
Receiver Logic.....	131
Clock Resource Sharing	133
Spartan-6 Devices.....	134
Transmitter Logic	134
Receiver Logic.....	136
Clock Resource Sharing	137
Virtex-5 Devices	138
Transmitter Logic	138
Receiver Logic.....	140
Clock Resource Sharing	141
Virtex-4 Devices	143
Transmitter Logic	143
Receiver Logic.....	144
Clock Resource Sharing	146
.....	147
Spartan-3, Spartan-3A, and Spartan-3A DSP Devices	148
Transmitter Logic	148
Receiver Logic.....	150
Clock Resource Sharing	152

Chapter 12: Physical Interfaces for Tri-speed (10 Mb/s, 100 Mb/s and 1 Gb/s) Ethernet MAC IP Core

Gigabit Media Independent Interface (GMII)	156
GMII Transmitter Interface.....	156
GMII Receive Interface	158
Virtex-6 Devices	158
Spartan-6 Devices	159
Virtex-5 Devices	160
Virtex-4, Spartan-3, Spartan-3E, and Spartan-3A Devices.....	162
Multiple Core Instantiations.....	165
Multiple Core Instances in Virtex-6 Devices.....	165
Multiple Core Instances in Spartan-6 Devices	167
Multiple Core Instances in Virtex-5 Devices.....	169
Multiple Core Instances in Virtex-4, Spartan-3, Spartan-3E, Spartan-3A and Spartan-3A DSP Devices	171

Reduced Gigabit Media Independent Interface (RGMII)	173
Virtex-6 Devices	173
Transmitter Logic	173
Receiver Logic	175
Clock Resource Sharing	177
Spartan-6 Devices	178
Transmitter Logic	178
Receiver Logic	179
Clock Resource Sharing	180
Virtex-5 Devices	182
Transmitter Logic	182
Receiver Logic	184
Clock Resource Sharing	185
Virtex-4 Devices	187
Transmitter Logic	187
Receiver Logic	189
Clock Resource Sharing	191
Spartan-3, Spartan-3A, and Spartan-3A DSP Devices	193
Transmitter Logic	193
Receiver Logic	195
Clock Resource Sharing	197

Chapter 13: Constraining the Core

Device, Package, and Speed Grade Selection	199
I/O Location Constraints	199
Placement Constraints	200
Timing Constraints	200
PERIOD(s) for Clock Nets	200
Transmitter Clock Constrains	200
Receiver Clock Constraints	201
IDELAYCTRL Reference Clock Constraints	201
Management Clock Constraints	201
MDIO Logic	202
Timespecs for Critical Logic	202
Flow Control	202
Configuration Logic	203
Constraints when Implementing an External GMII	204
GMII IOB Constraints	204
GMII Input Setup/Hold Timing	205
Virtex-6 Devices	205
Spartan-6 Devices	206
Virtex-5 Devices	207
Virtex-4 Devices	208
Spartan-3, Spartan-3E, and Spartan-3A Devices	209
Understanding Timing Reports for GMII Setup/Hold Timing	210
Virtex-6, Spartan-6, Spartan-3, Virtex-4 and Devices	210
Virtex-5 Devices	211
Constraints when Implementing an External RGMII	212
RGMII IOB Constraints	213
RGMII Input Setup/Hold Timing	214
Virtex-6 Devices	214
Spartan-6 Devices	215

Virtex-5 Devices	216
Virtex-4 Devices	218
Spartan-3, Spartan-3E, and Spartan-3A Devices	219
Understanding Timing Reports for RGMII Setup/Hold Timing	220
Virtex-6 and Virtex-5 Devices	220
Spartan-6, Virtex-4 and Spartan-3 Devices	222

Chapter 14: Interfacing to Other Xilinx Ethernet Cores

Ethernet 1000BASE-X PCS/PMA or SGMII Core	225
Ethernet Statistics Core	225
Ethernet AVB Endpoint	226

Chapter 15: Implementing Your Design

Pre-implementation Simulation	227
Using the Simulation Model	227
VHDL	227
Verilog	227
Synthesis	227
XST - VHDL	227
XST - Verilog	228
Implementation	228
Generating the Xilinx Netlist	228
Mapping the Design	228
Placing and Routing the Design	229
Static Timing Analysis	229
Generating a Bitstream	229
Post-Implementation Simulation	229
Generating a Simulation Model	229
VHDL	229
Verilog	229
Using the Model	230
Other Implementation Information	230

Chapter 16: Quick Start Example Design

Overview	231
Generating the Core	232
Implementing the Example Design	234
Linux	234
Windows	234
Running the Simulation	234
Functional Simulation	234
VHDL Simulation	234
Verilog Simulation	235
Timing Simulation	235
VHDL Simulation	235
Verilog Simulation	236
What's Next?	236

Chapter 17: Detailed Example Design

Directory and File Contents	238
<project directory>	238
<project directory>/<component name>	238
<component name>/doc	239
<component name>/example design	239
example design/fifo	240
example_design/physical	240
<component name>/implement	241
implement/results	241
<component name>/simulation	242
simulation/functional	242
simulation/timing	243
Implementation and Test Scripts	244
Implementation Scripts for Timing Simulation	244
Test Scripts For Functional Simulation	244
Test Scripts For Timing Simulation	245
Example Design	246
HDL Example Design	246
VHDL	246
Verilog	246
10 Mb/s /100 Mb/s/1 Gb/s Ethernet FIFO	247
VHDL	247
Verilog	247
rx_client_fifo	248
tx_client_fifo	248
Address Swap Module	249
VHDL	249
Verilog	249
Demonstration Test Bench	250
Test Bench Functionality	250
VHDL	250
Verilog	250
Core with Management Interface	251
Core with No Management Interface	252
Changing the Test Bench	252
Changing Frame Data	252
Changing Frame Error Status	252
Changing the Tri-Mode Ethernet MAC Configuration	253
Performing MDIO accesses if 10 Mb/s is not supported	253
VHDL	253
Verilog	253

Appendix A: Using the Client Side FIFO

Overview of LocalLink Interface	255
Receive FIFO Operation	257
LocalLink Interface	257
Transmit FIFO Operation	258
LocalLink Interface	258
Clock Requirements	258
User Interface Data Width Conversion	259

Appendix B: Core Verification, Compliance and Interoperability

Verification by Simulation	261
Hardware Verification	261

Appendix C: Core Latency

Transmit Path Latency	263
Receive Path Latency	263

Appendix D: Calculating the DCM Phase Shift or IODelay Tap Setting

DCM Usage	265
DCM Phase Shifting Requirements	265
Finding the Ideal Phase Shift Value	266
IODelay Usage	266
IODelay Tap Setting Requirements	266
Finding the Ideal Tap Setting Value	267

Appendix E: Differences between the Embedded Tri-Mode Ethernet MACs and the Soft TEMAC Solution IP Core

Virtex-6 Device	269
Features Exclusive to the Embedded Tri-Mode Ethernet MAC	269
Features Exclusive to Soft 10/100/1000 Mb/s, 1000 Mb/s and 10/100 Mb/s IP Cores	270
Virtex-5 and Virtex-4 Devices	270
Features Exclusive to the Embedded Tri-Mode Ethernet MAC	270
Features Exclusive to Soft 10/100/1000 Mb/s, 1000 Mb/s and 10/100 Mb/s IP Cores	271

Appendix F: Upgrading from the 1-Gigabit Ethernet MAC LogiCORE

Generate the TEMAC Core	273
Block Level Port Differences	274
Management Register Differences	275
Feature Differences	275
Core Netlist Port Differences	275

Appendix G: 10/100/1000 Mb/s Operation Without Using Clock Enables

Background	277
Port Map Changes	277
Client Interface	278
Physical Interfaces for 10/100/1000 Mb/s Operation (No Clock Enables)	278
GMII/MII	279
Transmitter Logic	279
Receiver Logic	281
RGMII	284
Transmitter Logic	284
Receiver Logic	288
Multiple Cores	290
Spartan-3 Device BUFGMUX Usage	291
Required Constraints	293

Schedule of Figures

Preface: About This Guide

Chapter 1: Introduction

Chapter 2: Licensing the Core

Chapter 3: Ethernet Overview

<i>Figure 3-1: Typical Application: Ethernet Switch or Router</i>	38
<i>Figure 3-2: Typical Application: Ethernet Communications Port for Embedded Processor</i>	38
<i>Figure 3-3: IEEE Std 802.3-2008 Ethernet Model</i>	39
<i>Figure 3-4: Standard Ethernet Frame Format</i>	40
<i>Figure 3-5: Ethernet VLAN Frame Format</i>	40

Chapter 4: Generating the Core

<i>Figure 4-1: Core Customization Screen</i>	45
--	----

Chapter 5: Tri-Mode Ethernet MAC Overview

<i>Figure 5-1: Tri-Mode Ethernet MAC Block Diagram</i>	52
--	----

Chapter 6: Designing with the Core

<i>Figure 6-1: Tri-Mode Ethernet MAC Core Example Design</i>	62
--	----

Chapter 7: Client Interface

<i>Figure 7-1: Understanding the Timing Diagrams</i>	65
<i>Figure 7-2: Normal Frame Reception</i>	66
<i>Figure 7-3: Frame Reception with Error</i>	67
<i>Figure 7-4: Frame Reception with In-Band FCS Field</i>	68
<i>Figure 7-5: Reception of a VLAN Tagged Frame</i>	69
<i>Figure 7-6: Receiver Statistics Vector Timing</i>	71
<i>Figure 7-7: Normal Frame Transmission Across Client Interface</i>	74
<i>Figure 7-8: Frame Transmission with Client-Supplied FCS</i>	75
<i>Figure 7-9: Frame Transmission with Underrun</i>	76
<i>Figure 7-10: Back-to-Back Frame Transmission</i>	77
<i>Figure 7-11: Transmission of a VLAN Tagged Frame</i>	77
<i>Figure 7-12: Collision Handling: Frame Retransmission Required</i>	79
<i>Figure 7-13: Collision Handling: No Frame Retransmission Required</i>	79
<i>Figure 7-14: Interframe Gap Adjustment</i>	80

<i>Figure 7-15: Transmitter Statistics Vector Timing</i>	80
--	----

Chapter 8: Flow Control

<i>Figure 8-1: The Requirement for Flow Control</i>	83
<i>Figure 8-2: MAC Control Frame Format</i>	84
<i>Figure 8-3: Pause Request Timing with Clock Enables</i>	85
<i>Figure 8-4: Flow Control Implementation Triggered from FIFO Occupancy</i>	88

Chapter 9: Configuration and Status

<i>Figure 9-1: Configuration Register Write Timing</i>	98
<i>Figure 9-2: Configuration Register Read Timing</i>	99
<i>Figure 9-3: Address Table Write Timing</i>	100
<i>Figure 9-4: Address Table Read Timing</i>	101
<i>Figure 9-5: A Typical MDIO-managed System</i>	102
<i>Figure 9-6: MDIO Write Transaction</i>	103
<i>Figure 9-7: MDIO Read Transaction</i>	103
<i>Figure 9-8: External MDIO Interface</i>	104
<i>Figure 9-9: Internal and External MDIO Interfaces</i>	105
<i>Figure 9-10: MDIO Access Through Management Interface</i>	106

Chapter 10: Physical Interface for the 10 Mb/s and 100 Mb/s Only Ethernet MAC IP Core

<i>Figure 10-1: MII Transmitter, Receiver and Clock Logic For All Devices</i>	115
---	-----

Chapter 11: Physical Interfaces for 1 Gb/s Only Ethernet MAC IP Core

<i>Figure 11-1: GMII Transmitter Logic and Clock Logic</i>	118
<i>Figure 11-2: GMII Receiver Logic and Clock Logic for Virtex-6 Devices</i>	120
<i>Figure 11-3: GMII Receiver Logic and Clock Logic for Spartan-6 Devices</i>	121
<i>Figure 11-4: GMII Receiver Logic and Clock Logic for Virtex-5 Devices</i>	123
<i>Figure 11-5: GMII Receiver Logic and Clock Logic for Virtex-4, Spartan-3, Spartan-3E, Spartan-3A and Spartan-3A DSP Devices</i>	124
<i>Figure 11-6: Clock Resource Sharing for 1 Gb/s GMII in Virtex-6 Devices</i>	126
<i>Figure 11-7: Clock Resource Sharing for 1-Gb/s GMII in Spartan-6 Devices</i>	127
<i>Figure 11-8: Clock Resource Sharing for 1-Gb/s GMII in Virtex-5 Devices</i>	128
<i>Figure 11-9: Clock Resource Sharing for 1-Gb/s GMII in Virtex-4, Spartan-3, Spartan-3E, Spartan-3A and Spartan-3A DSP Devices</i>	129
<i>Figure 11-10: RGMII Transmitter Logic and Clock Logic for Virtex-6 Devices</i>	131
<i>Figure 11-11: RGMII Receiver Logic and Clock Logic for Virtex-6 Devices</i>	132
<i>Figure 11-12: Clock Resource Sharing for 1-Gb/s RGMII in Virtex-6 Devices</i>	133
<i>Figure 11-13: RGMII Transmitter Logic and Clock Logic for Spartan-6 Devices</i>	135
<i>Figure 11-14: RGMII Receiver Logic and Clock Logic for Spartan-6 Devices</i>	136
<i>Figure 11-15: Clock Resource Sharing for 1Gb/s RGMII in Spartan-6 Devices</i>	137
<i>Figure 11-16: RGMII Transmitter Logic and Clock Logic for Virtex-5 Devices</i>	139

<i>Figure 11-17: RGMII Receiver Logic and Clock Logic for Virtex-5 Devices.</i>	141
<i>Figure 11-18: Clock Resource Sharing for 1-Gb/s RGMII in Virtex-5 Devices</i>	142
<i>Figure 11-19: RGMII Transmitter Logic and Clock Logic for Virtex-4 Devices.</i>	144
<i>Figure 11-20: RGMII Receiver Logic and Clock Logic for Virtex-4 Devices.</i>	145
<i>Figure 11-21: Clock Resource Sharing for 1-Gb/s RGMII in Virtex-4 Devices</i>	147
<i>Figure 11-22: RGMII Transmitter Logic and Clock Logic for Spartan-3, Spartan-3A and Spartan-3A DSP Devices</i>	149
<i>Figure 11-23: RGMII Receiver Logic and Clock Logic for Spartan-3, Spartan-3A and Spartan-3A DSP Devices</i>	151
<i>Figure 11-24: Clock Resource Sharing for 1-Gb/s RGMII in Spartan-3, Spartan-3A and Spartan-3A DSP Devices</i>	153

Chapter 12: Physical Interfaces for Tri-speed (10 Mb/s, 100 Mb/s and 1 Gb/s) Ethernet MAC IP Core

<i>Figure 12-1: GMII Transmitter Logic and Clock Logic.</i>	157
<i>Figure 12-2: GMII Receiver Logic and Clock Logic for Virtex-6 Devices</i>	159
<i>Figure 12-3: GMII Receiver Logic and Clock Logic for Spartan-6 Devices.</i>	160
<i>Figure 12-4: GMII Receiver Logic and Clock Logic for Virtex-5 Devices</i>	161
<i>Figure 12-5: GMII Receiver Logic and Clock Logic for Virtex-4, Spartan-3, Spartan-3E, Spartan-3A and Spartan-3A DSP Devices</i>	163
<i>Figure 12-6: Clock Resource Sharing for 1-Gb/s GMII in Virtex-6 Devices</i>	166
<i>Figure 12-7: Clock Resource Sharing for 1-Gb/s GMII in Spartan-6 Devices</i>	168
<i>Figure 12-8: Clock Resource Sharing for 1-Gb/s GMII in Virtex-5 Devices</i>	170
<i>Figure 12-9: Clock Resource Sharing for 1-Gb/s GMII in Virtex-4, Spartan-3, Spartan-3E, Spartan-3A and Spartan-3A DSP Devices</i>	172
<i>Figure 12-10: RGMII Transmitter Logic and Clock Logic for Virtex-6 Devices.</i>	174
<i>Figure 12-11: RGMII Receiver Logic and Clock Logic for Virtex-6 Devices.</i>	176
<i>Figure 12-12: Clock Resource Sharing for 1-Gb/s RGMII in Virtex-6 Devices</i>	177
<i>Figure 12-13: RGMII Transmitter Logic and Clock Logic for Spartan-6 Devices</i>	179
<i>Figure 12-14: RGMII Receiver Logic and Clock Logic for Spartan-6 Devices</i>	180
<i>Figure 12-15: Clock Resource Sharing for RGMII in Spartan-6 Devices</i>	181
<i>Figure 12-16: RGMII Transmitter Logic and Clock Logic for Virtex-5 Devices.</i>	183
<i>Figure 12-17: RGMII Receiver Logic and Clock Logic for Virtex-5 Devices.</i>	185
<i>Figure 12-18: Clock Resource Sharing for 1-Gb/s RGMII in Virtex-5 Devices</i>	186
<i>Figure 12-19: RGMII Transmitter Logic and Clock Logic for Virtex-4 devices</i>	188
<i>Figure 12-20: RGMII Receiver Logic and Clock Logic for Virtex-4 Devices.</i>	190
<i>Figure 12-21: Clock Resource Sharing for 1-Gb/s RGMII in Virtex-4 Devices</i>	192
<i>Figure 12-22: RGMII Transmitter Logic and Clock Logic for Spartan-3, Spartan-3A and Spartan-3A DSP Devices</i>	194
<i>Figure 12-23: RGMII Receiver Logic and Clock Logic for Spartan-3E, Spartan-3A and Spartan-3A DSP Devices</i>	196
<i>Figure 12-24: Clock Resource Sharing for 1-Gb/s RGMII in Spartan-3, Spartan-3A and Spartan-3A DSP Devices</i>	198

Chapter 13: Constraining the Core

<i>Figure 13-1: Input GMII Timing</i>	205
<i>Figure 13-2: Timing Report Setup/Hold</i>	212
<i>Figure 13-3: Input RGMII Timing</i>	214
<i>Figure 13-4: Timing Report Setup/Hold</i>	222

Chapter 14: Interfacing to Other Xilinx Ethernet Cores

Chapter 15: Implementing Your Design

Chapter 16: Quick Start Example Design

<i>Figure 16-1: Default Example Design and Test Bench</i>	231
<i>Figure 16-2: Tri-Mode Ethernet MAC Main Screen</i>	233

Chapter 17: Detailed Example Design

<i>Figure 17-1: HDL Example Design</i>	246
<i>Figure 17-2: Frame Transfer across LocalLink Interface</i>	247
<i>Figure 17-3: Modification of Frame Data by Address Swap Module</i>	249
<i>Figure 17-4: Demonstration Test Bench</i>	250

Appendix A: Using the Client Side FIFO

<i>Figure A-1: Typical 10 M/100 M/1 G Ethernet FIFO Implementation</i>	255
<i>Figure A-2: Frame Transfer across LocalLink Interface</i>	256
<i>Figure A-3: Frame Transfer with Flow Control</i>	256

Appendix B: Core Verification, Compliance and Interoperability

Appendix C: Core Latency

Appendix D: Calculating the DCM Phase Shift or IODelay Tap Setting

Appendix E: Differences between the Embedded Tri-Mode Ethernet MACs and the Soft TEMAC Solution IP Core

Appendix F: Upgrading from the 1-Gigabit Ethernet MAC LogiCORE

Appendix G: 10/100/1000 Mb/s Operation Without Using Clock Enables

<i>Figure G-1: GMII/MII Transmitter Logic</i>	279
<i>Figure G-2: GMII/MII Transmit Clock Generation</i>	280
<i>Figure G-3: GMII Receiver Logic</i>	281
<i>Figure G-4: GMII/MII Receive Clock Generation for Spartan-3 and Virtex-4 Devices</i>	282
<i>Figure G-5: GMII/MII Receive Clock Generation for Virtex-5 Devices</i>	283
<i>Figure G-6: RGMII Transmitter Logic</i>	284
<i>Figure G-7: RGMII Transmit Clock Generation for Spartan-3 Devices</i>	285
<i>Figure G-8: RGMII Transmit Clock Generation for Virtex-4 Devices</i>	286
<i>Figure G-9: RGMII Transmit Clock Generation for Virtex-5 Devices</i>	287
<i>Figure G-10: RGMII Receiver Logic</i>	288
<i>Figure G-11: RGMII Receive Clock Generation for Spartan-3 and Virtex-4 Devices</i> ..	289
<i>Figure G-12: RGMII Receive Clock Generation for Virtex-5 Devices</i>	290
<i>Figure G-13: Suggested BUFGMUX Scheme</i>	291
<i>Figure G-14: Alternative BUFGMUX Scheme</i>	292

Schedule of Tables

Chapter 1: Introduction

Chapter 2: Licensing the Core

Chapter 3: Ethernet Overview

Chapter 4: Generating the Core

<i>Table 4-1: XCO File Values and Default Values.</i>	48
---	----

Chapter 5: Tri-Mode Ethernet MAC Overview

<i>Table 5-1: Client Interface Signal Pins</i>	54
<i>Table 5-2: Client Interface Signal Pins</i>	56
<i>Table 5-3: Client Interface Signal Pins</i>	56
<i>Table 5-4: Optional Management Interface Signal Pinout</i>	57
<i>Table 5-5: Alternative to the Optional Management Interface: Configuration Vector Signal Pinout</i>	57
<i>Table 5-6: Address Filter Unicast Address</i>	58
<i>Table 5-7: Clock, Speed Indication and Reset Signals</i>	58
<i>Table 5-8: GMII/MII Interface Signal Pinout</i>	59
<i>Table 5-9: MDIO Interface Signal Pinout</i>	60

Chapter 6: Designing with the Core

Chapter 7: Client Interface

<i>Table 7-1: Abbreviations Used in Timing Diagrams</i>	66
<i>Table 7-2: Bit Definition for the Receiver Statistics Vector</i>	71
<i>Table 7-3: Bit Definition for the Transmitter Statistics Vector</i>	81

Chapter 8: Flow Control

Chapter 9: Configuration and Status

<i>Table 9-1: Management Interface Transaction Types</i>	89
<i>Table 9-2: Configuration Registers</i>	90
<i>Table 9-3: Receiver Configuration Word 0</i>	91
<i>Table 9-4: Receiver Configuration Word 1</i>	91
<i>Table 9-5: Transmitter Configuration Word</i>	93
<i>Table 9-6: Flow Control Configuration Word</i>	94
<i>Table 9-7: MDIO Configuration Word</i>	94

<i>Table 9-8: MAC Speed Configuration Word</i>	95
<i>Table 9-9: Unicast Address (Word 0)</i>	96
<i>Table 9-10: Unicast Address (Word 1)</i>	96
<i>Table 9-11: Address Table Configuration (Word 0)</i>	97
<i>Table 9-12: Address Table Configuration (Word 1)</i>	97
<i>Table 9-13: Address Filter Mode</i>	98
<i>Table 9-14: Abbreviations and Terms</i>	103
<i>Table 9-15: Configuration Vector Bit Definition</i>	108

Chapter 10: Physical Interface for the 10 Mb/s and 100 Mb/s Only Ethernet MAC IP Core

Chapter 11: Physical Interfaces for 1 Gb/s Only Ethernet MAC IP Core

Chapter 12: Physical Interfaces for Tri-speed (10 Mb/s, 100 Mb/s and 1 Gb/s) Ethernet MAC IP Core

Chapter 13: Constraining the Core

<i>Table 13-1: Supported Speed Grades</i>	199
<i>Table 13-2: Input GMII Timing</i>	205
<i>Table 13-3: Input RGMII Timing</i>	214

Chapter 14: Interfacing to Other Xilinx Ethernet Cores

Chapter 15: Implementing Your Design

Chapter 16: Quick Start Example Design

Chapter 17: Detailed Example Design

<i>Table 17-1: Project Directory</i>	238
<i>Table 17-2: Component Name Directory</i>	238
<i>Table 17-3: Doc Directory</i>	239
<i>Table 17-4: Example Design Directory</i>	239
<i>Table 17-5: FIFO Directory</i>	240
<i>Table 17-6: Physical Directory</i>	240
<i>Table 17-7: Implement Directory</i>	241
<i>Table 17-8: Results Directory</i>	241
<i>Table 17-9: Simulation Directory</i>	242
<i>Table 17-10: Functional Directory</i>	242
<i>Table 17-11: Timing Directory</i>	243

Appendix A: Using the Client Side FIFO

<i>Table A-1: Receive FIFO LocalLink Interface</i>	257
<i>Table A-2: Transmit FIFO LocalLink Interface</i>	258

Appendix B: Core Verification, Compliance and Interoperability

Appendix C: Core Latency

Appendix D: Calculating the DCM Phase Shift or IODelay Tap Setting

Appendix E: Differences between the Embedded Tri-Mode Ethernet MACs and the Soft TEMAC Solution IP Core

Appendix F: Upgrading from the 1-Gigabit Ethernet MAC LogiCORE

<i>Table F-1: Rx Statistics Vector Differences</i>	274
<i>Table F-2: Tx Statistics Vector Differences</i>	274
<i>Table F-3: MDIO Configuration Word Differences between the GEMAC and TEMAC</i>	275
<i>Table F-4: GEMAC to TEMAC Netlist Port Comparison</i>	275

Appendix G: 10/100/1000 Mb/s Operation Without Using Clock Enables

<i>Table G-1: Additional Clock Ports</i>	278
--	-----

About This Guide

The *Tri-Mode Ethernet MAC User Guide* describes the function and operation of the LogiCORE™ IP Tri-Mode Ethernet MAC (TEMAC) solution, as well as information about designing, customizing, and implementing the solution.

Guide Contents

This guide contains these chapters and appendixes:

- [About this Guide](#) introduces the organization and purpose of the user guide and the conventions used in this document.
- [Chapter 1, Introduction](#), describes the core and related information, including recommended design experience, additional resources, technical support, and submitting feedback to Xilinx.
- [Chapter 2, Licensing the Core](#), provides information about installing and licensing the core.
- [Chapter 3, Ethernet Overview](#), provides a brief introduction to Ethernet and Ethernet terminology.
- [Chapter 4, Generating the Core](#), describes how to generate the core and defines the available customization options.
- [Chapter 5, Tri-Mode Ethernet MAC Overview](#), provides an overview of the core features, architecture and discusses the core interface signals.
- [Chapter 6, Designing with the Core](#), provides general guidelines for creating designs using the core.
- [Chapter 7, Client Interface](#), provides information about using the client-side interface of the core.
- [Chapter 8, Flow Control](#), details flow control usage and capabilities of the core.
- [Chapter 9, Configuration and Status](#), describes how to operate the Management Interface.
- [Chapter 10, Physical Interface for the 10 Mb/s and 100 Mb/s Only Ethernet MAC IP Core](#), describes connecting the TEMAC with a MII to support 10 Mb/s or 100 Mb/s ethernet speeds.
- [Chapter 11, Physical Interfaces for 1 Gb/s Only Ethernet MAC IP Core](#), describes connecting the TEMAC with a GMII/MII or RGMII to support 1 Gb/s ethernet speed.
- [Chapter 12, Physical Interfaces for Tri-speed \(10 Mb/s, 100 Mb/s and 1 Gb/s\) Ethernet MAC IP Core](#), describes connecting the TEMAC with a GMII/MII or RGMII to support 10 Mb/s, 100 Mb/s and 1 Gb/s ethernet speeds.
- [Chapter 13, Constraining the Core](#), describes the constraints associated with the core.

- [Chapter 14, Interfacing to Other Xilinx Ethernet Cores](#), describes how to interface the core to the Ethernet 1000BASE-X PCS/PMA or SGMII core to provide SGMII functionality. In addition, the integration of the core with the Ethernet Statistics core is discussed.
- [Chapter 15, Implementing Your Design](#), provides instructions for setting up the synthesis, simulation, and implementation environment, and how to generate a bitstream.
- [Chapter 16, Quick Start Example Design](#), describes how to quickly generate the example design using the default parameters.
- [Chapter 17, Detailed Example Design](#), provides detailed information about the example design and demonstration test bench.
- [Appendix A, Using the Client Side FIFO](#), describes the operation of the FIFO included in the core example design.
- [Appendix B, Core Verification, Compliance and Interoperability](#), describes how the core was verified and certified for compliance, as well as its inter operability with other devices.
- [Appendix C, Core Latency](#), defines the core latency.
- [Appendix D, Calculating the DCM Phase Shift or IODelay Tap Setting](#), provides instructions for calculating a DCM phase-shift value to meet input setup and hold timing; DCMs are used with Spartan®-3, Spartan-3E, Spartan-3A, Spartan-3A DSP and Virtex®-4 families.
- [Appendix E, Differences between the Embedded Tri-Mode Ethernet MACs and the Soft TEMAC Solution IP Core](#), describes the differences between the Embedded Tri-Mode Ethernet MAC blocks, available in selected Virtex-6, Virtex-5 and Virtex-4 devices, and the soft IP cores, Tri-Mode Ethernet MAC (TEMAC), solutions provided by the CORE Generator™ tool.
- [Appendix F, Upgrading from the 1-Gigabit Ethernet MAC LogiCORE](#).
- [Appendix G, 10/100/1000 Mb/s Operation Without Using Clock Enables](#), describes the legacy mode for use of the core without clock enables for tri-speed ethernet operation.

Additional Resources

To find additional documentation, see the Xilinx website at:

www.xilinx.com/support/documentation/index.htm.

To search the Answer Database of silicon, software, and IP questions and answers, or to create a technical support WebCase, see the Xilinx website at:

www.xilinx.com/support

Conventions

This document uses the following conventions. An example illustrates each convention.

Typographical

These typographical conventions are used in this document:

Convention	Meaning or Use	Example
Courier font	Messages, prompts, and program files that the system displays. Signal names in text also.	speed grade: - 100
Courier bold	Literal commands that you enter in a syntactical statement	ngdbuild <i>design_name</i>
Helvetica bold	Commands that you select from a menu	File Ø Open
	Keyboard shortcuts	Ctrl+C
<i>Italic font</i>	Variables in a syntax statement for which you must supply values	ngdbuild <i>design_name</i>
	References to other manuals	See the <i>User Guide</i> for details.
	Emphasis in text	If a wire is drawn so that it overlaps the pin of a symbol, the two nets are <i>not</i> connected.
Dark Shading	Items that are not supported or reserved	This feature is not supported
Square brackets []	An optional entry or parameter. However, in bus specifications, such as bus[7:0] , they are required.	ngdbuild [<i>option_name</i>] <i>design_name</i>
Braces { }	A list of items from which you must choose one or more	lowpwr = { on off }
Vertical bar	Separates items in a list of choices	lowpwr = { on off }
Angle brackets < >	User-defined variable or in code samples	<directory name>
Vertical ellipsis . . .	Repetitive material that has been omitted	IOB #1: Name = QOUT' IOB #2: Name = CLKIN' . . .
Horizontal ellipsis ...	Repetitive material that has been omitted	allow block <i>block_name</i> <i>loc1</i> <i>loc2</i> ... <i>locn</i> ;

Convention	Meaning or Use	Example
Notations	The prefix '0x' or the suffix 'h' indicate hexadecimal notation	A read of address 0x00112975 returned 45524943h.
	An '_n' means the signal is active low	usr_teof_n is active low.

Online Document

These linking conventions are used in this document:

Convention	Meaning or Use	Example
Blue text	Cross-reference link to a location in the current document	See the section Guide Contents for details. See Title Formats in Chapter 1 for details.
Blue, underlined text	Hyperlink to a website (URL)	Go to www.xilinx.com for the latest speed files.

List of Acronyms

This table describes acronyms used in this manual.

Acronym	Spelled Out
AVB	Audio Video Bridging
CLB	Configurable Logic Block
CML	Current Mode Logic
CRC	Cyclic Redundancy Check
CSMA/CD	Carrier Sense Multiple Access with Collision Detection
DA	Destination Address
DCM	Digital Clock Manager
DDR	Double Data Rate
DMA	Direct Memory Access
DSP	Digital Signal Processor
EDK	Embedded Development Kit
FCS	Frame Check Sequence
FIFO	First In First Out
FPGA	Field Programmable Gate Array
Gb/s	Gigabits per second
GEMAC	Gigabit Ethernet Media Access Controller
GMII	Gigabit Media Independent Interface
GUI	Graphical User Interface

Acronym	Spelled Out
HDL	Hardware Description Language
HSTL	High-Speed Transistor Logic
I/F	Interface
IFG	Interframe GAP
IO	Input/Output
IOB	Input/Output Block
IP	Intellectual Property
LAN	Local Area Network
L/T	Length Type
LUT	Lookup Table
LVTTL	Low Voltage Transistor Transistor Logic
MAC	Media Access Controller
Mb/s	Megabits per second
MDC	Management Data Clock
MMD	MDIO Managed Device
MDIO	Management Data Input/Output
MGT	Multi-Gigabit Transceiver
MHz	Mega Hertz
MII	Media Independent Interface
ms	milliseconds
NCD	Native Circuit Description
NGC	Native Generic Circuit
NGD	Native Generic Database
ns	nanoseconds
OSI	Open Systems Interconnection
PCB	Printed Circuit Board
PCF	Physical Constraints File
PCS	Physical Coding Sublayer
PHY	physical-side interface
PLB	Processor Local Bus
PMA	Physical Medium Attachment
PMD	Physical Medium Dependent
RGMII	Reduced Gigabit Media Independent Interface

Acronym	Spelled Out
RTL	Resistor Transistor Logic
SA	Source Address
SFP	Small Form-Factor Pluggable
SGMII	Serial Gigabit Media Independent Interface
STA	Station Management Entity
TBI	Ten-Bit-Interface
TCP/IP	TCP/IP Transmission Control Protocol/Internet Protocol
TEMAC	Tri-Mode Ethernet MAC
TIG	Timing Ignore
TWR	Timing Wizard Report
UCF	User Constraints File
VHDL	VHSIC Hardware Description Language (VHSIC an acronym for Very High-Speed Integrated Circuits)
VLAN	Virtual LAN (Local Area Network)
XCO	Xilinx [®] CORE Generator [™] software core source file
XST	Xilinx Synthesis Technology

Introduction

The Tri-Mode Ethernet MAC (TEMAC) solution comprises the 10/100/1000 Mb/s, 1 Gb/s and 10/100 Mb/s IP cores which are fully-verified designs that support Verilog-HDL and VHDL. In addition, the example design provided with the core is in both Verilog and VHDL.

This chapter introduces the TEMAC solution and provides related information, including recommended design experience, additional resources, technical support, and submitting feedback to Xilinx.

About the Core

The TEMAC solution is generated through the Xilinx® CORE Generator™ software, included in the latest IP Update on the Xilinx IP Center. For detailed information about the core, see the TEMAC [product page](#). For information about system requirements and licensing the core, see [Chapter 2, Licensing the Core](#).

System Requirements

Windows

- Windows XP Professional 32-bit/64-bit
- Windows Vista Business 32-bit/64-bit

Linux

- Red Hat Enterprise Linux WS v4.0 32-bit/64-bit
- Red Hat Enterprise Desktop v5.0 32-bit/64-bit (with Workstation Option)
- SUSE Linux Enterprise (SLE) desktop and server v10.1 32-bit/64-bit

Software

- ISE® software v13.1

Recommended Design Experience

Although the TEMAC solution is fully-verified, the challenge associated with implementing a complete design varies depending on the configuration and functionality of the application. For best results, previous experience building high performance, pipelined FPGA designs using Xilinx implementation software and User Constraint Files (UCF) is recommended. Contact your local Xilinx representative for a closer review and estimation for your specific requirements.

Additional Core Resources

For details and updates about the core, see the data sheet, available from the TEMAC [product page](#). From the document directory, available after generating the core, all product documentation, including the release notes, are available.

Related Xilinx Ethernet Products and Services

See the Ethernet Products and Services page at:

www.xilinx.com/products/design_resources/conn_central/protocols/gigabit_ethernet.htm

Specifications

- *IEEE 802.3-2008*
- *Reduced Gigabit Media Independent Interface (RGMII), version 2.0*

Technical Support

For technical support, see support.xilinx.com/. Questions are routed to a team of engineers with expertise using the TEMAC solution.

Xilinx provides technical support for use of this product as described in the *Tri-Mode Ethernet MAC User Guide*. Xilinx cannot guarantee timing, functionality, or support of this product for designs that do not follow these guidelines.

Feedback

Xilinx welcomes comments and suggestions about the TEMAC solution and the documentation supplied with the core.

Tri-Mode Ethernet MAC Core

For comments or suggestions about the core, submit a WebCase from www.xilinx.com/support/clearxpress/websupport.htm. Be sure to include the following information:

- Product name
- Core version number
- Configuration mode (10/100/1000 Mb/s, 1 Gb/s, 10/100 Mb/s)
- Explanation of your comments

Document

For comments or suggestions about the core, submit a WebCase from www.xilinx.com/support/clearxpress/websupport.htm. Be sure to include the following information:

- Document title
- Document number
- Page number(s) to which your comments refer
- Explanation of your comments

Licensing the Core

The Tri-Mode Ethernet MAC (TEMAC) solution consists of three Xilinx® LogiCORE™ IP cores. This chapter provides licensing instructions for the 10/100/1000 Mb/s Tri-Mode Ethernet MAC, 1Gb/s Ethernet MAC and the 10/100 Mb/s Ethernet MAC.

You must obtain the appropriate licenses before using the cores in your designs. These three IP cores are provided under the terms of the [Xilinx LogiCORE IP Site License Agreement](#) or [Xilinx LogiCORE IP Project License Agreement](#), which conforms to the terms of the [SignOnce](#) IP License standard defined by the Common License Consortium. Purchase of the Tri-Mode Ethernet MAC core license includes licensing the 10/100/1000 Mb/s Tri-Mode, 1Gb/s and the 10/100 Mb/s Ethernet MAC. Purchase of the 10/100 MAC core license only entitles full access to the 10/100 Mb/s IP. Purchase of a core entitles you to technical support and access to updates for a period of one year.

Before you Begin

This chapter assumes that you have installed all required software specified on the [product page](#) for this core.

License Options

The TEMAC solution provides three licensing options. After installing the required Xilinx ISE® software and IP Service Packs, choose a license option.

Simulation Only

The Simulation Only Evaluation license key is provided with the Xilinx CORE Generator™ tool. This key lets you assess core functionality with either the example design provided with the TEMAC solution, or alongside your own design and demonstrates the various interfaces to the core in simulation. (Functional simulation is supported by a dynamically generated HDL structural model.)

Full System Hardware Evaluation

The Full System Hardware Evaluation license is available at no cost and lets you fully integrate the core into an FPGA design, place and route the design, evaluate timing, and perform back-annotated gate-level simulation of the TEMAC solution using the example design and the demonstration test bench provided with the core.

In addition, the license lets you generate a bitstream from the placed and routed design, which can then be downloaded to a supported device and tested in hardware. The core can be tested in the target device for a limited time before *timing out* (ceasing to function) at which time it can be reactivated by reconfiguring the device.

Full

The Full license key is available when you purchase the core and provides full access to all core functionality both in simulation and in hardware, including: Functional simulation support

- Back annotated gate-level simulation support
- Functional simulation support
- Full implementation support including place and route and bitstream generation
- Full functionality in the programmed device with no time outs

Obtaining your License Key

This section contains information about obtaining a simulation, full system hardware, and full license keys.

Simulation License

No action is required to obtain the Simulation Only Evaluation license key; it is provided by default with the Xilinx CORE Generator software.

Full System Hardware Evaluation License

To obtain a Full System Hardware Evaluation license, do the following:

1. Navigate to the [product page](#) for this core.
2. Click Evaluate.
3. Follow the instructions to install the required Xilinx ISE software and IP Service Packs.

Obtaining a Full License

To obtain a Full license key, you must purchase a license for the core. After doing so, click the “Access Core” link on the Xilinx.com IP core product page for further instructions.

Installing your License File

The Simulation Only Evaluation license key is provided with the ISE software CORE Generator system and does not require installation of an additional license file. For the Full System Hardware Evaluation license and the Full license, an email will be sent to you containing instructions for installing your license file. Additional details about IP license key installation can be found in the ISE Design Suite Installation, Licensing and Release Notes document.

Ethernet Overview

This chapter provides an overview of typical Ethernet applications and the Ethernet protocol.

This chapter contains the following sections:

- [Typical Ethernet Application Overview](#)
- [Ethernet Protocol Overview](#)

Typical Ethernet Application Overview

Typical applications for the Ethernet MAC include:

- [Ethernet Switch or Router](#)
- [Ethernet Communications Port for an Embedded Processor](#)

Ethernet Switch or Router

[Figure 3-1](#) illustrates a typical application for a single Ethernet MAC. The PHY side of the core is connected to an off-the-shelf Ethernet PHY device, which performs the BASE-T standard at 1 Gb/s, 100 Mb/s, and 10 Mb/s speeds. The PHY device can be connected using any of the following supported interfaces: GMII/MII, RGMII, or, by additionally using the *Ethernet 1000BASE-X PCS/PMA or SGMII LogiCORE™*, SGMII.

The client side of the Ethernet MAC is connected to a FIFO to complete a single Ethernet port. This port is connected to a Switch or Routing matrix, which can contain several ports.

The CORE Generator™ tool provides an example design for the TEMAC solution for any of the supported physical interfaces. A FIFO example is also generated, which can be used as the FIFO in the illustration, for a typical application

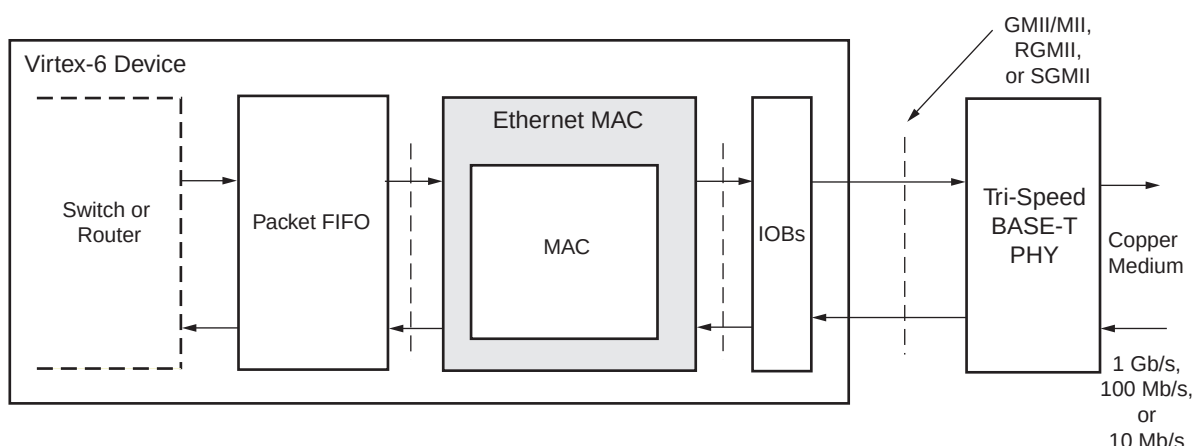


Figure 3-1: Typical Application: Ethernet Switch or Router

Ethernet Communications Port for an Embedded Processor

Figure 3-2 illustrates a typical application for a single Ethernet MAC. The PHY side of the core is connected to an off-the-shelf Ethernet PHY device, which performs the BASE-T standard at 1 Gb/s, 100 Mb/s, and 10 Mb/s speeds. The PHY device can be connected using any of the following supported interfaces: GMII/MII, RGMII, or, by additionally using the *Ethernet 1000BASE-X PCS/PMA or SGMII LogiCORE*, SGMII.

The client side of the MAC is connected to a processor system via a processor DMA engine. This processor could be running a communications stack, such as the TCP/IP protocol. For applications such as this, see the Xilinx Platform Studio (XPS), Embedded Development Kit (EDK) IP portfolio. This portfolio contains additional IP to connect the client interface of the MAC to the DMA port of a processor. *XPS LL TEMAC Data Sheet (DS537)*, describes the XPS_LL_TEMAC, which can be instantiated for an intended processor application.

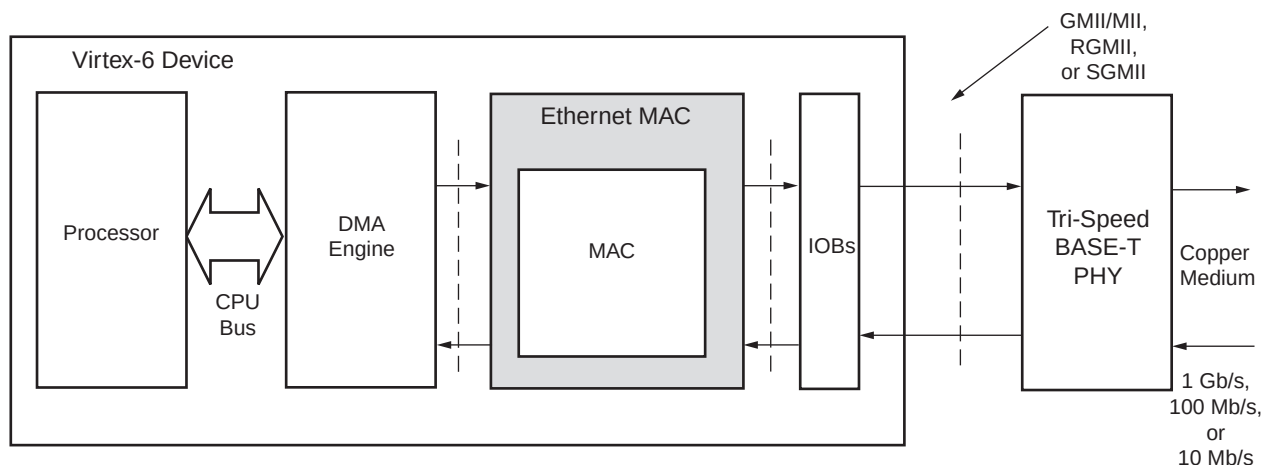


Figure 3-2: Typical Application: Ethernet Communications Port for Embedded Processor

Ethernet Protocol Overview

This section gives an overview of where the Ethernet MAC fits into an Ethernet system and provides a description of some basic Ethernet terminology.

Ethernet Sublayer Architecture

Figure 3-3 illustrates the relationship between the OSI reference model and the Ethernet MAC, as defined in IEEE Std 802.3-2008. The grayed-in layers show the functionality that the Ethernet MAC handles. Figure 3-3 also shows where the supported physical interfaces fit into the architecture.

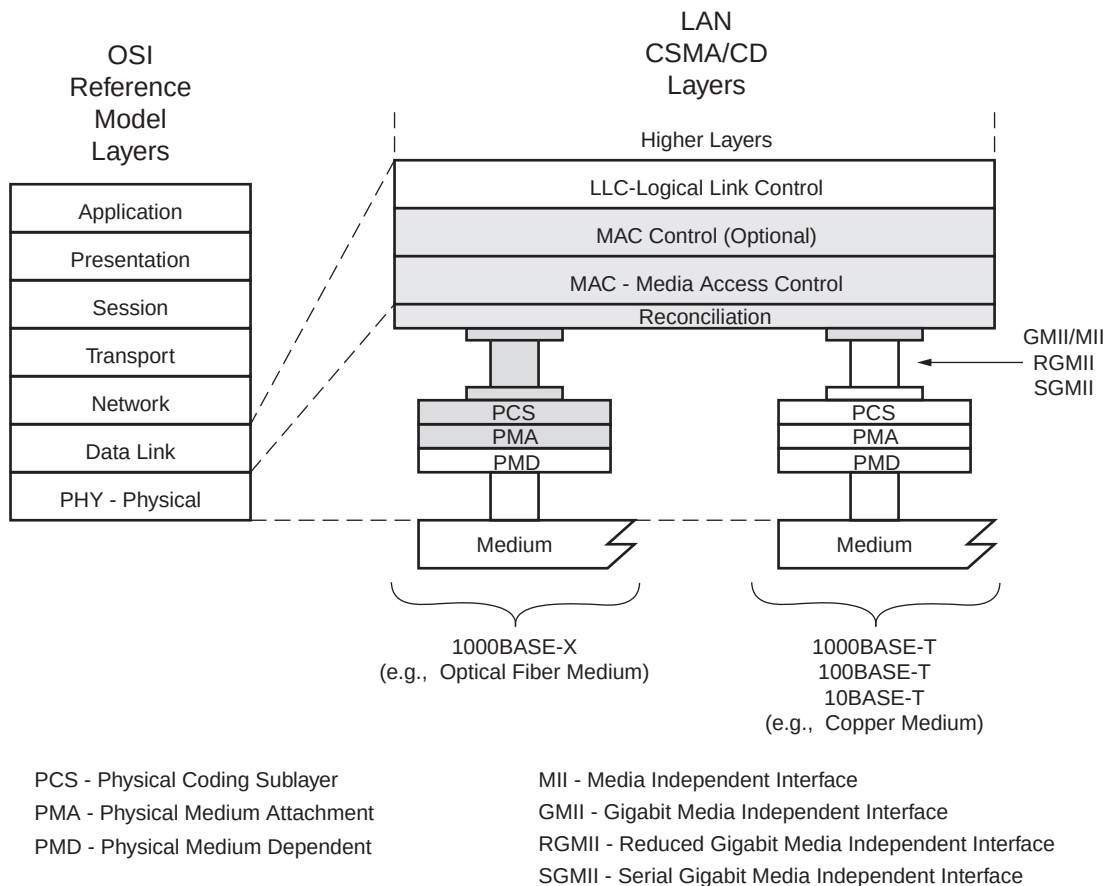


Figure 3-3: IEEE Std 802.3-2008 Ethernet Model

MAC and MAC CONTROL Sublayer

The Ethernet MAC is defined in IEEE Std 802.3-2008, clauses 2, 3, and 4. A MAC is responsible for the Ethernet framing protocols described in Ethernet Data Format and error detection of these frames. The MAC is independent of and can connect to any type of physical layer device.

The MAC CONTROL sublayer is defined in IEEE Std 802.3-2008, clause 31. This provides real-time flow control manipulation of the MAC sublayer.

Both the MAC CONTROL and MAC sublayers are provided by the Ethernet MAC in all modes of operation.

Physical Sublayers PCS, PMA, and PMD

The combination of the Physical Coding Sublayer (PCS), the Physical Medium Attachment (PMA), and the Physical Medium Dependent (PMD) sublayer constitute the physical layers for the protocol. Two main physical standards are specified:

- **BASE-T** PHYs provide a link between the MAC and copper mediums. This functionality is not offered within the TEMAC. However, external BASE-T PHY devices are readily available on the market. These can connect to the Ethernet MAC, using GMII/MII, RGMII, or, by additionally using the *Ethernet 1000BASE-X PCS/PMA or SGMII LogiCORE*, SGMII interfaces.
- **BASE-X** PHYs provide a link between the MAC and (usually) fiber optic mediums. The TEMAC is capable of supporting the 1 Gb/s BASE-X standard; 1000BASE-X PCS and PMA sublayers can be offered by connecting the TEMAC to the *Ethernet 1000BASE-X PCS/PMA or SGMII LogiCORE*.

Ethernet Data Format

Ethernet data is encapsulated in frames, as shown in Figure 3-4, for standard Ethernet frames. The fields in the frame are transmitted from left to right. The bytes within the fields are transmitted from left to right (from least significant bit to most significant bit unless specified otherwise). The Ethernet MAC can handle jumbo Ethernet frames where the data field can be much larger than 1500 bytes.

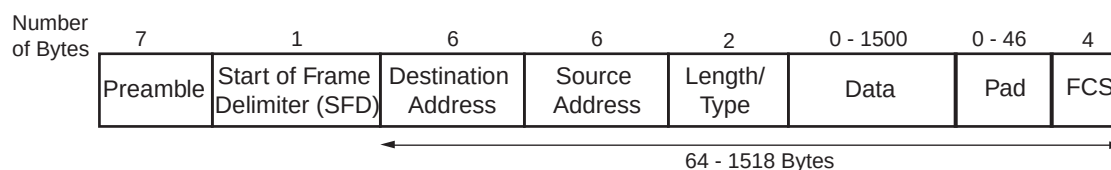


Figure 3-4: Standard Ethernet Frame Format

The Ethernet MAC can also accept VLAN frames. The VLAN frame format is shown in Figure 3-5. If the frame is a VLAN type frame, the Ethernet MAC accepts four additional bytes.

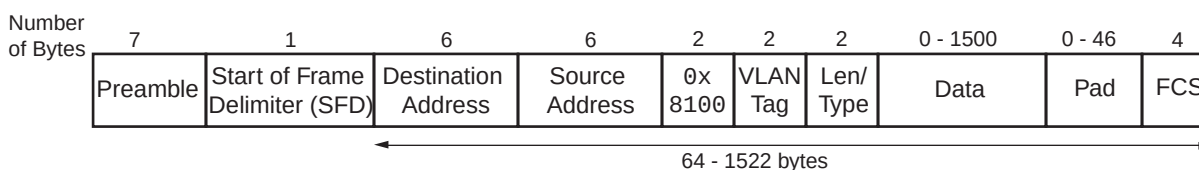


Figure 3-5: Ethernet VLAN Frame Format

Ethernet PAUSE/flow control frames can be transmitted and received by the Ethernet MAC. Figure 8-2, page 84 shows how a PAUSE/flow control frame differs from the standard Ethernet frame format.

The following subsections describe the individual fields of an Ethernet frame and some basic functionality of the Ethernet MAC.

Preamble

For transmission, this field is automatically inserted by the Ethernet MAC. The preamble field was historically used for synchronization and contains seven bytes with the pattern 0x55, transmitted from left to right. For reception, this field is always stripped from the incoming frame, before the data is passed to the client. The Ethernet MAC can receive Ethernet frames, even if the preamble does not exist, as long as a valid start of frame delimiter is available.

Start of Frame Delimiter

The start of frame delimiter field marks the start of the frame and must contain the pattern 0xD5.

For transmission on the physical interface, this field is automatically inserted by the Ethernet MAC. For reception, this field is always stripped from the incoming frame before the data is passed to the client.

MAC Address Fields

MAC Address

The least significant bit of a MAC address determines if the address is an individual/unicast (0) or group/multicast (1) address. Multicast addresses are used to group logically related stations. The broadcast address (destination address field is all 1s) is a multicast address that addresses all stations on the LAN. The Ethernet MAC supports transmission and reception of unicast, multicast, and broadcast packets.

The address is transmitted in an ethernet frame least significant bit first: so the bit representing an individual or group address is the first bit to appear in an address field of an ethernet frame.

Destination Address

This MAC Address field is the first field of the Ethernet frame that is always provided in the packet data for transmissions and is always retained in the receive packet data. It provides the MAC address of the intended recipient on the network.

Source Address

This MAC Address field is the second field of the Ethernet frame that is always provided in the packet data for transmissions and is always retained in the receive packet data. It provides the MAC address of the frame's initiator on the network.

For transmission, the source address of the Ethernet frame should always be provided by the client because it is unmodified by the TEMAC.

Length/Type

The value of this field determines if it is interpreted as a length or a type field, as defined by IEEE Std 802.3-2008. A value of 1536 decimal or greater is interpreted by the Ethernet MAC as a type field.

When used as a length field, the value in this field represents the number of bytes in the following data field. This value does not include any bytes that can be inserted in the pad field following the data field.

A length/type field value of 0x8100 indicates that the frame is a VLAN frame, and a value of 0x8808 indicates a PAUSE MAC control frame.

For transmission, the Ethernet MAC does not perform any processing of the length/type field.

For reception, if this field is a length field, the Ethernet MAC receive engine interprets this value and removes any padding in the pad field (if necessary). If the field is a length field and length/type checking is enabled, the Ethernet MAC compares the length against the actual data field length and flags an error if a mismatch occurs. If the field is a type field, the Ethernet MAC ignores the value and passes it along with the packet data with no further processing. The length/type field is always retained in the receive packet data.

Data

The data field can vary from 0 to 1,500 bytes in length for a normal frame. The Ethernet MAC can handle jumbo frames of any length.

This field is always provided in the packet data for transmissions and is always retained in the receive packet data.

Pad

The pad field can vary from 0 to 46 bytes in length. This field is used to ensure that the frame length is at least 64 bytes in length (the preamble and SFD fields are not considered part of the frame for this calculation), which is required for successful CSMA/CD operation. The values in this field are used in the frame check sequence calculation but are not included in the length field value, if it is used. The length of this field and the data field combined must be at least 46 bytes. If the data field contains 0 bytes, the pad field is 46 bytes. If the data field is 46 bytes or more, the pad field has 0 bytes.

For transmission, this field can be inserted automatically by the Ethernet MAC or can be supplied by the client. If the pad field is inserted by the Ethernet MAC, the FCS field is calculated and inserted by the Ethernet MAC. If the pad field is supplied by the client, the FCS can be either inserted by the Ethernet MAC or provided by the client, as indicated by a configuration register bit.

For reception, if the length/type field has a length interpretation, any pad field in the incoming frame is not be passed to the client, unless the Ethernet MAC is configured to pass the FCS field on to the client.

FCS

The value of the FCS field is calculated over the destination address, source address, length/type, data, and pad fields using a 32-bit Cyclic Redundancy Check (CRC), as defined in IEEE Std 802.3-2008 para. 3.2.8:

$$G(x) = x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x^1 + x^0$$

The CRC bits are placed in the FCS field with the x^{31} term in the left-most bit of the first byte, and the x^0 term is the right-most bit of the last byte (that is, the bits of the CRC are transmitted in the order $x^{31}, x^{30}, \dots, x^1, x^0$).

For transmission, this field can be either inserted automatically by the Ethernet MAC or supplied by the client, as indicated by a configuration register bit.

For reception, the incoming FCS value is verified on every frame. If an incorrect FCS value is received, the Ethernet MAC indicates to the client that it has received a bad frame. The FCS field can either be passed on to the client or be dropped by the Ethernet MAC, as indicated by a configuration register bit.

Frame Transmission and Interframe Gap

Frames are transmitted over the Ethernet medium with an interframe gap, as specified by the IEEE Std 802.3-2008, to be 96 bit times (9.6 μ s for 10 Mb/s, 0.96 μ s for 100 Mb/s, and 96 ns for 1 Gb/s). This value is a minimum and can be increased with a resulting decrease in throughput.

The process for frame transmission is different for half-duplex and full-duplex systems.

Half-Duplex Frame Transmission

In a half-duplex system, the CSMA/CD media access method defines how two or more stations share a common medium.

1. Even when it has nothing to transmit, the Ethernet MAC monitors the Ethernet medium for traffic by watching the carrier sense signal (CRS) from the external PHY. Whenever the medium is busy (CRS = 1), the Ethernet MAC defers to the passing frame by delaying any pending transmission of its own.
2. After the last bit of the passing frame (when the carrier sense signal changes from TRUE to FALSE), the Ethernet MAC starts the timing of the interframe gap.
3. The Ethernet MAC resets the interframe gap timer if the carrier sense becomes TRUE during the period defined by "interframe gap part 1 (IFG1)." IEEE Std 802.3-2008 states that this should be the first 2/3 of the interframe gap timing interval (64 bit times) but can be shorter and as small as zero. The purpose of this option is to support a possible brief failure of the carrier sense signal during a collision condition and is described in paragraph 4.2.3.2.1 of the IEEE standard.
4. The Ethernet MAC does not reset the interframe gap timer if carrier sense becomes TRUE during the period defined by "interframe gap part 2 (IFG2)" to ensure fair access to the bus. IEEE Std 802.3-2008 states that this should be the last 1/3 of the interframe gap timing interval.

If, after initiating a transmission, the message collides with the message of another station (COL = 1), then each transmitting station intentionally continues to transmit (jam) for an additional predefined period (32 bit times for 10/100 Mb/s) to ensure propagation of the collision throughout the system. The station remains silent for a random amount of time (back off) before attempting to transmit again.

A station can experience a collision during the beginning of its transmission (the collision window) before its transmission has had time to propagate to all stations on the bus. After the collision window has passed, a transmitting station has acquired the bus. Subsequent collisions (late collisions) are avoided because all other (properly functioning) stations are assumed to have detected the transmission and are deferring to it.

Full-Duplex Frame Transmission

In a full-duplex system, there is a point-to-point dedicated connection between two Ethernet devices, capable of simultaneous transmit and receive with no possibility of collisions. The Ethernet MAC does not use the carrier sense signal from the external PHY because the medium is not shared, and the Ethernet MAC only needs to monitor its own transmissions. After the last bit of an Ethernet MAC frame transmission, the Ethernet MAC starts the interframe gap timer and defers transmissions until the IFG count completes. The minimum value supported for the IFG depends on the TEMAC solution options and the current mode of operation. If the TEMAC solution has been built with half-duplex support then the IFG delay will be 96 bit times, or when IFG Adjustment is enabled, the greater of 64 bit times, and the value presented on `CLIENTEMACTXIFGDELAY`. If the TEMAC solution has been built with only full-duplex support then the IFG delay will be 96 bit times, or when IFG Adjustment is enabled, the greater of 32 bit times, and the value presented on `CLIENTEMACTXIFGDELAY`.

Generating the Core

The TEMAC solution which comprises the 10/100/1000 Mb/s, 1 Gb/s and 10/100 Mb/s IP cores are generated through the Xilinx® CORE Generator™ tool using a graphical user interface (GUI). This chapter describes the GUI options used to generate and customize the core.

GUI Interface

Figure 4-1 displays the TEMAC core customization screen.

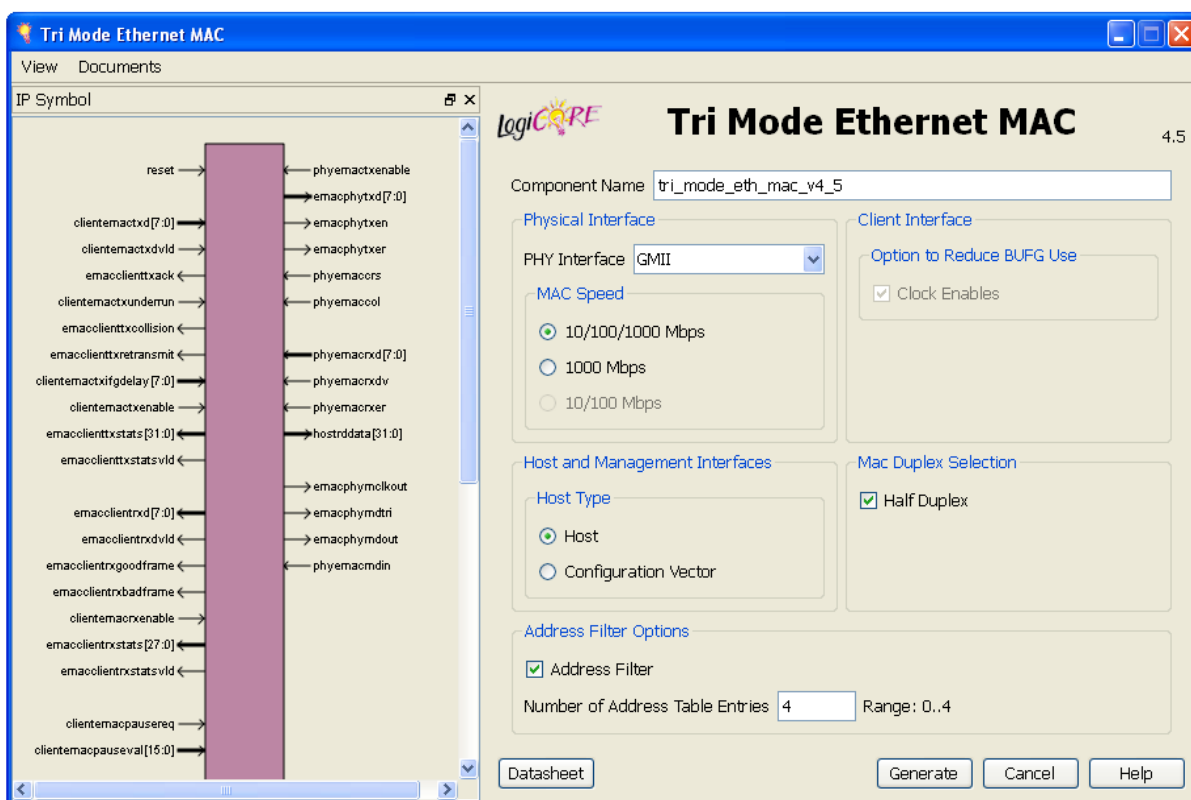


Figure 4-1: Core Customization Screen

For general help starting and using the CORE Generator tool on your system, see the documentation supplied with the Xilinx ISE® software, including the *CORE Generator Guide* at www.xilinx.com/support/software_manuals.htm.

Component Name

The component name is used as the base name of the output files generated for the core. Names must begin with a letter and must be composed from the following characters: a through z, 0 through 9 and “_”.

Physical Interface

Three physical interface types are available for the core:

- GMII. The Gigabit Media Independent Interface (GMII) is defined by the IEEE802.3 specification; it can provide support for Ethernet operation at 10 Mb/s, 100 Mb/s and 1 Gb/s speeds.
- MII. The Media Independent Interface (MII) is defined by the IEEE802.3 specification; it can provide support for Ethernet operation at 10 Mb/s and 100 Mb/s speeds.
- RGMII. The Reduced Gigabit Media Independent Interface (RGMII) is, effectively, a Double Data Rate version of GMII; it can provide support for Ethernet operation at 10 Mb/s, 100 Mb/s and 1 Gb/s speeds.

The choice of physical interface determines the content of the example design delivered with the core where the external GMII, MII or RGMII is described in HDL.

There is no change in the core netlist for GMII/MII verses RGMII operation.

The default is to use GMII.

MAC Speed

The TEMAC solution can provide support for 1 Gb/s speed only operation; 10 Mb/s and 100 Mb/s speed operation; full tri-speed operation (10 Mb/s, 100 Mb/s and 1 Gb/s speed capability).

The available choice for speed support selection is dependent on the chosen Physical Interface:

- If GMII or RGMII is selected, then Tri-speed operation and 1 Gb/s only operation are available for selection.
- If MII is selected, then only 10 Mb/s and 100 Mb/s operation is available.

Clock Enables

This option is only available for selection when the MAC speed is selected for tri-speed operation. The default and recommendation for tri-speed designs is to use clock enables; this is the most efficient use of device clock resources. For further information on clock and clock enable resources, see [Chapter 12, Physical Interfaces for Tri-speed \(10 Mb/s, 100 Mb/s and 1 Gb/s\) Ethernet MAC IP Core](#).

Operation at tri-speed without using the clock enables is inefficient in its use of device clock resources; this is a legacy mode of operation and is described in [Appendix G, 10/100/1000 Mb/s Operation Without Using Clock Enables](#).

At 1 Gb/s-only operation, the clock enable option is not available for selection and can be ignored. For further information on clock resources for 1 Gb/s only operation, see [Chapter 11, Physical Interfaces for 1 Gb/s Only Ethernet MAC IP Core](#).

At 10 Mb/s and 100 Mb/s only operation, clock enables are always provided and so this option is not available for selection. For further information on clock resources for 10 Mb/s and 100 Mb/s only operation, see [Chapter 10, Physical Interface for the 10 Mb/s and 100 Mb/s Only Ethernet MAC IP Core](#).

Half Duplex

The TEMAC solution always provides support for full-duplex Ethernet. However, to provide half-duplex operation, further FPGA logic resources are required. Because many applications require only full-duplex support, the half-duplex logic is therefore optional.

When the core is generated with half-duplex logic, full or half-duplex operation can be selected using TEMAC configuration.

The default is to include half-duplex support.

Note: If the selected family/physical interface requires the use of a DCM on the physical interface receive path, as defined in the physical interface implementation sections, then 1G half-duplex is not supported.

Management Interface

Select this option if you wish to include the optional Management Interface for TEMAC configuration (see [Using the Optional Management Interface, page 89](#)). If this option is not selected, the core is generated with a replacement configuration vector (see [Accessing Configuration without the Management Interface, page 108](#)).

The default is to have the Management Interface.

Address Filter

It is possible to generate the core with an address filter, which prevents the reception of frames that are not addressed to this MAC. See [Address Filter, page 70](#).

The default is to use the Address Filter.

Number of Address Table Entries

The Address Filter can be generated with an address table that holds up to 4 additional valid MAC addresses. You can select an integer between 0 and 4 to define the number of addresses that are present in the table.

The default is to use 4 address table entries.

Parameter Values in the XCO File

An XCO file is produced by CORE Generator software whenever a core is customized and generated: it records all options used in the generation of the core, and lists these as parameters. Existing or manually created XCO files can be imported into a CORE Generator software project.

XCO file parameter names and their values are identical to the names and values shown in the GUI, except that underscore characters (_) are used instead of spaces. The text in an XCO file is case insensitive.

[Table 4-1](#) shows the XCO file parameters and values, and summarizes the GUI defaults. The following is an example of the CSET parameters in an XCO file:

```
CSET component_name=tri_mode_eth_mac_v4_5
CSET physical_interface=GMII
CSET mac_speed=Tri_speed
CSET clock_enables=true
CSET half_duplex=true
CSET management_interface=true
CSET address_filter=true
CSET number_of_address_table_entries=4
```

Table 4-1: XCO File Values and Default Values

Parameter	XCO File Values	Default GUI Setting
component_name	ASCII text starting with a letter and based upon the following character set: a..z, 0..9 and _	tri_mode_eth_mac_v4_5
physical_interface	One of the following keywords: mii, gmii, rgmii	gmii
mac_speed	One of the following keywords: tri_speed, 1000_Mb/s, 10_100_Mb/s	tri_speed
clock_enables	One of the following keywords: true, false	true
half_duplex	One of the following keywords: true, false	true
management_interface	One of the following keywords: true, false	true
address_filter	One of the following keywords: true, false	true
number_of_address_table_entries	Integer in the range 0 - 4	4

Output Generation

The output files generated from the CORE Generator tool are placed in the CORE Generator tool project directory. The list of output files includes

- the netlist file
- supporting CORE Generator software files
- release notes and other documentation
- subdirectories containing example design files
- scripts to run the core through the Xilinx back-end tools and to simulate the core using the Mentor Graphics ModelSim, Cadence IES, or Synopsys simulators.

Tri-Mode Ethernet MAC Overview

This chapter introduced the TEMAC solution architecture including all interfaces and the major functional blocks.

This chapter contains the following sections:

- [Key Features](#)
- [Architecture Description](#)
- [Core Interfaces](#)

Key Features

The key features of the TEMAC solution are:

- Designed to the IEEE Std 802.3-2008 specification
- Supports three separate IP cores
 - 10/100/1000 Mb/s Ethernet MAC
 - 1 Gb/s Ethernet MAC
 - 10/100 Mb/s Ethernet MAC
- Configurable duplex operation
- Support for Media Independent Interface (MII), Gigabit Media Independent Interface (GMII), and Reduced Gigabit Media Independent Interface (RGMII).
- Management Data Input/Output (MDIO) interface to manage objects in the physical layer
- User-accessible raw statistic vector outputs
- Support for VLAN frames
- Configurable interframe gap (IFG) adjustment in full-duplex operation
- Configurable in-band Frame Check Sequence (FCS) field passing on both transmit and receive paths
- Auto padding on transmit and stripping on receive paths
- Configured and monitored through a generic management interface
- Configurable flow control through Ethernet MAC Control PAUSE frames; symmetrically or asymmetrically enabled
- Configurable support for jumbo frames of any length
- Configurable receive address filter for unicast and broadcast addresses

Note: Virtex®-6 devices support GMII and MII at 2.5 V only; Virtex-5, Virtex-4, Spartan®-6 and Spartan-3 devices support GMII and MII at 3.3 V or lower.

In addition, the 10/100/1000 Mb/s or 1 Gb/s Ethernet MAC IP core can be connected to the *Ethernet 1000BASE-X PCS/PMA or SGMII LogiCORE™* IP core to provide Ethernet MAC functionality with a Serial-GMII (SGMII) or to extend the MAC functionality to include the PCS/PMA physical sublayers for 1000BASE-X operation.

Architecture Description

Figure 5-1 illustrates a block diagram of the core.

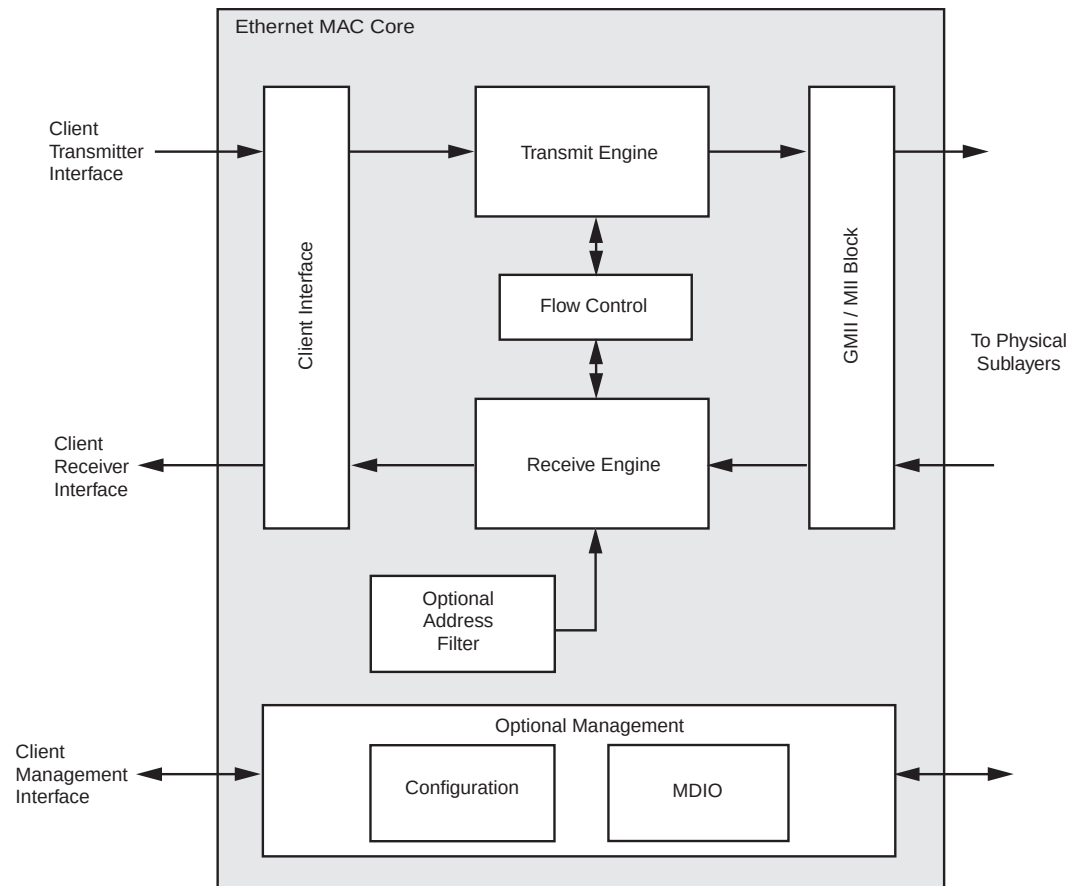


Figure 5-1: Tri-Mode Ethernet MAC Block Diagram

Core Components

The major functional blocks of the MAC are:

Transmit Engine

The Transmit Engine accepts Ethernet frame data from the Client Transmitter Interface, adds the preamble field to the start of the frame, adds padding bytes (if required) to ensure that the frame meets the minimum frame length requirements, and adds the frame check sequence (when configured to do so). The transmitter also ensures that the inter-frame spacing between successive frames is at least the minimum specified. The frame is then converted into a format that is compatible with GMII/MII and sent to the GMII/MII Block.

The transmitter client interface of the core is defined in [Chapter 7, Client Interface](#).

Receive Engine

The Receive Engine accepts Ethernet frame data from the GMII Block, removes the preamble field at the start of the frame, removes padding bytes and Frame Check Sequence (if required, and when configured to do so). The receiver also performs error detection on the received frame using information such as the frame check sequence field, received GMII error codes, and legal frame size boundaries.

The receiver client interface of the core is defined in [Chapter 7, Client Interface](#).

Flow Control

The Flow Control block is designed to clause 31 of the *IEEE 802.3-2008* standard. The MAC can be configured to send pause frames and to act upon their reception during full-duplex operation. These two behaviors can be configured independently. For further information, see [Chapter 8, Flow Control](#).

Address Filter

The Address Filter checks the address of incoming frames into the receiver. If the Address Filter is enabled, the device does not pass frames that do not contain one of a set of known addresses to the client.

Optional Management Interface and MDIO

The optional processor-independent Management Interface has standard address, data, and control signals. It can be used as is, or you can apply a logical shim to interface to common bus architectures. See [Chapter 9, Configuration and Status](#).

This interface is used to access the following blocks.

- **Configuration Register** After power up or reset, the client may reconfigure the core parameters from their defaults. Configuration changes can be written at any time.
- **MDIO Interface** The Management Interface is also used to access the MDIO interface of the core; this interface is typically connected to the MDIO port of a physical layer device (PHY) to access its configuration and status registers. The MDIO format is defined in *IEEE802.3* clause 22.

When the Management Interface is omitted, configuration of the core can still be made via an alternative configuration vector.

New configuration settings are sampled at the beginning of frame transmission/reception by the transmitter and receiver engines respectively.

GMII/MII Block

This implements GMII/MII style signaling for the physical interface of the core and is typically attached to a physical layer device (PHY), either off-chip or internally integrated.

When connecting to external PHY devices, either of GMII/MII or RGMII interfaces can be used: these are selectable from the CORE Generator™ tool GUI (see [Chapter 4, Generating the Core](#)). These external interfaces are described in the following chapters:

- [Chapter 10, Physical Interface for the 10 Mb/s and 100 Mb/s Only Ethernet MAC IP Core](#)
- [Chapter 11, Physical Interfaces for 1 Gb/s Only Ethernet MAC IP Core](#)
- [Chapter 12, Physical Interfaces for Tri-speed \(10 Mb/s, 100 Mb/s and 1 Gb/s\) Ethernet MAC IP Core](#)

Alternatively, the GMII/MII interface of the core can be connected internally to the *Ethernet 1000BASE-X PCS/PMA or SGMII* core to provide an SGMII or to perform the 1000BASE-X physical standard. See [Chapter 14, Interfacing to Other Xilinx Ethernet Cores](#)

Core Interfaces

Client Side Interface Signals

Transmitter Interface

[Table 5-1](#) describes the client-side transmitter signals of the core. These signals are used to transmit data from the client to the core. See [Transmitting Outbound Frames, page 73](#).

The Transmitter Interface is designed to be connected to internal device logic only. Attempting to add external ports to this interface results in a breakdown of the handshaking protocol used by this interface.

Table 5-1: Client Interface Signal Pins

Signal	Direction	Description
clientemactxd[7:0]	Input	Frame data to be transmitted is supplied on this port.
clientemactxdvld	Input	Control signal for clientemactxd port.
clientemactxifgdelay[7:0]	Input	Control signal for configurable interframe gap adjustment. See Interframe Gap Adjustment: Full-Duplex Mode Only, page 79 for timing diagrams.
emacclienttxack	Output	Handshaking signal. Asserted when the current data on clientemactxd has been accepted. See Transmitting Outbound Frames, page 73 for timing diagrams.

Table 5-1: Client Interface Signal Pins (Continued)

Signal	Direction	Description
clientemactxenable	Input	Transmitter clock enable signal. This signal is present when: - The TEMAC solution is generated for 10/100 Mb/s only ethernet speed support. - The TEMAC solution is generated for tri-speed (10.100/1000 Mb/s) ethernet support when using the optional clock enables.
clientemactxunderrun	Input	Asserted by client to force MAC core to corrupt the current frame.
emacclienttxcollision	Output	Asserted by the MAC core to signal a collision on the medium and that any transmission in progress should be aborted. This signal is only present when the core is generated with half-duplex support.
emacclienttxretransmit	Output	When asserted at the same time as the <code>emacclienttxcollision</code> signal, this signals to the client that the aborted frame should be resupplied to the MAC core for retransmission. This signal is only present when the core is generated with half-duplex support.
emacclienttxstats[31:0]	Output	This gives information on the last frame transmitted. See Transmitter Statistics Vector, page 80 for vector contents.
emacclienttxstatsvld	Output	Asserted at end of frame transmission, indicating that the <code>emacclienttxstats</code> is valid.

Receiver Interface

[Table 5-2](#) describes the client-side transmit signals of the core. These signals are used to transmit data from the client to the core. See [Receiving Inbound Frames](#), page 66.

Table 5-2: Client Interface Signal Pins

Signal	Direction	Description
emacclientrxdata[7:0]	Output	Frame data received is supplied on this port.
emacclientrxdatavalid	Output	Control signal for the emacclientrxdata port.
emacclientrxgoodframe	Output	Asserted at end of frame reception to indicate that the frame should be processed by the MAC client. See Normal Frame Reception , page 66.
emacclientrxbadframe	Output	Asserted at end of frame reception to indicate that the frame should be discarded by the MAC client. See Frame Reception with Errors , page 67.
clientemacrxenable	Input	Receiver clock enable signal This signal is present when: - The TEMAC solution is generated for 10/100 Mb/s only ethernet speed support. - The TEMAC solution is generated for tri-speed(10/100/1000 Mb/s) ethernet support when using the optional clock enables.
emacclientrxstats[27:0]	Output	Provides information about the last frame received. See Receiver Statistics Vector , page 71 for the vector contents.
emacclientrxstatsvalid	Output	Asserted at end of frame reception, indicating that the emacclientrxstats is valid.

Flow Control Interface

[Table 5-3](#) describes the signals used by the client to request a flow control action from the transmit engine. See [Chapter 8, Flow Control](#).

Table 5-3: Client Interface Signal Pins

Signal	Direction	Description
clientemacpausereq	Input	Pause request: sends a pause frame down the link. See Transmitting a Pause Control Frame , page 85.
clientemacpauseval[15:0]	Input	Pause value: inserted into the parameter field of the transmitted pause frame.

Management Interface Signals

Table 5-4 describes the Management Interface and support signals. These signals are used by the client to configure the MAC core and to read the status of configuration bits. See [Using the Optional Management Interface, page 89](#).

Table 5-4: Optional Management Interface Signal Pinout

Signal	Direction	Description
hostclk	Input	Clock for Management Interface
hostopcode[1:0]	Input	Defines operation to be performed over MDIO interface. Bit 1 is also used in configuration register access. See Using the Management Interface, page 98 .
hostaddr[9:0]	Input	Address of register to be accessed
hostwrddata[31:0]	Input	Data to write to register
hostrddata[31:0]	Output	Data read from register
hostmiimsel	Input	When asserted, the MDIO interface is accessed. When disasserted, the MAC internal configuration is accessed.
hostreq	Input	Used to signal a transaction on the MDIO interface. See Using the Management Interface, page 98 .
hostmiimrdy	Output	When high, the MDIO interface has completed any pending transaction and is ready for a new transaction.

Configuration Vector Signals

If the Management Interface is not present, the configuration of the core is carried out by a configuration vector. ([Table 5-5](#).)

Table 5-5: Alternative to the Optional Management Interface: Configuration Vector Signal Pinout

Signal	Direction	Description
tieemacconfigvec[67:0]	Input	The Configuration Vector is used to replace the functionality of the MAC Configuration Registers when the Management Interface is not used. See Accessing Configuration without the Management Interface, page 108 .

Address Filter Signals

If the optional address filter is included in the core, you can specify a unicast address for the MAC by setting the `tieemacunicastaddr` signal. If the Management Interface is present, this can be overwritten by writing to the unicast address register. See [Using the Management Interface](#), page 98.

Table 5-6: Address Filter Unicast Address

Signal	Direction	Description
<code>tieemacunicastaddr[47:0]</code>	Input	This vector is used to set the default address for the MAC. See Address Filter , page 70.

Clock, Speed Indication, and Reset Signals

[Table 5-7](#) describes the clock signals that are input to the core and the outputs that can be used to select between the three operating speeds. The clock signals are generated in the block level of the example design provided with the core.

Table 5-7: Clock, Speed Indication and Reset Signals

Signal	Direction	Description
<code>txgmiimiiclk</code>	Input	The transmitter clock for the core. See the appropriate section: • Chapter 10, Physical Interface for the 10 Mb/s and 100 Mb/s Only Ethernet MAC IP Core • Chapter 11, Physical Interfaces for 1 Gb/s Only Ethernet MAC IP Core • Chapter 12, Physical Interfaces for Tri-speed (10 Mb/s, 100 Mb/s and 1 Gb/s) Ethernet MAC IP Core
<code>rxgmiimiiclk</code>	Input	The receiver clock for the core. See the appropriate section: • Chapter 10, Physical Interface for the 10 Mb/s and 100 Mb/s Only Ethernet MAC IP Core • Chapter 11, Physical Interfaces for 1 Gb/s Only Ethernet MAC IP Core • Chapter 12, Physical Interfaces for Tri-speed (10 Mb/s, 100 Mb/s and 1 Gb/s) Ethernet MAC IP Core
<code>speedis100</code>	Output	This output is asserted when the core is operating at 100 Mb/s. It is derived from either bits 30 and 31 of the MAC Speed Configuration register (See Configuration Register Map , page 90) if the optional Management Interface is present. If the Management Interface is not present, this is derived from configuration vector bits 65 and 66.

Table 5-7: Clock, Speed Indication and Reset Signals

Signal	Direction	Description
speedis10100	Output	This output is asserted when the core is operating at either 10 Mb/s or 100 Mb/s. It is derived from either bits 30 and 31 of the MAC Speed Configuration register (see Configuration Register Map, page 90) if the optional Management Interface is present. If the Management Interface is not present, this is derived from configuration vector bits 65 and 66.
reset	Input	Asynchronous reset for entire core.

Physical Interface Signals

GMII/MII

[Table 5-8](#) describes the GMII/MII signals of the MAC core. These are typically attached to a PHY module, either off-chip or internally integrated. The GMII is defined in *IEEE 802.3* clause 35; MII in clause 22. The GMII/MII physical interface, together with logic to convert these signals to RGMII format, is described in the Physical Interface chapters of this Guide.

Table 5-8: GMII/MII Interface Signal Pinout

Signal	Direction	Description
phyemactxenable	Input	Transmitter clock enable signal for the GMII/MII logic within the core. This signal is present when: - The TEMAC solution is generated for 10/100 Mb/s only ethernet speed support. - The TEMAC solution is generated for tri-speed(10/100/1000 Mb/s) ethernet support when using the optional clock enables. Even when present, this signal is only utilized when used with an RGMII. At all other times it should be tied high.
emacphytxd[7:0]	Output	Transmit data from MAC This bus is 8-bits wide when the core is generated for 1 Gig only or tri-speed ethernet support. This bus is 4-bits wide when the core is generated for 10 Mb/s and 100 Mb/s only ethernet support.
emacphytxen	Output	Data Enable control signal from MAC
emacphytxer	Output	Error control signal from MAC

Table 5-8: GMII/MII Interface Signal Pinout (Continued)

Signal	Direction	Description
phyemaccrs	Input	Carrier Sense Control signal to MAC. This signal is only present when the core is generated with half-duplex support.
phyemacol	Input	Collision Control signal to MAC. This signal is only present when the core is generated with half-duplex support.
phyemacrx[7:0]	Input	Received data to MAC. This bus is 8-bits wide when the core is generated for 1 Gig only or tri-speed ethernet support. This bus is 4-bits wide when the core is generated for 10 Mb/s and 100 Mb/s only ethernet support.
phyemacrxdv	Input	Data Valid control signal to MAC
phyemacrxe	Input	Error control signal to MAC
corehassgmii	Input	Tie this input high if the core is interfaced to the Ethernet 1000BASE-X PCS/PMA or SGMII core in SGMII mode. See Ethernet 1000BASE-X PCS/PMA or SGMII Core , page 225 for details.

MDIO Signals

[Table 5-9](#) describes the MDIO (MII Management, defined in *IEEE 802.3* clause 22) interface signals of the MAC core. These signals are present whenever the optional Management Interface is used; they are typically connected to the MDIO port of a PHY device, either off-chip or an SoC-integrated core. The MDIO format is defined in *IEEE 802.3* clause 22. For further information, see [MDIO Interface](#), page 101.

Table 5-9: MDIO Interface Signal Pinout

Signal	Direction	Description
emacphymclkout	Output	MDIO Management Clock: derived from hostclk on the basis of supplied configuration data when the optional Management Interface is used. See Accessing PHY Configuration Registers, via MDIO using the Management Interface , page 106.
emacphymdin	Input	Input data signal for communication with PHY configuration and status. Tie high if unused.
emacphymdout	Output	Output data signal for communication with PHY configuration and status.
emacphymdtri	Output	3-state control for MDIO signals; '0' signals that the value on emacphymdout should be asserted onto the MDIO bus.

Designing with the Core

This chapter provides guidelines for creating designs using the TEMAC solution. For details about the example design included with the TEMAC solution, see [Chapter 17, Detailed Example Design](#).

General Design Guidelines

This section describes the steps required to turn the TEMAC solution into a fully-functioning design integrated with user application logic. It is important to recognize that not all designs require all the design steps defined in this chapter. The following sections discuss the design steps required for various implementations; follow the logic design guidelines carefully.

Design Steps

Generate the core using the CORE Generator™ software. See [Chapter 4, Generating the Core](#).

Using the Example Design as a Starting Point

The core is delivered through the CORE Generator software with an HDL example design built around the core, allowing the functionality of the core to be demonstrated using either a simulation package or in hardware, if placed on a suitable board. [Figure 6-1](#) is a block diagram of the example design. For details about the example design, see [Chapter 17, Detailed Example Design](#).

The example design illustrates how to:

- Instantiate the core from HDL.
- Source and use the client-side interface ports of the core from application logic.
- Connect the physical-side interface of the core (GMII/MII or RGMII) to device IOBs creating an external interface. (See the Physical Interface chapters in this document)
- Derive the clock logic, required for the core (See the Physical Interface chapters in this document).

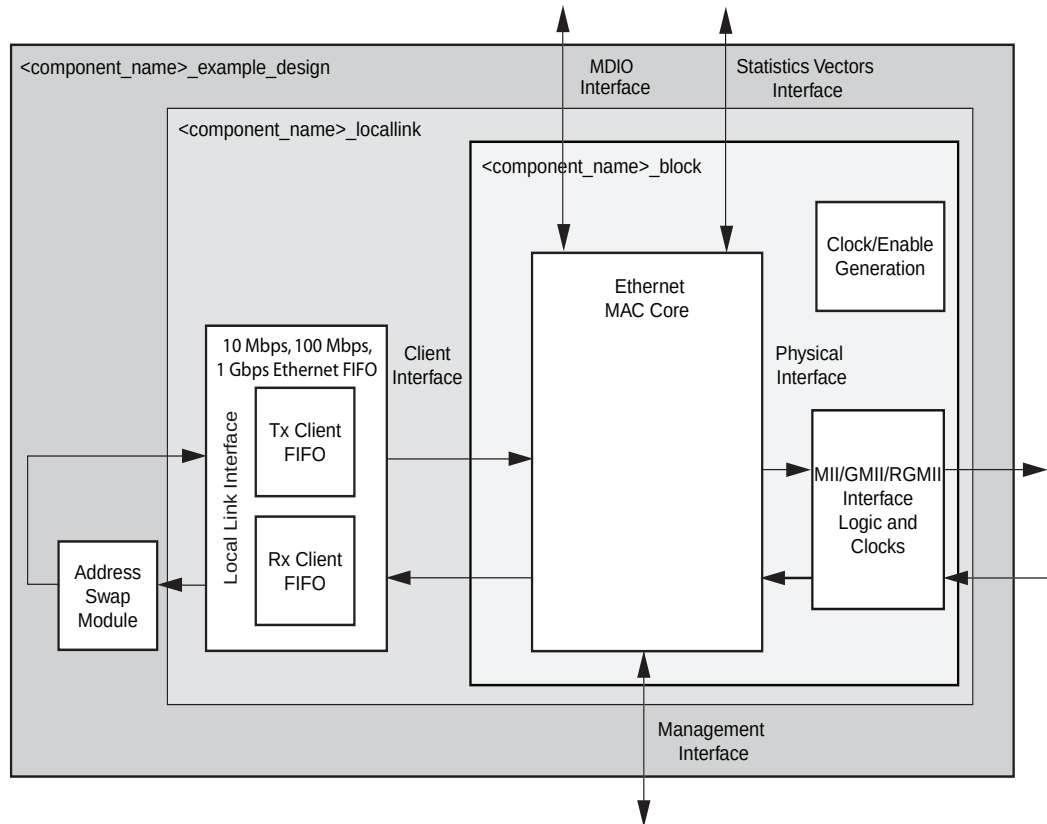


Figure 6-1: Tri-Mode Ethernet MAC Core Example Design

Using the example design as a starting point, you can do the following:

- Edit the HDL top level of the example design file to:
 - Change the clocking scheme.
 - Add/remove IOBs as required.
 - Replace the client loopback logic with your specific application logic.
 - Adapt the 10 Mb/s, 100 Mb/s, 1 Gb/s Ethernet FIFO to suit your specific application (see [Using the Client Side FIFO](#)).

- Synthesize the entire design.

The Xilinx® Synthesis Tool (XST) script and Project file in the `/implement` directory can be adapted to include any HDL files you want to add.

- Run the `implement` script in the `/implement` directory to create a top-level netlist for the design. The script can also run the Xilinx tools `map`, `par`, and `bitgen`, creating a bitstream that can be downloaded to a Xilinx device. SimPrim-based simulation models for the entire design are also produced by the `implement` scripts.
- Simulate the entire design using the demonstration test bench provided as a template in the `/simulation` directory.
- Download the bitstream to a target device.

Implementing the Tri-Mode Ethernet MAC in Your Application

The example design can be studied as an example of how to do the following:

- Instantiate the core from HDL.
- Source and use the client-side interface ports of the core from application logic.
- Connect the physical-side interface of the core (GMII/MII or RGMII) to device IOBs to create an external interface.
- Derive the required clock logic.

After working with the example design and this User Guide, you can write your own HDL application, using single or multiple instances of the core.

You can synthesize the entire design using any synthesis tool. The core is pre-synthesized and is delivered as an NGC netlist (which appears as a black box to synthesis tools).

Run the Xilinx tools **map**, **par**, and **bitgen** to create a bitstream that can be downloaded to a Xilinx device. Care must be taken to constrain the design correctly, and the UCF produced by the CORE Generator software should be used as the basis for the your own UCF. See [Chapter 15, Implementing Your Design](#).

You can simulate the entire design and download the bitstream to the target device.

Keep it Registered

To simplify timing and increase system performance in an FPGA design, keep all inputs and outputs registered between your application and the core. This means that all inputs and outputs from your application should come from, or connect to, a flip-flop. While registering signals may not be possible for all paths, it simplifies timing analysis and makes it easier for the Xilinx tools to place-and-route the design.

Recognize Timing Critical Signals

The UCF provided with the example design identifies the critical signals and timing constraints that should be applied. See [Chapter 13, Constraining the Core](#), for details.

Use Supported Design Flows

The core is pre-synthesized and delivered as an NGC netlist. The example implementation scripts provided use XST 13.1 as the synthesis tool for the HDL example design. Other synthesis tools can be used for your application logic. The core is always unknown to the synthesis tool and should appear as a black box. For post synthesis, only ISE® v13.1 tools are supported.

Make Only Allowed Modifications

You should not modify the core. Any modifications can have adverse effects on system timing and protocol compliance. Supported user configurations of the core can only be made by selecting the options in the CORE Generator tool when the core is generated. See [Chapter 4, Generating the Core](#).

Client Interface

This chapter provides a detailed description of the client-side data-flow interface. This interface must be used by the client-side logic to initiate frame transmission and accept frame reception to and from the core.

Introduction to the Timing Diagrams

The core may or may not use clock enables. This is summarized by these rules:

- When the TEMAC solution is generated for 1 Gb/s only speed support, clock enables are not used.
- When the TEMAC solution is generated for 10 Mb/s and 100 Mb/s only speed support, clock enables are always used.
- When the TEMAC solution is generated for tri-speed support, clock enables may or may not be used according to the setting of [Clock Enables in Chapter 4](#).

To avoid the duplication of timing diagrams for usage with and without clock enables, the following timing diagrams in this document use vertical dashed lines to represent a *clock-enabled cycle*. This is illustrated in [Figure 7-1](#) for the receiver case. The transmitter case is the same: simply replace rxgmiimiiclk with txgmiimiiclk and clientemacrxcenable with clientemactxcenable.

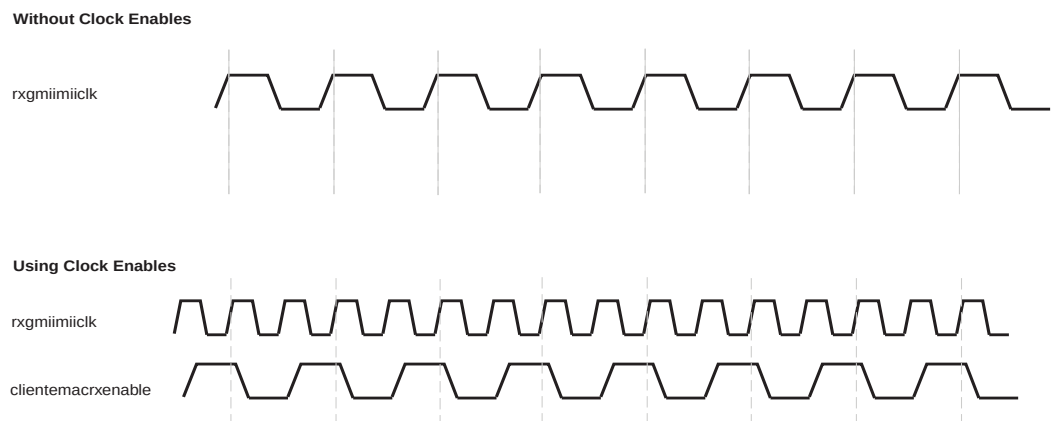


Figure 7-1: Understanding the Timing Diagrams

The definitions and abbreviations used in this chapter are described in [Table 7-1](#).

Table 7-1: Abbreviations Used in Timing Diagrams

Abbreviation	Definition
DA	Destination address; 6 bytes
SA	Source address; 6 bytes
L/T	Length/type field; 2 bytes
FCS	Frame check sequence; 4 bytes

Receiving Inbound Frames

Received Ethernet frames are presented to the client logic on the receiver subset of the Client-Side Interface. For port definition, see [Receiver Interface, page 56](#).

All receiver signals are synchronous to the `rxgmiimiclk` clock. If a clock enable is present, this is the `clientemacrxenable` signal.

Normal Frame Reception

[Figure 7-2](#) shows the timing of a normal inbound frame transfer. The client must be prepared to accept data at any time; there is no buffering within the MAC to allow for latency in the receive client. After frame reception begins, data is transferred on consecutive clock-enabled cycles to the receive client until the frame is complete. The MAC asserts the `emacclientrxgoodframe` signal to indicate that the frame was successfully received and that the frame should be analyzed by the client.

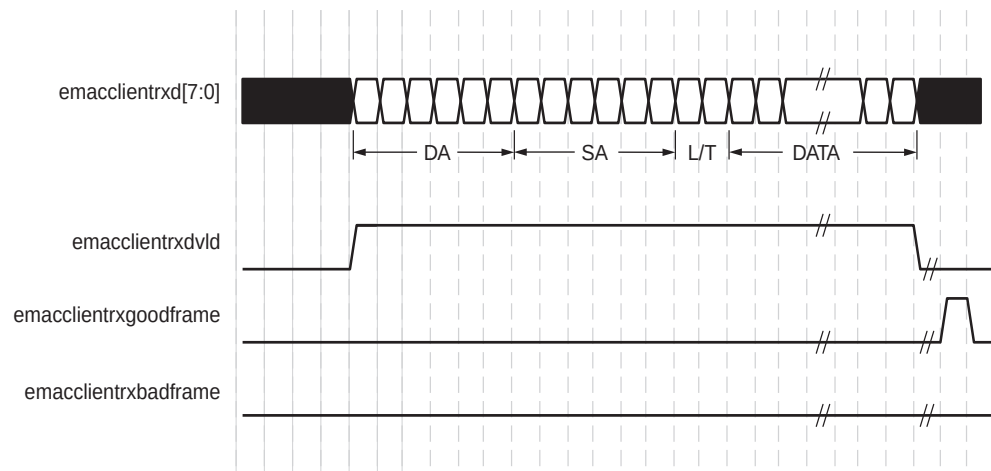


Figure 7-2: Normal Frame Reception

Frame parameters (destination address, source address, length/type and optionally FCS) are supplied on the data bus according to the timing diagram. The abbreviations are described in [Table 7-1](#).

If the length/type field in the frame has the length interpretation, and this indicates that the inbound frame has been padded to meet the Ethernet minimum frame size specification, then this padding is not passed to the client in the data payload. The exception to this is in the case where FCS passing is enabled. See [Client-Supplied FCS Passing](#).

When client-supplied FCS passing is disabled, `emaccliintrxdvld` = '0' between frames for the duration of the padding field (if present), the FCS field, carrier extension (if present), the interframe gap following the frame, and the preamble field of the next frame. When client-supplied FCS passing is enabled, `emaccliintrxdvld` = '0' between frames for the duration of carrier extension (if present), the interframe gap, and the preamble field of the following frame.

emaccliintrxgoodframe and emaccliintrxbadframe Timing

Although Figure 7-2 illustrates the `emaccliintrxgoodframe` signal asserted shortly after the last valid data on `emaccliintrxd`, this is not always the case. The `emaccliintrxgoodframe` or `emaccliintrxbadframe` signals are asserted only after all frame checks are completed. This is after the FCS field has been received (and after reception of carrier extension, if present).

Therefore, either `emaccliintrxgoodframe` or `emaccliintrxbadframe` is asserted following frame reception at the beginning of the interframe gap.

Frame Reception with Errors

Figure 7-3 illustrates an unsuccessful frame reception (for example, a fragment frame or a frame with an incorrect FCS). In this case, the `emaccliintrxbadframe` signal is asserted to the client at the end of the frame. It is then the responsibility of the client to drop the data already transferred for this frame.

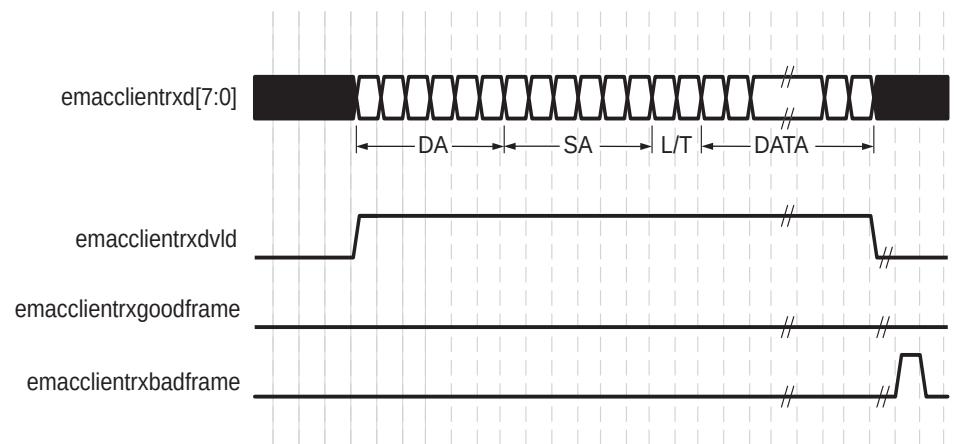


Figure 7-3: Frame Reception with Error

The following conditions cause the assertion of `emaccliintrxbadframe`:

- FCS errors occur.
- Packets are shorter than 64 bytes (undersize or fragment frames).
- Jumbo frames are received when jumbo frames are not enabled.
- VLAN frames of length 1519-1522 are received when VLAN frames are not enabled.
- A value of 0x0000 to 0x002D is in the type/length field. In this situation, the frame should be padded to minimum length. If it is not padded to exactly minimum frame length, the frame is marked as bad (when length/type checking is enabled).
- A value of 0x002E to 0x0600 is in the type/length field, but the real length of the received frame does not match this value (when length/type checking is enabled).

- Any control frame that is received is not exactly the minimum frame length (unless control frame length checks are disabled: see ["Receiving a Pause Control Frame" on page 86](#)).
- `phyemacrxer` is asserted at any point during frame reception.
- An error code is received in the 1-Gigabit frame extension field.
- A valid pause frame, addressed to the MAC, is received when flow control is enabled. See [Overview of Flow Control](#) for details.

Client-Supplied FCS Passing

If the MAC core is configured to pass the FCS field to the client (see [Configuration Register Map, page 90](#)), it is handled as displayed in [Figure 7-4](#).

In this case, any padding inserted into the frame to meet Ethernet minimum frame length specifications is left intact and passed to the client.

Even though the FCS is passed up to the client, it is also verified by the MAC core, and `emacclientrxbadframe` is asserted if the FCS check fails.

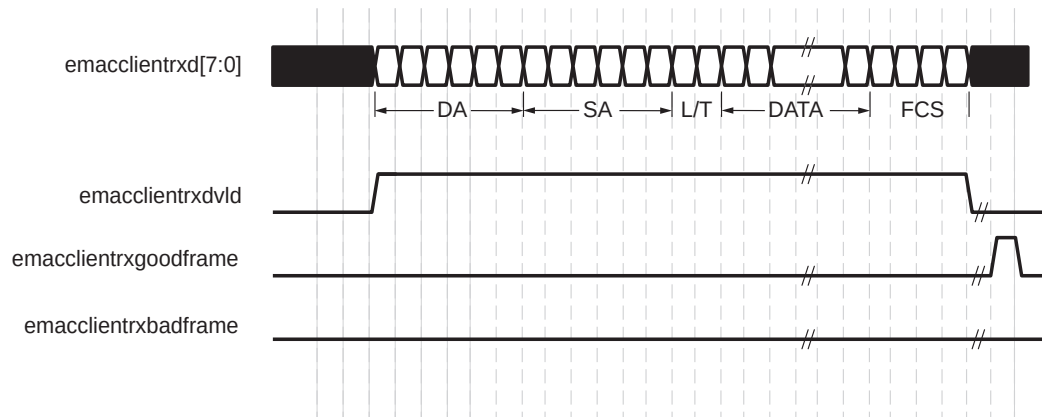


Figure 7-4: Frame Reception with In-Band FCS Field

VLAN Tagged Frames

The reception of a VLAN tagged frame (if enabled, see [Configuration Register Map, page 90](#)) can be seen in [Figure 7-5](#). This frame is identified as being a VLAN frame by the inclusion of the VLAN type tag (81-00), located in the first two bytes following the Source Address. This is followed by the Tag Control Information bytes, V1 and V2. The length/type field after the tag control information is not checked for errors. More information on the interpretation of these bytes can be found in *IEEE 802.3-2008* standard.

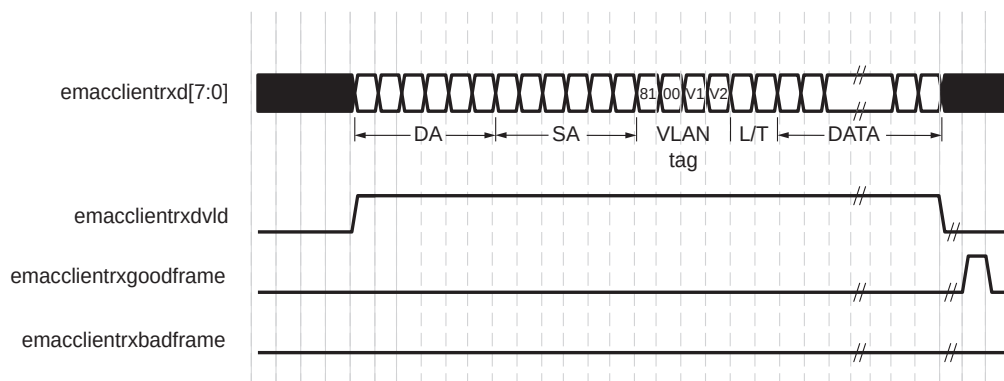


Figure 7-5: Reception of a VLAN Tagged Frame

Maximum Permitted Frame Length

The maximum legal length of a frame specified in *IEEE 802.3-2008* is 1518 bytes for non-VLAN tagged frames. VLAN tagged frames can be extended to 1522 bytes. When jumbo frame handling is disabled and the core receives a frame which exceeds the maximum legal length, `emacclientrxbadframe` is asserted. When jumbo frame handling is enabled, frames which are longer than the legal maximum are received in the same way as shorter frames.

For details on enabling and disabling jumbo frame handling, see [Configuration Register Map, page 90](#).

Length/Type Field Error Checks

Enabled

Default operation is with the length/type error checking enabled. In this mode, the following checks are made on all frames received. If either of these checks fail, the frame is marked as bad.

- A value in the length/type field that is greater than or equal to decimal 46 but less than decimal 1536 (a Length interpretation) is checked against the actual data length received.
- A value in the length/type field that is less than decimal 46 is checked to see that the data field is padded to exactly 46 bytes (so that the resultant frame is minimum frame size: 64 bytes total in length).

Furthermore, if padding is indicated (the length/type field is less than decimal 46) and [Client-Supplied FCS Passing](#) is disabled, then the length value in the length/type field is used to deassert `emacclientrxvalid` after the indicated number of data bytes so that the padding bytes are removed from the frame.

Disabled

When the length/type error checking is disabled (see ["Configuration Register Definition" on page 91](#)) and the length/type field has a length interpretation, the MAC does not check the length value against the actual data length received. A frame containing only this error is marked as good. However, if the length/type field is less than decimal 46, the MAC marks a frame as bad if it is not the minimum frame size of 64 bytes.

Furthermore, if padding is indicated and [Client-Supplied FCS Passing](#) is disabled, then a length value in the length/type field is not used to deassert `emaccliентrxdvld`. Instead `emaccliентrxdvld` is deasserted before the start of the FCS field; in this way any padding is not removed from the frame.

Address Filter

If the optional address filter is included in the core, the MAC is able to reject frames that do not contain a recognized address in their destination address field. If a frame is rejected, the `emaccliентrxdvld` signal is not asserted for the duration of the frame. In addition neither `emaccliентrxgoodframe` or `emaccliентrxbadframe` are asserted at the end of the frame. The statistics vectors are still output with a valid pulse at the end of the rejected frame.

If the address filter is not in promiscuous mode, it rejects frames in which the destination address does not meet any of the following criteria:

- It is equal to the broadcast address defined in the *IEEE 802.3-2008* specification (0xFFFF_FFFF_FFFF).
- It is equal to the pause multicast address defined in the *IEEE 802.3-2008* specification. (0x0100_00C2_8001)
- The destination address field contains the Pause frame MAC source address specified in Receiver Configuration Word 0 and Word 1 ([Table 9-3](#) and [Table 9-4](#)).
- It is equal to the MAC Unicast Address. When the optional Management Interface is present, this is contained in the unicast address configuration registers ([Table 9-9](#) and [Table 9-10](#)). If the Management Interface is not present, the unicast address is input on the `tieemacunicastaddr` input.
- It matches any of the addresses stored in the MAC address table (see ["Optional Address Table" on page 97](#)). The address table is only present when the MAC contains the optional Management Interface.

Receiver Statistics Vector

The statistics for the frame received are contained within the `emacclientrxstats` output. Table 7-2 defines the bit field for the vector.

All bit fields, with the exception of `BYTE_VALID` are valid only when the `emacclientrxstatsvld` is asserted, as illustrated in Figure 7-6. `BYTE_VALID` is significant on every receiver clock-enabled cycle.



Figure 7-6: Receiver Statistics Vector Timing

Table 7-2: Bit Definition for the Receiver Statistics Vector

<code>emacclientrxstats</code>	Name	Description
27	ADDRESS_MATCH	If the optional address filter is included in the core, this bit is asserted if the address of the incoming frame matches one of the stored or pre-set addresses in the address filter. If the address filter is omitted from the core or is configured in promiscuous mode, this line is held high.
26	ALIGNMENT_ERROR	Asserted at speeds below 1 Gb/s if the frame contains an odd number of nibbles and the FCS for the frame is invalid.
25	LENGTH/TYPE Out of Range	If the length/type field contained a length value that did not match the number of MAC client data bytes received and the length/type field checks are enabled, then this bit is asserted. This bit is also asserted if the length/type field is less than 46, and the frame is not padded to exactly 64 bytes. This is independent of whether or not the length/type field checks are enabled.

Table 7-2: Bit Definition for the Receiver Statistics Vector (*Continued*)

emacclientrxstats	Name	Description
24	BAD_OPCODE	Asserted if the previous frame was error-free and contained the special control frame identifier in the length/type field, but contained an opcode that is unsupported by the MAC (any opcode other than PAUSE).
23	FLOW_CONTROL_FRAME	Asserted if the previous frame was error-free, contained the special control frame identifier in the length/type field, contained a destination address that matched either the MAC Control multicast address or the configured source address of the MAC, contained the supported PAUSE opcode, and was acted upon by the MAC.
22	BYTE_VALID	Asserted if a MAC frame byte (destination address to FCS inclusive) is in the process of being received. This is valid on every clock cycle. Do not use this as an enable signal to indicate that data is present on emacclientrx[7:0].
21	VLAN_FRAME	Asserted if the previous frame contained a VLAN identifier in the length/type field when receiver VLAN operation is enabled.
20	OUT_OF_BOUNDS	Asserted if the previous frame exceeded the specified <i>IEEE 802.3-2008</i> maximum legal length (see Maximum Permitted Frame Length, page 69). This is only valid if jumbo frames are disabled.
19	CONTROL_FRAME	Asserted if the previous frame contained the special control frame identifier in the length/type field.
18 down to 5	FRAME_LENGTH_COUNT	The length of the previous frame in number of bytes. The count will stick at 16368 for any jumbo frames larger than this value.
4	MULTICAST_FRAME	Asserted if the previous frame contained a multicast address in the destination address field.
3	BROADCAST_FRAME	Asserted if the previous frame contained the broadcast address in the destination address field.

Table 7-2: Bit Definition for the Receiver Statistics Vector (Continued)

emacclientrxstats	Name	Description
2	FCS_ERROR	Asserted if the previous frame received was correctly aligned but had an incorrect FCS value or the MAC detected error codes during frame reception.
1	BAD_FRAME ¹	Asserted if the previous frame received contained errors.
0	GOOD_FRAME ¹	Asserted if the previous frame received was error-free.

1. If the length/type field error checks are disabled, a frame which has an actual data length that does not match the length/type field value is marked as a GOOD_FRAME providing no additional errors were detected. See [Length/Type Field Error Checks](#), page 69 for details.

Transmitting Outbound Frames

Ethernet frames to be transmitted are presented to the client logic on the transmitter subset of the Client-Side Interface. For port definition, see [Transmitter Interface](#), page 54.

All transmitter signals are synchronous to the txgmiiiclk clock. If a clock enable is present, this is the clientemactxenablesignal.

Normal Frame Transmission

The timing of a normal outbound frame transfer can be seen in [Figure 7-7](#). When the client wants to transmit a frame, it places the first column of data onto the clientemactxd port and asserts a '1' onto clientemactxdvld.

When the MAC core has read this first byte of data, it asserts the emacclienttxack signal; on the next and subsequent clock-enabled rising edge, the client must provide the remainder of the data for the frame.

The end of frame is signalled to the MAC core by taking clientemactxdvld low.

For maximum flexibility in switching applications, the Ethernet frame parameters (destination address, source address, length/type and optionally FCS) are encoded within the same data stream that the frame payload is transferred upon, rather than on separate ports.

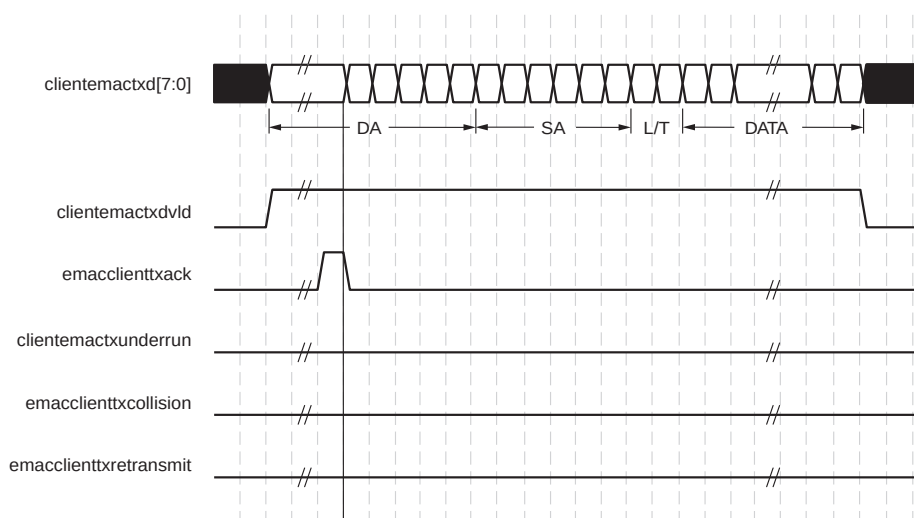


Figure 7-7: Normal Frame Transmission Across Client Interface

Padding

When fewer than 46 bytes of data are supplied by the client to the MAC core, the transmitter module adds padding up to the minimum frame length. The exception to this is when the MAC core is configured for client-passed FCS; in this case the client must also supply the padding to maintain the minimum frame length.

Client-Supplied FCS Passing

If the MAC core is configured to have the FCS field passed in by the client, the transmission timing is as depicted in [Figure 7-8](#). In this case, it is the responsibility of the client to ensure that the frame meets the Ethernet minimum frame length requirements. If frame length requirements are not met, the core will append zeroes at the end of the supplied frame to meet the minimum frame length. Although this does not cause the [Transmitter Statistics Vector](#) to indicate a bad frame, it results in an errored frame as received by the link partner MAC (due to the detection of an FCS error).

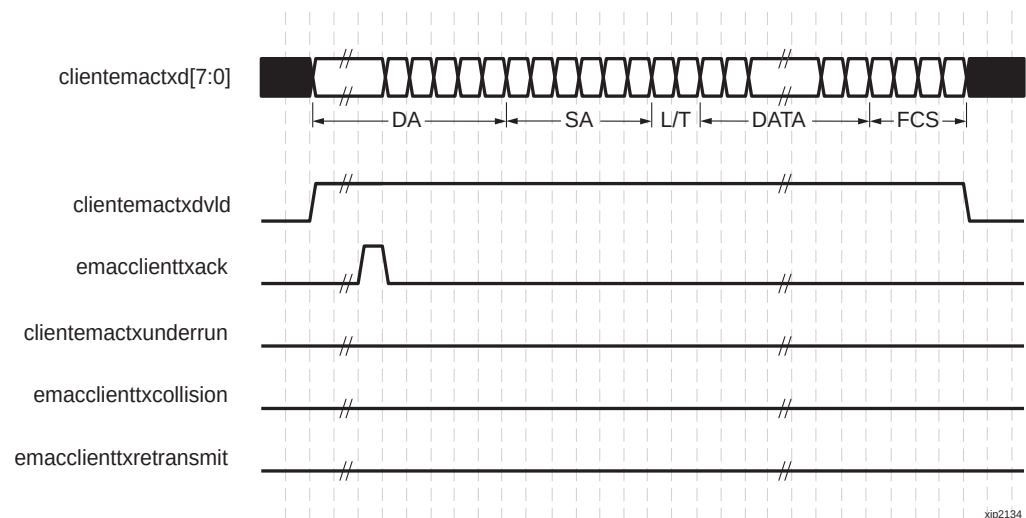


Figure 7-8: Frame Transmission with Client-Supplied FCS

xip2134

Client Underrun

Figure 7-9 shows an example of the timing for an aborted transfer. This can occur, for example, if a FIFO connected to the client interface empties before a frame is completed. When the client asserts `clientemactxunderrun` during a frame transmission, the MAC core inserts an error code to corrupt the current frame and then falls back to idle transmission. It is the responsibility of the client to re-queue the aborted frame for transmission.

The `clientemactxunderrun` signal can be asserted at any time during active frame transmission. This active frame transmission range can be defined as:

- Following any clock enabled cycle after the assertion of the `emacclienttxack` signal: this is shown by the solid vertical line on Figure 7-9 nearest the left-hand side of the figure.
- At any point during frame transmission up to 1 clock enabled cycle after `clientemactxdvld` goes low: this is shown by the solid vertical line on Figure 7-9 nearest the right-hand side of the figure.

After an underrun occurs, `clientemactxdvld` must be deasserted for at least one clock enabled cycle before transmission can recommence.

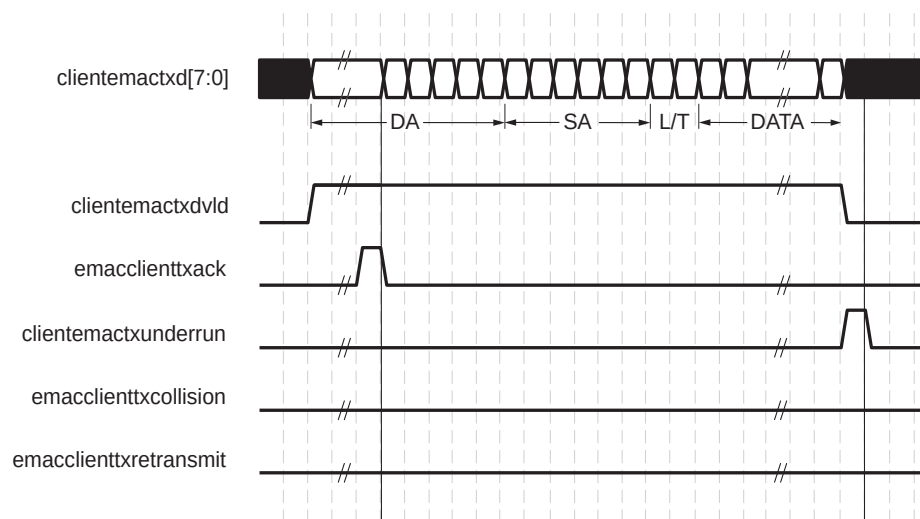


Figure 7-9: Frame Transmission with Underrun

Back-to-Back Transfers

Figure 7-10 shows the MAC client immediately ready to transmit a second frame of data following completion of its first frame. In this figure, the end of the first frame is shown on the left. On the clock-enabled cycle immediately following the final byte of the first frame, `clientemactxdvld` is taken low by the client, and is taken high one clock-enabled cycle later to indicate that the first byte of the destination address of the second frame is on `clientemactxd` awaiting transmission.

When the MAC core is ready to accept data, `emacclienttxack` is asserted and the transmission continues in the same manner as in the case of the single frame. The MAC core defers the assertion of `emacclienttxack` appropriately to comply with inter-packet gap requirements and flow control requests.

If the MAC core is operating at 1 Gb/s in half-duplex mode, the timing shown in [Figure 7-10](#) is required to take advantage of frame bursting; the MAC core is only guaranteed to retain control of the medium if the `clientemactxdvld` signal is low for a single clock-enabled cycle. For details on frame bursting, see *IEEE 802.3-2008*.

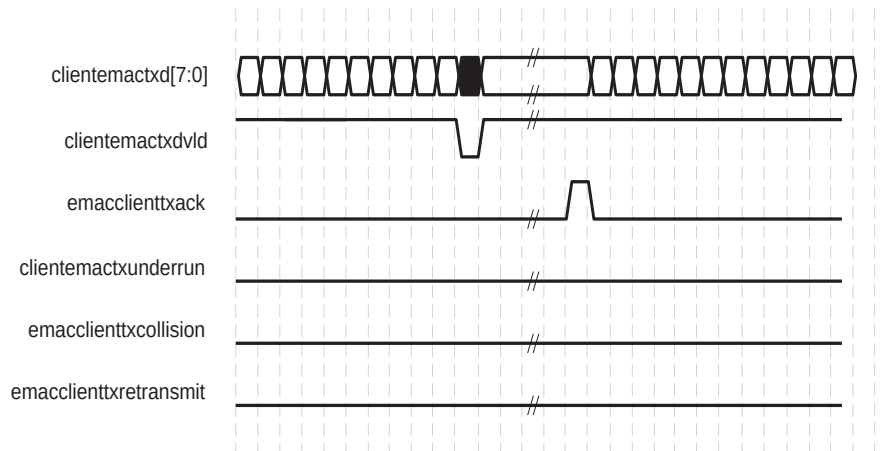


Figure 7-10: Back-to-Back Frame Transmission

VLAN Tagged Frames

Transmission of a VLAN tagged frame (if enabled) can be seen in [Figure 7-11](#). The handshaking signals across the interface do not change; however, the VLAN type tag 81-00 must be supplied by the client to signify that the frame is VLAN tagged. The client also supplies the two bytes of Tag Control Information, V1 and V2, at the appropriate times in the data stream. More information on the contents of these two bytes can be found in *IEEE 802.3-2008*.

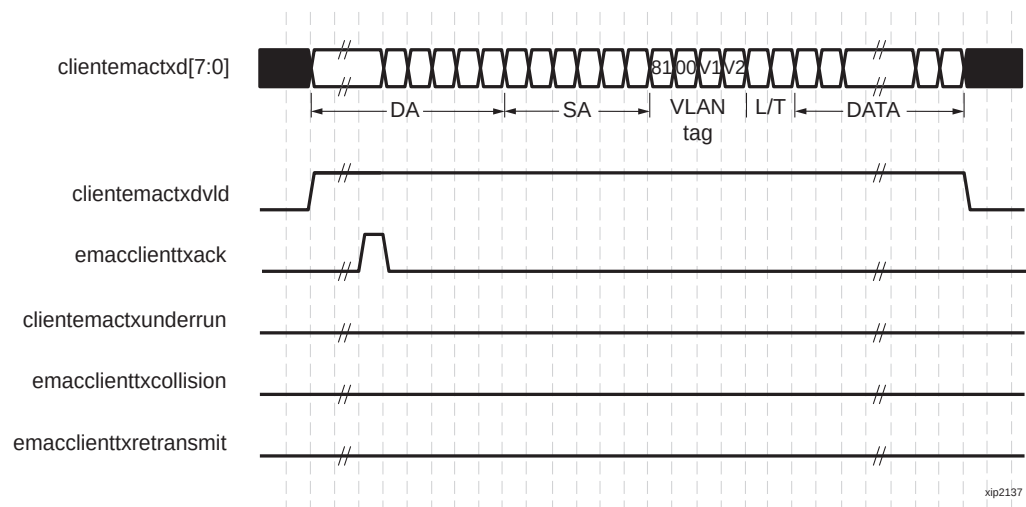


Figure 7-11: Transmission of a VLAN Tagged Frame

kip2137

Maximum Permitted Frame Length

The maximum legal length of a frame specified in *IEEE 802.3-2008* is 1518 bytes for non-VLAN tagged frames. VLAN tagged frames can be extended to 1522 bytes. When jumbo frame handling is disabled and the client attempts to transmit a frame which exceeds the maximum legal length, the MAC core inserts an error code to corrupt the current frame and the frame is truncated to the maximum legal length. When jumbo frame handling is enabled, frames which are longer than the legal maximum are transmitted error-free.

For details on enabling and disabling jumbo frame handling, see [Configuration Register Map, page 90](#).

Frame Collisions: Half-Duplex Operation Only

In half-duplex Ethernet operation, collisions occur on the medium as a matter of course; this is how the arbitration algorithm is fulfilled. In the case of a collision, the MAC core signals to the client that data may need to be resupplied as follows.

- If there is a collision, the `emacclienttxcollision` signal is set to '1' by the MAC core. If a frame is in progress, the client must abort the transfer and take `clientemactxdvld` to '0.'
- If the `emacclienttxretransmit` signal is '1' in the same clock-enabled cycle that the `emacclienttxcollision` signal is '1,' the client must then resubmit the previous frame to the MAC core for retransmission; `clientemactxdvld` must be asserted to the MAC core within 8 clock-enabled cycles of the `emacclienttxretransmit` signal: if `clientemactxdvld` is asserted later than this, the MAC assumes that the client frame is not retransmitted and the *number of retransmission attempts* counter within the MAC is reset. This case is illustrated in [Figure 7-12](#).

If any frame presented to the client interface is shorter than the collision window (slot time) as defined in IEEE Std 802.3-2008, a retransmission request can occur after the end of the frame as observed on the client interface. Therefore, the client logic (which may have queued a subsequent frame for transmission) may then have to rewind back to the previous frame. The previous frame data should be re-supplied on the `clientemactxd[7:0]` port within the same 8 clock-enabled cycles illustrated in [Figure 7-12](#).

For reference only: the collision window (slot time) is 64 clock-enabled cycles when operating at 10 Mb/s and 100 Mb/s speeds (corresponding to 64-bytes of frame data), and 512 clock-enabled cycles when operating at 1 Gb/s speed (corresponding to 512-bytes of frame data).

- If the `emacclienttxretransmit` signal is '0' in the same clock-enabled cycle that the `emacclienttxcollision` signal is '1,' the number of retries for this frame has exceeded the Ethernet specification or the collision has been classed as late, and the frame should be dropped by the client. The client can then make any new frame available to the MAC for transmission without timing restriction. This case is illustrated in [Figure 7-13](#).

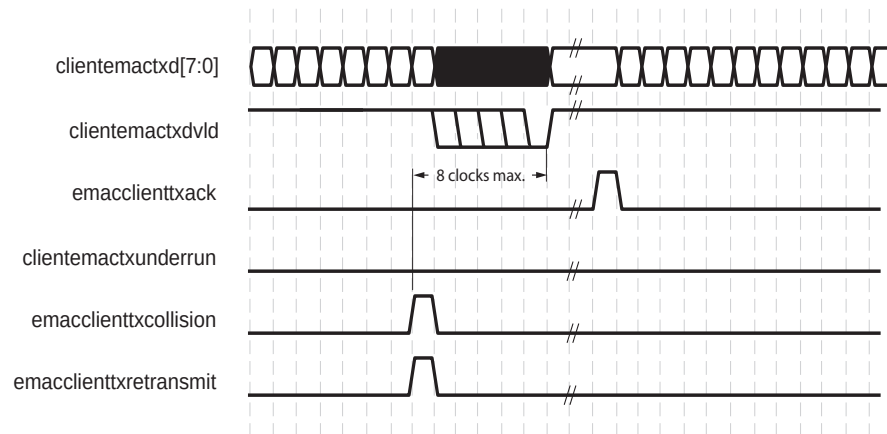


Figure 7-12: Collision Handling: Frame Retransmission Required

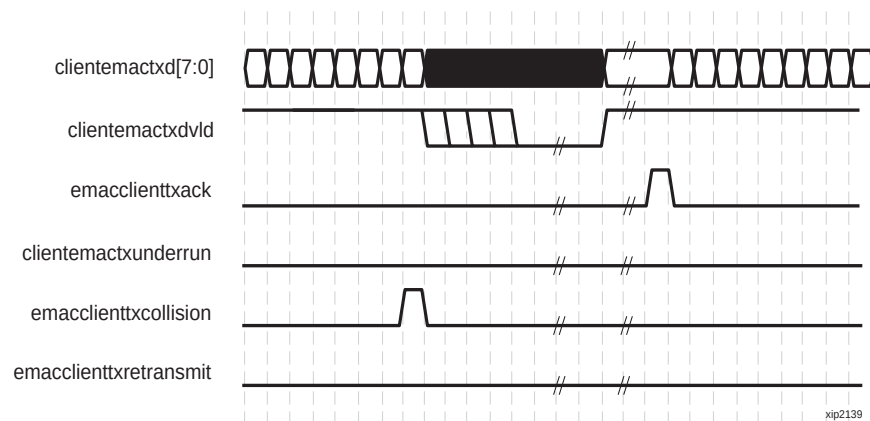


Figure 7-13: Collision Handling: No Frame Retransmission Required

Interframe Gap Adjustment: Full-Duplex Mode Only

A configuration bit in the transmitter control register (see [Configuration Register Map, page 90](#)) allows you to control the length of the interframe gap transmitted by the MAC on the physical interface. If this function is selected, the MAC exerts back pressure on the client interface to delay the transmission of the next frame until the requested number of idle cycles has elapsed. The number of idle cycles is controlled by the value on the `clientemactxifgdelay` port seen at the start of frame transmission on the client interface. [Figure 7-14](#) shows the MAC operating in this mode.

The minimum interframe gap supported is dependent upon the support of half-duplex operation. If half duplex is supported, the minimum IFG possible is 8 transmit clock cycles. If the MAC only supports full-duplex operation then this reduces the minimum possible IFG to 4 transmit clock cycles. In both cases the interframe gap used when the **Interframe Gap Adjust Enable** bit is set to '0' is the minimum value as specified in the *IEEE 802.3-2008*. This corresponds to 12 transmit clock cycles on the GMMI/MII interface. The value on the `clientemactxifgdelay` port must be larger than 4 or 8 to have an effect as described previously.

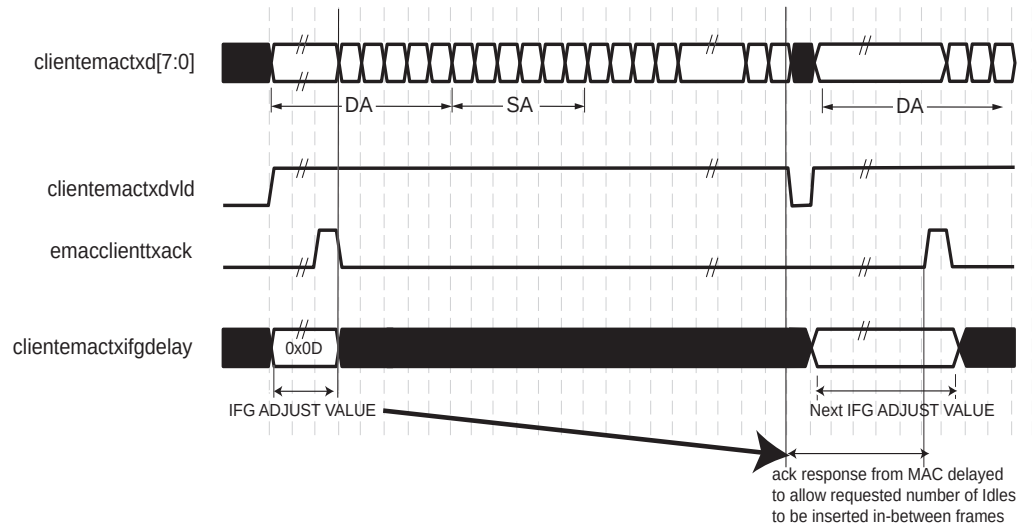


Figure 7-14: Interframe Gap Adjustment

Transmitter Statistics Vector

The statistics for the frame transmitted are contained within the `emacclienttxstats` output. The bit field definition for the Vector is defined in [Table 7-3](#).

All bit fields, with the exception of `BYTE_VALID` are valid only when the `emacclienttxstatsvld` is asserted, as illustrated in [Figure 7-15](#). `BYTE_VALID` is significant on every transmitter clock-enabled cycle.

`emacclienttxstats` bits 28 down to 20 inclusive are for half-duplex only and is set to logic 0 when operating in full-duplex mode.

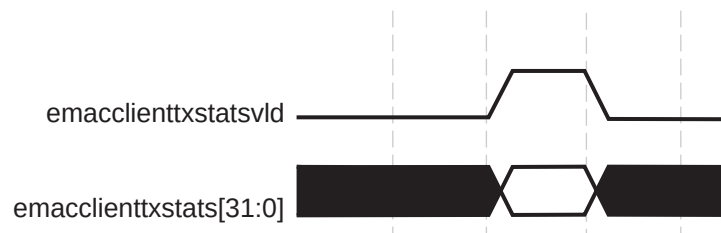


Figure 7-15: Transmitter Statistics Vector Timing

Table 7-3: Bit Definition for the Transmitter Statistics Vector

emacclientxstats	Name	Description
31	PAUSE_FRAME_TRANSMITTED	Asserted if the previous frame was a pause frame that the MAC itself initiated in response to a clientemacpausereq assertion.
30	BYTE_VALID	Asserted if a MAC frame byte (DA to FCS inclusive) is in the process of being transmitted. This is valid on every clock cycle. Do not use this as an enable signal to indicate that data is present on emacphytxd[7:0].
29	Reserved	Returns logic 0.
28 down to 25	TX_ATTEMPTS[3:0]	The number of attempts that have been made to transmit the previous frame. This is a 4-bit number: 0 should be interpreted as 1 attempt; 1 as 2 attempts, up until 15 as 16 attempts.
24	Reserved	Returns logic 0.
23	EXCESSIVE_COLLISION	Asserted if a collision has been detected on each of the last 16 attempts to transmit the previous frame.
22	LATE_COLLISION	Asserted if a late collision occurred during frame transmission.
21	EXCESSIVE_DEFERRAL	Asserted if the previous frame was deferred for an excessive amount of time as defined by the constant "maxDeferTime" in IEEE 802.3-2008.
20	TX_DEFERRED	Asserted if transmission of the frame was deferred.
19	VLAN_FRAME	Asserted if the previous frame contained a VLAN identifier in the length/type field when transmitter VLAN operation is enabled.

Table 7-3: Bit Definition for the Transmitter Statistics Vector (*Continued*)

emacclientxstats	Name	Description
18 down to 5	FRAME_LENGTH_COUNT	The length of the previous frame in number of bytes. The count will stick at 16368 for any jumbo frames larger than this value.
4	CONTROL_FRAME	Asserted if the previous frame had the special MAC Control Type code 88-08 in the length/type field.
3	UNDERRUN_FRAME	Asserted if the previous frame contained an underrun error.
2	MULTICAST_FRAME	Asserted if the previous frame contained a multicast address in the destination address field.
1	BROADCAST_FRAME	Asserted if the previous frame contained a broadcast address in the destination address field.
0	SUCCESSFUL_FRAME	Asserted if the previous frame was transmitted without error.

Flow Control

This chapter describes the operation of the flow control logic of the TEMAC solution. The flow control block is designed to clause 31 of the *IEEE 802.3-2008* standard. In full duplex mode the MAC can be configured to transmit pause requests and to act on their reception; these modes of operation can be independently enabled or disabled. (See [Configuration Register Map](#), page 90.)

Overview of Flow Control

Flow Control Requirement

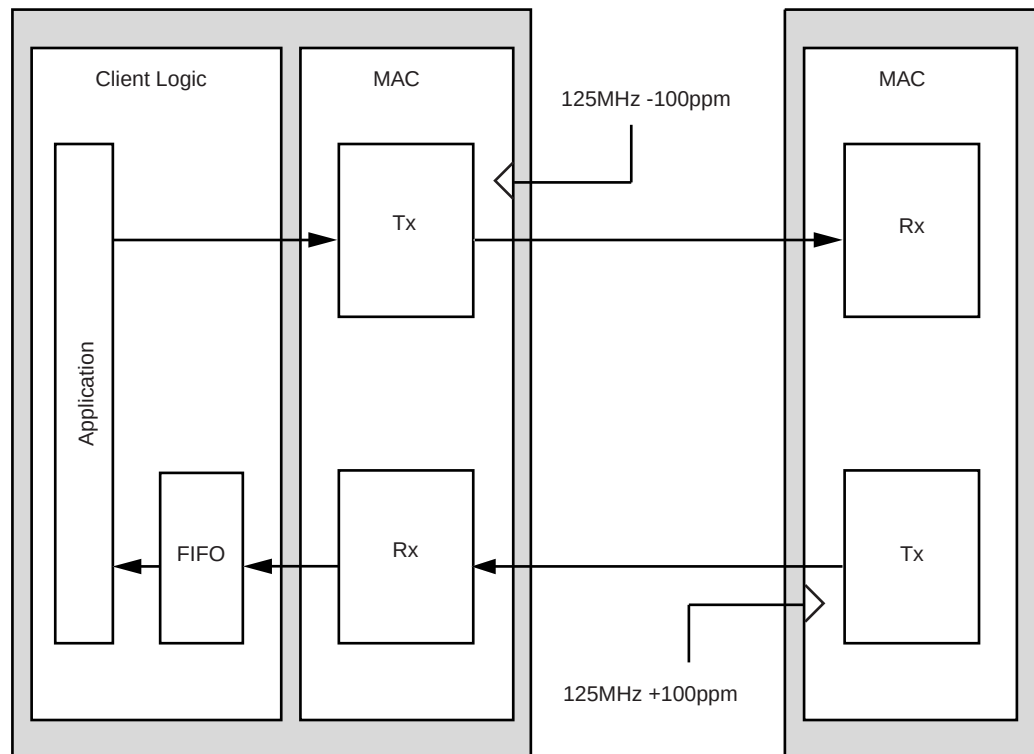


Figure 8-1: The Requirement for Flow Control

Figure 8-1 illustrates the requirement for Flow Control at 1 Gb/s. The MAC on the right side of the figure has a reference clock slightly faster than the nominal 125 MHz. The MAC on the left side of the figure has a reference clock slightly slower than the nominal 125 MHz. This results in the MAC on the left side of the figure not being able to match the full line rate of the MAC on the right side (due to clock tolerances). The MAC at the left is illustrated as performing a loopback implementation, which results in the FIFO filling up over time. Without Flow Control, this FIFO eventually fills and overflows, resulting in the corruption or loss of ethernet frames. Flow Control is one solution to this problem.

Flow Control Basics

A MAC may transmit a Pause Control frame to request that its link partner cease transmission for a specific period of time. For example, the left MAC in Figure 8-1 may initiate a pause request when its client FIFO (illustrated) reaches a nearly full state.

A MAC should respond to received Pause Control frames by ceasing transmission of frames for the period of time defined in the received pause control frame. For example, the right MAC of Figure 8-1 may cease transmission after receiving the Pause Control frame transmitted by the left MAC. In a well designed system, the right MAC ceases transmission before the client FIFO of the left MAC overflows to provide time to empty the FIFO to a safe level before resuming normal operation. This practice safeguards the system against FIFO overflow conditions and frame loss.

Pause Control Frames

Control frames are a special type of ethernet frame defined in clause 31 of the IEEE 802.3 standard. Control frames are identified from other frame types by a defined value placed into the length/type field (the MAC Control Type code). Figure 8-2 illustrates control frame format.

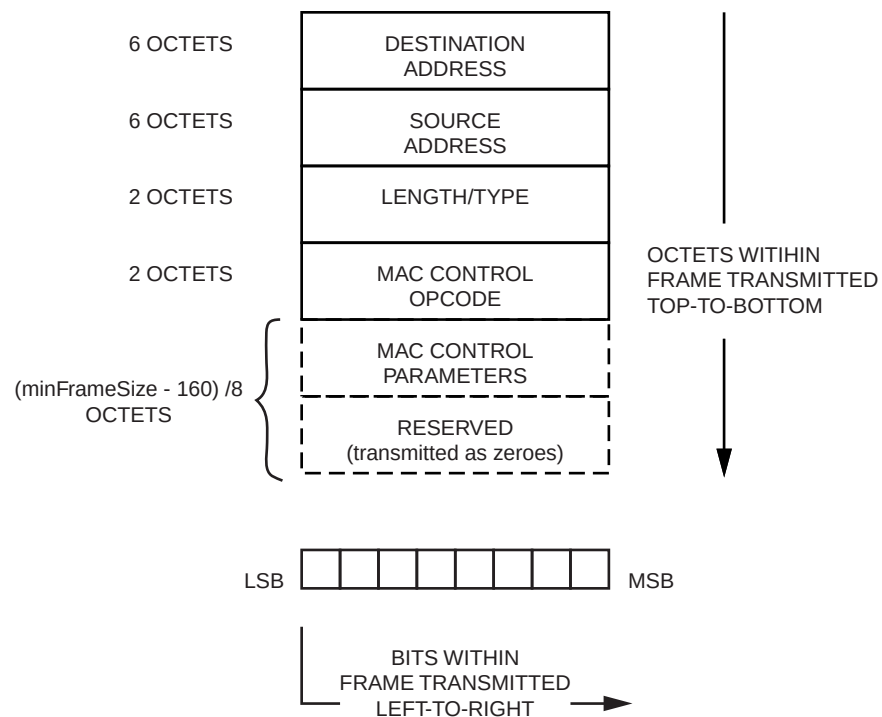


Figure 8-2: MAC Control Frame Format

A Pause Control frame is a special type of Control frame, identified by a defined value placed into the MAC Control opcode field.

Note: MAC Control OPCODES other than for Pause (Flow Control) frames have recently been defined for Ethernet Passive Optical Networks.

The MAC Control Parameter field of the Pause Control frame contains a 16-bit field which contains a binary value directly relating to the duration of the pause. This defines the number of *pause_quantum* (512 bit times of the particular implementation). At 1 Gb/s, a single *pause_quantum* corresponds to 512 ns. At 100 Mb/s, a single *pause_quantum* corresponds to 5120 ns, and at 10 Mb/s, a single *pause_quantum* corresponds to 51200 ns.

Flow Control Operation of the TEMAC

Transmitting a Pause Control Frame

Core-Initiated Pause Request

If the core is configured to support transmit flow control, the client may initiate a flow control frame by asserting `clientemacpausereq` while the pause value is on the `clientemacpauseval` bus. If the core is generated with the `clock_enable` option set to false, these signals are synchronous to `txgmiimiclk`. [Figure 8-3](#) illustrates this timing.

Pause request signals are synchronous to the `txgmiimiclk` clock. If a clock enable is present, this is the `clientemactxenble` signal.

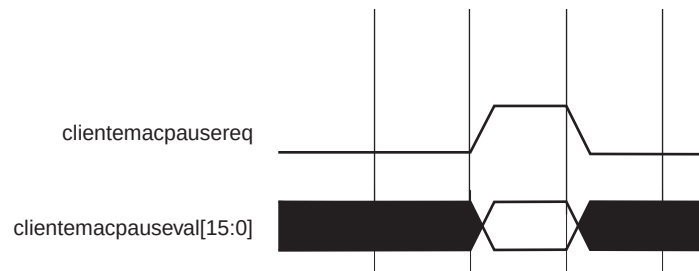


Figure 8-3: Pause Request Timing with Clock Enables

This action causes the core to construct and transmit a Pause Control frame on the link with the following MAC Control frame parameters (see [Figure 8-2](#)):

- The destination address used is an *IEEE 802.3* globally assigned multicast address (which any Flow Control capable MAC will respond to).
- The source address used is the configurable Pause Frame MAC Address (see ["Configuration Register Map" on page 90](#)).
- The value sampled from the `clientemacpauseval[15:0]` port at the time of the `clientemacpausereq` assertion is encoded into the MAC Control Parameter field to select the duration of the pause (in units of *pause_quantum*).

If the transmitter is currently inactive at the time of the pause request, this Pause Control frame is transmitted immediately. If the transmitter is currently busy, the current frame being transmitted is allowed to complete; the Pause Control frame will then follow in preference to any pending client supplied frame.

A Pause Control frame initiated by this method is transmitted even if the transmitter itself has ceased transmission in response to receiving an inbound pause request.

Note: Only a single pause control frame request is stored by the transmitter. If the `clientemacpausereq` signal is asserted numerous times in a short time period (before the control pause frame transmission has had a chance to begin), only a single pause control frame is transmitted. The `clientemacpauseval[15:0]` value used is the most recent value sampled.

Client-Initiated Pause Request

For maximum flexibility, flow control logic can be disabled in the core and alternatively implemented in the client logic connected to the core. Any type of Control frame can be transmitted through the core via the client interface using the same transmission procedure as a standard ethernet frame (see [Transmitting Outbound Frames, page 73](#)).

Receiving a Pause Control Frame

Core-Initiated Response to a Pause Request

An error-free Control frame is a received frame matching the format of [Figure 8-2](#). It must pass all standard receiver frame checks (for example, FCS field checking); in addition, the control frame received must be exactly 64-bytes in length (from destination address through to the FCS field inclusive). This is minimum legal ethernet MAC frame size and the defined size for control frames.

Any Control frame received that does not conform to these checks contains an error, and it is passed to the receiver client with the `emacclientrxbadframe` signal asserted.

Pause Frame Reception Disabled

When pause control reception is disabled, an error-free control frame is received through the client interface with the `emacclientrxgoodframe` signal asserted. In this way, the frame is passed to the client logic for interpretation (see [Client-Initiated Response to a Pause Request, page 87](#)).

Pause Frame Reception Enabled

When pause control reception is enabled and an error-free frame is received by the MAC core, the following frame decoding functions are performed:

1. The destination address field is matched against the *IEEE 802.3* globally assigned control multicast address (01-80-C2-00-00-01) or the configurable Pause Frame MAC Address (see [Configuration Register Map, page 90](#)).
2. The length/type field is matched against the MAC Control Type code.
3. If the second match is TRUE, the OPCODE field contents are matched against the Ethernet MAC control OPCODE for pause frames.

If all the previously listed checks are true, and the frame is of minimum legal size OR larger and control frame length checking is disabled (see [Table 9-4, page 91](#)), the 16-bit binary value in the MAC control parameters field of the control frame is then used to inhibit transmitter operation for the required number of *pause_quantum*. This inhibit is implemented by delaying the assertion of `clientemactxack` at the transmitter client interface until the requested pause duration has expired. Because the received pause frame has been acted upon, it is passed to the client with `emacclientrxbadframe` asserted to indicate to the client that can now be dropped.

If the second match is true and the frame is not exactly 64 bytes in length (when control frame length checking is enabled), the reception of any frame is considered to be an invalid control frame. This frame is ignored by the flow control logic and passed to the client with `emaccliентrxbadframe` asserted. In this case `emaccliентrxbadframe` is asserted even if flow control is not enabled.

If any of the previously listed checks are false, the frame is ignored by the Flow Control logic and passed up to the client logic for interpretation by marking it with `emaccliентrxgoodframe` asserted. It is then the responsibility of the MAC client logic to decode, act on (if required) and drop this control frame.

Note: Any frame in which the length/type field contains the MAC Control Type in the length/type field should be dropped by the receiver client logic. All Control frames are indicated by `emaccliентrxstats` bit 19 (see [Receiver Statistics Vector, page 71](#)).

Client-Initiated Response to a Pause Request

For maximum flexibility, flow control logic can be disabled in the core and alternatively implemented in the client logic connected to the core. Any type of error-free Control frame is then passed through the core with the `emaccliентrxgoodframe` signal asserted. In this way, the frame is passed to the client for interpretation. It is then the responsibility of the client to drop this control frame and to act on it by ceasing transmission through the core, if applicable.

Flow Control Implementation Example

This explanation is intended to describe a simple (but crude) example of a Flow Control implementation to introduce the concept.

Consider the system illustrated in [Figure 8-1](#). To summarize the example, the MAC on the left-hand side of the figure cannot match the full line rate of the right-hand MAC due to clock tolerances. Over time, the FIFO illustrated will fill and overflow. The aim is to implement a Flow Control method which, over a long time period, reduces the full line rate of the right-hand MAC to average that of the lesser full line rate capability of the left-hand MAC.

Method

1. Choose a FIFO nearly full to occupancy threshold (7/8 occupancy is used in this description). When the occupancy of the FIFO exceeds this occupancy, initiate a single pause control frame with 0xFFFF used as the *pause_quantum* duration (0xFFFF is placed on `cliентmacpauseval[15:0]`). This is the maximum pause duration. This causes the right-hand MAC to cease transmission and the FIFO of the left-hand MAC starts to empty.
2. Choose a second FIFO occupancy threshold (3/4 is used in this description). When the occupancy of the FIFO falls below this occupancy, initiate a second pause control frame with 0x0000 used as the *pause_quantum* duration (0x0000 is placed on `cliентmacpauseval[15:0]`). This indicates a zero pause duration, and upon receiving this pause control frame, the right-hand MAC immediately resumes transmission (it does not wait for the original requested pause duration to expire). This pause control frame can therefore be considered a "pause cancel" command.

Operation

Figure 8-4 illustrates the FIFO occupancy over time.

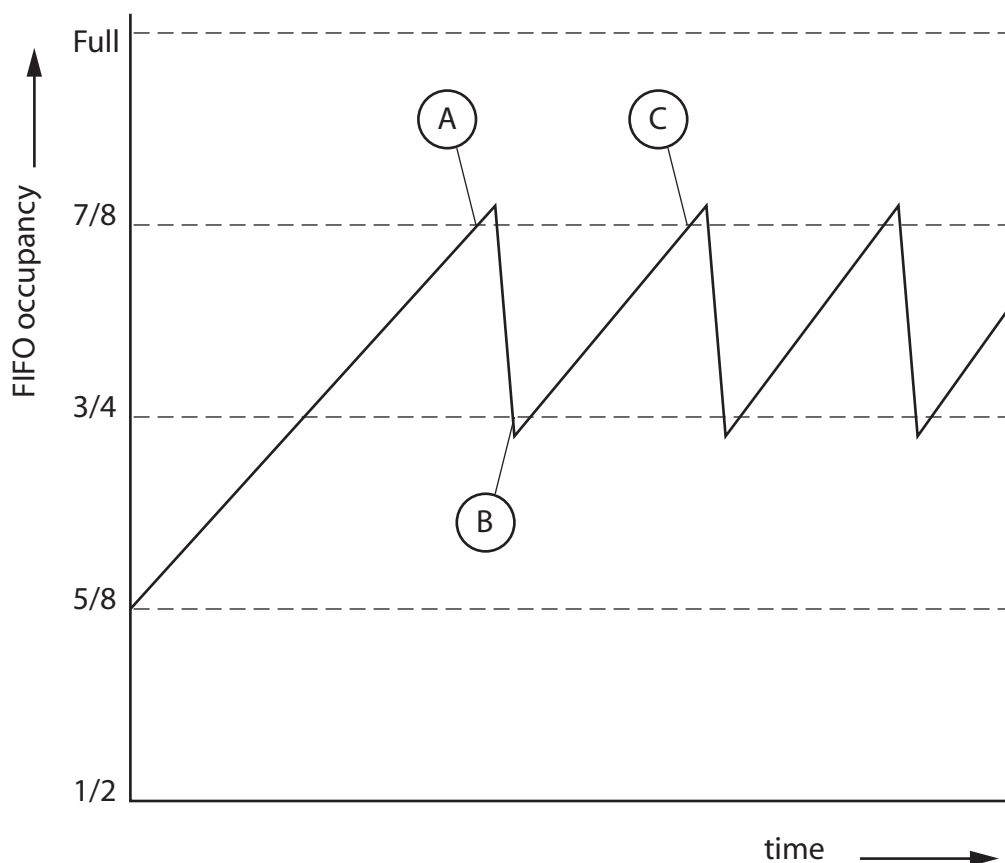


Figure 8-4: Flow Control Implementation Triggered from FIFO Occupancy

The following text describes the sequence of flow control operation in this given example.

1. The average FIFO occupancy of the left-hand MAC gradually increases over time due to the clock tolerances. At point A, the occupancy has reached the threshold of $7/8$ occupancy. This triggers the maximum duration pause control frame request.
2. Upon receiving the pause control frame, the right-hand MAC ceases transmission.
3. After the right-hand MAC ceases transmission, the occupancy of the FIFO attached to the left-hand MAC rapidly empties. The occupancy falls to the second threshold of $3/4$ occupancy at point B. This triggers the zero duration pause control frame request (the pause cancel command).
4. Upon receiving this second pause control frame, the right-hand MAC resumes transmission.
5. Normal operation resumes and the FIFO occupancy again gradually increases over time. At point C, this cycle of Flow Control repeats.

Configuration and Status

This chapter provides general guidelines for configuring and monitoring the core, including a detailed description of the client-side management interface and registers present in the core. It also describes the alternative to the optional management interface which is the Configuration Vector. See the appropriate section:

- [Using the Optional Management Interface](#)
- [Accessing Configuration without the Management Interface](#)

Finally, a section is included to discuss unusual configuration options:

- [TEMAC Configuration Settings](#)

Using the Optional Management Interface

The Management Interface is a processor-independent interface with standard address, data, and control signals. It can be used as is, or a wrapper (not supplied) can be applied to interface to common bus architectures. For port definition, see [Management Interface Signals](#), page 57.

This interface is used for:

- Configuration of the MAC core
- Access through the MDIO interface to the management registers located in the PHY attached to the MAC core. See [MDIO Interface](#) for an introduction.

The Management Interface is accessed differently depending on the type of transaction; a truth table showing which access method is required for each transaction type is shown in [Table 9-1](#). These access methods are described in the following sections.

Table 9-1: Management Interface Transaction Types

Transaction	HOST_MIIM_SEL	HOST_ADDR[9]	HOST_OPCODE[1]	HOST_OPCODE[0]
Configuration Read	0	1	1	X
Configuration Write	0	1	0	X
MIIM Read	1	X	1	0
MIIM Write	1	X	0	1

hostclk

The Management Interface clock, `hostclk`, is used to derive the MDIO clock, `emacphymclkout`, and for this reason is subject to the following frequency restriction:

$$\geq 10 \text{ MHz}$$

To save on clock resources, it can be tied to the user-supplied 125 MHz input clock.

Configuring the core to derive the `emacphymclkout` signal from this clock is detailed in [Accessing PHY Configuration Registers, via MDIO using the Management Interface](#), page 106.

Configuration Register Map

After power up or reset, the client may reconfigure the core parameters from their defaults, such as flow control support. Configuration changes can be written at any time. Both the receiver and transmitter logic only sample configuration changes at the start of frame transmission/reception. The exceptions to this are the configurable resets which take effect immediately.

Configuration of the core is performed through a register bank accessed through the Management Interface. The configuration registers available in the core are detailed in [Table 9-2](#). As can be seen, the address has some implicit *don't care* bits; any access to an address in the ranges shown perform a 32-bit read or write from the same configuration word.

Table 9-2: Configuration Registers

Address	Description
0x200-0x23F	Receiver Configuration (Word 0)
0x240-0x27F	Receiver Configuration (Word 1)
0x280-0x2BF	Transmitter Configuration
0x2C0-0x2FF	Flow Control Configuration
0x300-0x31F	MAC Speed Configuration
0x320-0x33F	Reserved
0x340-0x37F	Management Configuration
0x380-0x383	Unicast Address (Word 0) (if address filter is present)
0x384-0x387	Unicast Address (Word 1) (if address filter is present)
0x388-0x38B	Address Table Configuration (Word 0) (if address filter is present)
0x38C-0x38F	Address Table Configuration (Word 1) (if address filter is present)
0x390-0x3BF	Address Filter Mode (if address filter is present)

Configuration Register Definition

Receiver Configuration

The register contents for the two receiver configuration words can be seen in [Table 9-3](#) and [Table 9-4](#).

Table 9-3: Receiver Configuration Word 0

Bit	Default Value	Description
31-0	All 0s	Pause frame MAC Source Address[31:0] This MAC Address is used by the MAC to match against the destination address of any incoming flow control frames. It is also used by the flow control block as the source address (SA) for any outbound flow control frames. The address register is ordered so that the least significant bit of the MAC Address (IEEE802.3 definition) is stored in the least significant bit of the register. Consequently, bit 0 of this register differentiates between an individual or group (multicast) address. The transmission order within a MAC frame is to send the least significant bit of the MAC Address first. Consequently, bits 7-0 of this register represents the first byte to appear in frame transmission.

Table 9-4: Receiver Configuration Word 1

Bit	Default Value	Description
15-0	All 0s	Pause frame MAC Source Address[47:32] See description in Table 9-3 .
23-16	N/A	Reserved
24	0	Control Frame Length Check Disable When this bit is set to '1,' the core does not mark control frames as 'bad' if they are greater than the minimum frame length.
25	0	Length/Type Error Check Disable When this bit is set to '1,' the core does not perform the length/type field error checks as described in Length/Type Field Error Checks, page 69 . When this bit is set to '0,' the length/type field checks are performed; this is normal operation.
26	0	Half Duplex If '1,' the receiver operates in half- duplex mode. If '0,' the receiver operates in full- duplex mode.
27	0	VLAN Enable When this bit is set to '1,' VLAN tagged frames are accepted by the receiver.
28	1	Receiver Enable If set to '1,' the receiver block becomes operational. If set to '0,' the block ignores activity on the physical interface RX port.

Table 9-4: Receiver Configuration Word 1

Bit	Default Value	Description
29	0	In-band FCS Enable When this bit is '1,' the MAC receiver passes the FCS field up to the client as described in Client-Supplied FCS Passing, page 75 . When it is '0,' the client is not passed to the FCS. In both cases, the FCS is verified on the frame.
30	0	Jumbo Frame Enable When this bit is set to '1,' the MAC receiver accepts frames over the specified <i>IEEE 802.3-2008</i> maximum legal length. When this bit is '0,' the MAC only accepts frames up to the specified maximum.
31	0	Reset When this bit is set to '1,' the receiver is reset. The bit then automatically reverts to '0.' This reset also sets all of the receiver configuration registers to their default values.

Transmitter Configuration

The register contents for the Transmitter Configuration Word are described in [Table 9-5](#).

Table 9-5: Transmitter Configuration Word

Bit	Default Value	Description
24-0	N/A	Reserved
25	0	Interframe Gap Adjust Enable If '1,' the transmitter reads the value on the port clientemactxifgdelay at the start of frame transmission and adjust the interframe gap following the frame accordingly (see Interframe Gap Adjustment: Full-Duplex Mode Only , page 79). If '0,' the transmitter outputs a minimum interframe gap of at least twelve clock cycles, as specified in <i>IEEE 802.3-2008</i> .
26	0	Half Duplex If '1,' the transmitter operates in half-duplex mode.
27	0	VLAN Enable When this bit is set to '1,' the transmitter recognizes the transmission of VLAN tagged frames.
28	1	Transmit Enable When this bit is '1,' the transmitter is operational. When it is '0,' the transmitter is disabled.
29	0	In-band FCS Enable When this bit is '1,' the MAC transmitter expects the FCS field to be passed in by the client as described in Client-Supplied FCS Passing , page 75. When this bit is '0,' the MAC transmitter appends padding as required, compute the FCS and append it to the frame.
30	0	Jumbo Frame Enable When this bit is set to '1,' the MAC transmitter sends frames that are greater than the specified <i>IEEE 802.3-2008</i> maximum legal length. When this bit is '0,' the MAC only sends frames up to the specified maximum.
31	0	Reset When this bit is set to '1,' the transmitter is reset. The bit then automatically reverts to '0.' This reset also sets all of the transmitter configuration registers to their default values.

Flow Control Configuration

The register contents for the Flow Control Configuration Word are described in [Table 9-6](#).

Table 9-6: Flow Control Configuration Word

Bit	Default Value	Description
28-0	N/A	Reserved
29	1	Flow Control Enable (RX) When this bit is '1,' received flow control frames inhibit the transmitter operation as described in Receiving a Pause Control Frame, page 86 . When this bit is '0,' received flow control frames are always passed up to the client.
30	1	Flow Control Enable (TX) When this bit is '1,' asserting the <code>clientemacpausereq</code> signal sends a flow control frame out from the transmitter as described in Transmitting a Pause Control Frame, page 85 . When this bit is '0,' asserting the <code>clientemacpausereq</code> signal has no effect.
31	N/A	Reserved

MDIO Configuration

The register contents for the MDIO Configuration Word are described in [Table 9-7](#).

Table 9-7: MDIO Configuration Word

Bits	Default Value	Description
5-0	All 0s	Clock Divide[5:0] See Accessing PHY Configuration Registers, via MDIO using the Management Interface, page 106 .
6	0	MDIO Enable When this bit is '1,' the MDIO interface can be used to access attached PHY devices. When this bit is '0,' the MDIO interface is disabled and the MDIO signals remain inactive. A write to this bit only takes effect if Clock Divide is set to a non-zero value.
31-7	N/A	Reserved

TEMAC Operational Speed Configuration

The register contents for the MAC Speed Configuration Word, when the TEMAC solution has been generated with tri-speed support, are described in [Table 9-8](#).

When the TEMAC solution has been generated for only 1 Gb/s speed support, bits 31-3 are hard coded to the value "10".

When the TEMAC solution has been generated for only 10 Mb/s and 100 Mb/s speed support, bits 31-3 only accept the values of "00" to configure for 10 Mb/s operation, or "01" to configure for 100 Mb/s operation.

Caution! Please issue the core with a system-wide reset following a speed change.

Table 9-8: MAC Speed Configuration Word

Bits	Default Value	Description
29-0	N/A	Reserved
31-30	"10"	MAC Speed Configuration "00" - 10 Mb/s "01" - 100 Mb/s "10" - 1 Gb/s

Note: The setting of the MAC Speed Configuration register is not affected by a reset.

Address Filter Configuration

[Table 9-9](#) through [Table 9-13](#) describe the registers used to access the optional Address Filter configuration when the TEMAC solution is implemented with an Address Filter. If no address filter is present, these registers do not exist and return 0s for a read from the stated addresses.

Unicast Address Configuration

The register contents for the two unicast address registers are described in [Table 9-9](#) and [Table 9-10](#).

Table 9-9: Unicast Address (Word 0)

Bits	Default Value	Description
31-0	tieemacunicastaddr[31-0]	Address filter unicast address[31:0]. This MAC Address is used by the MAC to match against the destination address of any incoming frames. The address register is ordered so that the least significant bit of the MAC Address (IEEE802.3 definition) is stored in the least significant bit of the register. Consequently, bit 0 of this register differentiates between an individual or group (multicast) address. The transmission order within a MAC frame is to send the least significant bit of the MAC Address first. Consequently, bits 7-0 of this register represent the first byte to appear in frame transmission.

Table 9-10: Unicast Address (Word 1)

Bits	Default Value	Description
15-0	tieemacunicastaddr[47 downto 32]	Address filter unicast address[47:32]. See description in Table 9-9 .
31-16	N/A	Reserved

Optional Address Table

In addition to the unicast address, broadcast address and pause addresses, the address filter can optionally be generated to respond to up to four additional separate addresses. These are stored in an address table within the address filter. See [Address Filter, page 70](#). [Table 9-11](#) and [Table 9-12](#) show how the contents of the table are set.

.If the TEMAC solution is generated without Address Table functionality, the following registers do not exist and return 0s for a read from the stated addresses.

Table 9-11: Address Table Configuration (Word 0)

Bits	Default Value	Description
31-0	All 0s	MAC Address[31:0]. The lower 31 bits of the MAC Address that is to be written to the address table. The address register is ordered so that the least significant bit of the MAC Address (IEEE802.3 definition) is stored in the least significant bit of the register. Consequently, bit 0 of this register differentiates between an individual or group (multicast) address. The transmission order within a MAC frame is to send the least significant bit of the MAC Address first. Consequently, bits 7-0 of this register represent the first byte to appear in frame transmission.

Table 9-12: Address Table Configuration (Word 1)

Bits	Default Value	Description
15-0	All 0s	MAC Address[47:32]. The upper 16 bits of MAC address: see the description in Table 9-11.
17-16	All 0s	The location in the address table that the MAC address is to be written to or read from. There are up to 4 entries in the table (Location 0 to 3).
22-18	N/A	Reserved
23	0	Read not write This bit is set to '1' to read from the address table. If it is set to '1,' the contents of the table entry that is being accessed by bits 17-16 are output on the hostrddata bus in consecutive cycles (Least Significant Word first). If it is set to '0,' the data on bits 15-0 is written into the table at the address specified by bits 17-16.
31-24	N/A	Reserved

The contents of the address table mode register are described in [Table 9-13](#).

Table 9-13: Address Filter Mode

Bits	Default Value	Description
30-0	N/A	Reserved
31	1	Promiscuous Mode If this bit is set to '1,' the address filter is set to operate in promiscuous mode. All frames are passed to the receiver client regardless of the destination address.

Using the Management Interface

Accessing Configuration

Writing to the configuration registers through the Management Interface is depicted in [Figure 9-1](#). When accessing the configuration registers (for example, when `hostaddr[9] = '1'` and `hostmiimsel = '0'`), the upper bit of `hostopcode` functions as an Active Low write enable signal. The lower `hostopcode` bit is a *don't care* bit.

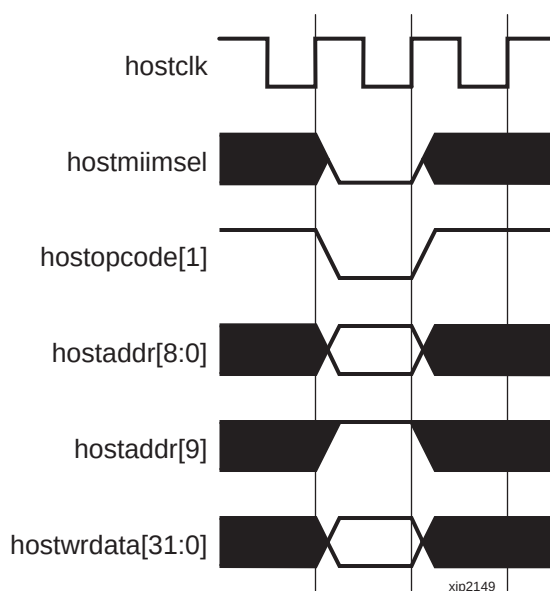


Figure 9-1: Configuration Register Write Timing

Reading from the configuration register words is similar, but the upper `hostopcode` bit should be '1,' as shown in [Figure 9-2](#). In this case, the contents of the register appear on `hostrddata` the `hostclk` edge after the register address is asserted onto `hostaddr`.

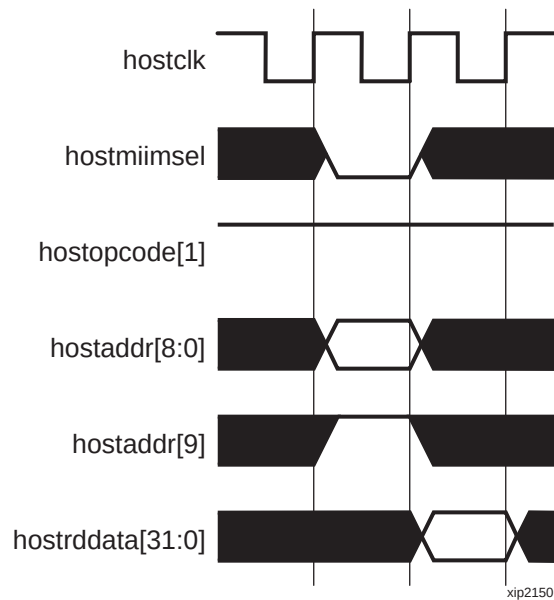


Figure 9-2: Configuration Register Read Timing

Accessing the MAC Address Table of the Address Filter

Access to the Unicast Address of the Address Filter follows the register access described in [Accessing Configuration](#) and illustrated in [Figure 9-1](#) and [Figure 9-2](#), using the appropriate Unicast Register address as defined in [Table 9-2](#).

Writing and reading to the MAC Address Table is less straightforward. To write to a specific entry in the table, you must first write the least significant 32-bits of the address into the address table configuration (Word 0) register. You then write the most significant 16-bits together with the location in the table (bits 17-16) to the address table configuration (Word 1) register with bit 23 (read not write) set to '0.' This is shown in Figure 9-3.

Although it is shown in the figure, there is no requirement for the two writes to be on adjacent cycles. However, following this second write, allow 1 wait state (a single period of `hostclk`) before accessing the MAC Address Table again: this single wait state requirement only applies to accesses to the MAC Address Table and any other configuration register that can be accessed immediately.

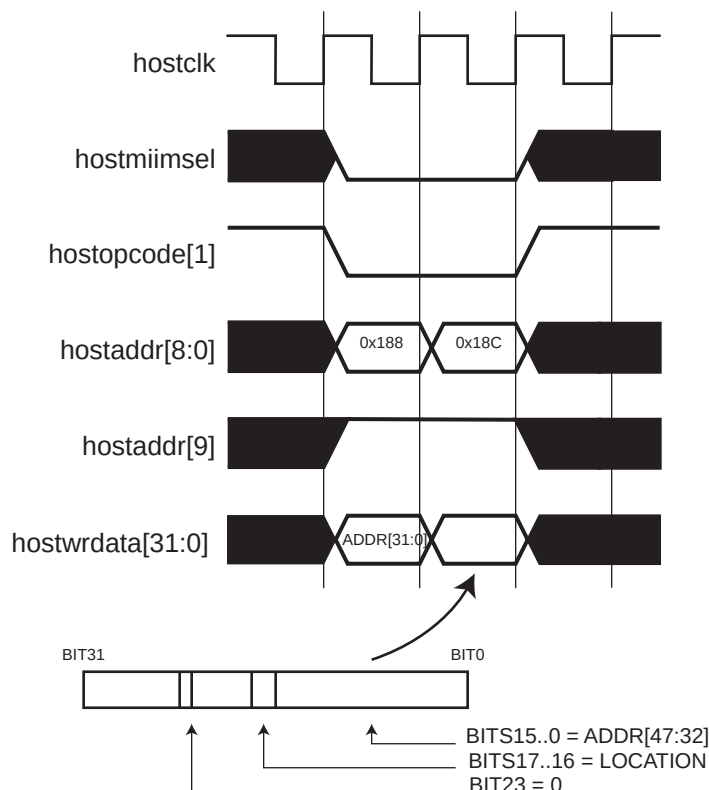


Figure 9-3: Address Table Write Timing

To read from the address table, you write to the address table configuration register (Word 1) with the location set to the desired table entry and bit 23 set to '1.' On the next cycle the least significant word appears on the `hostrrdata` bus. One cycle afterwards the most significant 16 bits are output on the lower 16 bits of the bus. This is shown in Figure 9-4.

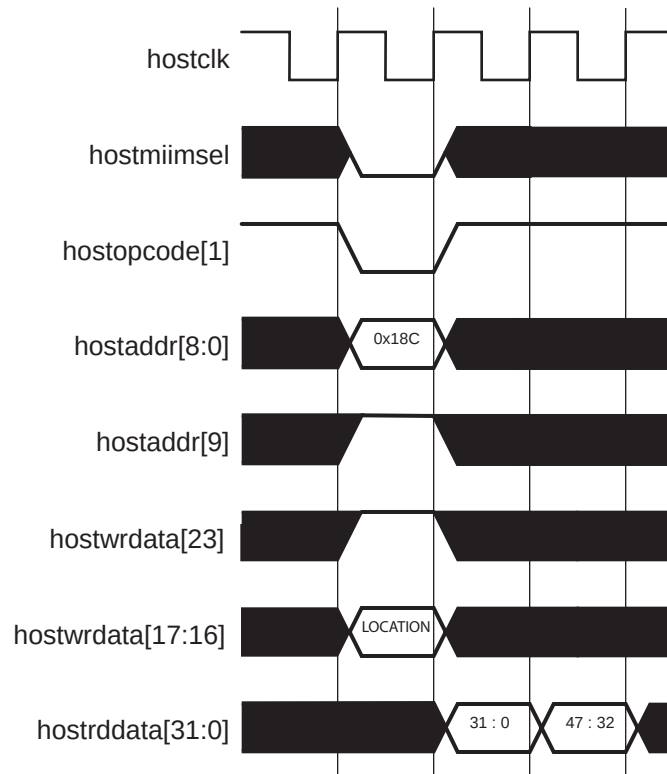


Figure 9-4: Address Table Read Timing

MDIO Interface

The MDIO is always present whenever the TEMAC solution is generated with Management Register support.

Introduction to MDIO

MDIO Bus System

The Management Data Input/Output (MDIO) interface for access to Ethernet PHY devices for 1 Gb/s operation and slower speeds is defined in *IEEE 802.3*, clause 22. This two-wire interface consists of a clock (MDC) and a shared serial data line (MDIO). The maximum permitted frequency of MDC is set at 2.5 MHz.

Figure 9-5 illustrates an example MDIO bus system.

An Ethernet MAC is shown as the MDIO bus master (the Station Management (STA) entity).

Two PHY devices are shown connected to the same bus, both of which are MDIO slaves (MDIO Managed Device (MMD) entities).

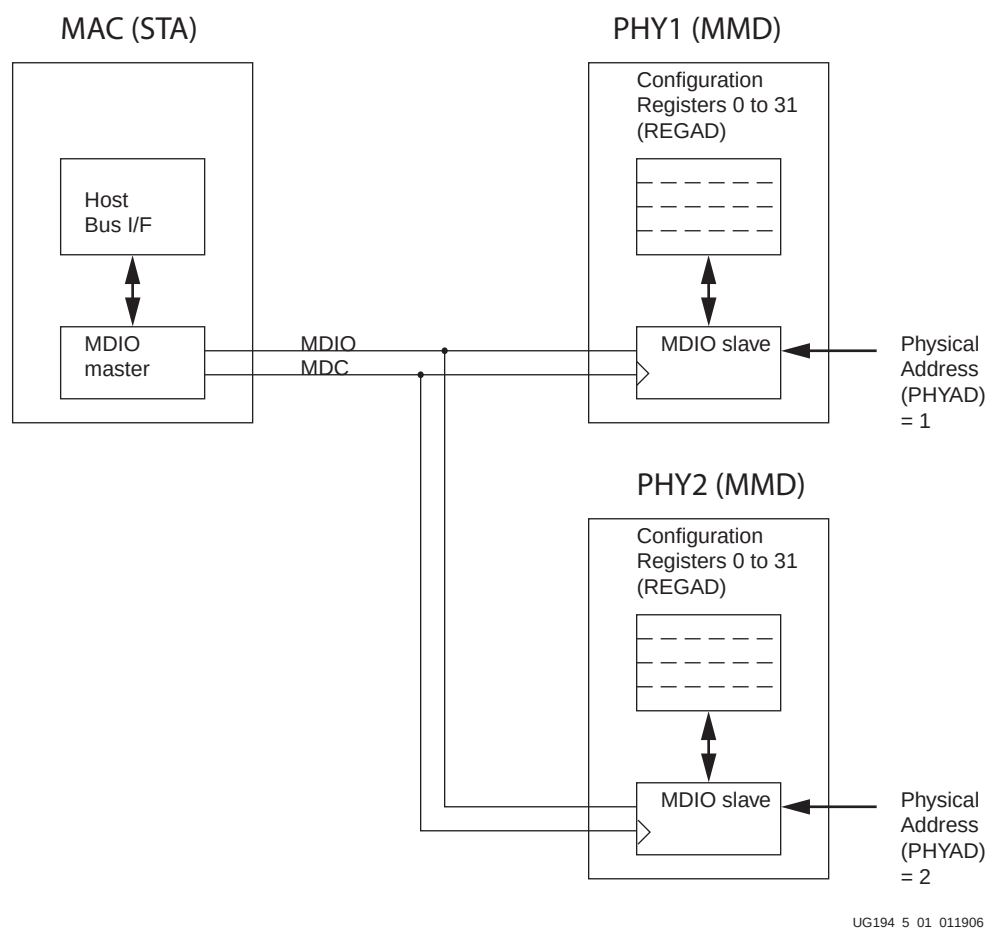


Figure 9-5: A Typical MDIO-managed System

The MDIO bus system is a standardized interface for accessing the configuration and status registers of Ethernet PHY devices. In the example illustrated, the Management Host Bus I/F of the Ethernet MAC is able to access the configuration and status registers of two PHY devices via the MDIO bus.

MDIO Transactions

All transactions, read or write, are initiated by the MDIO master. All MDIO slave devices, when addressed, must respond. MDIO transactions take the form of an MDIO frame, containing fields for transaction type, address and data. This MDIO frame is transferred across the MDIO wire synchronously to MDC. The abbreviations are used in this section are explained in [Table 9-14](#).

Table 9-14: Abbreviations and Terms

Abbreviation	Term
PRE	Preamble
ST	Start of frame
OP	Operation code
PHYAD	Physical address
REGAD	Register address
TA	Turnaround

Write Transaction

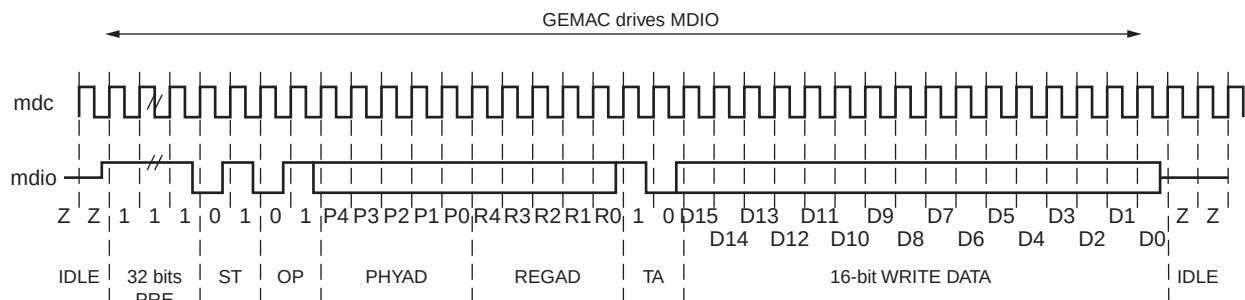


Figure 9-6: MDIO Write Transaction

[Figure 9-6](#) shows a Write transaction across the MDIO; this is defined by OP="01". The addressed PHY (PHYAD) device takes the 16-bit word in the data field and writes it to the register at REGAD.

Read Transaction

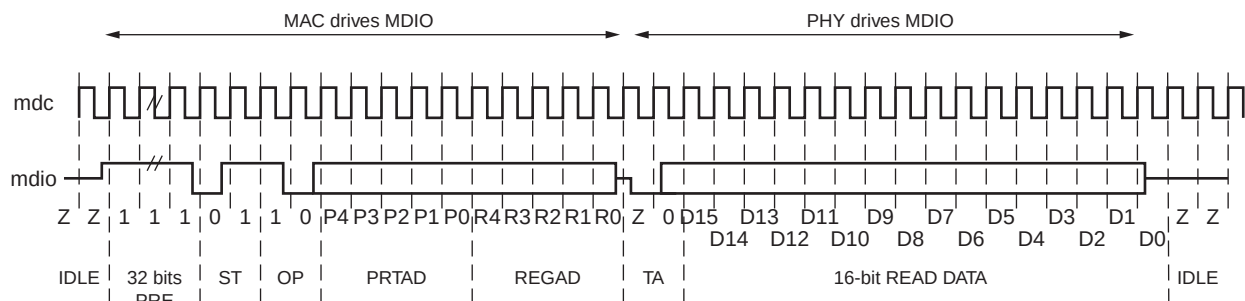


Figure 9-7: MDIO Read Transaction

Figure 9-7 shows a Read transaction; this is defined by OP="10". The addressed PHY (PHYAD) device returns the 16-bit word from the register at REGAD.

For details of the register map of PHY layer devices and a fuller description of the operation of the MDIO Interface itself, see *IEEE 802.3-2008*.

Connecting the TEMAC to an MDIO bus

Connecting the MDIO to an Internally Integrated PHY

The MDIO ports of the core (see Table 5-9) can be connected to the MDIO ports of an internally integrated physical layer device. For example, the MDIO ports of the Ethernet 1000BASE-X PCS/PMA or SGMII from Xilinx (see [Ethernet 1000BASE-X PCS/PMA or SGMII Core](#), page 225).

Connecting the MDIO to an External PHY

When the core is used to connect to an external PHY device using GMII/MII or RGMII, it is expected that the MDIO of the core will also be connected to the external PHY. This allows the configuration registers of the PHY to be accessed through the Management Interface of the core.

In this situation, `phyemacmdin`, `emacphyndout` and `emacphyndtri` must be connected to a Tri-State buffer to create a bidirectional wire, `mdio`. This Tri-State buffer can be either external to the FPGA, or internally integrated by using an IOB IOBUF component with an appropriate SelectIO™ interface standard for the external PHY. (This is illustrated in Figure 9-8.).

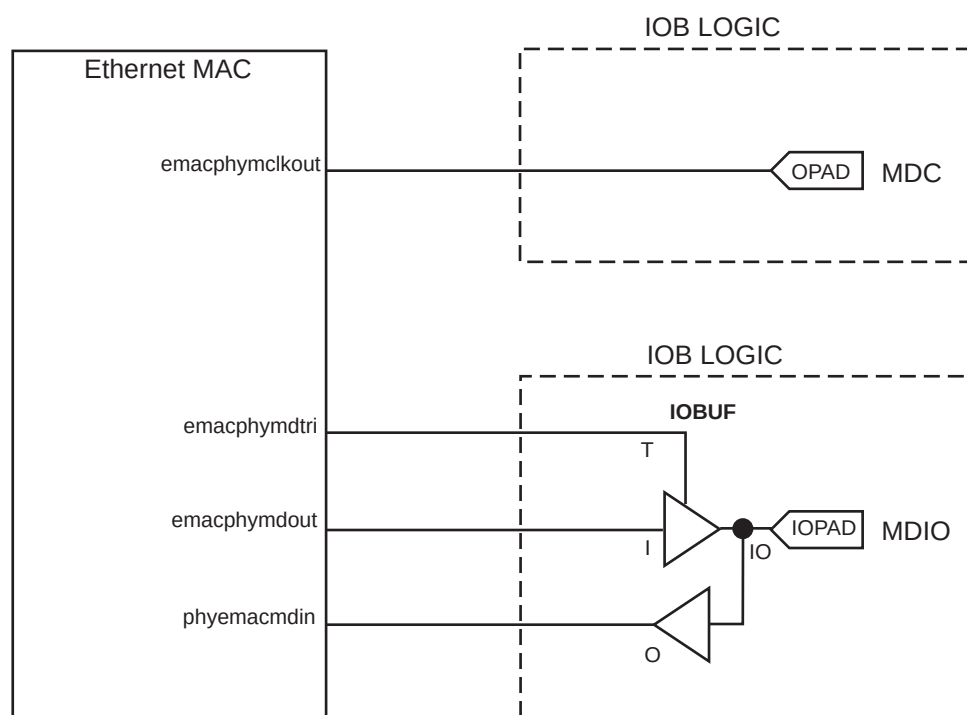


Figure 9-8: External MDIO Interface

Connecting the MDIO to an External and Internal PHY

The MDIO can connect to more than one device. If an internal PHY is present, for example the *Ethernet 1000BASE-X PCS/PMA* or *SGMII LogiCORE™*, performing as SGMII **and** it is desired to also connect the MDIO to an external PHY device, then an arbitration circuit is required. An example circuit is shown in [Figure 9-9](#). Both PHY devices must be assigned a unique non zero physical address (PHYAD).

This description is included only for completeness and this particular use case is not expected to be common.

Note: The SGMII specification itself contains a method of transferring PHY configuration information across the SGMII link. Therefore all relevant PHY information can be obtained directly from the internal SGMII core.

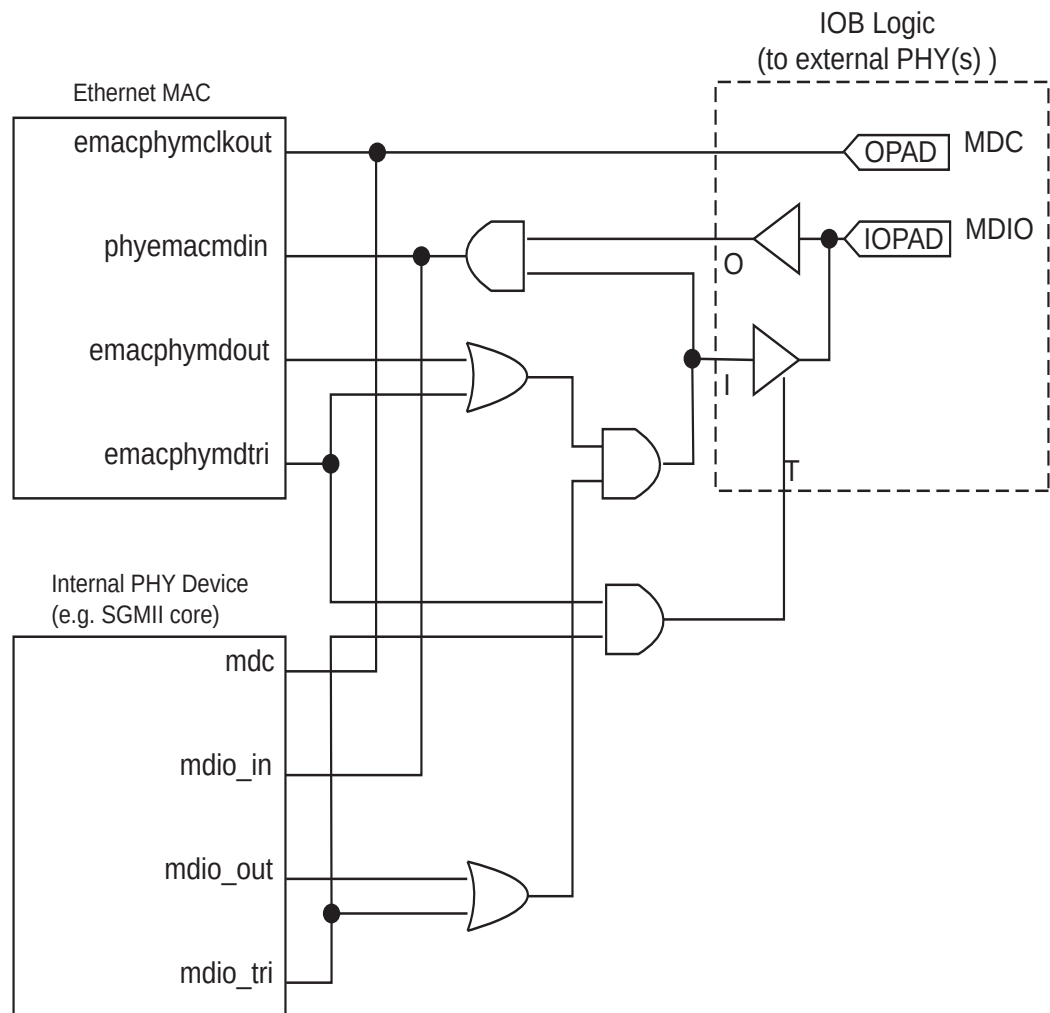


Figure 9-9: Internal and External MDIO Interfaces

Accessing PHY Configuration Registers, via MDIO using the Management Interface

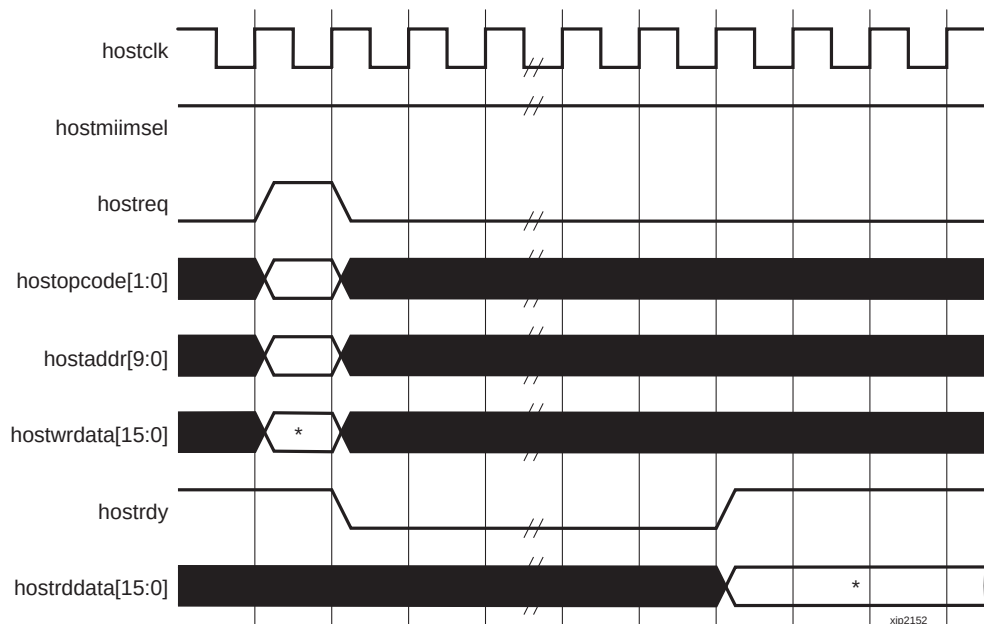
The Management Interface is also used to access the MDIO interface of the core. The MDIO interface supplies a clock to the connected PHY, `mdc`. This clock is derived from the `hostclk` signal using the value in the `Clock Divide[4:0]` configuration register. The frequency of `mdc` is given by the following equation:

$$f_{MDC} = \frac{f_{HOST_CLK}}{(1 + \text{Clock Divide}[4:0]) \times 2}$$

The frequency of `mdc` given by this equation should not exceed 2.5 MHz to comply with the *IEEE 802.3-2008* specification for this interface. To prevent `mdc` from being out of specification, the `Clock Divide[4:0]` value powers up at 00000, and while this value is in the register, it is impossible to enable the MDIO interface.

For details of the register map of PHY layer devices and a fuller description of the operation of the MDIO interface itself, see *IEEE 802.3-2008*.

Access to the MDIO interface through the Management Interface is depicted in the timing diagram in [Figure 9-10](#).



* If a read transaction is initiated, the hostrddata bus is valid at the point indicated. If a write transaction is initiated, the hostwrdata bus must be valid at the indicated point. Simultaneous read and write is not permitted.

Figure 9-10: MDIO Access Through Management Interface

For MDIO transactions, the following points apply:

- `hostmiimsel` is '1'
- `hostopcode[1:0]` maps to the OP (opcode) field of the MDIO frame
- `hostaddr` maps to the two address fields of the MDIO frame; PHYAD is `hostaddr[9:5]`, and REGAD is `host_addr[4:0]`
- `hostwrdata[15:0]` maps into the data field of the MDIO frame when performing a write operation
- The data field of the MDIO frame maps into `hostrddata[15:0]` when performing a read operation

The MAC core signals to the host that it is ready for an MDIO transaction by asserting `hostmiimrdy`. A read or write transaction on the MDIO is initiated by a pulse on the `hostreq` signal. This pulse is ignored if the MDIO interface already has a transaction in progress.

The MAC core then deasserts the `hostmiimrdy` signal while the transaction across the MDIO is in progress. When the transaction across the MDIO interface has been completed, the `hostmiimrdy` signal is asserted by the MAC core; if the transaction is a read, the data is also available on the `hostrddata[15:0]` bus at this time.

Accessing Configuration without the Management Interface

If the optional management interface is omitted from the core, all of the relevant configuration settings described in [Table 9-3](#) through [Table 9-8](#) are brought out of the core as signals. These signals are bundled into the `tieemacconfigvec[67:0]` signal as described in [Table 9-15](#).

These signals may be permanently set by connecting to logic 0 or 1, or may be changed by your application at any time; however, with the exception of the reset signals, any changes do not take effect until the current frame has completed transmission or reception.

The Clock heading in [Table 9-15](#) denotes which clock domain the configuration signal is registered into before use by the core. It is not necessary to drive the signal from this clock domain

Note: MDIO functionality is not included in this configuration.

Table 9-15: Configuration Vector Bit Definition

Bit(s)	Configuration Register cross reference	Clock	Description
47:0	Receiver Configuration Word 0 bits 31-0 and Receiver Configuration Word 1 bits 15-0	rxgmiimiiclk	Pause frame MAC Source Address[47:0] This MAC Address is used by the MAC core to match against the destination address of any incoming flow control frames, and as the source address for any outbound flow control frames. The bits in this vector field are ordered so that the least significant bit of the MAC Address (IEEE802.3 definition) is stored in the least significant bit of this vector field. Consequently, bit 0 of this field differentiates between an individual or group (multicast) address. The transmission order within a MAC frame is to send the least significant bit of the MAC Address first. Consequently, bits 7-0 of this vector field represents the first byte to appear in frame transmission
48	Receiver Configuration Word 1 bit 26	rxgmiimiiclk	Receiver Half Duplex If '1,' the receiver operates in half-duplex mode. If '0,' the receiver operates in full-duplex mode. If the TEMAC has been generated without half-duplex support then this input to the core is unused.
49	Receiver Configuration Word 1 bit 27	rxgmiimiiclk	Receiver VLAN Enable When this bit is set to '1,' VLAN tagged frames are accepted by the receiver.
50	Receiver Configuration Word 1 bit 28	rxgmiimiiclk	Receiver Enable If set to '1,' the receiver block is operational. If set to '0,' the block ignores activity on the physical interface RX port.

Table 9-15: Configuration Vector Bit Definition (Continued)

Bit(s)	Configuration Register cross reference	Clock	Description
51	Receiver Configuration Word 1 bit 29	rxgmiimiiclk	Receiver In-band FCS Enable When this bit is '1,' the MAC receiver passes the FCS field up to the client as described in Client-Supplied FCS Passing, page 68 . When it is '0,' the MAC receiver does not pass the FCS field. In both cases, the FCS field is verified on the frame.
52	Receiver Configuration Word 1 bit 30	rxgmiimiiclk	Receiver Jumbo Frame Enable When this bit is '0,' the receiver does not pass frames longer than the maximum legal frame size specified in <i>IEEE 802.3-2008</i> (Maximum Permitted Frame Length, page 78). When it is '1,' the receiver does not have an upper limit on frame size.
53	Receiver Configuration Word 1 bit 31	N/A	Receiver Reset When this bit is '1,' the receiver is held in reset. This signal is an input to the reset circuit for the receiver block.
54	Transmitter Configuration Word bit 25	txgmiimiiclk	Transmitter Interframe Gap Adjust Enable If '1,' and the MAC is set to operate in full-duplex mode, then the transmitter reads the value of the clientemactxifgdelay port and set the Interframe Gap accordingly. If '0,' the transmitter always inserts at least the legal minimum interframe gap.
55	Transmitter Configuration Word bit 26	txgmiimiiclk	Transmitter Half Duplex If '1,' the transmitter operates in half-duplex mode. If '0,' the transmitter operates in full-duplex mode. If the TEMAC solution has been generated without half-duplex support, this input to the core is unused.
56	Transmitter Configuration Word bit 27	txgmiimiiclk	Transmitter VLAN Enable When this bit is set to '1,' the transmitter allows the transmission of VLAN tagged frames.
57	Transmitter Configuration Word bit 28	txgmiimiiclk	Transmitter Enable When this bit is '1,' the transmitter is operational. When it is '0,' the transmitter is disabled.

Table 9-15: Configuration Vector Bit Definition (Continued)

Bit(s)	Configuration Register cross reference	Clock	Description
58	Transmitter Configuration Word bit 29	txgmiimiiclk	Transmitter In-Band FCS Enable When this bit is '1,' the MAC transmitter expects the FCS field to be passed in by the client as described in Client-Supplied FCS Passing, page 75 . When it is '0,' the MAC transmitter appends padding as required, compute the FCS and append it to the frame.
59	Transmitter Configuration Word bit 30	txgmiimiiclk	Transmitter Jumbo Frame Enable When this bit is '1,' the MAC transmitter allows frames larger than the maximum legal frame length specified in <i>IEEE 802.3-2008</i> to be sent. When set to '0,' the MAC transmitter only allows frames up to the legal maximum to be sent.
60	Transmitter Configuration Word bit 31	N/A	Transmitter Reset When this bit is '1,' the MAC transmitter is held in reset. This signal is an input to the reset circuit for the transmitter block.
61	Flow Control Configuration Word bit 29	txgmiimiiclk	Transmit Flow Control Enable When this bit is '1,' asserting the <code>clientemacpausereq</code> signal causes the MAC core to send a flow control frame out from the transmitter as described in Transmitting a Pause Control Frame, page 85 . When this bit is '0,' asserting the <code>clientemacpausereq</code> signal has no effect.
62	Flow Control Configuration Word bit 30	rxgmiimiiclk	Receive Flow Control Enable When this bit is '1,' received flow control frames inhibit the transmitter operation as described in Receiving a Pause Control Frame, page 86 . When it is '0,' received flow frames are passed up to the client.
63	Receiver Configuration Word 1 bit 25	rxgmiimiiclk	Length/Type Error Check Disable When this bit is '1,' the core does not perform the length/type field error checks as described in Length/Type Field Error Checks, page 69 . When it is set to '0,' the length/type field checks are performed; this is normal operation.

Table 9-15: Configuration Vector Bit Definition (Continued)

Bit(s)	Configuration Register cross reference	Clock	Description
64	Address Filter Mode bit 31	rxgmiimiiclk	Address Filter Enable When this bit is '0,' the address filter is enabled. If it is set to '1,' the address filter operates in promiscuous mode. If the TEMAC solution has been generated without the optional Address Filter, this input to the core is unused.
66:65	MAC Speed Configuration Word bits 31 - 30	txgmiimiiclk and rxgmiimiiclk	MAC Speed Configuration "00" - 10 Mb/s "01" - 100 Mb/s "10" - 1 Gb/s When the TEMAC solution is generated for only 1 Gb/s speed support, these inputs is unused. When the TEMAC solution is generated for only 10 Mb/s or 100 Mb/s speed support, only bit 65 is used to differentiated the speed: bit 66 is unused. Caution! Please issue the core with a system-wide reset following a speed change.
67	Receiver Configuration Word 1 bit 24	rxgmiimiiclk	Control Frame Length Check Disable When this bit is set to '1,' the core does not mark control frames as 'bad' if they are greater than the minimum frame length.

TEMAC Configuration Settings

This section discusses unusual configuration options. These can be set by either method (Management I/F or the alternative Configuration Vector).

Half-Duplex Configuration Settings

When the core is generated with half-duplex capability, the transmitter and receiver can be independently configured between full and half-duplex modes. This functionality is made available for full flexibility in unusual applications, for example, ethernet protocol testers.

However, for legal and predictable behavior in ethernet networks:

- Always configure transmitter and receiver duplex modes identically.

Half-Duplex and Flow Control Configuration Settings

The IEEE802.3 specification defines the flow control functionality only for full-duplex applications.

Configuration of the TEMAC allows Flow Control functionality and Duplex mode to be configured independently. However, Flow Control is only enabled in full-duplex mode:

- When operating half-duplex mode, always disable Flow Control.
- When operating in full-duplex mode, Flow Control can optionally be enabled.

MAC Address Settings

Under all core generation settings, the core contains a configurable Pause frame MAC Source Address (see [Receiver Configuration](#)). This [MAC Address](#) is used by the flow control logic; received pause frames are matched against this address appearing in the Destination Address field; pause frames initiated by the core place this MAC Address into the Source Address field of a transmitted pause frame.

When the TEMAC solution is generated with the optional address filter, the core contains a configurable Unicast Address (see [Unicast Address Configuration](#)). This is used by the Address Filter to match against this address appearing in the Destination Address field of all received frames.

The core, for full flexibility, allows the Pause frame MAC Source Address and the Unicast Address (when present) to be configured independently. However, under standard network operating conditions:

- the Pause frame MAC Source Address should be set to the Unicast Address

Physical Interface for the 10 Mb/s and 100 Mb/s Only Ethernet MAC IP Core

The HDL example design supplied with the 10 Mb/s or 100 Mb/s only IP core, provides an MII interface. This is typically used to connect the MAC to an external PHY device.

The Media Independent Interface (MII), defined in IEEE Std 802.3-2008, clause 22 is a parallel interface that connects a 10-Mb/s and/or 100-Mb/s capable MAC to the physical sublayers.

Virtex®-6 devices support MII at 2.5 V only; Virtex-5, Virtex-4, Spartan®-6 and Spartan-3 devices support MII at 3.3 V or lower.

MII Transmitter Interface

The logic required to implement the MII transmitter logic is illustrated in [Figure 10-1](#).

`mii_tx_clk` is provided by the external PHY device connected to the MII. As illustrated, this is placed onto global clock routing to provide the clock for all transmitter logic, both within the core and for the client side logic which connects to the transmitter client I/F of the core. Alternatively, for devices containing regional clock resources, the BUFG of [Figure 10-1](#) can be replaced with a BUFR primitive.

Closely linked to the transmitter clock logic is the use of the `tx_enable` clock enable derivation. This must be used as a standard clock enable by all flip-flops in the Tx Client I/F logic. At 10 Mb/s and 100 Mb/s operation this clock enable is high on alternative clock cycles; this is required to match the data rates between the 4-bit data width of the MII and the 8-bit data width of the client I/F.

[Figure 10-1](#) also illustrates how to use the physical transmitter interface of the core to create an external MII. The signal names and logic shown in this figure exactly match those delivered with the example design. [Figure 10-1](#) shows that the output transmitter signals are registered in device IOBs before driving them to the device pads.

MII Receiver Interface

The logic required to implement the MII receiver logic is also illustrated in [Figure 10-1](#).

`mii_rx_clk` is provided by the external PHY device connected to the MII. As illustrated, this is placed onto global clock routing to provide the clock for all receiver logic, both within the core and for the client-side logic which connects to the receiver client I/F of the TEMAC. Alternatively, for devices containing regional clock resources, the BUFG of [Figure 10-1](#) can be replaced with a BUFR primitive.

Closely linked to the receiver clock logic is the use of the `rx_enable` clock enable derivation. This must be used as a standard clock enable by all flip-flops in the Rx Client I/F logic. At 10 Mb/s and 100 Mb/s operation this clock enable is high on alternative clock cycles; this is required to match the data rates between the 4-bit data width of the MII and the 8-bit data width of the client I/F.

[Figure 10-1](#) also illustrates how to use the physical receiver interface of the core to create an external MII. The signal names and logic shown in this figure exactly match those delivered with the example design. [Figure 10-1](#) shows that the input receiver signals are registered in device IOBs before routing them to the core.

Multiple Core Instances with the MII

Because both `mii_tx_clk` and `mii_rx_clk` are both sourced by the external PHY device connected to the MII, it is not possible to share transmitter or receiver clock resources across multiple instantiations of the core. Each instance of the core requires its own independent clocking resources. Therefore the logic of [Figure 10-1](#) must be duplicated for each instance of the core.

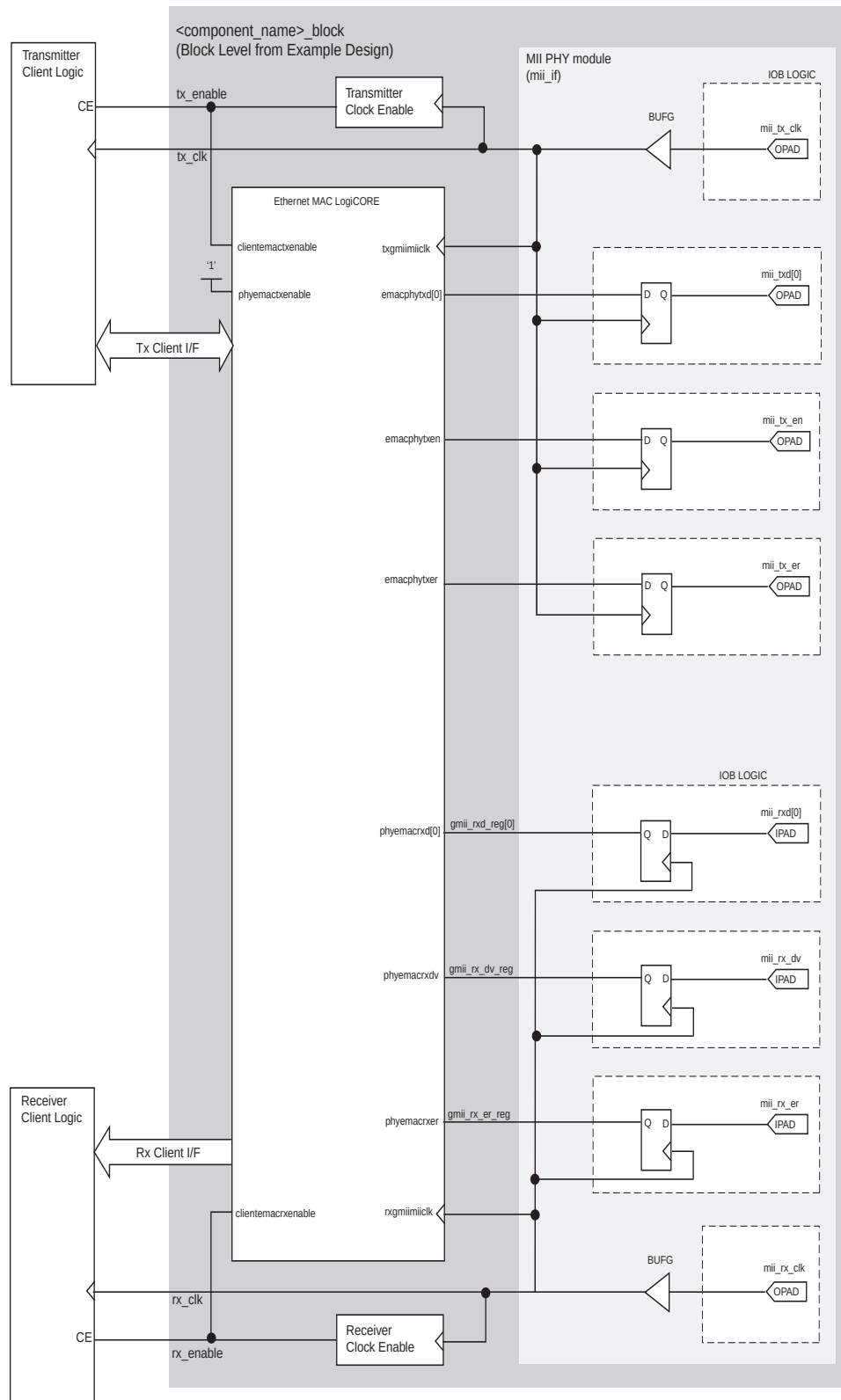


Figure 10-1: MII Transmitter, Receiver and Clock Logic For All Devices

Physical Interfaces for 1 Gb/s Only Ethernet MAC IP Core

The HDL example design supplied with the core, for 1 Gb/s only operation provides either a GMII or RGMII interface. These are typically used to connect the MAC to an external PHY device.

The Gigabit Media Independent Interface (GMII), defined in IEEE Std 802.3-2008, clause 35, is used to connect a 1-Gb/s capable MAC to the physical sublayers.

Virtex®-6 devices support GMII at 2.5 V only; Virtex-5, Virtex-4, Spartan®-6 and Spartan-3 devices support GMII at 3.3 V or lower.

The Reduced Gigabit Media Independent Interface (RGMII) is an alternative to the GMII and achieves a 50% reduction in the pin count compared with GMII. Therefore, this is often favored over GMII by PCB designers. This configuration is achieved with the use of double-data-rate (DDR) flip-flops.

See the appropriate section:

- [Gigabit Media Independent Interface \(GMII\)](#)
- [Clock Resource Sharing in Virtex-4, Spartan-3, Spartan-3E, Spartan-3A and Spartan-3A DSP Devices](#)

Gigabit Media Independent Interface (GMII)

GMII Transmitter Interface

The logic required to implement the GMII transmitter logic is illustrated in [Figure 11-1](#). `gtx_clk` is a user-supplied 125 MHz reference clock source. As illustrated, this is placed onto global clock routing to provide the clock for all transmitter logic, both within the core and for the client side logic which connects to the transmitter client I/F of the core.

[Figure 11-1](#) illustrates how to use the physical transmitter interface of the core to create an external GMII. The signal names and logic shown in this figure exactly match those delivered with the example design for a Virtex-5 device when the GMII is selected. If other families are chosen, equivalent primitives specific to that family are used in the example design.

[Figure 11-1](#) shows that the output transmitter signals are registered in device IOBs before driving them to the device pads. The logic required to forward the transmitter clock is also shown. This logic uses an IOB output Double-Data-Rate (DDR) register so that the clock signal produced incurs exactly the same delay as the data and control signals.

This clock signal, `gmii_tx_clk`, is inverted with respect to `gtx_clk` so that the rising edge of `gmii_tx_clk` occurs in the centre of the data valid window, therefore maximizing setup and hold times across the interface.

The half-duplex signals `gmii_col` and `gmii_crs` are asynchronous to the transmit clock. These are routed through PADs and IOBs and then input to the core.

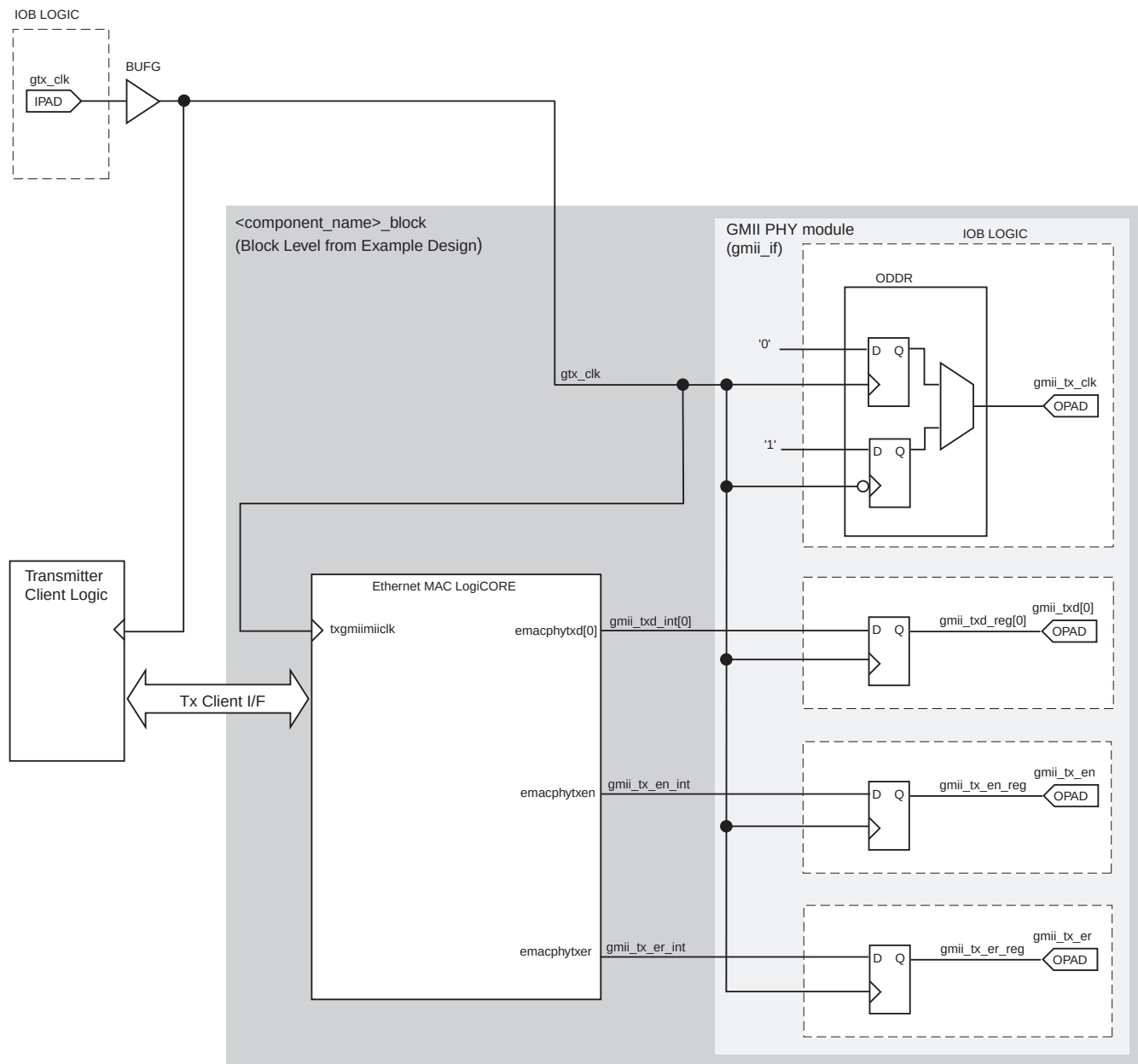


Figure 11-1: GMII Transmitter Logic and Clock Logic

GMII Receive Interface

The logic required to implement the GMII receiver logic is described in the following sections. Logical implementation is different for different device families. See the specific family section:

- [Virtex-6 Devices](#)
- [Spartan-6 Devices](#)
- [Virtex-5 Devices](#)
- [Virtex-4, Spartan-3, Spartan-3E, and Spartan-3A Devices](#)

Virtex-6 Devices

In this implementation, a BUFIO is used to provide the lowest form of clock routing delay from input clock to input GMII Rx signal sampling at the device IOBs. Note, however, that this creates placement constraints: a BUFIO capable clock input pin must be selected, and all other input GMII Rx signals must be placed in the respective BUFIO region. The *Virtex-6 FPGA User Guide* should be consulted.

The input clock is also placed onto regional clock routing using the BUFR component as illustrated. This regional clock then provides the clock for all receiver logic, both within the core and for the client side logic which connects to the receiver client I/F of the core.

The IODELAY elements can be adjusted to fine-tune the setup and hold times at the GMII IOB input flip-flops. The delay is applied to the IODELAY element using constraints in the UCF; these can be edited if desired. See [Chapter 13, Constraining the Core](#) for details.

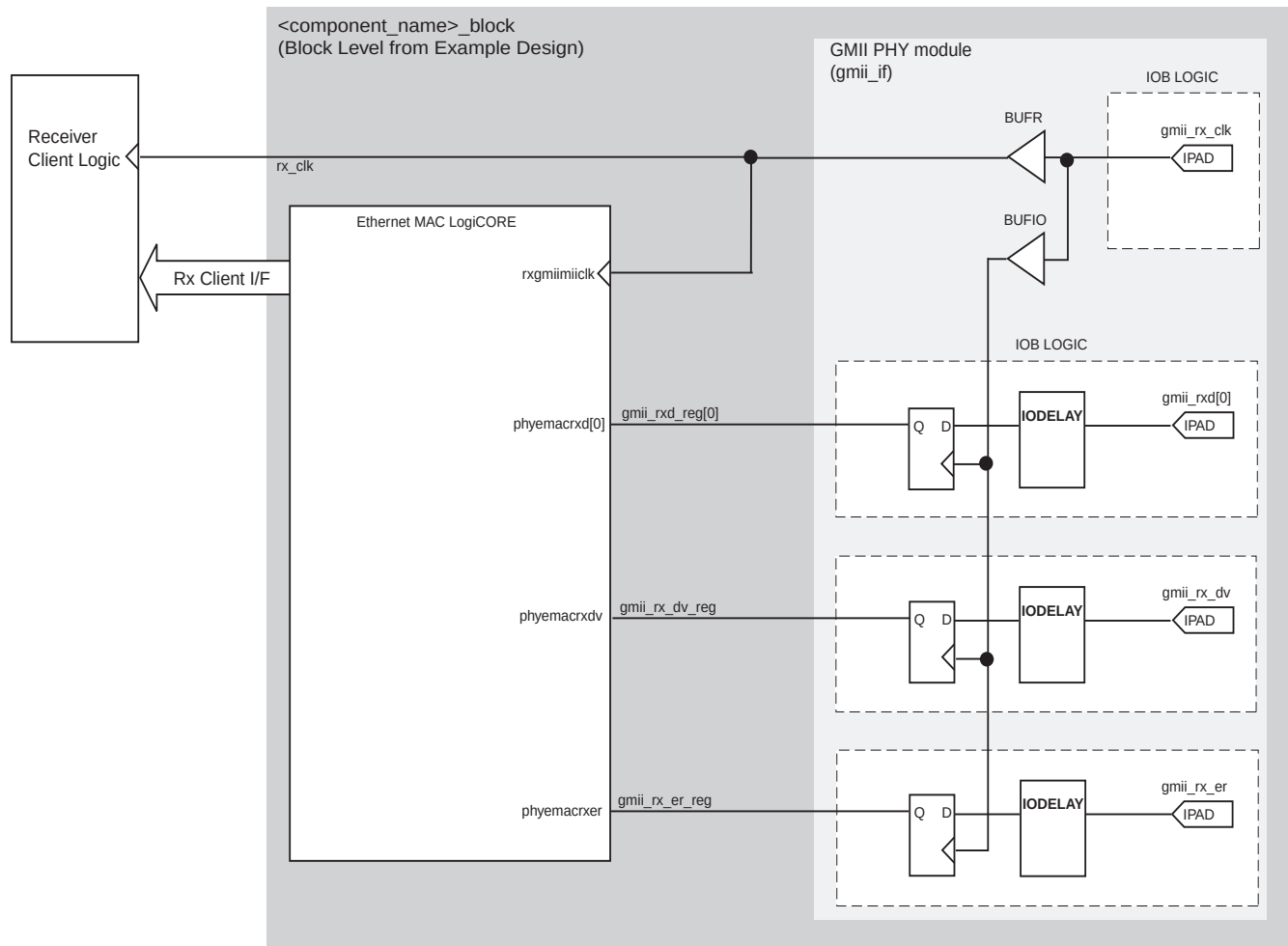


Figure 11-2: GMII Receiver Logic and Clock Logic for Virtex-6 Devices

Spartan-6 Devices

In this implementation, a BUFIO2 is used to provide the lowest form of clock routing delay from input clock to input GMII Rx signal sampling at the device IOBs. Note, however, that this creates placement constraints: a BUFIO2 capable clock input pin must be selected, and all other input GMII Rx signals must be placed in the respective BUFIO2 region. The *Spartan-6 FPGA User Guide* should be consulted.

The input clock is also placed onto global clock routing using the BUFG component as illustrated. This clock then provides the clock for all receiver logic, both within the core and for the client side logic which connects to the receiver client I/F of the core.

The IODELAY2 elements can be adjusted to fine-tune the setup and hold times at the GMII IOB input flip-flops. The delay is applied to the IODELAY2 element using constraints in the UCF; these can be edited if desired. See [Chapter 13, Constraining the Core](#) for details.

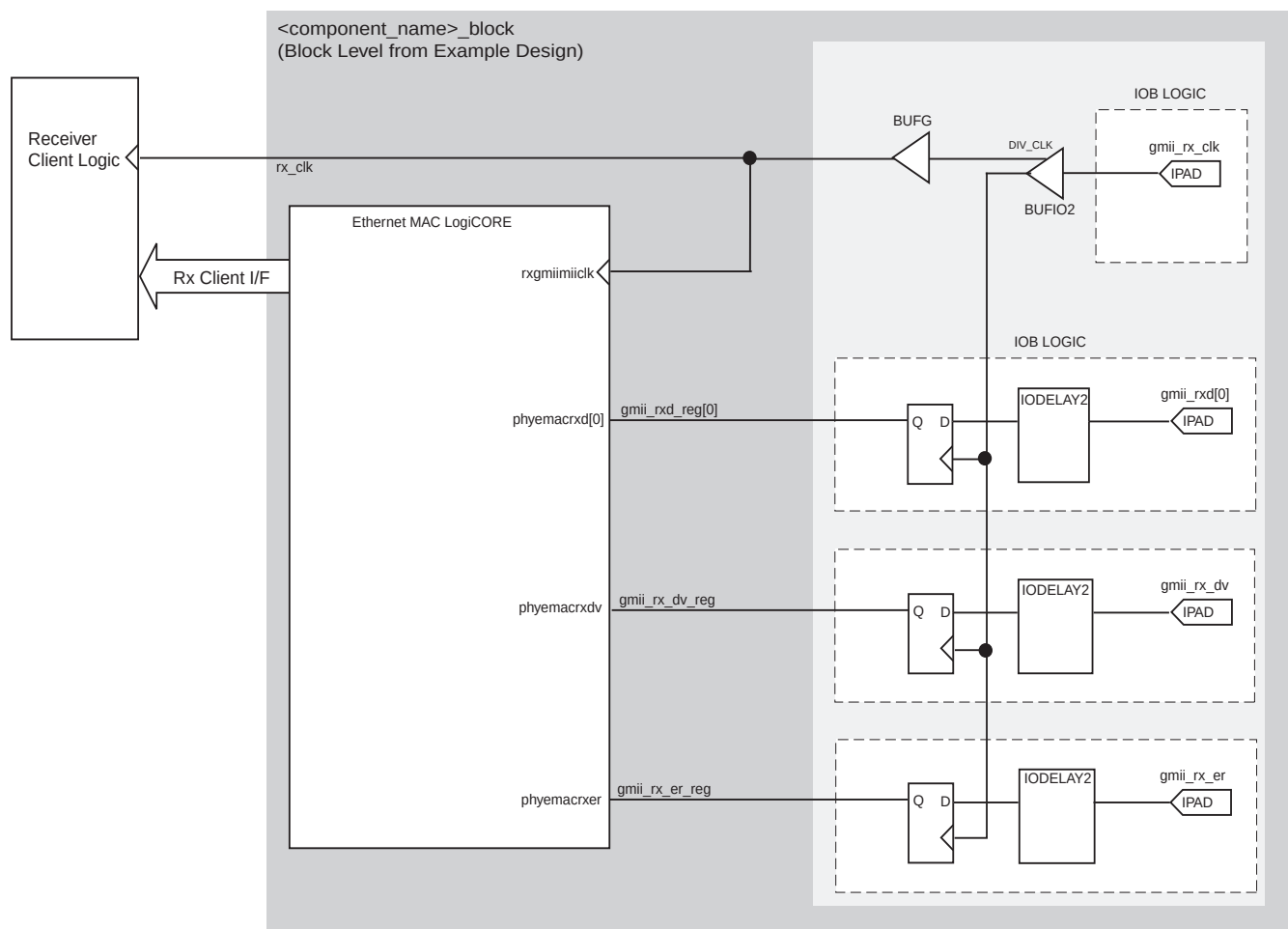


Figure 11-3: GMII Receiver Logic and Clock Logic for Spartan-6 Devices

Virtex-5 Devices

Three possible solutions are available for Virtex-5 devices:

1. A DCM can be used on the `gmi_i_rx_clk` clock path, exactly as described for [Virtex-4, Spartan-3, Spartan-3E, and Spartan-3A Devices](#)
2. Regional and BUFIO clock routing can be used as described in the section for [Virtex-6 Devices](#).
3. Input Delay Elements can be used on the GMII data and clock path, using global clock routing and as illustrated in [Figure 11-4](#).

The design for Case 3 provides a simpler solution than Case 1 and fully supports all operating modes; it also allows for a more flexible pinout than for Case 2. It has therefore been chosen as the default example design and is described in this section.

As illustrated in [Figure 11-4](#), IODELAY components can be used on the data and control signals. These can be used to either shift the input clock, `gmi_i_rx_clk`, or the data and control signals to meet the setup and hold requirements and to allow for any bus skew across the data and control inputs. The IODELAY components are used in fixed delay mode, where the attribute `IDELAY_VALUE` determines the tap delay value. An `IDELAYCTRL` primitive must be instantiated for this mode of operation. See the *Virtex-5 FPGA User Guide* for details about using the `IDELAYCTRL` and `IDELAY` components. For details on setting the `IDELAY_VALUE`s, see [Chapter 13, Constraining the Core](#).

The `gmii_rx_clk` clock received through the IODELAY component, placed onto global clock routing, is then used to provide the clock for all receiver logic, both within the core and for the client side logic which connects to the receiver client I/F of the core.

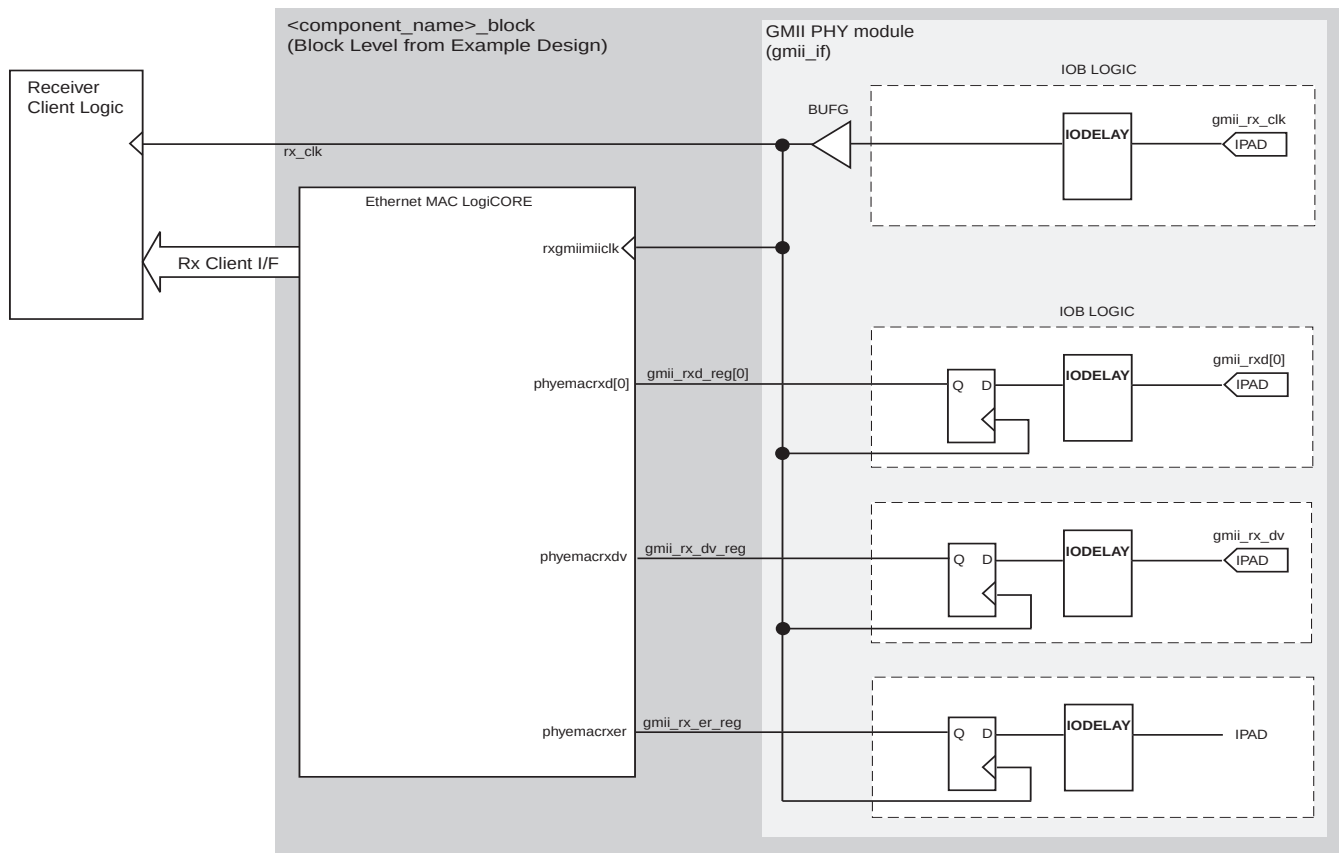


Figure 11-4: GMII Receiver Logic and Clock Logic for Virtex-5 Devices

Virtex-4, Spartan-3, Spartan-3E, and Spartan-3A Devices

In Virtex-4, Spartan-3, Spartan-3E, Spartan-3A and Spartan-3A DSP devices, a DCM must be used on the `gmii_rx_clk` clock path as illustrated in Figure 11-5 to meet the GMII input setup and hold requirements. This logic is implemented by the example design delivered with the core (all signal names and logic match).

Phase shifting can then be applied to the DCM to fine-tune the setup and hold times of the input GMII receiver signals which are sampled at the GMII IOB input flip-flops; fixed-phase shift is applied to the DCM via the example UCF for the example design. See Chapter 13, *Constraining the Core*.

A limitation of using a DCM on this interface is that, as a DCM is sensitive to a change in the input clock, 1 Gb/s Half Duplex is not supported. This is due to the nature of the rx clock in this mode of operation as it is sourced by whichever device has control of the media and a PPM shift is to be expected. This causes the DCM to lose lock, rendering the receive interface inactive.

The clock produced by the DCM, placed onto global clock routing, is used to provide the clock for all receiver logic, both within the core and for the client-side logic which connects to the receiver client I/F of the core.

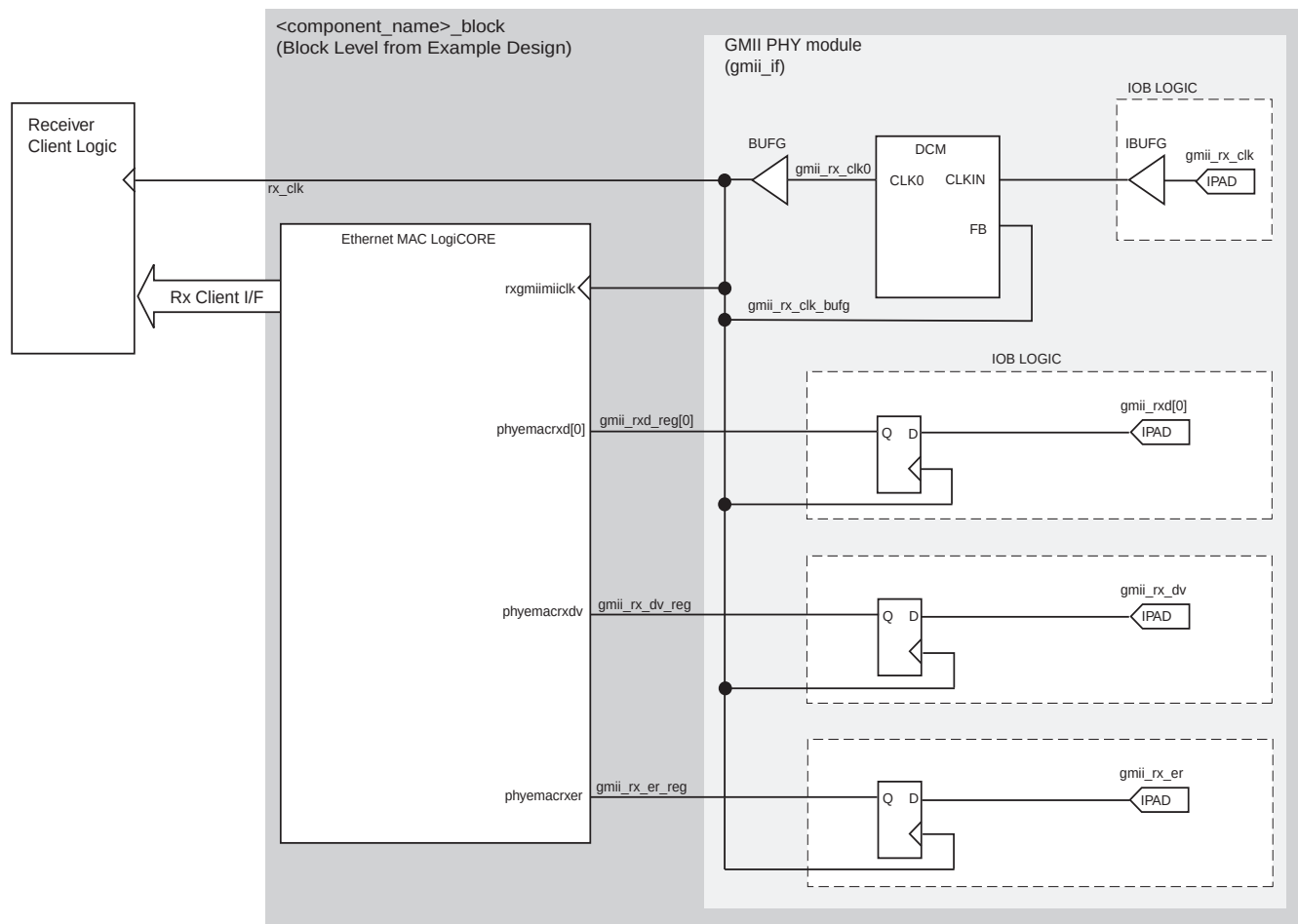


Figure 11-5: GMII Receiver Logic and Clock Logic for Virtex-4, Spartan-3, Spartan-3E, Spartan-3A and Spartan-3A DSP Devices

DCM Reset Circuitry

A DCM reset module, not illustrated in Figure 11-5, is also present and is instantiated in the example design next to the DCM. Because this logic must be reliable whatever the reset/locked status of the DCM, the module requires a reliable reference clock. In the example design for GMII, the global transmitter clock is therefore used (`gtx_clk` from Figure 11-1). This reset circuitry generates an appropriate reset pulse, based on the family, for the receiver DCM under the following conditions:

- The locked signal from the DCM is constantly monitored. Following a high-to-low transition on this signal, indicating that the DCM has lost lock, a reset is issued.
- A timeout counter is enabled when the DCM is in the loss of lock state. If, following the timeout period, the DCM has not obtained lock, another DCM reset is issued. This timeout counter will time a > 1ms interval. This timeout functionality is required for DCMs connected to Ethernet PHYs because the PHYs may source discontinuous clocks under certain network conditions (for example, when no ethernet cable is connected).

Spartan-3 Devices

For Spartan-3 devices, the reset pulse is transferred into the DCM input clock (`gmii_rx_clk` from [Figure 11-5](#)). Here it is extended to three DCM clock periods duration and routed to the reset input of the DCM.

Virtex-4 Devices

For Virtex-4 devices, the generated reset from within the DCM reset module is further complicated. Here the DCM reset pulse duration must be asserted for a minimum of 200 ms (see the *Virtex-4 FPGA Data Sheet*). Consequently, a 200 ms duration timer is also included in the DCM reset module to extend the reset pulse. This reset signal is output from the DCM reset module and **should** be routed to the DCM reset input port.

Caution! In the example designs for Virtex-4 devices the 200 ms reset pulse from the DCM reset module is NOT connected directly to the DCM; this signal must be connected when implementing the core in real hardware.

For Virtex-4 FPGA example designs, the reason why the 200 ms input is not routed directly to the DCM is so that simulations can occur in a timely manner.

The 200 ms reset pulse takes the signal name `reset_200ms` in the example design HDL files. It is routed from the DCM reset module instantiation up through all levels of hierarchy to the top level. It is then left unconnected in the example design instantiation from within the demonstration test bench. Furthermore, an accompanying signal, `reset_200ms_in`, is routed down from the top level of the example design hierarchy to the DCM instantiation, where it is connected to the DCM reset. From the demonstration test bench, this `reset_200ms_in` signal is driven for a short duration to enable fast simulation start-up. However, to re-iterate, when implementing the design in real hardware, the DCM reset signal must be connected correctly:

- This can be achieved by connecting the `reset_200ms` signal to the `reset_200ms_in` signal at any level of example design HDL hierarchy.

Clock Sharing across Multiple Cores with GMII for 1 Gb/s Operation

When multiple instances of the core are instantiated in a design, transmitter clock resources can be shared across all core instances; receiver clock resources cannot be shared and are independent for each core instance. See the appropriate section:

- [Clock Resource Sharing in Virtex-6 Devices](#)
- [Clock Resource Sharing in Spartan-6 Devices](#)
- [Clock Resource Sharing in Virtex-5 Devices](#)
- [Clock Resource Sharing in Virtex-4, Spartan-3, Spartan-3E, Spartan-3A and Spartan-3A DSP Devices](#)

Clock Resource Sharing in Virtex-6 Devices

[Figure 11-6](#) illustrates clock resource sharing across multiple instantiations of the core when using GMII at 1 Gb/s in Virtex-6 devices. For all instantiations, `gtx_clk` can be shared between multiple cores, resulting in a common clock domain across the device.

The receiver clocks cannot be shared. Each core is provided with its own local version of `gmii_rx_clk` from the connected external PHY device as illustrated.

[Figure 11-6](#) illustrates only two cores. However, more can be added using the same principal. This is done by instantiating the cores using the block level (from the example

design) and sharing `gtx_clk` across all instantiations. The receiver clock, which cannot be shared, is unique for every instance of the core.

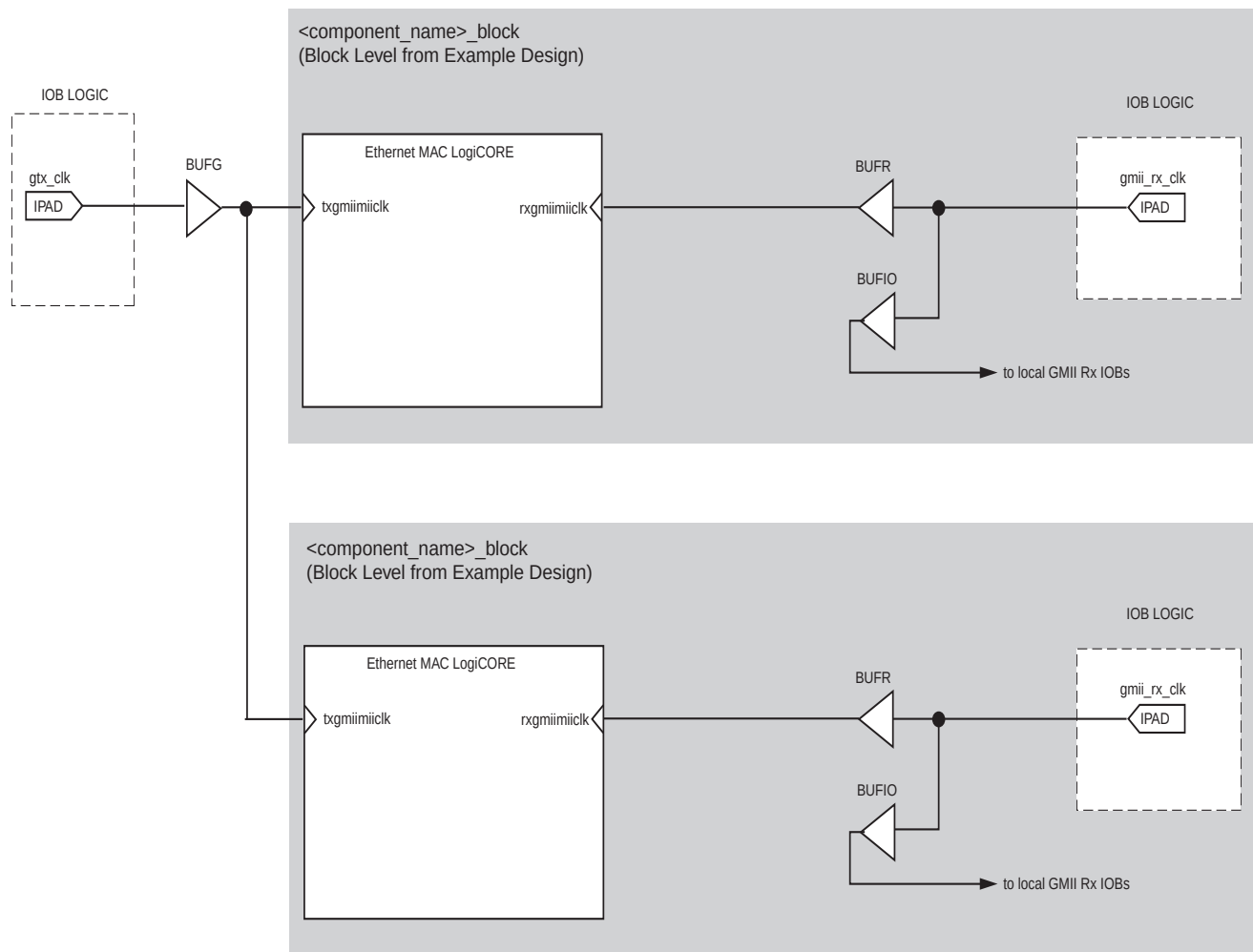


Figure 11-6: Clock Resource Sharing for 1 Gb/s GMII in Virtex-6 Devices

Clock Resource Sharing in Spartan-6 Devices

Figure 11-7 illustrates clock resource sharing across multiple instantiations of the core when using GMII at 1 Gb/s in Spartan-6 devices. For all instantiations, `gtx_clk` can be shared between multiple cores, resulting in a common clock domain across the device.

The receiver clocks cannot be shared. Each core is provided with its own local version of `gmii_rx_clk` from the connected external PHY device as illustrated.

Figure 11-7 illustrates only two cores. However, more can be added using the same principal. This is done by instantiating the cores using the block level (from the example design) and sharing `gtx_clk` across all instantiations. The receiver clock, which cannot be shared, is unique for every instance of the core.

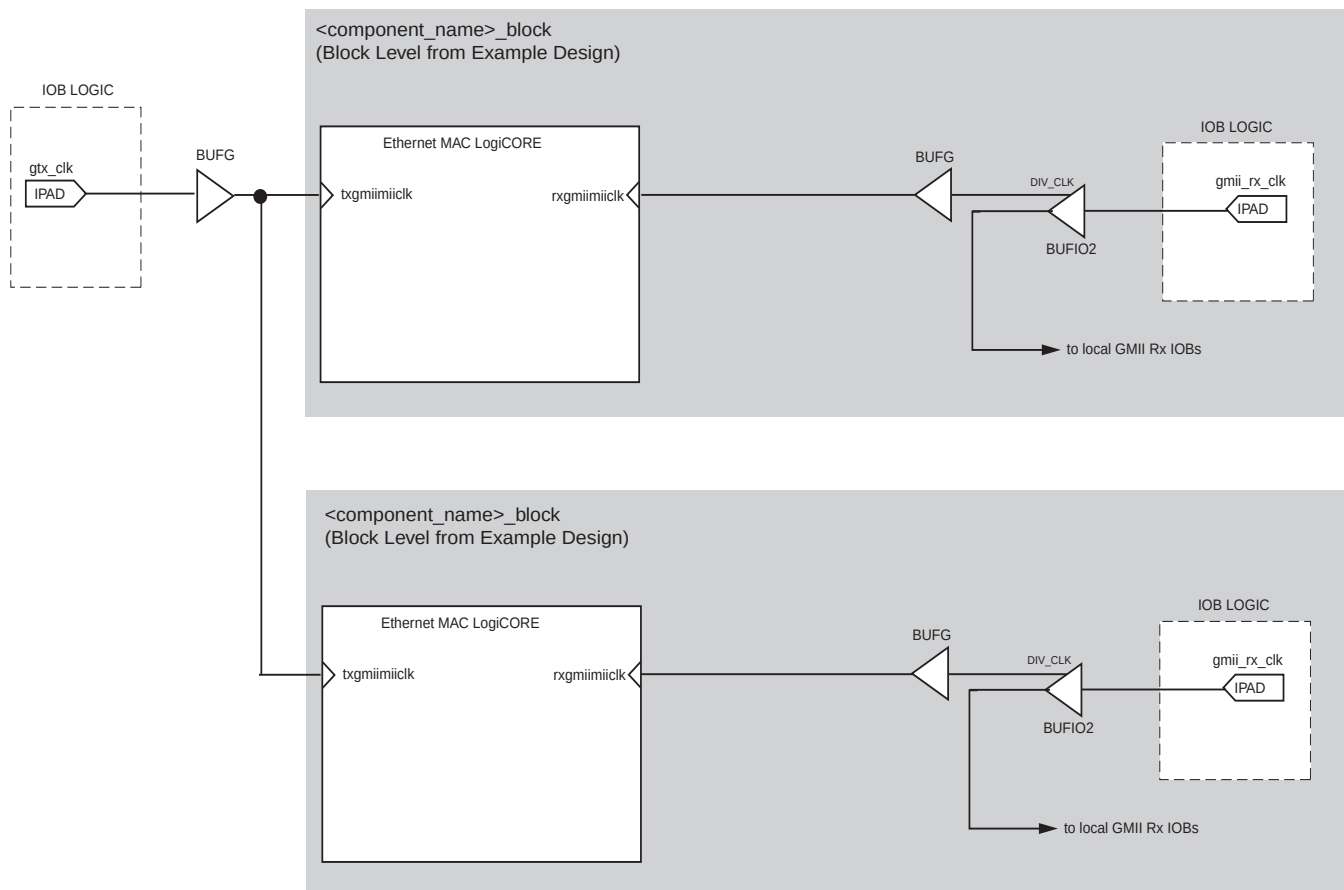


Figure 11-7: Clock Resource Sharing for 1-Gb/s GMII in Spartan-6 Devices

Clock Resource Sharing in Virtex-5 Devices

Figure 11-8 illustrates clock resource sharing across multiple instantiations of the core when using GMII at 1 Gb/s in Virtex-5 devices. For all instantiations, `gtx_clk` can be shared between multiple cores, resulting in a common clock domain across the device.

The receiver clocks cannot be shared. Each core is provided with its own local version of `gmii_rx_clk` from the connected external PHY device as illustrated.

Figure 11-8 illustrates only two cores. However, more can be added using the same principal. This is done by instantiating the cores using the block level (from the example design) and sharing `gtx_clk` across all instantiations. The receiver clock, which cannot be shared, is unique for every instance of the core.

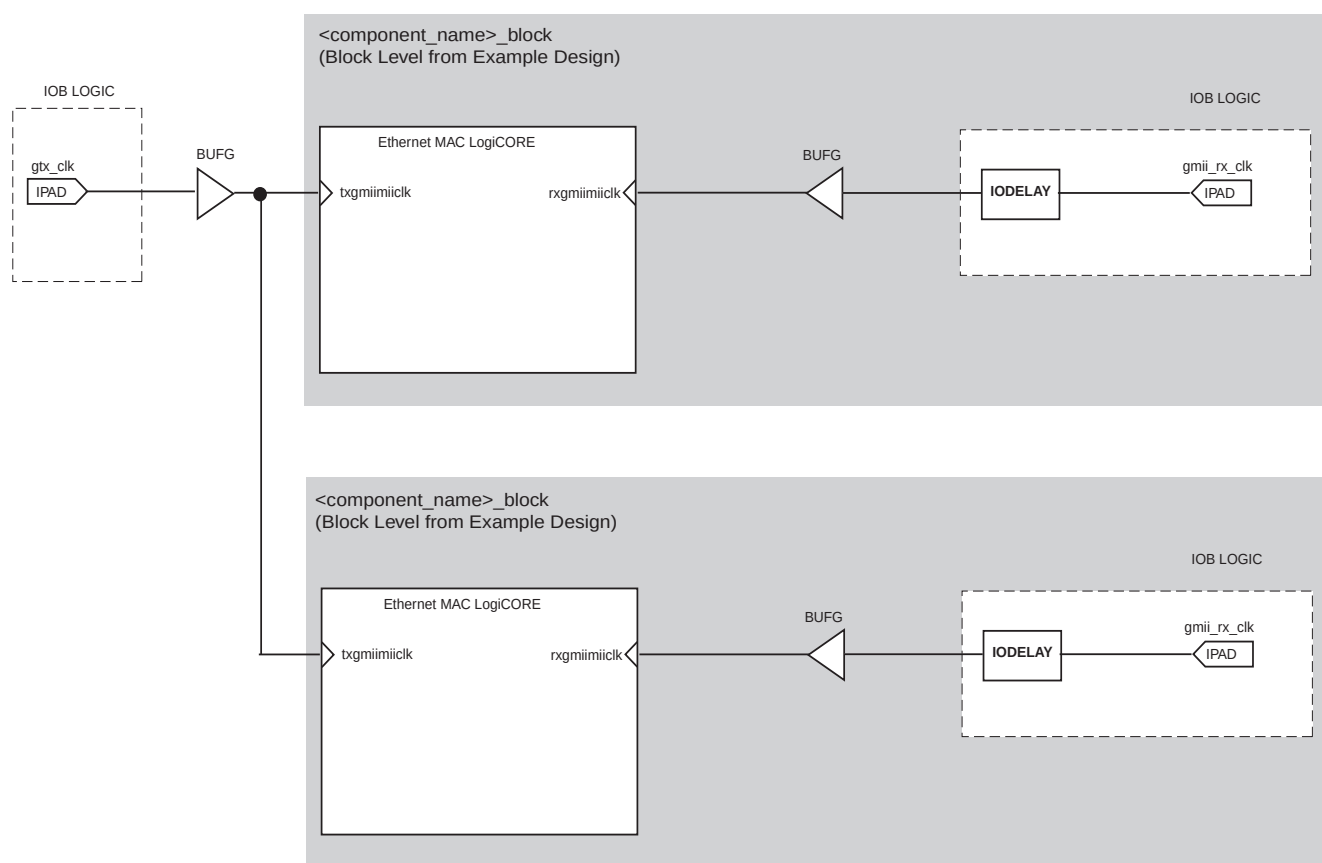


Figure 11-8: Clock Resource Sharing for 1-Gb/s GMII in Virtex-5 Devices

Clock Resource Sharing in Virtex-4, Spartan-3, Spartan-3E, Spartan-3A and Spartan-3A DSP Devices

Figure 11-9 illustrates clock resource sharing across multiple instantiations of the core when using GMII at 1 Gb/s in Virtex-4, Spartan-3, Spartan-3E, Spartan-3A and Spartan-3A DSP devices. For all instantiations, `gtx_clk` can be shared between multiple cores, resulting in a common clock domain across the device.

The receiver clocks cannot be shared. Each core is provided with its own local version of `gmii_rx_clk` from the connected external PHY device as illustrated.

Figure 11-9 illustrates only two cores. However, more can be added using the same principal. This is done by instantiating the cores using the block level (from the example design) and sharing `gtx_clk` across all instantiations. The receiver clock, which cannot be shared, is unique for every instance of the core.

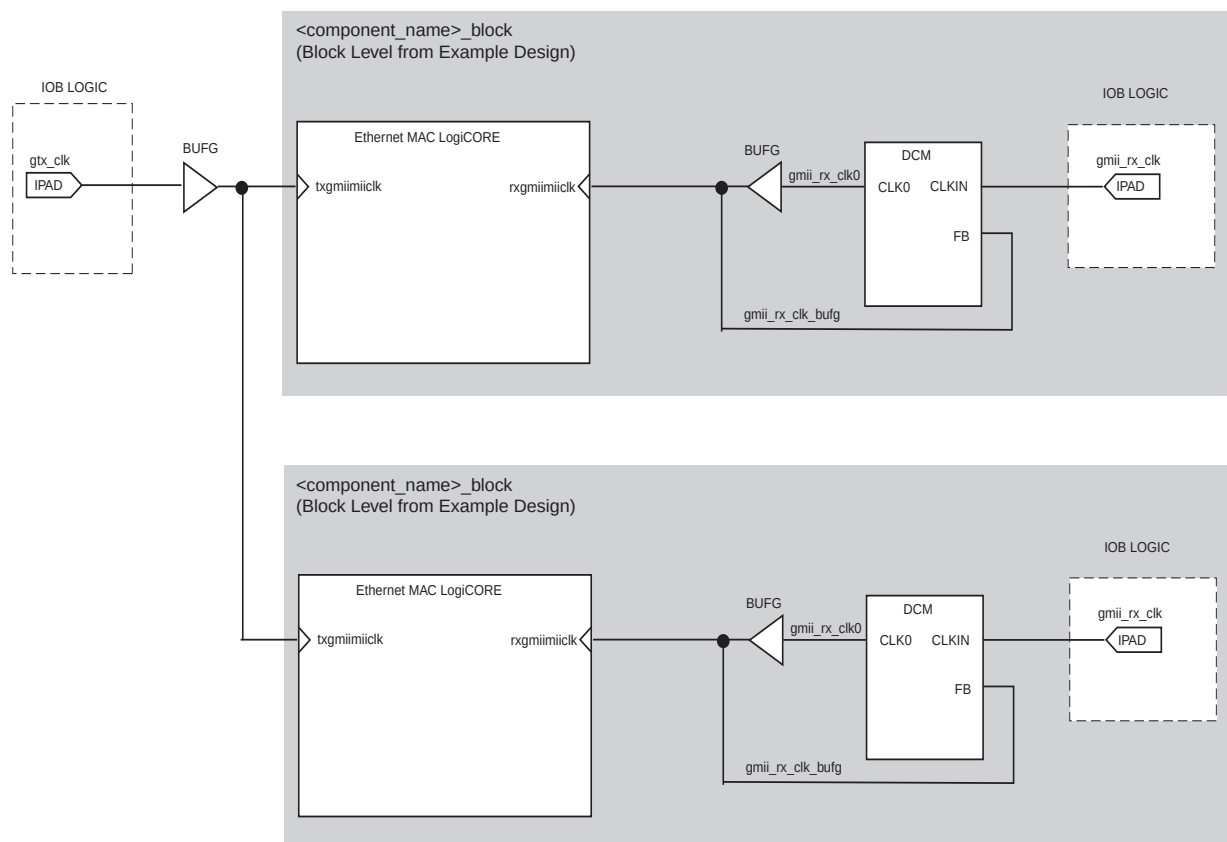


Figure 11-9: Clock Resource Sharing for 1-Gb/s GMII in Virtex-4, Spartan-3, Spartan-3E, Spartan-3A and Spartan-3A DSP Devices

Reduced Gigabit Media Independent Interface (RGMII)

The logic required to implement the RGMII logic is described in the following sections. Logical implementation is different for different device families. See the specific family section:

- [Virtex-6 Devices](#)
- [Spartan-6 Devices](#)
- [Virtex-5 Devices](#)
- [Virtex-4 Devices](#)
- [Spartan-3, Spartan-3A, and Spartan-3A DSP Devices](#)

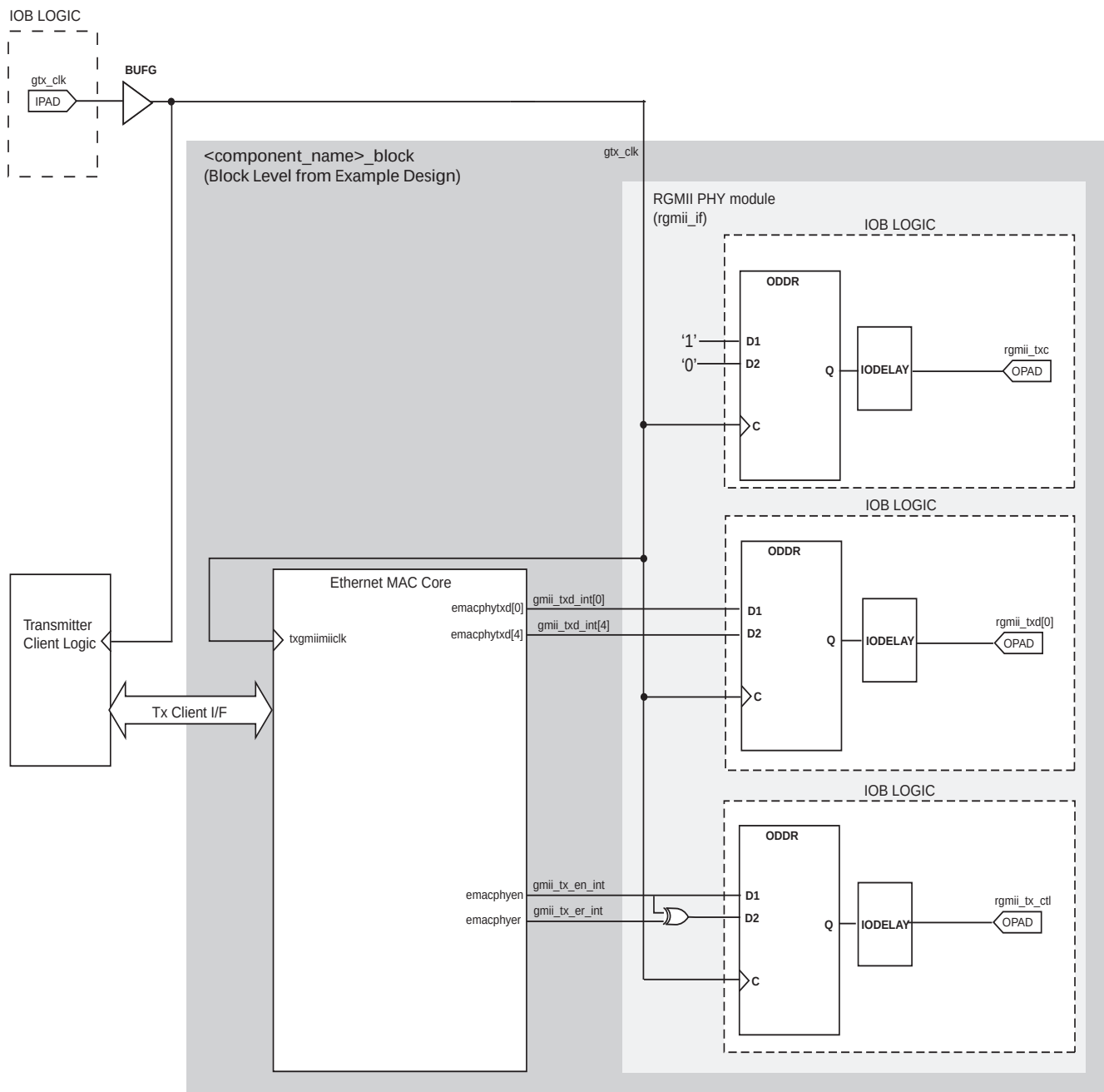
Virtex-6 Devices

Transmitter Logic

The logic required to implement the RGMII transmitter logic is illustrated in [Figure 11-10](#). `gtx_clk` is a user-supplied 125 MHz reference clock source which is placed onto global clock routing to provide the clock for all transmitter logic, both within the core and for the client side logic which connects to the transmitter client I/F of the core.

[Figure 11-10](#) illustrates how to use the physical transmitter interface of the core to create an external RGMII. The signal names and logic shown in this figure exactly match those delivered with the example design. [Figure 11-10](#) shows that the output transmitter signals are registered in device IOBs, using DDR registers, before driving them to the device pads.

The logic required to forward the transmitter clock is also shown. This logic uses an IOB output Double-Data-Rate (DDR) register so that the clock signal produced incurs exactly the same delay as the data and control signals. However, the clock signal is then routed through an output delay element (IODELAY) before connecting to the device pad. The result of this is to create a 2 ns delay, which places the `rgmii_txc` forwarded clock in the centre of the data valid window for forwarded RGMII data and control signals.



The input clock is also placed onto regional clock routing using the BUFR component as illustrated. This regional clock then provides the clock for all receiver logic, both within the core and for the client side logic which connects to the receiver client I/F of the core.

The IODELAY elements can be adjusted to fine-tune the setup and hold times at the RGMII IOB input flip-flops. The delay is applied to the IODELAY element using constraints in the UCF; these can be edited if desired. See [Chapter 13, Constraining the Core](#) for details.

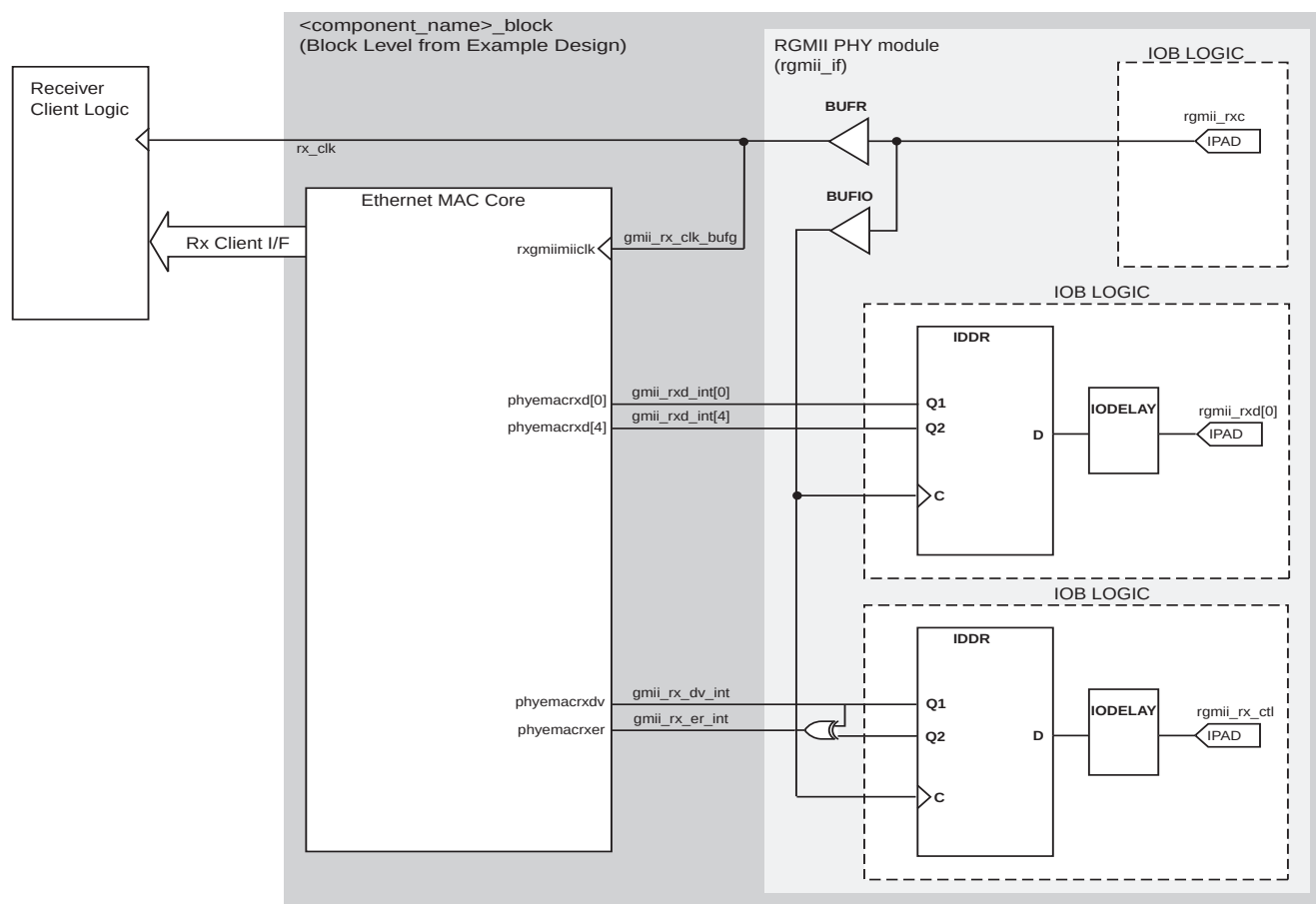


Figure 11-11: RGMII Receiver Logic and Clock Logic for Virtex-6 Devices

Clock Resource Sharing

Figure 11-12 illustrates clock resource sharing across multiple instantiations of the core when using RGMII at 1 Gb/s in Virtex-6 devices. For all instantiations, `gtx_clk` can be shared between multiple cores, resulting in a common clock domain across the device.

The receiver clocks cannot be shared. Each core is provided with its own local version of `rgmii_rxc` from the connected external PHY device as illustrated.

Figure 11-12 illustrates only two cores. However, more can be added using the same principal. This is done by instantiating the cores using the block level (from the example design) and sharing `gtx_clk` across all instantiations. The receiver clock, which cannot be shared, is unique for every instance of the core.

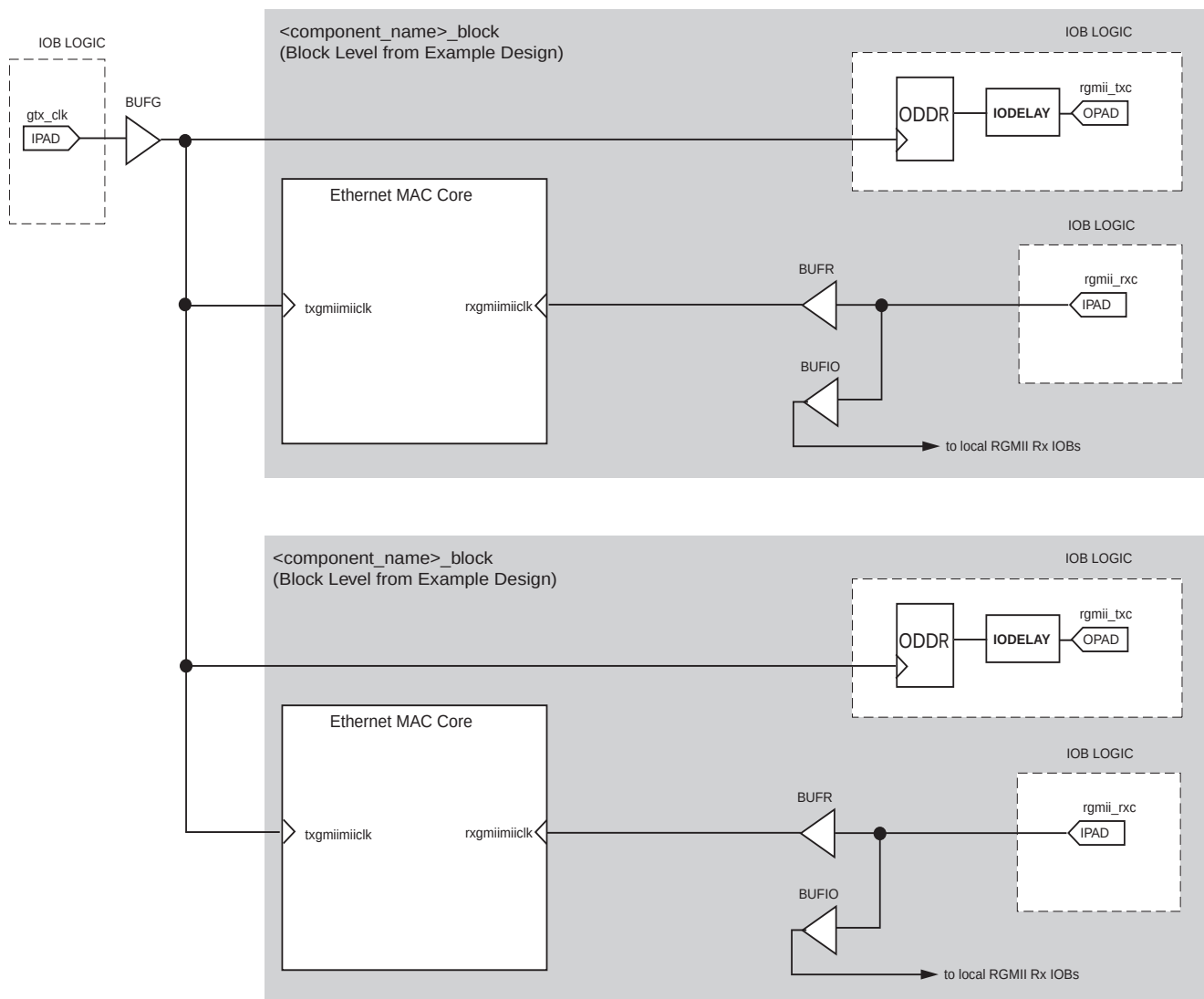


Figure 11-12: Clock Resource Sharing for 1-Gb/s RGMII in Virtex-6 Devices

Spartan-6 Devices

Transmitter Logic

The logic required to implement the RGMII transmitter logic is illustrated in [Figure 11-13](#). `gtx_clk` is a user-supplied 125 MHz reference clock source which is placed onto global clock routing to provide the clock for all transmitter logic, both within the core and for the client side logic which connects to the transmitter client I/F of the core.

[Figure 11-13](#) illustrates how to use the physical transmitter interface of the core to create an external RGMII. The signal names and logic shown in this figure exactly match those delivered with the example design. [Figure 11-13](#) shows that the output transmitter signals are registered in device IOBs, using DDR registers, before driving them to the device pads.

The logic required to forward the transmitter clock is also shown. This logic uses an IOB output Double-Data-Rate (DDR) register so that the clock signal produced incurs exactly the same delay as the data and control signals. However, the clock signal is then routed through an output delay element (IODELAY2) before connecting to the device pad. The result of this is to create a 2 ns delay, which places the `rgmii_txc` forwarded clock in the centre of the data valid window for forwarded RGMII data and control signals.

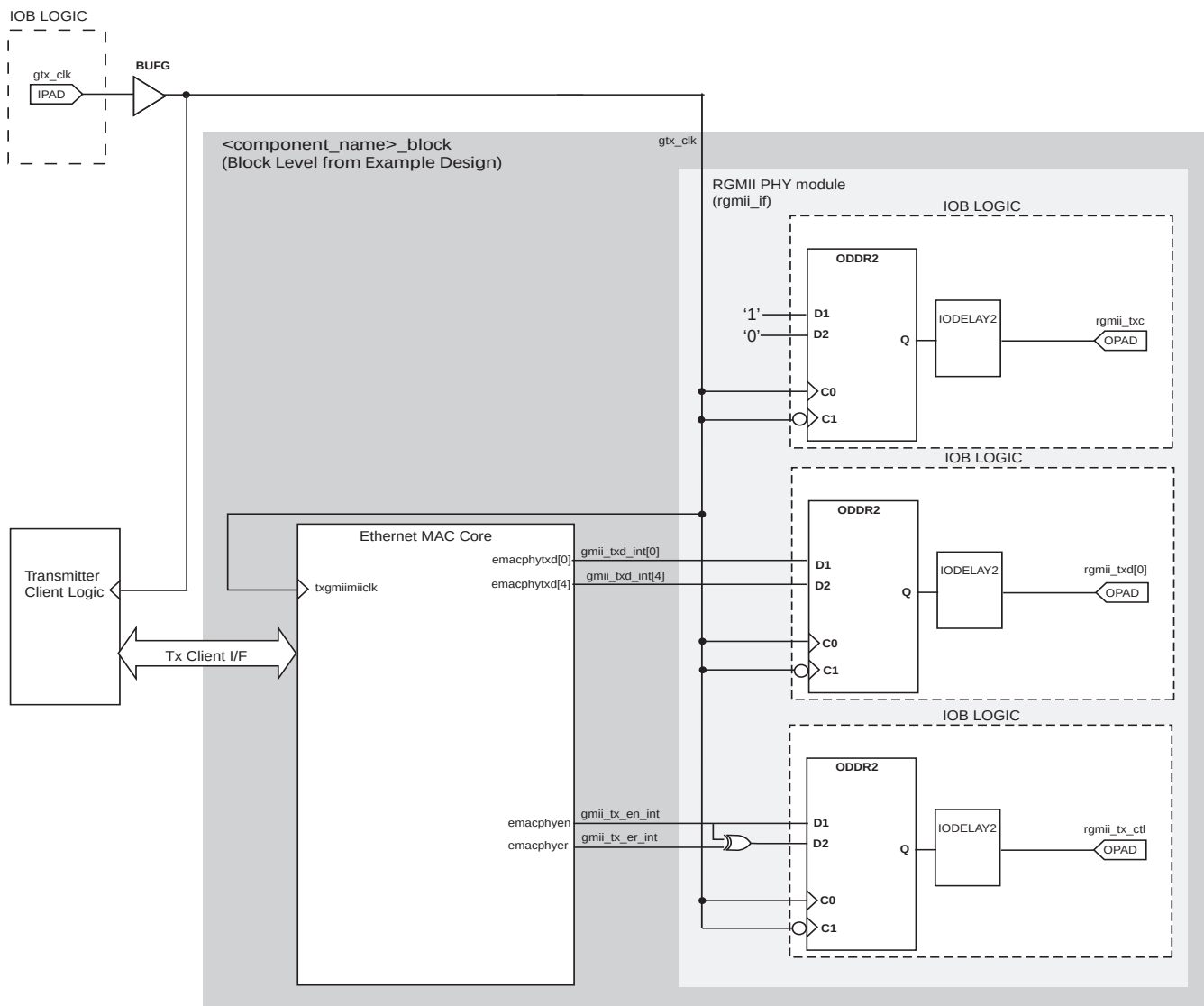


Figure 11-13: RGMII Transmitter Logic and Clock Logic for Spartan-6 Devices

Receiver Logic

In Spartan-6 devices, a BUFG is used on the `rgmii_rxc` clock path with IODELAY2s on the data paths as illustrated in Figure 11-14 to meet the RGMII input setup and hold requirements. This logic is implemented by the example design delivered with the core (all signal names and logic match).

The tap delays of the individual IODELAY2s can then be adjusted to fine-tune the setup and hold times of the input RGMII receiver signals which are sampled at the RGMII IOB flip-flops; a fixed tap delay is applied to the IODELAY2s via the example UCF for the example design. See Chapter 13, *Constraining the Core*.

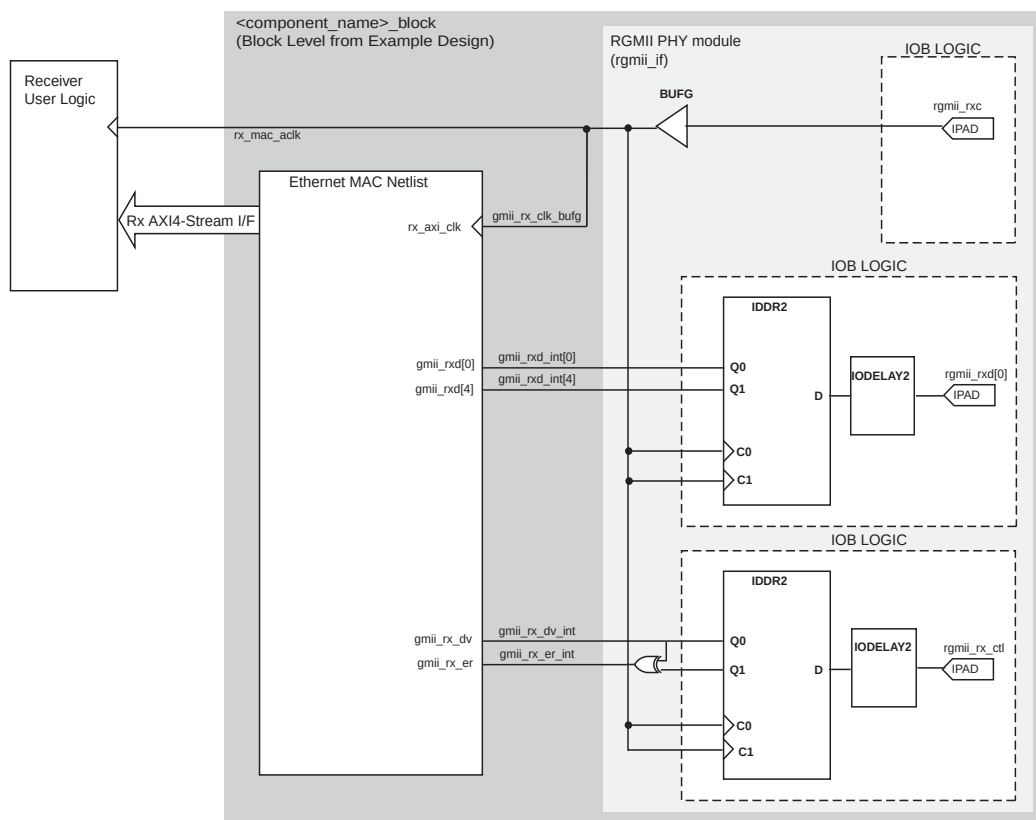


Figure 11-14: RGMII Receiver Logic and Clock Logic for Spartan-6 Devices

Clock Resource Sharing

Figure 11-15 illustrates clock resource sharing across multiple instantiations of the core when using RGMII at 1 Gb/s in Spartan-6 devices. For all instantiations, `gtx_clk` can be shared between multiple cores, resulting in a common clock domain across the device.

The receiver clocks cannot be shared. Each core is provided with its own local version of `rgmii_rxc` from the connected external PHY device as illustrated.

Figure 11-15 illustrates only two cores. However, more can be added using the same principal. This is done by instantiating the cores using the block level (from the example design) and sharing `gtx_clk` across all instantiations. The receiver clock, which cannot be shared, is unique for every instance of the core.

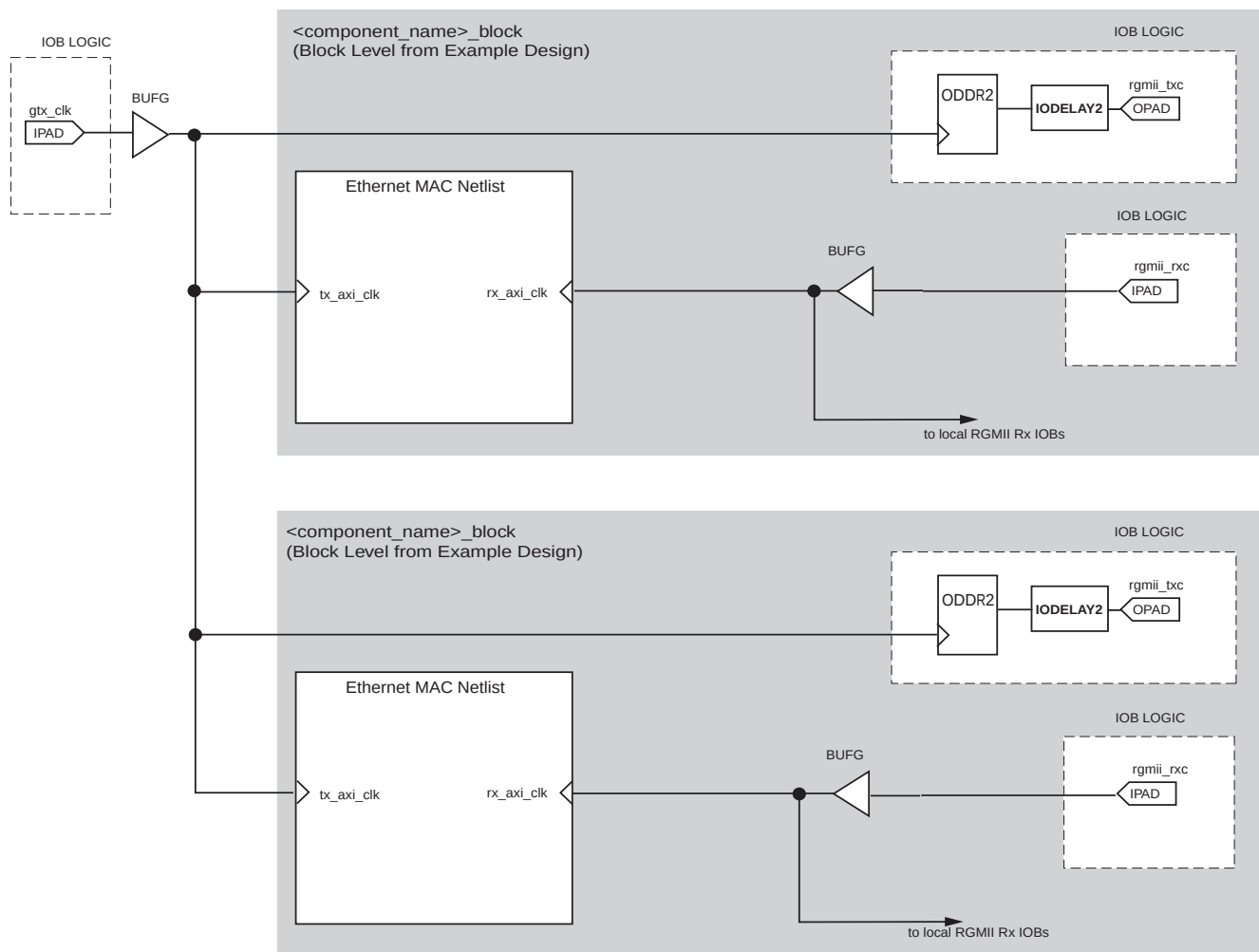


Figure 11-15: Clock Resource Sharing for 1Gb/s RGMII in Spartan-6 Devices

Virtex-5 Devices

Transmitter Logic

The logic required to implement the RGMII transmitter logic is illustrated in [Figure 11-16](#). `gtx_clk` is a user-supplied 125 MHz reference clock source which is placed onto global clock routing to provide the clock for all transmitter logic, both within the core and for the client-side logic which connects to the transmitter client I/F of the core.

[Figure 11-16](#) illustrates how to use the physical transmitter interface of the core to create an external RGMII. The signal names and logic shown in this figure exactly match those delivered with the example design. [Figure 11-16](#) shows that the output transmitter signals are registered in device IOBs, using DDR registers, before driving them to the device pads.

The logic required to forward the transmitter clock is also shown. This logic uses an IOB output Double-Data-Rate (DDR) register so that the clock signal produced incurs exactly the same delay as the data and control signals. However, the clock signal is then routed through an output delay element (IODELAY) before connecting to the device pad. The result of this is to create a 2 ns delay, which places the `rgmii_txc` forwarded clock in the centre of the data valid window for forwarded RGMII data and control signals.

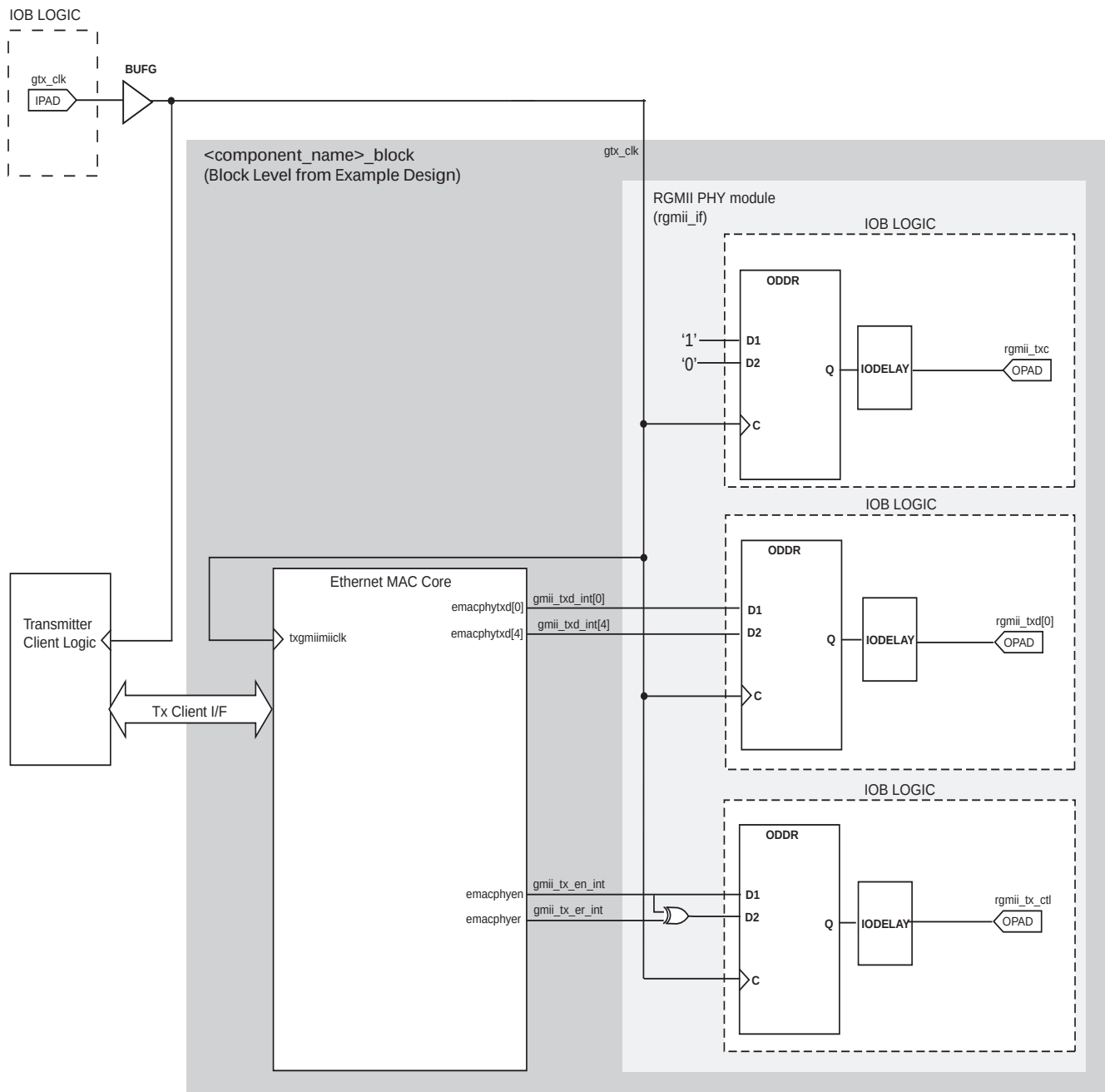


Figure 11-16: RGMII Transmitter Logic and Clock Logic for Virtex-5 Devices

Receiver Logic

Three possible solutions are available for Virtex-5 devices:

1. A DCM can be used on the `rgmii_rxc` clock path, exactly as described for [Spartan-3, Spartan-3A, and Spartan-3A DSP Devices](#).
2. Regional and BUFIO clock routing can be used as described in the section for [Virtex-6 Devices](#).
3. Input Delay Elements can be used on the RGMII data and clock path, using global clock routing and as illustrated in [Figure 11-17](#).

The design for Case 3 provides a simpler solution than Case 1 and fully supports all operating modes; it also allows for a more flexible pinout than for Case 2. It has therefore been chosen as the default example design and is described in this section.

As illustrated in [Figure 11-17](#), IODELAY components can be used on the data and control signals. These can be used to either shift the input clock, `rgmii_rxc`, or the data and control signals to meet the setup and hold requirements and to allow for any bus skew across the data and control inputs. The IODELAY components are used in fixed delay mode, where the attribute `IDELAY_VALUE` determines the tap delay value. An `IDELAYCTRL` primitive must be instantiated for this mode of operation. See the *Virtex-5 FPGA User Guide* for details about using the `IDELAYCTRL` and `IDELAY` components. For details on setting the `IDELAY_VALUE`s, see [Chapter 13, Constraining the Core](#).

The `rgmii_rxc` clock received through the IODELAY component, placed onto global clock routing, is then used to provide the clock for all receiver logic, both within the core and for the client side logic which connects to the receiver client I/F of the core.

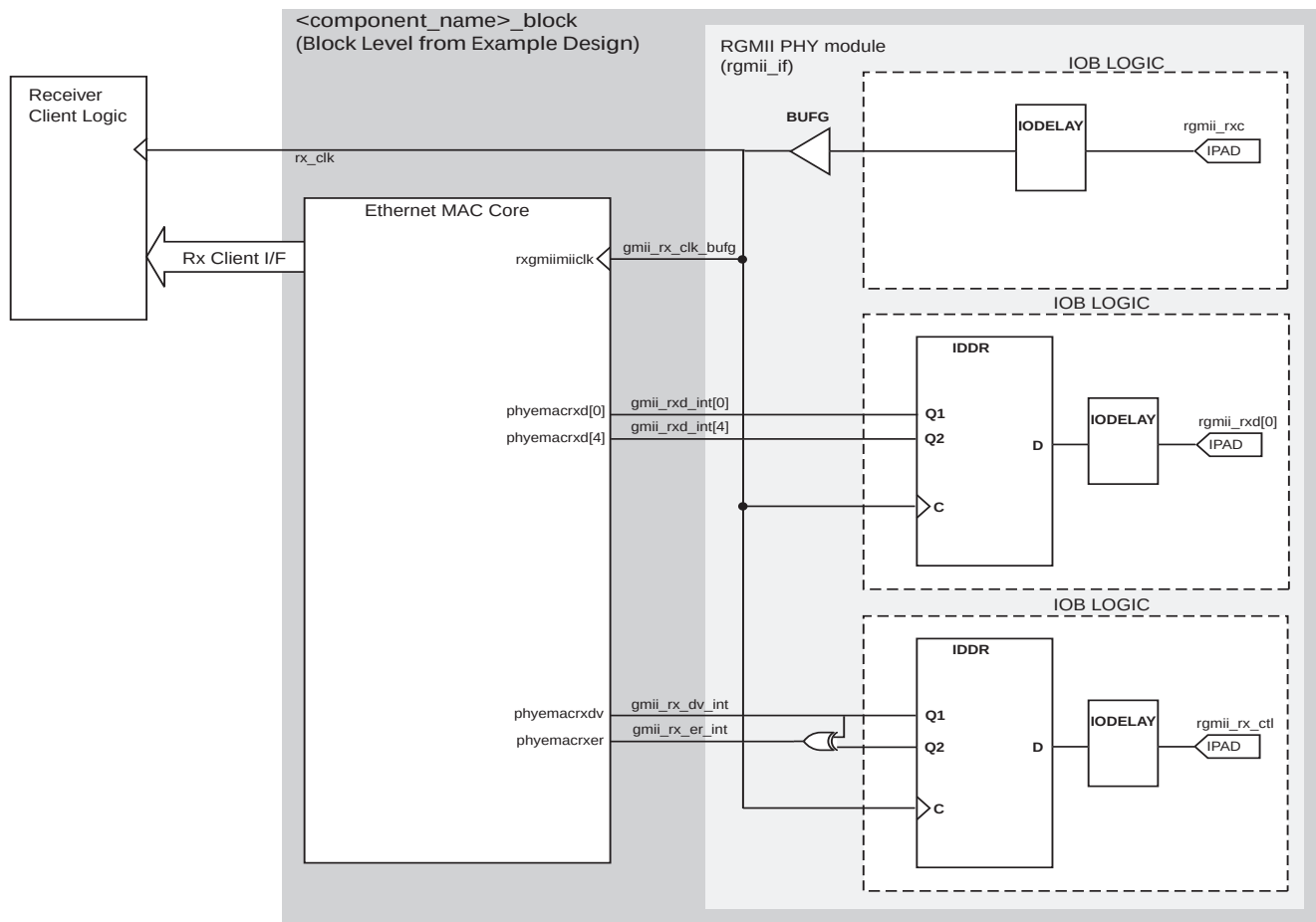


Figure 11-17: RGMI Receiver Logic and Clock Logic for Virtex-5 Devices

Clock Resource Sharing

Figure 11-18 illustrates clock resource sharing across multiple instantiations of the core when using RGMI at 1 Gb/s in Virtex-5 devices. For all instantiations, `gtx_clk` can be shared between multiple cores, resulting in a common clock domain across the device.

The receiver clocks cannot be shared. Each core is provided with its own local version of `rgmii_rxc` from the connected external PHY device as illustrated.

Figure 11-18 illustrates only two cores. However, more can be added using the same principal. This is done by instantiating the cores using the block level (from the example design) and sharing `gtx_clk` across all instantiations. The receiver clock, which cannot be shared, is unique for every instance of the core.

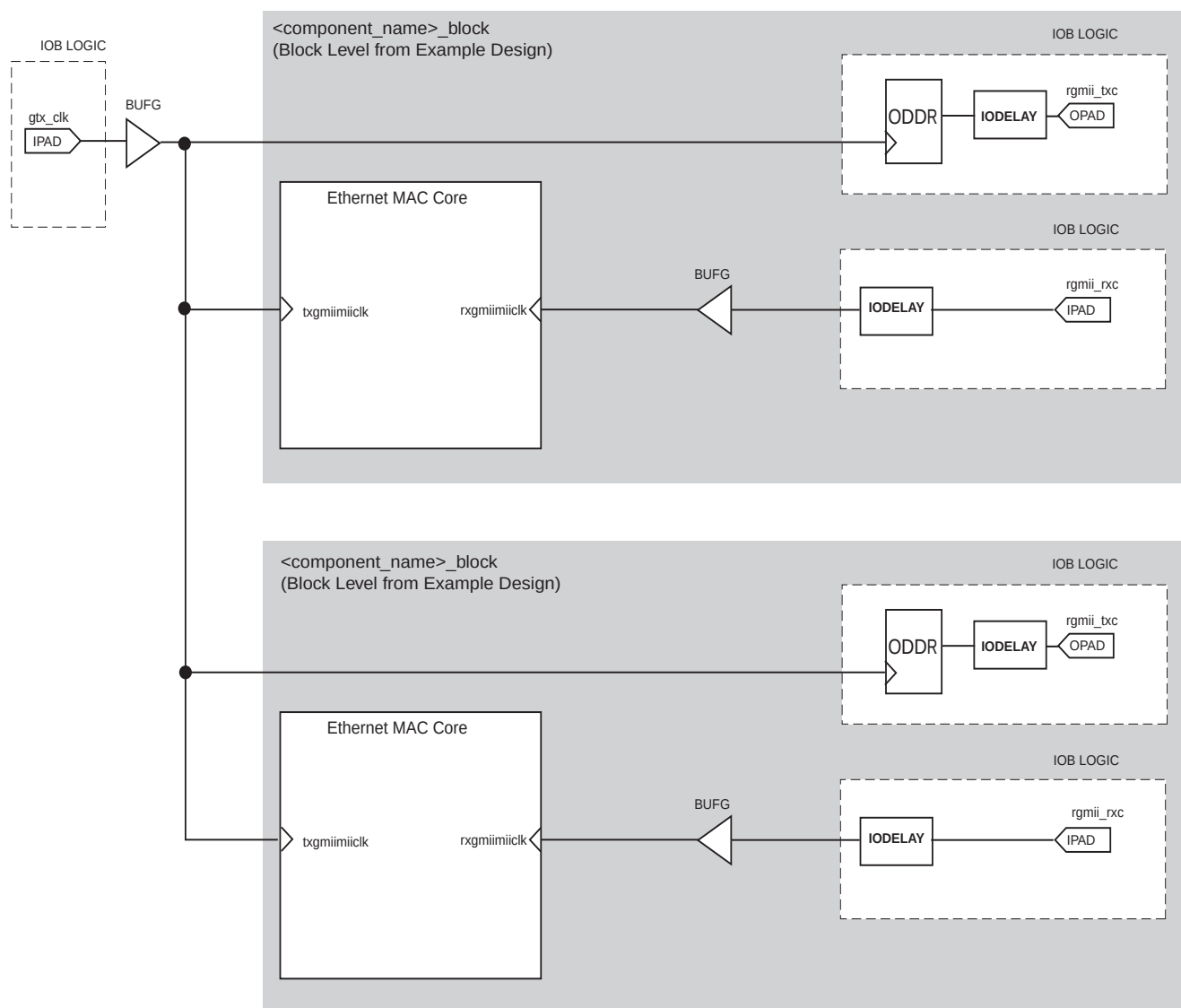


Figure 11-18: Clock Resource Sharing for 1-Gb/s RGMII in Virtex-5 Devices

Virtex-4 Devices

Transmitter Logic

The logic required to implement the RGMII transmitter logic is illustrated in [Figure 11-19](#). `gtx_clk` is a user-supplied 125 MHz reference clock source. As illustrated, this is routed to a DCM before being placed onto global clock routing using a BUFG to provide the clock for all transmitter logic, both within the core and for the client-side logic which connects to the transmitter client I/F of the core.

[Figure 11-19](#) illustrates how to use the physical transmitter interface of the core to create an external RGMII. The signal names and logic shown in this figure exactly match those delivered with the example design for a Virtex-4 device when the RGMII is selected. If other families are chosen, equivalent primitives specific to that family are used in the example design.

[Figure 11-19](#) shows that the output transmitter signals are registered in device IOBs, using DDR registers, before driving them to the device pads.

The logic required to forward the transmitter clock is also shown. This logic uses an IOB output Double-Data-Rate (DDR) register so that the clock signal produced incurs exactly the same delay as the data and control signals. However, the clock source used for this logic is derived from the DCM CLK90 output. The result of this is to create a clock signal which is 90 degrees out of phase with respect to the CLK0 equivalent. This places the `rgmii_txc` forwarded clock in the centre of the data valid window for forwarded RGMII data and control signals.

The use of the BUFGMUX shown, with one input connected to the DCM CLK90 output is included so that a reliable 125 MHz clock source is always provided on global routing (when the DCM is held in reset, the DCM input clock is instead selected). This is required to always provide a reliable clock for the receiver logic DCM; see [DCM Reset Circuitry](#).

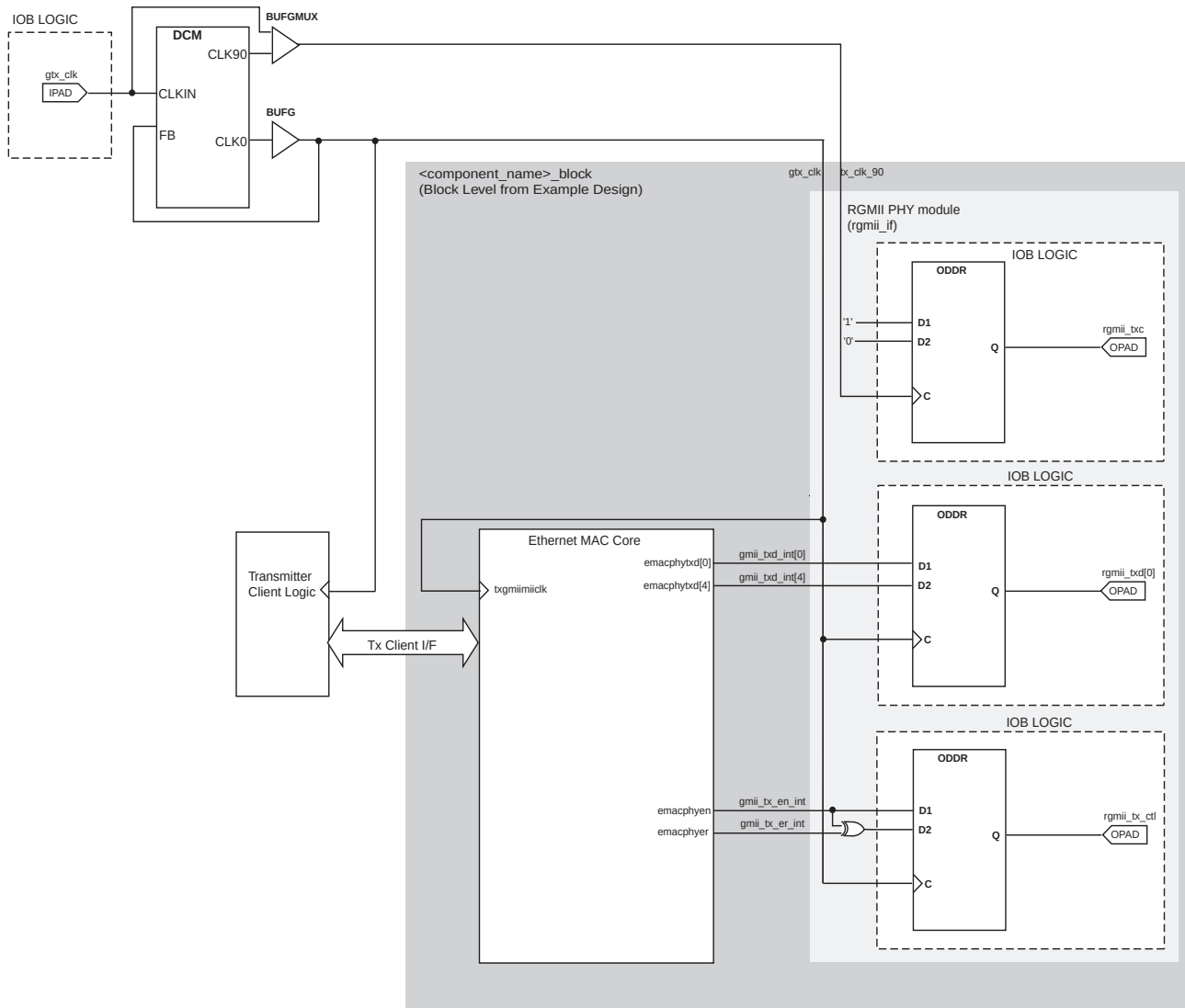


Figure 11-19: RGMII Transmitter Logic and Clock Logic for Virtex-4 Devices

Receiver Logic

In Virtex-4 devices, a DCM must be used on the `rgmii_rxc` clock path as illustrated in [Figure 11-20](#) to meet the RGMII input setup and hold requirements. This logic is implemented by the example design delivered with the core (all signal names and logic match).

Phase shifting can then be applied to the DCM to fine-tune the setup and hold times of the input RGMII receiver signals which are sampled at the RGMII IOB input flip-flops; fixed-phase shift is applied to the DCM via the example UCF for the example design. See [Chapter 13, Constraining the Core](#).

A limitation of using a DCM on this interface is that, as a DCM is sensitive to a change in input clock, 1 Gb/s Half Duplex is not supported. This is due to the nature of the rx clock in this mode of operation as it is sourced by whichever device has control of the media and a PPM shift is to be expected. This causes the DCM to lose lock, rendering the receive interface inactive. This is not a problem at 10M/100 Mb/s operation as the DCM is bypassed.

The clock produced by the DCM, placed onto global clock routing, is used to provide the clock for all receiver logic, both within the core and for the client-side logic which connects to the receiver client I/F of the core.

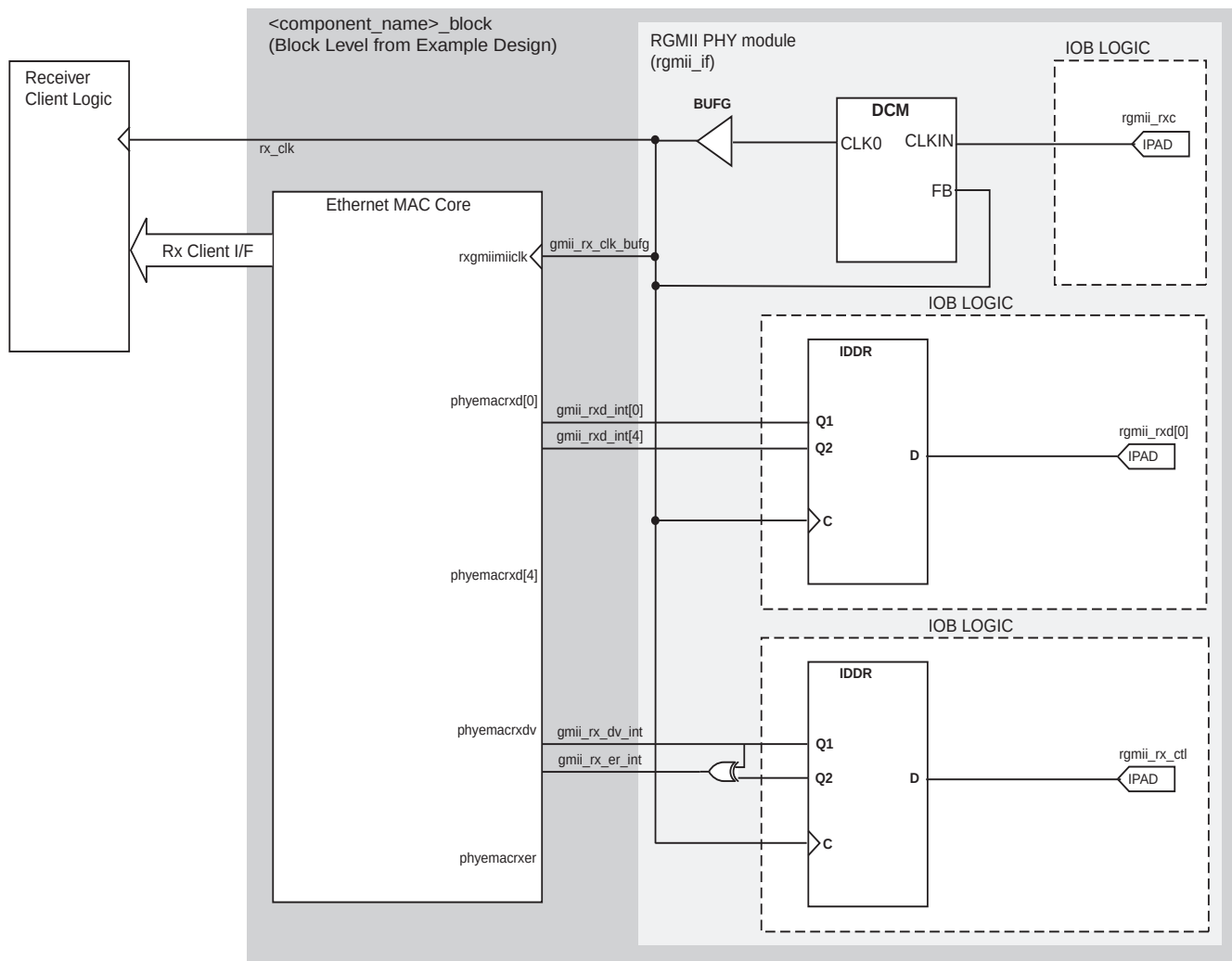


Figure 11-20: RGMI Receiver Logic and Clock Logic for Virtex-4 Devices

DCM Reset Circuitry

A DCM reset module, not illustrated in Figure 11-20, is also present and is instantiated in the example design next to the DCM. Because this logic must be reliable whatever the reset/locked status of the DCM, the module requires a reliable reference clock. In the example design for RGMII, the global transmitter clock is therefore used. This reset circuitry generates an appropriate reset pulse, based on the family, for the receiver DCM under the following conditions:

- The **locked** signal from the DCM is constantly monitored. Following a high-to-low transition on this signal, indicating that the DCM has lost lock, a reset is issued.
- A timeout counter is enabled when the DCM is in the loss of lock state. If, following the timeout period, the DCM has not obtained lock, another DCM reset is issued. This timeout counter will time a > 1 ms interval. This timeout functionality is required for DCMs connected to Ethernet PHYs because the PHYs may source discontinuous clocks under certain network conditions (for example, when no ethernet cable is connected).

For Virtex-4 devices, the generated reset from within the DCM reset module is further complicated. Here the DCM reset pulse duration must be asserted for a minimum of 200 ms (see the *Virtex-4 FPGA Data Sheet*). Consequently, a 200 ms duration timer is also included in the DCM reset module to extend the reset pulse. This reset signal is output from the DCM reset module and **should** be routed to the DCM reset input port.

Caution! In the example designs for Virtex-4 devices, the 200 ms reset pulse from the DCM reset module is NOT connected directly to the DCM; this signal must be connected when implementing the core in real hardware.

For Virtex-4 FPGA example designs, the reason why the 200 ms input is not routed directly to the DCM is so that simulations can occur in a timely manner.

The 200 ms reset pulse takes the signal name `reset_200ms` in the example design HDL files. It is routed from the DCM reset module instantiation up through all levels of hierarchy to the top level. It is then left unconnected in the example design instantiation from within the demonstration test bench. Furthermore, an accompanying signal, `reset_200ms_in`, is routed down from the top level of the example design hierarchy to the DCM instantiation, where it is connected to the DCM reset. From the demonstration test bench, this `reset_200ms_in` signal is driven for a short duration to enable fast simulation start-up. However, to re-iterate, when implementing the design in real hardware, the DCM reset signal must be connected correctly:

- **This can be achieved by connecting the `reset_200ms` signal to the `reset_200ms_in` signal at any level of example design HDL hierarchy.**

Clock Resource Sharing

Figure 11-21 illustrates clock resource sharing across multiple instantiations of the core when using RGMII at 1 Gb/s in Virtex-4 devices. `gtx_clk` should be routed to a single DCM; the DCM outputs `CLK0` and `CLK90` should both be connected onto global clock routing resources by using `BUFGMUX`/`BUFG` components as illustrated. These two transmitter global clocks can be shared between multiple cores.

The receiver clocks cannot be shared. Each core is provided with its own local version of `rgmii_rxc` from the connected external PHY device as illustrated.

Figure 11-21 illustrates only two cores. However, more can be added using the same principal. This is done by instantiating the cores using the block level (from the example design) and sharing `gtx_clk` across all instantiations. The receiver clock, which cannot be shared, is unique for every instance of the core.

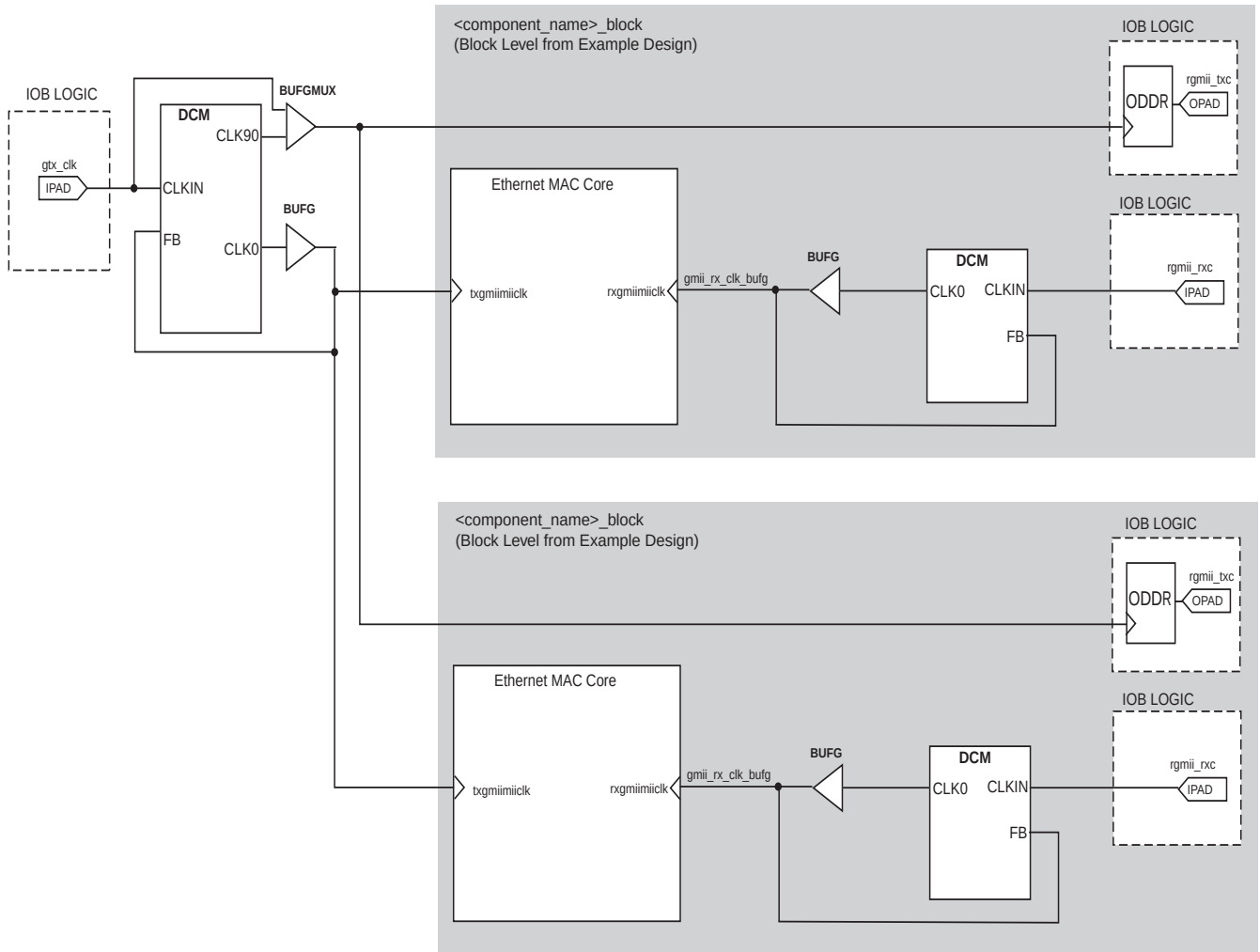


Figure 11-21: Clock Resource Sharing for 1-Gb/s RGMII in Virtex-4 Devices

Spartan-3, Spartan-3A, and Spartan-3A DSP Devices

Transmitter Logic

The logic required to implement the RGMII transmitter logic is illustrated in [Figure 11-22](#). `gtx_clk` is a user-supplied 125 MHz reference clock source. As illustrated, this is routed to a DCM before being placed onto global clock routing to provide the clock for all transmitter logic, both within the core and for the client-side logic which connects to the transmitter client I/F of the core.

[Figure 11-22](#) illustrates how to use the physical transmitter interface of the core to create an external RGMII. The signal names and logic shown in this figure exactly match those delivered with the example design for a Spartan-3 device when the RGMII is selected.

[Figure 11-22](#) shows that the output transmitter signals are registered in device IOBs, using DDR registers, before driving them to the device pads.

The logic required to forward the transmitter clock is also shown. This logic uses an IOB output Double-Data-Rate (DDR) register so that the clock signal produced incurs exactly the same delay as the data and control signals. However, the clock source used for this logic is derived from the DCM CLK90 output. The result of this is to create a clock signal which is 90 degrees out of phase with respect to the CLK0 equivalent. This places the `rgmii_txc` forwarded clock in the centre of the data valid window for forwarded RGMII data and control signals.

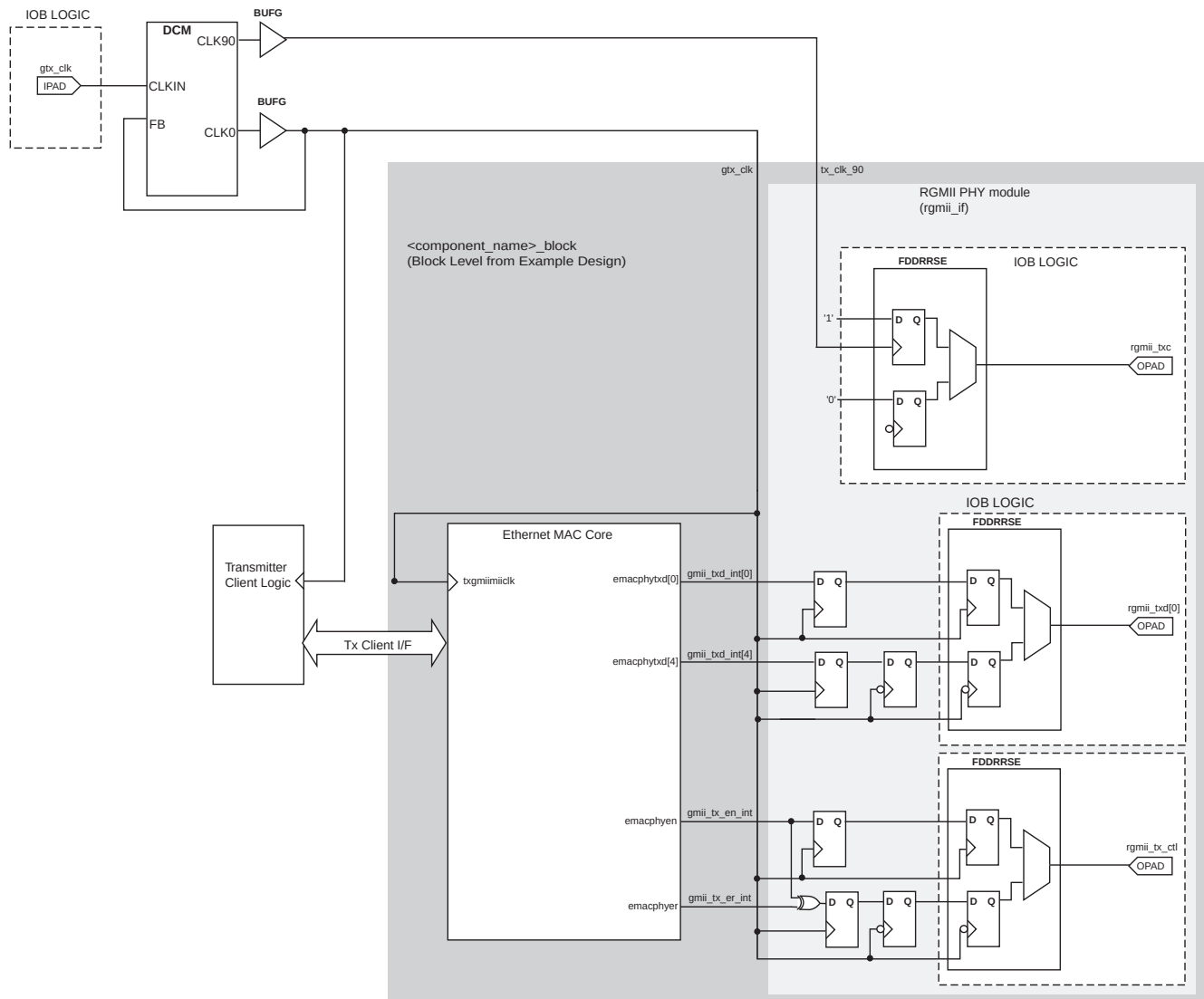


Figure 11-22: RGMII Transmitter Logic and Clock Logic for Spartan-3, Spartan-3A and Spartan-3A DSP Devices

Receiver Logic

In Spartan-3, Spartan-3A and Spartan-3A DSP devices, a DCM must be used on the `rgmii_rxc` clock path as illustrated in [Figure 11-23](#) to meet the RGMII input setup and hold requirements. This logic is implemented by the example design delivered with the core (all signal names and logic match).

Phase shifting can then be applied to the DCM to fine-tune the setup and hold times of the input RGMII receiver signals which are sampled at the RGMII IOB input flip-flops; fixed-phase shift is applied to the DCM via the example UCF for the example design. See [Chapter 13, Constraining the Core](#).

A limitation of using a DCM on this interface is that, as a DCM is sensitive to a change in the input clock, 1 Gb/s Half Duplex is not supported. This is due to the nature of the rx clock in this mode of operation as it is sourced by whichever device has control of the media and a PPM shift is to be expected. This causes the DCM to lose lock, rendering the receive interface inactive.

The clock produced by the DCM, placed onto global clock routing, is used to provide the clock for all receiver logic, both within the core and for the client-side logic which connects to the receiver client I/F of the core.

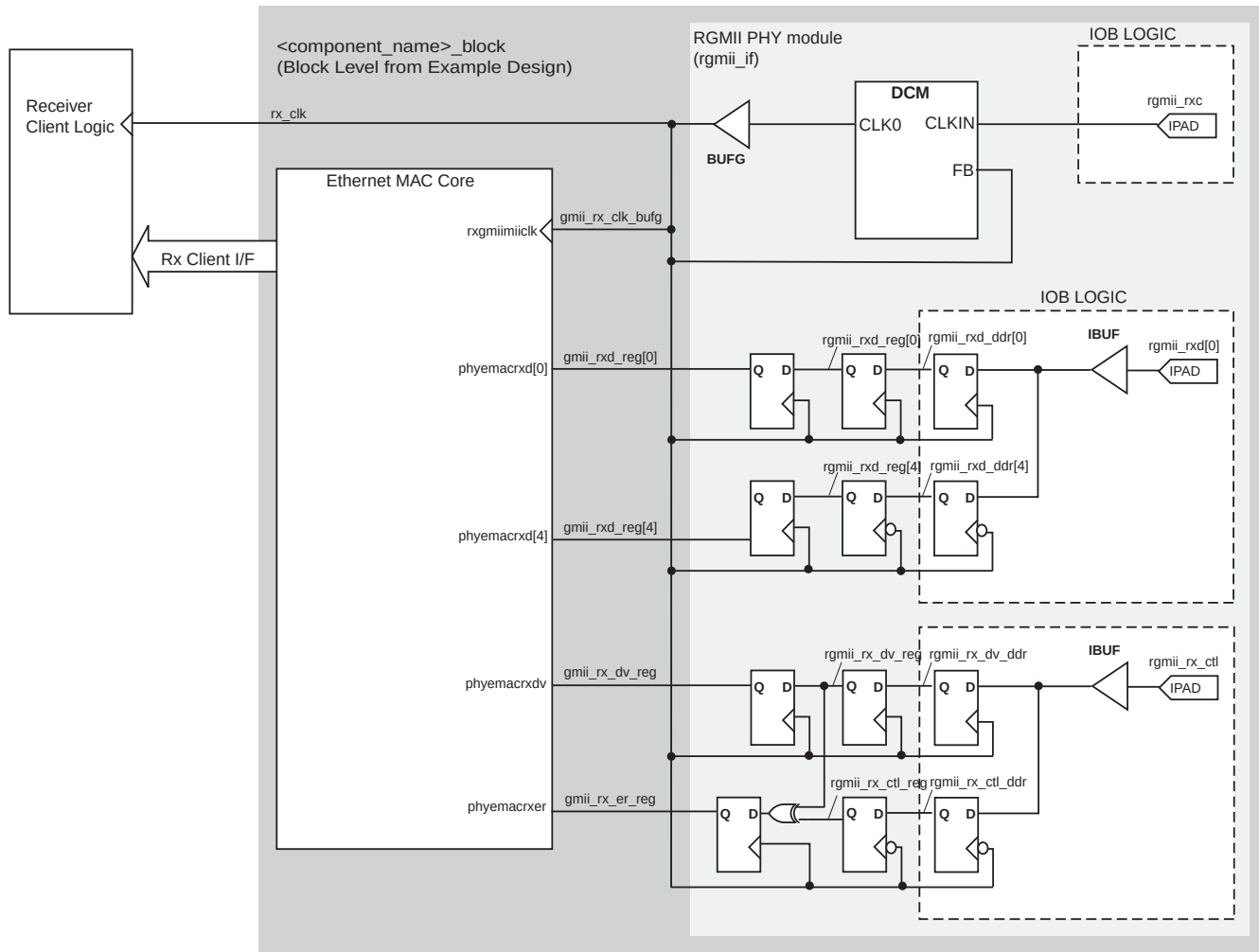


Figure 11-23: RGMII Receiver Logic and Clock Logic for Spartan-3, Spartan-3A and Spartan-3A DSP Devices

DCM Reset Circuitry

A DCM reset module, not illustrated in [Figure 11-23](#), is also present and is instantiated in the example design next to the DCM. Because this logic must be reliable whatever the reset/locked status of the DCM, the module requires a reliable reference clock. In the example design for RGMII, the global transmitter clock is therefore used. This reset circuitry generates an appropriate reset pulse, based on the family, for the receiver DCM under the following conditions:

- The `locked` signal from the DCM is constantly monitored. Following a high to low transition on this signal, indicating that the DCM has lost lock, a reset is issued.
- A timeout counter is enabled when the DCM is in the loss of lock state. If, following the timeout period, the DCM has not obtained lock, another DCM reset is issued. This timeout counter will time a > 1 ms interval. This timeout functionality is required for DCMs connected to Ethernet PHYs because the PHYs may source discontinuous clocks under certain network conditions (for example, when no ethernet cable is connected).

For Spartan-3 devices, the reset pulse is transferred into the DCM input clock (`gmii_rx_clk` from [Figure 11-23](#)). Here it is extended to three DCM clock periods duration and routed to the reset input of the DCM.

Clock Resource Sharing

[Figure 11-24](#) illustrates clock resource sharing across multiple instantiations of the core when using RGMII at 1 Gb/s in Spartan-3, Spartan-3A and Spartan-3A DSP devices. `gtx_clk` should be routed to a single DCM; the DCM outputs `CLK0` and `CLK90` should both be connected onto global clock routing resources by using `BUFG` components. These two transmitter global clocks can be shared between multiple cores as illustrated.

The receiver clocks cannot be shared. Each core is provided with its own local version of `rgmii_rxc` from the connected external PHY device as illustrated.

[Figure 11-24](#) illustrates only two cores. However, more can be added using the same principal. This is done by instantiating the cores using the block level (from the example design) and sharing `gtx_clk` across all instantiations. The receiver clock, which cannot be shared, is unique for every instance of the core.

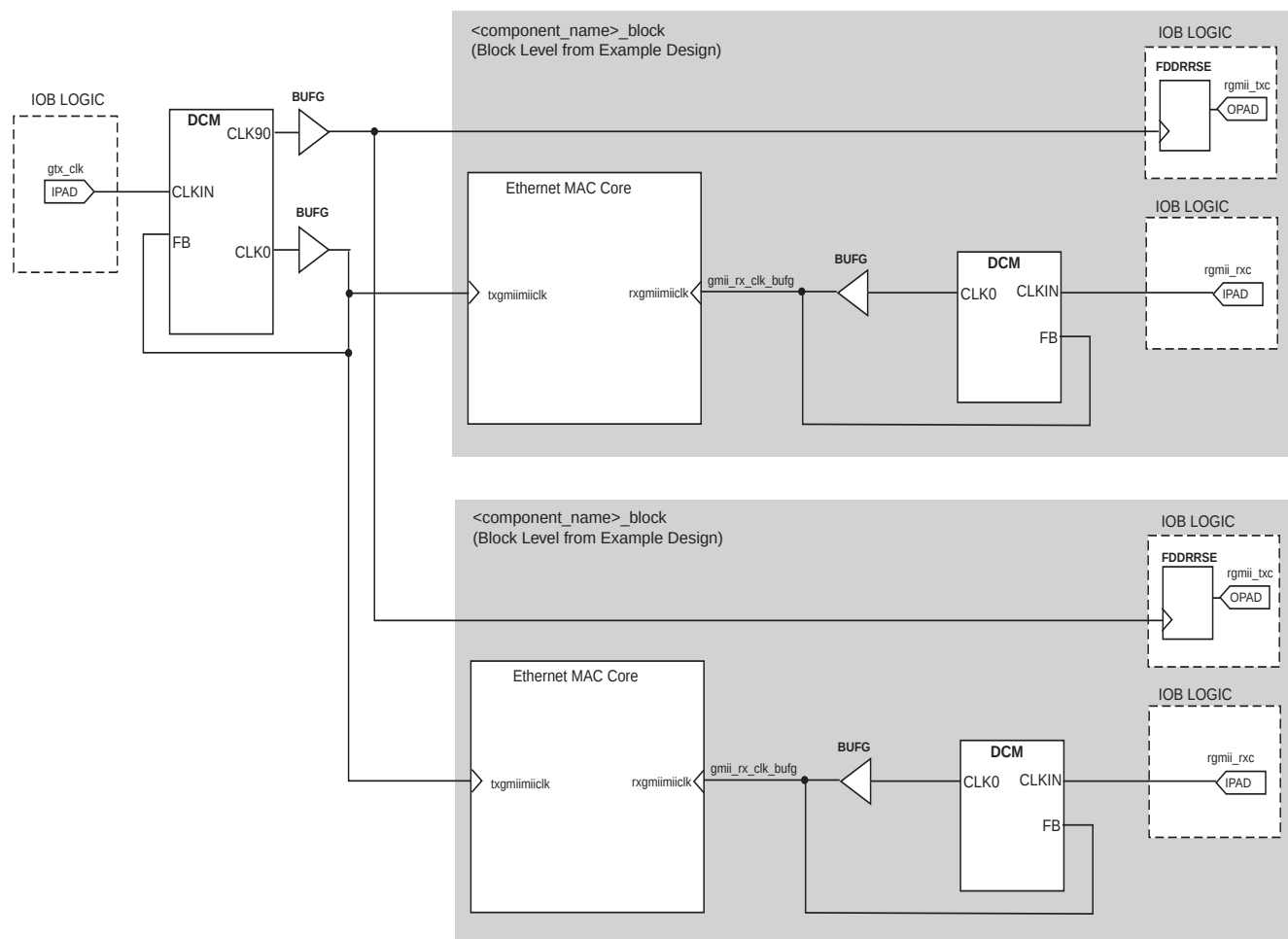


Figure 11-24: Clock Resource Sharing for 1-Gb/s RGMII in Spartan-3, Spartan-3A and Spartan-3A DSP Devices

Physical Interfaces for Tri-speed (10 Mb/s, 100 Mb/s and 1 Gb/s) Ethernet MAC IP Core

The HDL example design supplied with the core, for Tri-speed operation, provides either a GMII or RGMII interface. These are typically used to connect the MAC to an external PHY device.

The Media Independent Interface (MII), defined in IEEE Std 802.3-2008, clause 22, is a parallel interface that connects a 10 Mb/s and/or 100 Mb/s capable MAC to the physical sublayers. The Gigabit Media Independent Interface (GMII), defined in IEEE Std 802.3-2008, clause 35, is an extension of the MII and is used to connect a 1-Gb/s capable MAC to the physical sublayers. MII can be considered a subset of GMII, and as a result, GMII/MII can carry Ethernet traffic at 10 Mb/s, 100 Mb/s and 1 Gb/s.

Virtex®-6 devices support GMII and MII at 2.5 V only; Virtex-5, Virtex-4, Spartan®-6 and Spartan-3 devices support GMII and MII at 3.3 V or lower.

The Reduced Gigabit Media Independent Interface (RGMII) is an alternative to the GMII/MII. RGMII can carry Ethernet traffic at 10 Mb/s, 100 Mb/s and 1 Gb/s and achieves a 50% reduction in the pin count compared with GMII; this is achieved with the use of double-data-rate (DDR) flip-flops. RGMII is therefore often favored over GMII by PCB designers. A further advantage of the RGMII implementation is that, unlike GMII/MII, clock resources for the transmitter can be shared across multiple core instances. This results in significant clock resource savings when implementing multiple cores in a design.

See the appropriate section:

- [Gigabit Media Independent Interface \(GMII\)](#)
- [Multiple Core Instances in Virtex-4, Spartan-3, Spartan-3E, Spartan-3A and Spartan-3A DSP Devices](#)

Note: Only the use of the 10/100/1000 Mb/s Ethernet MAC when using clock enables is discussed in this chapter. Use of the core for tri-speed operation without clock enables is a legacy mode: see [Appendix G, 10/100/1000 Mb/s Operation Without Using Clock Enables](#).

Gigabit Media Independent Interface (GMII)

GMII Transmitter Interface

The logic required to implement the GMII transmitter logic is illustrated in [Figure 12-1](#). `gtx_clk` is a user-supplied 125 MHz reference clock source for use at 1 Gb/s. `mii_tx_clk` is sourced by the external PHY device for use at 10 Mb/s and 100 Mb/s speeds. Consequently a global clock multiplexer, a BUFGMUX, is used to switch the clock source depending on the operating speed. The output from this BUFGMUX provides the transmitter clock for the core and client logic as illustrated.

Closely linked to the clock logic is the use of the `tx_enable` clock enable derivation. This must be used as a standard clock enable by all flip-flops in the Tx Client I/F logic. At 1 Gb/s operation the clock enable is high at all times. At 10 Mb/s and 100 Mb/s operation this clock enable is high on alternative clock cycles; this is required to match the data rates between the 4-bit data width of the MII and the 8-bit data width of the client I/F.

[Figure 12-1](#) also illustrates how to use the physical transmitter interface of the core to create an external GMII. The signal names and logic shown in this figure exactly match those delivered with the example design for a Virtex-5 device when the GMII is selected. If other families are chosen, equivalent primitives specific to that family are used in the example design.

As shown in [Figure 12-1](#), the output transmitter signals are registered in device IOBs before driving them to the device pads. The logic required to forward the transmitter clock for 1 Gb/s operation is also shown. This logic uses an IOB output Double-Data-Rate (DDR) register so that the clock signal produced incurs exactly the same delay as the data and control signals. This clock signal, `gmii_tx_clk`, is inverted with respect to `gtx_clk` so that the rising edge of `gmii_tx_clk` occurs in the centre of the data valid window, therefore maximizing setup and hold times across the interface.

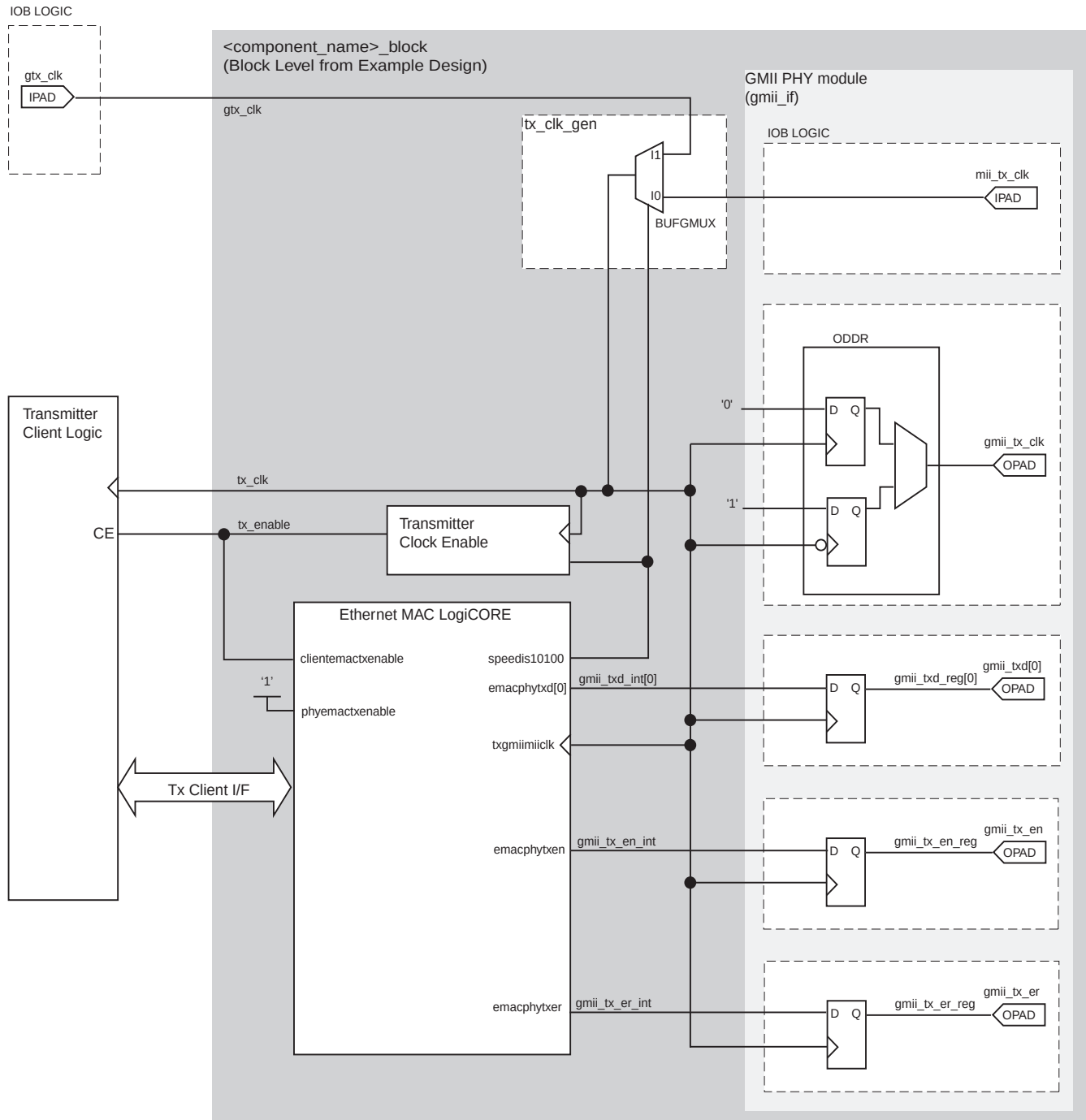


Figure 12-1: GMII Transmitter Logic and Clock Logic

GMII Receive Interface

The logic required to implement the GMII receiver logic is described in the following sections. Logical implementation varies for different device families. See the specific family section:

- [Virtex-6 Devices](#)
- [Spartan-6 Devices](#)
- [Virtex-5 Devices](#)
- [Virtex-4, Spartan-3, Spartan-3E, and Spartan-3A Devices](#)

Virtex-6 Devices

In this implementation, a BUFIO is used to provide the lowest form of clock routing delay from input clock to input GMII Rx signal sampling at the device IOBs. Note, however, that this creates placement constraints; a BUFIO capable clock input pin must be selected, and all other input GMII Rx signals must be placed in the respective BUFIO region. The *Virtex-6 FPGA User Guide* should be consulted.

The input clock is also placed onto regional clock routing using the BUFR component as illustrated. This regional clock then provides the clock for all receiver logic, both within the core and for the client-side logic which connects to the receiver client I/F of the core.

The IODELAY elements can be adjusted to fine-tune the setup and hold times at the GMII IOB input flip-flops. This will meet input setup and hold constraints at all three ethernet speeds. The delay is applied to the IODELAY element using constraints in the UCF; these can be edited if desired. See [Chapter 13, Constraining the Core](#) for details.

Closely linked to the clock logic is the use of the `rx_enable` clock enable derivation. This must be used as a standard clock enable by all flip-flops in the Rx Client I/F logic. At 1 Gb/s operation the clock enable is high at all times. At 10 Mb/s and 100 Mb/s operation this clock enable is high on alternative clock cycles; this is required to match the data rates between the 4-bit data width of the MII and the 8-bit data width of the client I/F.

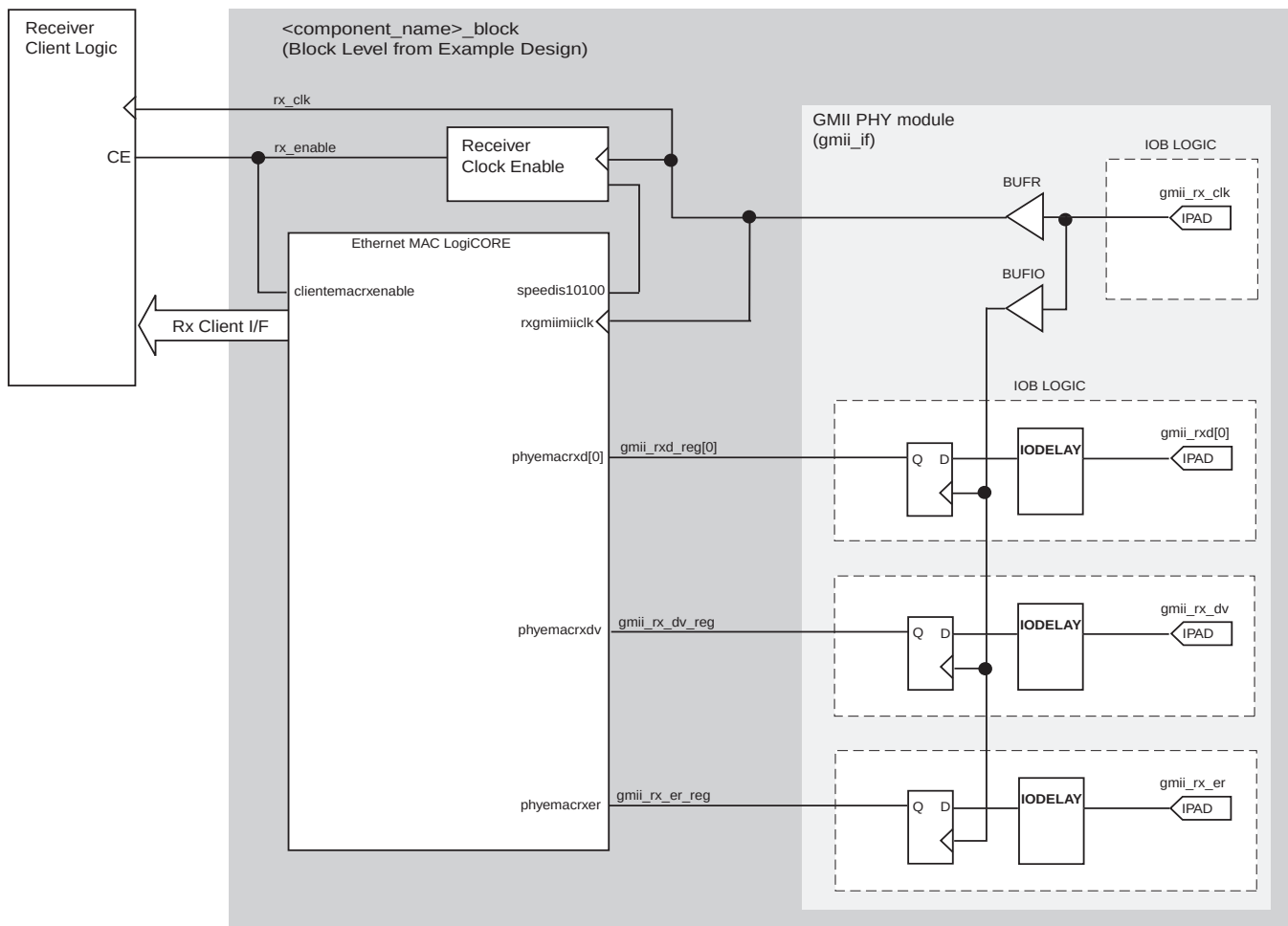


Figure 12-2: GMII Receiver Logic and Clock Logic for Virtex-6 Devices

Spartan-6 Devices

In this implementation, a BUFIO2 is used to provide the lowest form of clock routing delay from input clock to input GMII Rx signal sampling at the device IOBs. Note, however, that this creates placement constraints; a BUFIO2 capable clock input pin must be selected, and all other input GMII Rx signals must be placed in the respective BUFIO2 region. The *Spartan-6 FPGA User Guide* should be consulted.

The input clock is also placed onto global clock routing using the BUFG component as illustrated. This clock then provides the clock for all receiver logic, both within the core and for the client-side logic which connects to the receiver client I/F of the core.

The IODELAY2 elements can be adjusted to fine-tune the setup and hold times at the GMII IOB input flip-flops. This will meet input setup and hold constraints at all three ethernet speeds. The delay is applied to the IODELAY2 element using constraints in the UCF; these can be edited if desired. See [Chapter 13, Constraining the Core](#) for details.

Closely linked to the clock logic is the use of the rx_enable clock enable derivation. This must be used as a standard clock enable by all flip-flops in the Rx Client I/F logic. At 1 Gb/s operation the clock enable is high at all times.

At 10 Mb/s and 100 Mb/s operation this clock enable is high on alternative clock cycles; this is required to match the data rates between the 4-bit data width of the MII and the 8-bit data width of the client I/F.

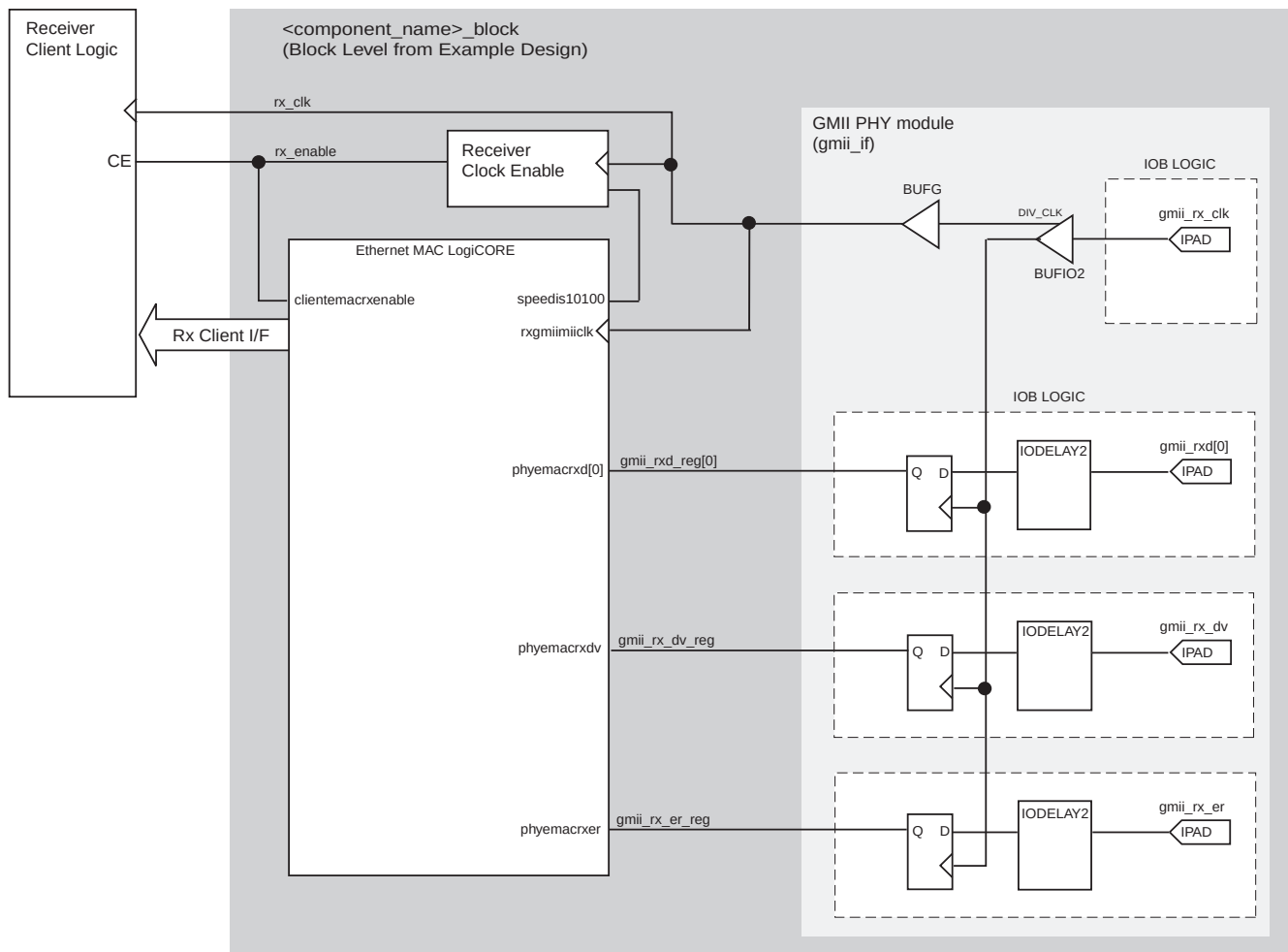


Figure 12-3: GMII Receiver Logic and Clock Logic for Spartan-6 Devices

Virtex-5 Devices

Three possible solutions are available for Virtex-5 devices:

1. A DCM can be used on the `gmii_rx_clk` clock path, exactly as described for [Virtex-4, Spartan-3, Spartan-3E, and Spartan-3A Devices](#).
2. Regional and BUFIO clock routing can be used as described in the section for [Virtex-6 Devices](#).
3. Input Delay Elements can be used on the GMII data and clock path, using global clock routing and as illustrated in [Figure 12-4](#).

The design for Case 3 provides a simpler solution than Case 1 and fully supports all operating modes, it also allows for a more flexible pinout than for Case 2. It has therefore been chosen as the default example design and is described in this section.

As illustrated in Figure 12-4, IODELAY components can be used on the data and control signals. These can be used to either shift the input clock, `gmii_rx_clk`, or the data and control signals to meet the setup and hold requirements at all ethernet speeds and to allow for any bus skew across the data and control inputs. The IODELAY components are used in fixed delay mode, where the attribute `IDELAY_VALUE` determines the tap delay value. An `IDELAYCTRL` primitive must be instantiated for this mode of operation. See the *Virtex-5 FPGA User Guide* for details about using the `IDELAYCTRL` and `IDELAY` components. For details on setting the `IDELAY_VALUE`s, see Chapter 13, *Constraining the Core*.

The `gmii_rx_clk` clock received through the IODELAY component, placed onto global clock routing, is then used to provide the clock for all receiver logic, both within the core and for the client-side logic which connects to the receiver client I/F of the core.

Closely linked to the clock logic is the use of the `rx_enable` clock enable derivation. This must be used as a standard clock enable by all flip-flops in the Rx Client I/F logic. At 1 Gb/s operation the clock enable is high at all times. At 10 Mb/s and 100 Mb/s operation this clock enable is high on alternative clock cycles; this is required to match the data rates between the 4-bit data width of the MII and the 8-bit data width of the client I/F.

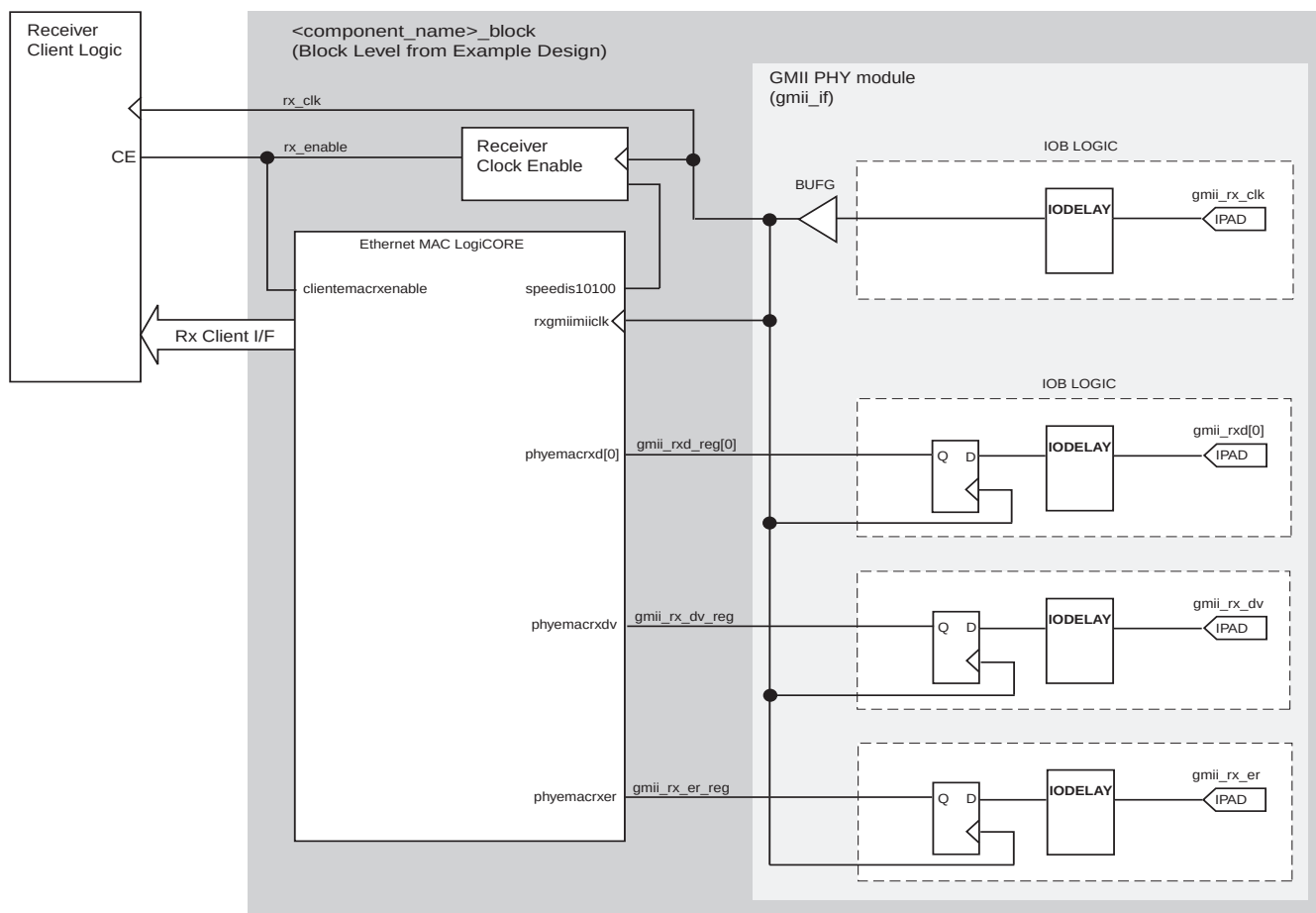


Figure 12-4: GMII Receiver Logic and Clock Logic for Virtex-5 Devices

Virtex-4, Spartan-3, Spartan-3E, and Spartan-3A Devices

In Virtex-4, Spartan-3, Spartan-3E, Spartan-3A and Spartan-3A DSP devices, a DCM must be used on the `gmii_rx_clk` clock path as illustrated in [Figure 12-5](#) to meet the GMII input setup and hold requirements when operating at 1 Gb/s. Phase shifting can then be applied to the DCM to fine-tune the setup and hold times of the input GMII receiver signals which are sampled at the GMII IOB input flip-flops; fixed-phase shift is applied to the DCM via the example UCF for the example design. See [Chapter 13, Constraining the Core](#).

A limitation of using a DCM on this interface is that, as a DCM is sensitive to a change in input clock, 1 Gb/s Half Duplex is not supported. This is due to the nature of the rx clock in this mode of operation as it is sourced by whichever device has control of the media and a PPM shift is to be expected. This causes the DCM to lose lock, rendering the receive interface inactive.

When operating at 10 Mb/s and 100 Mb/s, the DCM is bypassed and held in reset. This is achieved using the BUFGMUX global clock multiplexer as illustrated. It is a requirement to bypass the DCM since the clock frequency of `gmii_rx_clk` is 2.5 MHz when operating at 10 Mb/s: this is below the DCM low frequency threshold for most families. However, at the 10 Mb/s and 100 Mb/s operating speeds, input setup and hold margins increase appropriately and the input MII data can be sampled correctly without use of the DCM.

The output from the BUFGMUX provides the receiver clock for the core and client logic as illustrated.

Closely linked to the clock logic is the use of the `rx_enable` clock enable derivation. This must be used as a standard clock enable by all flip-flops in the Rx Client I/F logic. At 1 Gb/s operation the clock enable is high at all times. At 10 Mb/s and 100 Mb/s operation this clock enable is high on alternative clock cycles; this is required to match the data rates between the 4-bit data width of the MII and the 8-bit data width of the client I/F.

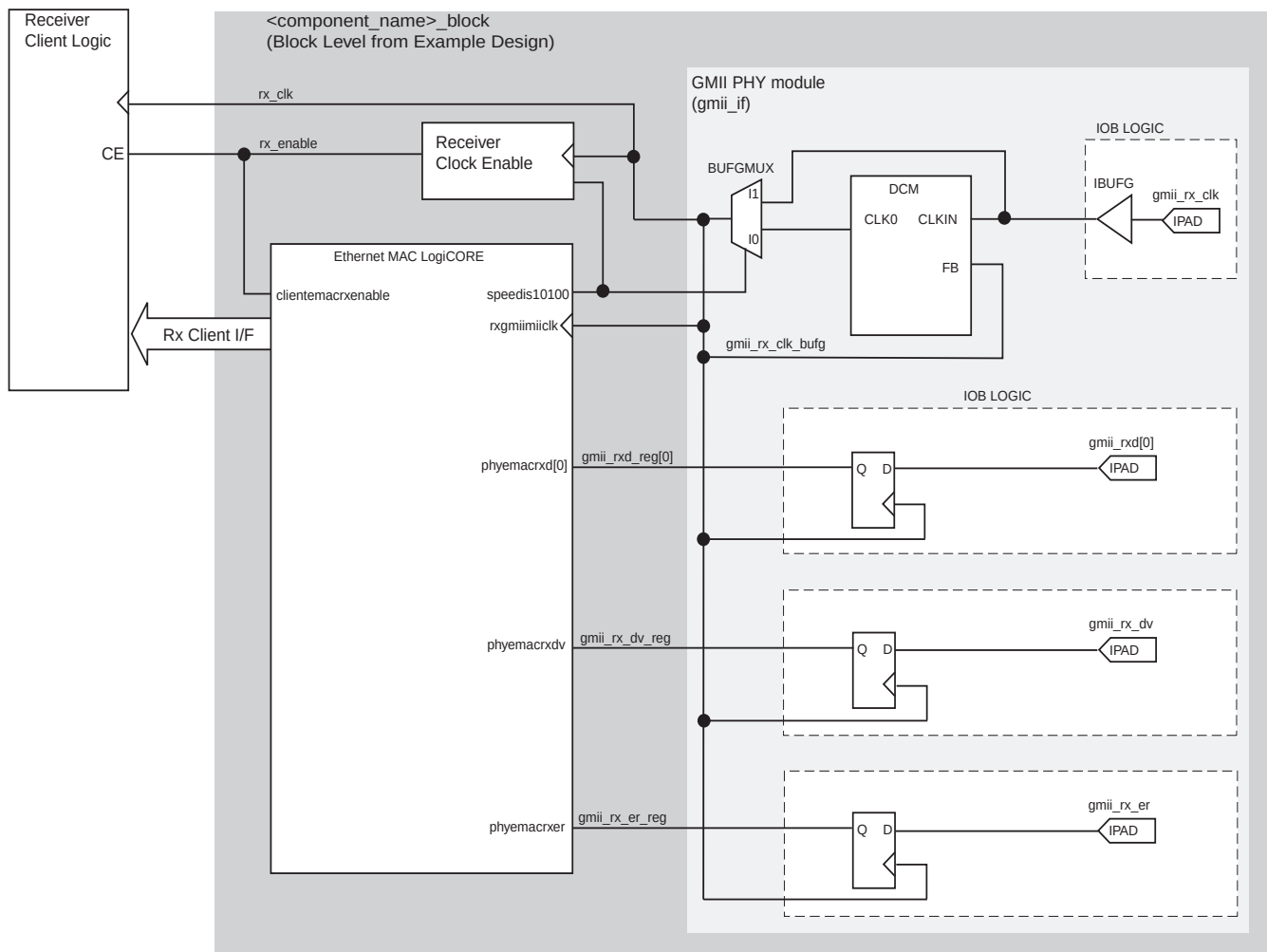


Figure 12-5: GMII Receiver Logic and Clock Logic for Virtex-4, Spartan-3, Spartan-3E, Spartan-3A and Spartan-3A DSP Devices

DCM Reset Circuitry

A DCM reset module, not illustrated in Figure 12-5, is also present and is instantiated in the example design next to the DCM. Because this logic must be reliable whatever the reset/locked status of the DCM, the module requires a reliable reference clock. In the example design for GMII, the global transmitter clock is therefore used (`gtx_clk` from Figure 12-1). This reset circuitry generates an appropriate reset pulse, based on the family, for the receiver DCM under the following conditions:

- The locked signal from the DCM is constantly monitored. Following a high to low transition on this signal, indicating that the DCM has lost lock, a reset is issued.
- A timeout counter is enabled when the DCM is in the loss of lock state. If, following the timeout period, the DCM has not obtained lock, another DCM reset is issued. This timeout counter will time a > 1 ms interval. This timeout functionality is required for DCMs connected to Ethernet PHYs because the PHYs may source discontinuous clocks under certain network conditions (for example, when no ethernet cable is connected).

Spartan-3 Devices

For Spartan-3 devices, the reset pulse is transferred into the DCM input clock (`gmii_rx_clk` from Figure 12-5). Here it is extended to three DCM clock periods duration and routed to the reset input of the DCM.

Virtex-4 Devices

For Virtex-4 devices, the generated reset from within the DCM reset module is further complicated. Here the DCM reset pulse duration must be asserted for a minimum of 200 ms (see the *Virtex-4 Data sheet*). Consequently, a 200 ms duration timer is also included in the DCM reset module to extend the reset pulse. This reset signal is output from the DCM reset module and **should** be routed to the DCM reset input port.

Caution! In the example designs for Virtex-4 devices, the 200 ms reset pulse from the DCM reset module is NOT connected directly to the DCM; this signal must be connected when implementing the core in real hardware.

For Virtex-4 FPGA example designs, the reason why the 200 ms input is not routed directly to the DCM is so that simulations can occur in a timely manner.

The 200 ms reset pulse takes the signal name `reset_200ms` in the example design HDL files. It is routed from the DCM reset module instantiation up through all levels of hierarchy to the top level. It is then left unconnected in the example design instantiation from within the demonstration test bench. Furthermore, an accompanying signal, `reset_200ms_in`, is routed down from the top level of the example design hierarchy to the DCM instantiation, where it is connected to the DCM reset. From the demonstration test bench, this `reset_200ms_in` signal is driven for a short duration to enable fast simulation start-up. However, to re-iterate, when implementing the design in real hardware, the DCM reset signal must be connected correctly:

- This can be achieved by connecting the `reset_200ms` signal to the `reset_200ms_in` signal at any level of example design HDL hierarchy.

Multiple Core Instantiations

Because both `mii_tx_clk` and `gmii_rx_clk` are both sourced by the external PHY device connected to the GMII/MII, it is not possible to share global transmitter or receiver clock resources across multiple instantiations of the core. Each instance of the core requires its own endpoint clocking resources. See the appropriate section:

- [Multiple Core Instances in Virtex-6 Devices](#)
- [Multiple Core Instances in Spartan-6 Devices](#)
- [Multiple Core Instances in Virtex-5 Devices](#)
- [Multiple Core Instances in Virtex-4, Spartan-3, Spartan-3E, Spartan-3A and Spartan-3A DSP Devices](#)

Note: RGMII provides a more optimal solution because it does allow transmitter clock resources to be shared. See [Multiple Core Instances in Virtex-4, Spartan-3, Spartan-3E, Spartan-3A and Spartan-3A DSP Devices](#).

Multiple Core Instances in Virtex-6 Devices

[Figure 12-6](#) illustrates multiple instances of the core in Virtex-6 devices. The 1 Gb/s transmitter reference clock source, `gtx_clk`, can be shared across all cores as illustrated. However, global transmitter and receiver clock resources cannot be shared and require independent BUFGMUX/BUFR elements as illustrated.

[Figure 12-6](#) illustrates only two cores. However, more can be added using the same principal. This is done by instantiating the cores using the block level (from the example design) and sharing the `gtx_clk` clock source across all instantiations.

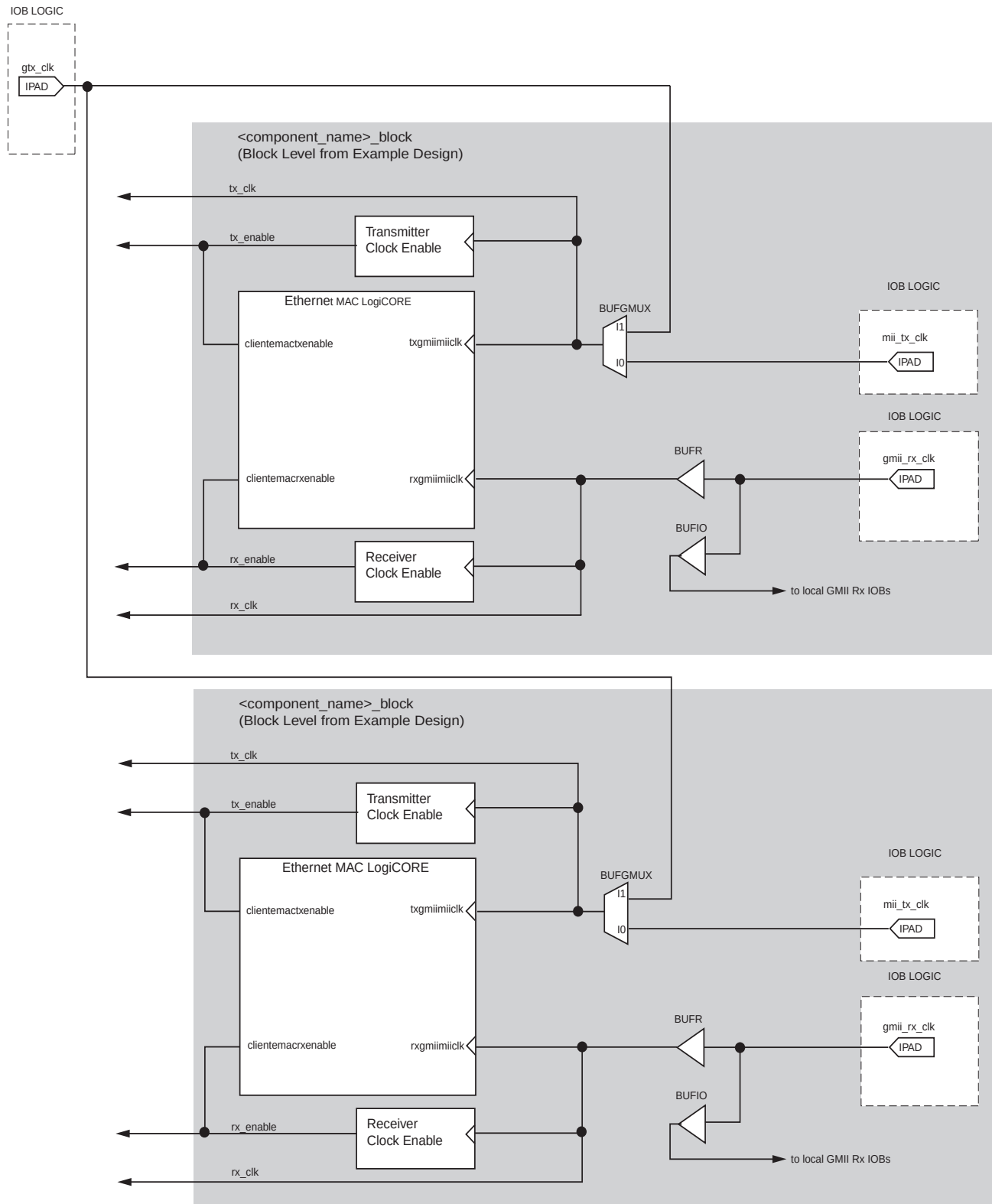


Figure 12-6: Clock Resource Sharing for 1-Gb/s GMII in Virtex-6 Devices

Multiple Core Instances in Spartan-6 Devices

Figure 12-7 illustrates multiple instances of the core in Virtex-6 devices. The 1-Gb/s transmitter reference clock source, `gtx_clk`, can be shared across all cores as illustrated. However, global transmitter and receiver clock resources cannot be shared and require independent BUFGMUX/BUFR elements as illustrated.

Figure 12-7 illustrates only two cores. However, more can be added using the same principal. This is done by instantiating the cores using the block level (from the example design) and sharing the `gtx_clk` clock source across all instantiations.

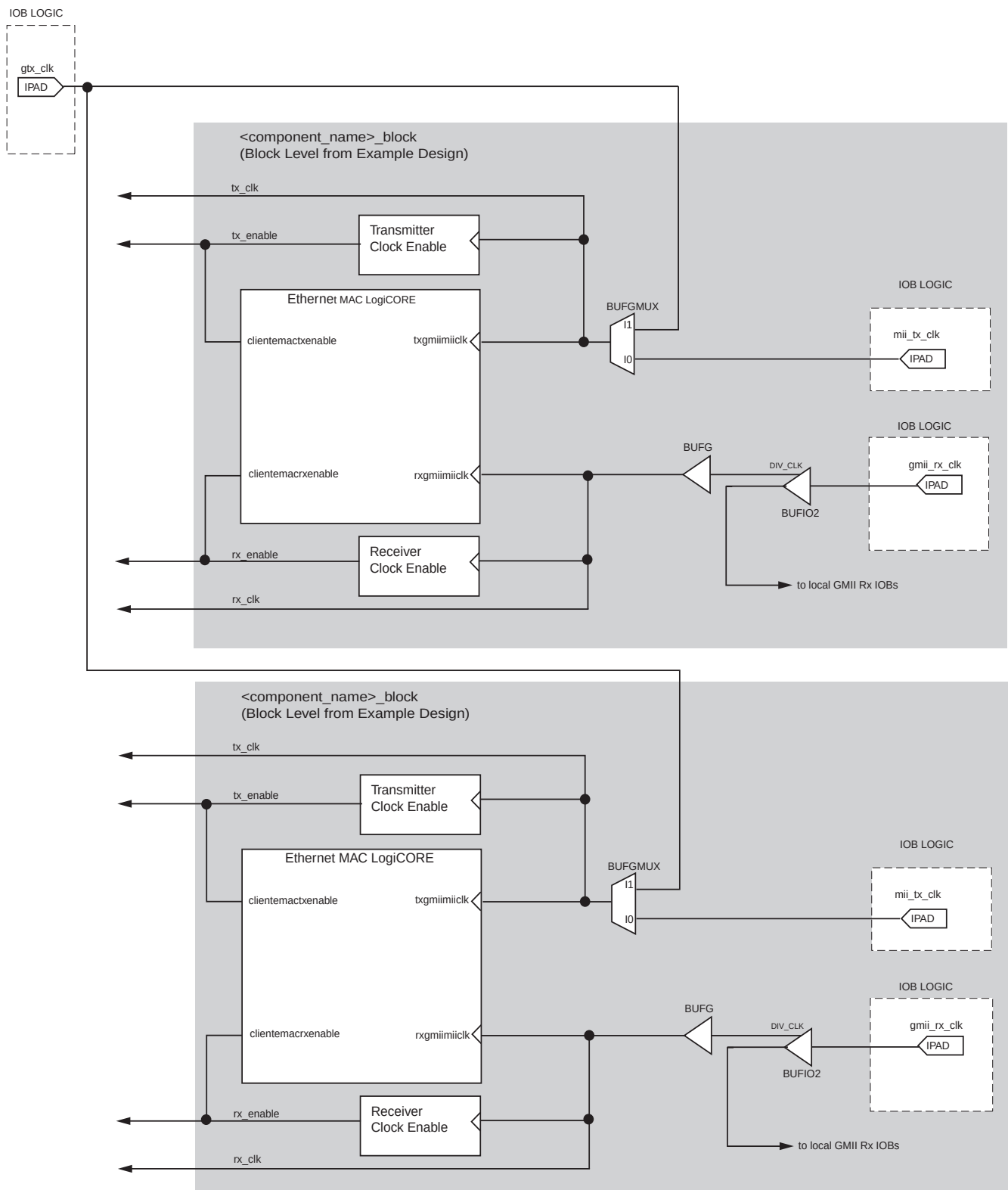


Figure 12-7: Clock Resource Sharing for 1-Gb/s GMII in Spartan-6 Devices

Multiple Core Instances in Virtex-5 Devices

Figure 12-8 illustrates multiple instances of the core in Virtex-5 devices. The 1 Gb/s transmitter reference clock source, `gtx_clk`, can be shared across all cores as illustrated. However, global transmitter and receiver clock resources cannot be shared and require independent BUFGMUX/BUFG elements as illustrated.

Figure 12-8 illustrates only two cores. However, more can be added using the same principal. This is done by instantiating the cores using the block level (from the example design) and sharing the `gtx_clk` clock source across all instantiations.

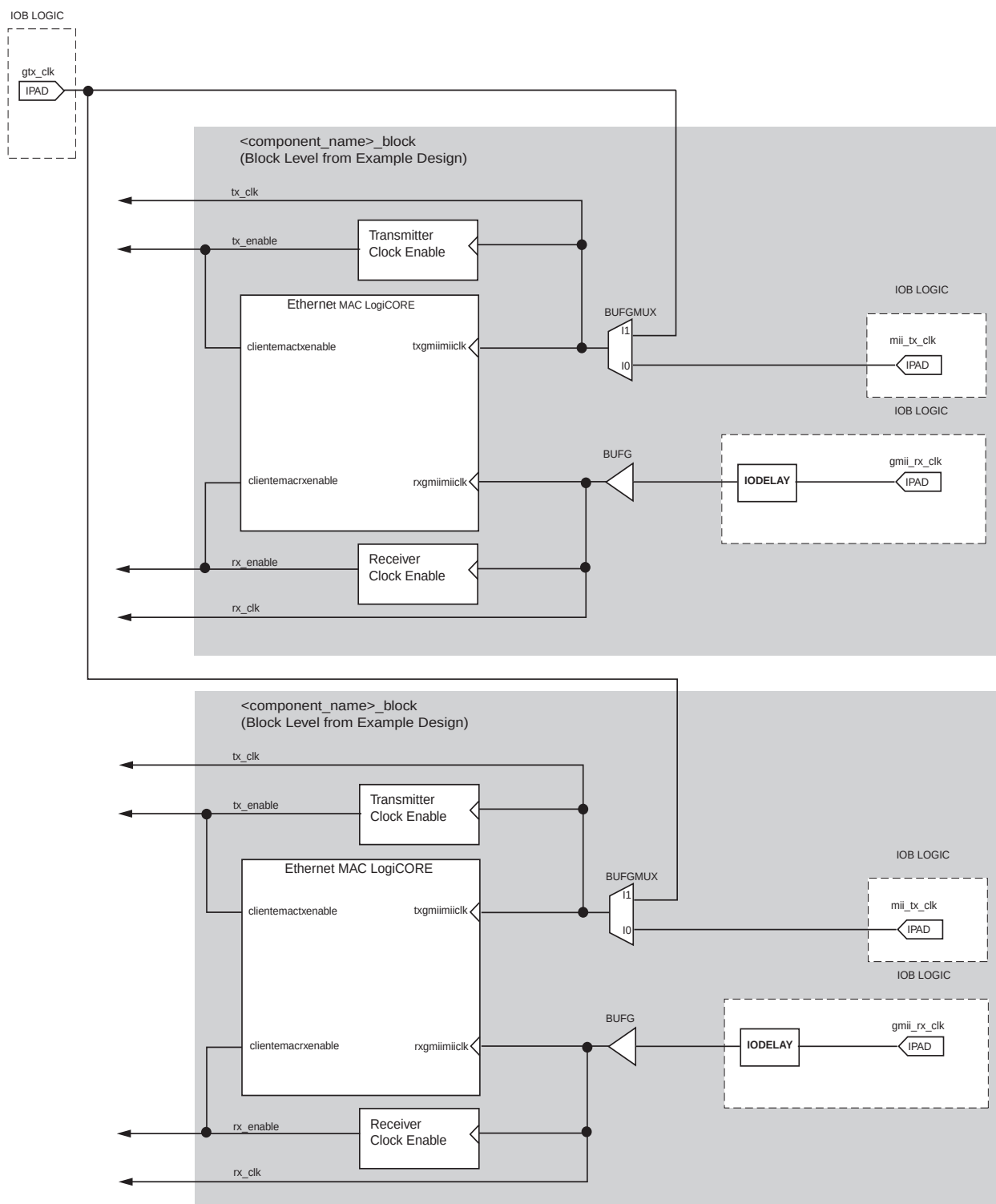


Figure 12-8: Clock Resource Sharing for 1-Gb/s GMII in Virtex-5 Devices

Multiple Core Instances in Virtex-4, Spartan-3, Spartan-3E, Spartan-3A and Spartan-3A DSP Devices

Figure 12-9 illustrates multiple instances of the core in Virtex-4, Spartan-3, Spartan-3E, Spartan-3A and Spartan-3A DSP devices. The 1 Gb/s transmitter reference clock source, `gtx_clk`, can be shared across all cores as illustrated. However, global transmitter and receiver clock resources cannot be shared and require independent BUFGMUX elements as illustrated.

Figure 12-9 illustrates only two cores. However, more can be added using the same principal. This is done by instantiating the cores using the block level (from the example design) and sharing the `gtx_clk` clock source across all instantiations.

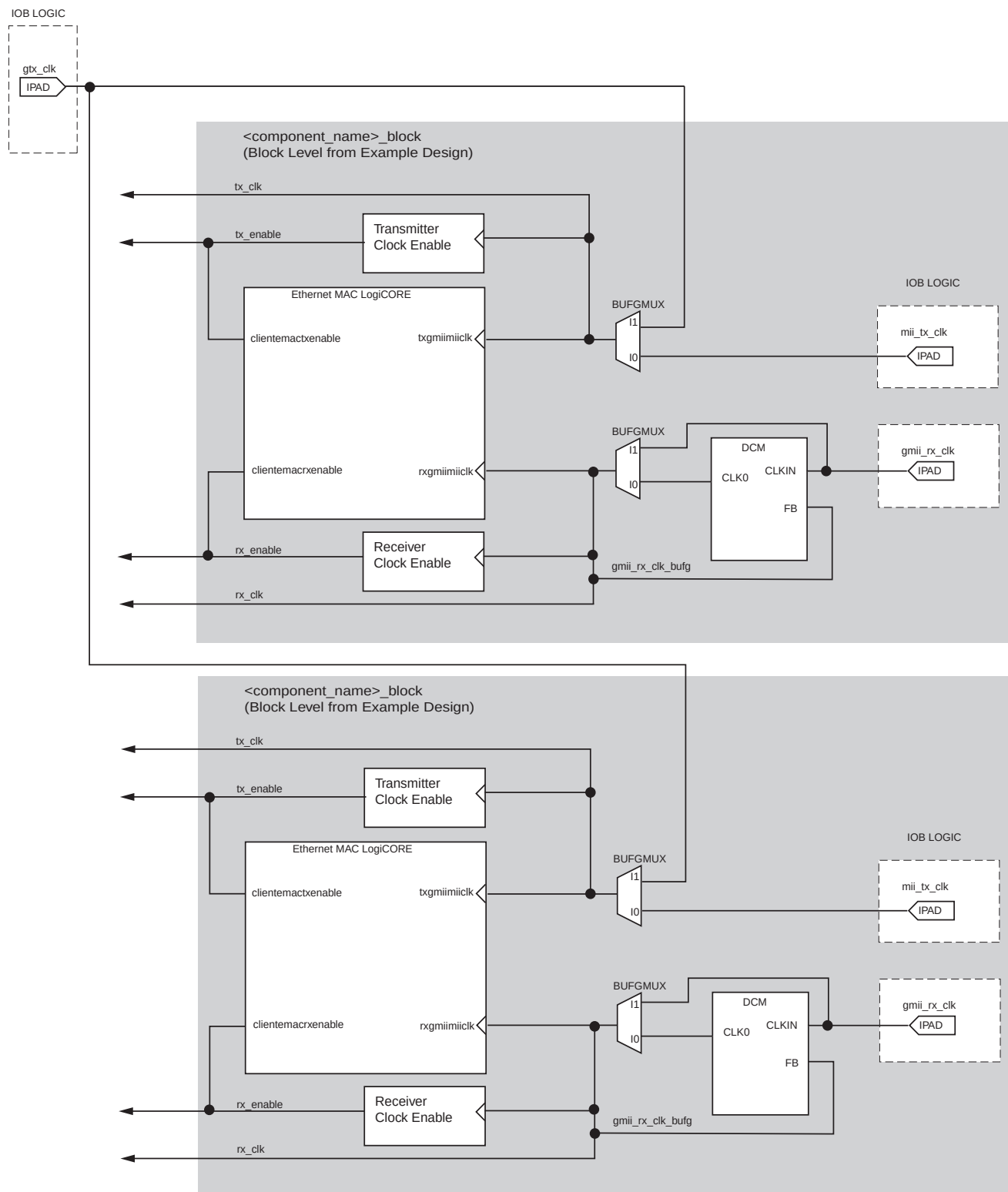


Figure 12-9: Clock Resource Sharing for 1-Gb/s GMII in Virtex-4, Spartan-3, Spartan-3E, Spartan-3A and Spartan-3A DSP Devices

Reduced Gigabit Media Independent Interface (RGMII)

The logic required to implement the RGMII logic is described in the following sections. Logical implementation varies for different device families. See the specific family section:

- [Virtex-6 Devices](#)
- [Spartan-6 Devices](#)
- [Virtex-5 Devices](#)
- [Virtex-4 Devices](#)
- [Spartan-3, Spartan-3A, and Spartan-3A DSP Devices](#)

Virtex-6 Devices

Transmitter Logic

The logic required to implement the RGMII transmitter logic is illustrated in [Figure 12-10](#). `gtx_clk` is a user-supplied 125 MHz reference clock source which is placed onto global clock routing to provide the clock for all transmitter logic, both within the core and for the client side logic which connects to the transmitter client I/F of the TEMAC.

As of version 4.1 of the core, this global 125 MHz is used to clock transmitter logic at all three ethernet speeds. The data rate difference between the three speeds is compensated for by the transmitter clock enable logic (the `enable_gen` module from the example design describes the required logic). The derived `tx_enable` signal must be used as a standard clock enable by all flip-flops in the Tx Client I/F logic. At 1 Gb/s operation the clock enable is high at all times. At 100 Mb/s the `tx_enable` signal is asserted for one 125 MHz clock cycle in every ten; at 10 Mb/s the `tx_enable` signal is asserted for one 125 MHz clock cycle in every hundred. The advantage of this approach is that it allows common transmitter global clocks to be shared across any number of instantiated cores; the operating speed is ensured solely by the clock enable logic specific to each instance of the core. See [Clock Resource Sharing](#).

[Figure 12-10](#) illustrates how to use the physical transmitter interface of the core to create an external RGMII. The signal names and logic shown in this figure exactly match those delivered with the example design. [Figure 12-10](#) shows that the output transmitter signals are registered in device IOBs, using DDR registers, before driving them to the device pads.

The logic required to forward the transmitter clock is also shown. This logic uses an IOB output Double-Data-Rate (DDR) register so that the clock signal produced incurs exactly the same delay as the data and control signals. However, the clock signal is then routed through an output delay element (IODELAY) before connecting to the device pad. The result of this is to create a 2 ns delay, which places the `rgmii_txc` forwarded clock in the centre of the data valid window for forwarded RGMII data and control signals when operating at 1 Gb/s ethernet speed. At 10 Mb/s and 100 Mb/s speeds, the `enable_gen` module will toggle the DDR input signals at the required frequency so that the forwarded `rgmii_txc` clock is always of the correct frequency for the forwarded data.

Receiver Logic

In this implementation, a BUFIO is used to provide the lowest form of clock routing delay from input clock to input RGMII Rx signal sampling at the device IOBs. Note, however, that this creates placement constraints: a BUFIO capable clock input pin must be selected, and all other input RGMII Rx signals must be placed in the respective BUFIO region. The *Virtex-6 FPGA User Guide* should be consulted.

The input clock is also placed onto regional clock routing using the BUFR component as illustrated. This regional clock then provides the clock for all receiver logic, both within the core and for the client side logic which connects to the receiver client I/F of the core.

The IODELAY elements can be adjusted to fine-tune the setup and hold times at the RGMII IOB input flip-flops. This will meet input setup and hold constraints at all three ethernet speeds. The delay is applied to the IODELAY element using constraints in the UCF; these can be edited if desired. See [Chapter 13, Constraining the Core](#).

Closely linked to the clock logic is the use of the rx_enable clock enable derivation. This must be used as a standard clock enable by all flip-flops in the Rx Client I/F logic. At 1 Gb/s operation the clock enable is high at all times. At 10 Mb/s and 100 Mb/s operation this clock enable is high on alternative clock cycles.

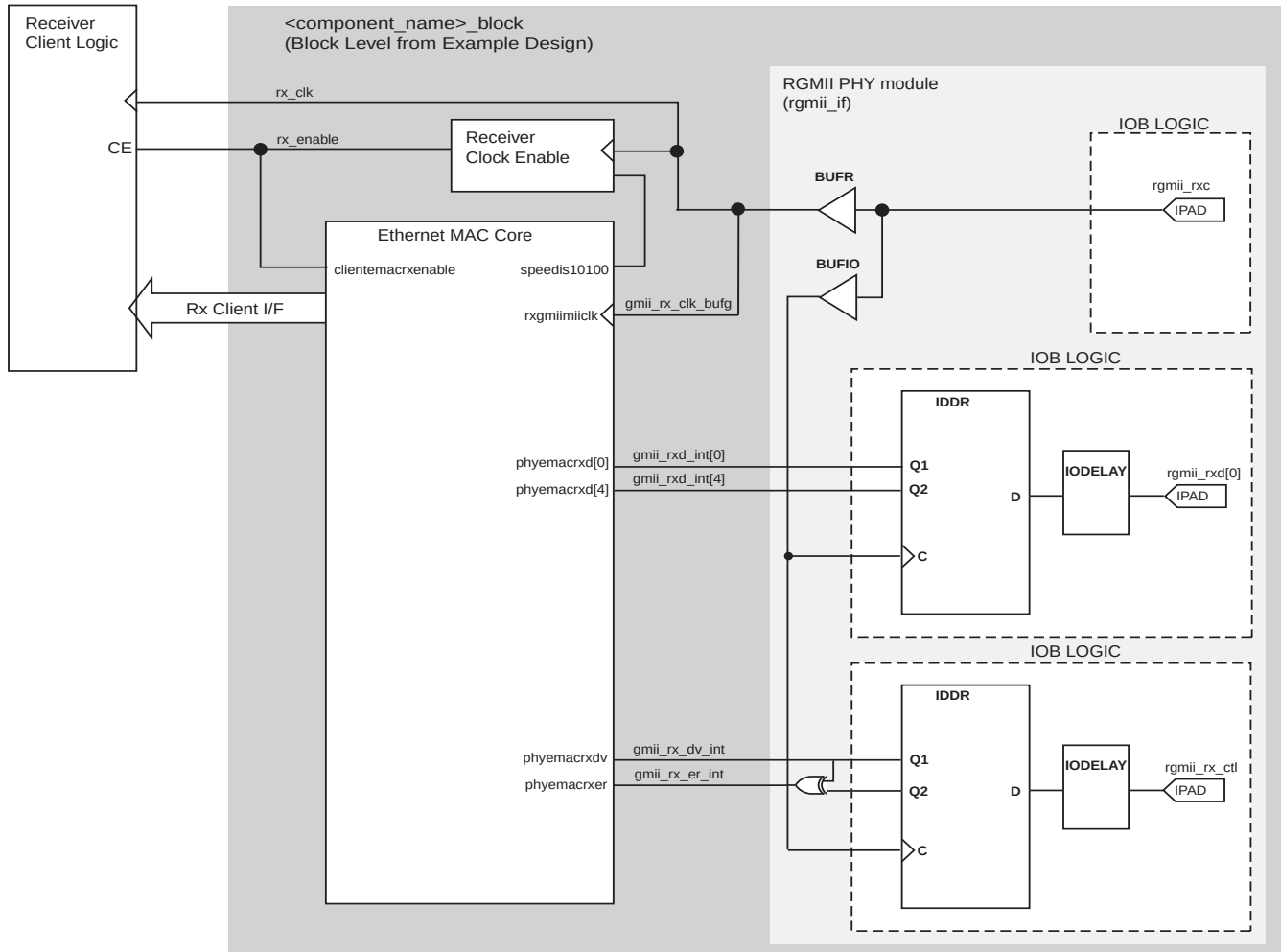


Figure 12-11: RGMII Receiver Logic and Clock Logic for Virtex-6 Devices

Clock Resource Sharing

Figure 12-12 illustrates clock resource sharing across multiple instantiations of the core when using RGMII in Virtex-6 devices. For all instantiations, `gtx_clk` can be shared between multiple cores, resulting in a common clock domain across the device.

The receiver clocks cannot be shared. Each core is provided with its own local version of `rgmii_rxc` from the connected external PHY device as illustrated.

Figure 12-12 illustrates only two cores. However, more can be added using the same principal. This is done by instantiating the cores using the block level (from the example design) and sharing `gtx_clk` across all instantiations. The receiver clock, which cannot be shared, is unique for every instance of the core.

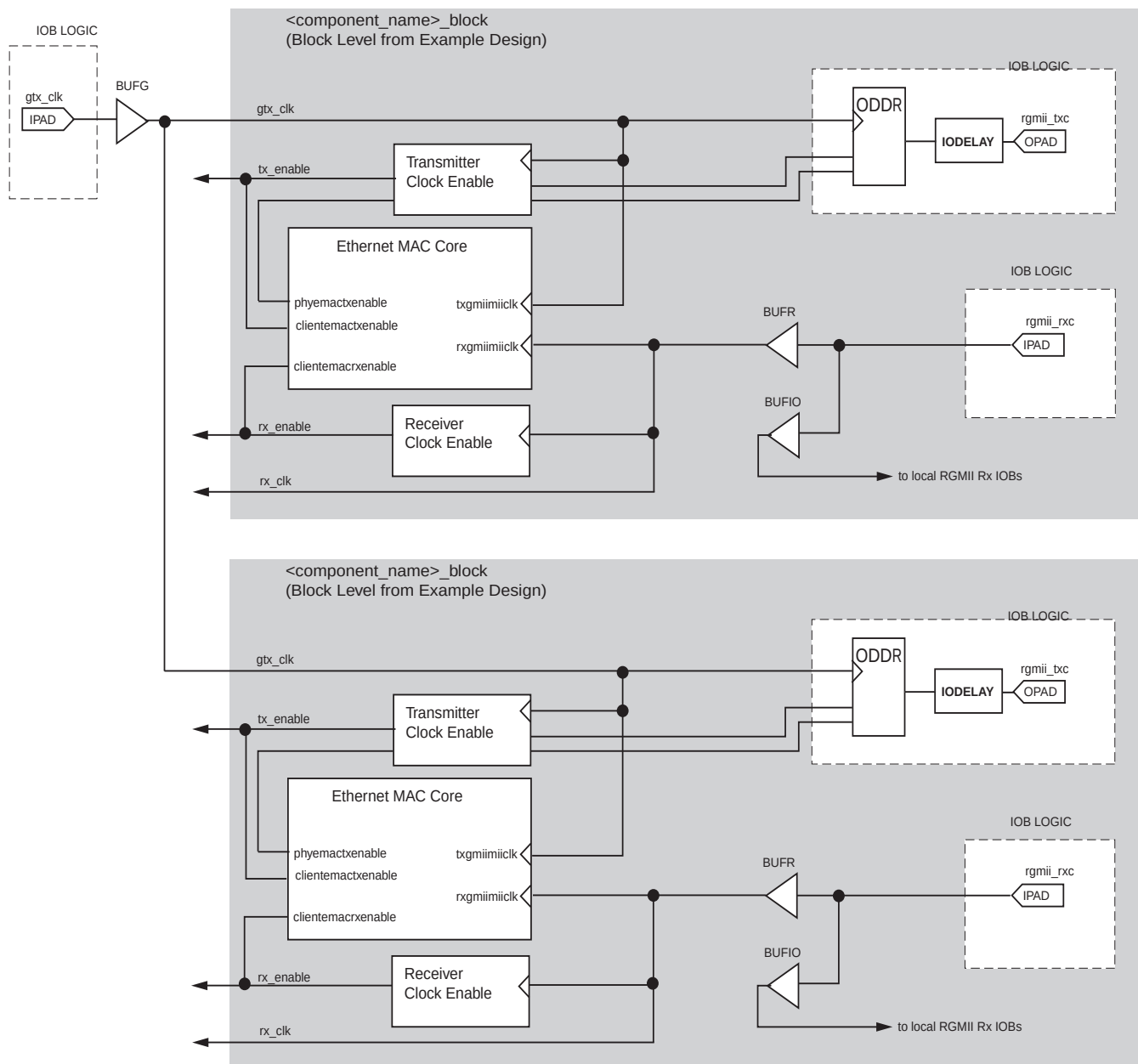


Figure 12-12: Clock Resource Sharing for 1-Gb/s RGMII in Virtex-6 Devices

Spartan-6 Devices

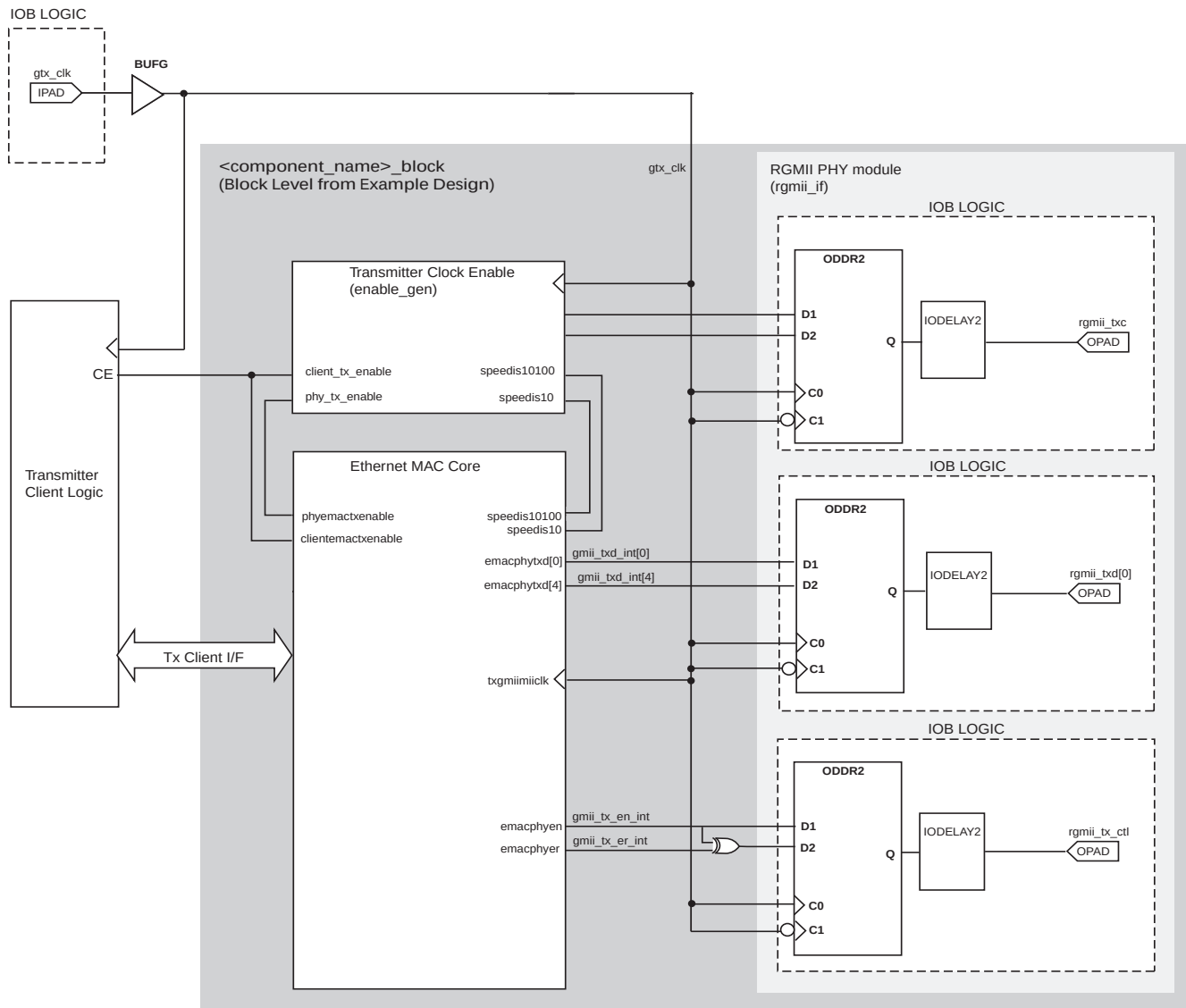
Transmitter Logic

The logic required to implement the RGMII transmitter logic is illustrated in [Figure 12-13](#). `gtx_clk` is a user-supplied 125 MHz reference clock source which is placed onto global clock routing to provide the clock for all transmitter logic, both within the core and for the client side logic which connects to the transmitter client I/F of the core.

As of version 4.1 of the core, this global 125 MHz is used to clock transmitter logic at all three ethernet speeds. The data rate difference between the three speeds is compensated for by the transmitter clock enable logic (the `enable_gen` module from the example design describes the required logic). The derived `tx_enable` signal must be used as a standard clock enable by all flip-flops in the Tx Client I/F logic. At 1 Gb/s operation the clock enable is high at all times. At 100 Mb/s the `tx_enable` signal is asserted for one 125 MHz clock cycle in every ten; at 10 Mb/s the `tx_enable` signal is asserted for one 125 MHz clock cycle in every hundred. The advantage of this approach is that it allows common transmitter global clocks to be shared across any number of instantiated cores; the operating speed is ensured solely by the clock enable logic specific to each instance of the core. See [Clock Resource Sharing](#).

[Figure 12-13](#) illustrates how to use the physical transmitter interface of the core to create an external RGMII. The signal names and logic shown in this figure exactly match those delivered with the example design. [Figure 12-13](#) shows that the output transmitter signals are registered in device IOBs, using DDR registers, before driving them to the device pads.

The logic required to forward the transmitter clock is also shown. This logic uses an IOB output Double-Data-Rate (DDR) register so that the clock signal produced incurs exactly the same delay as the data and control signals. However, the clock signal is then routed through an output delay element (IODELAY2) before connecting to the device pad. The result of this is to create a 2 ns delay, which places the `rgmii_txc` forwarded clock in the centre of the data valid window for forwarded RGMII data and control signals when operating at 1 Gb/s ethernet speed. At 10 Mb/s and 100 Mb/s speeds, the `enable_gen` module will toggle the DDR input signals at the required frequency so that the forwarded `rgmii_txc` clock is always of the correct frequency for the forwarded data.



Closely linked to the clock logic is the use of the rx_enable clock enable derivation. This must be used as a standard clock enable by all flip-flops in the Rx Client I/F logic. At 1 Gb/s operation the clock enable is high at all times. At 10 Mb/s and 100 Mb/s operation this clock enable is high on alternative clock cycles.

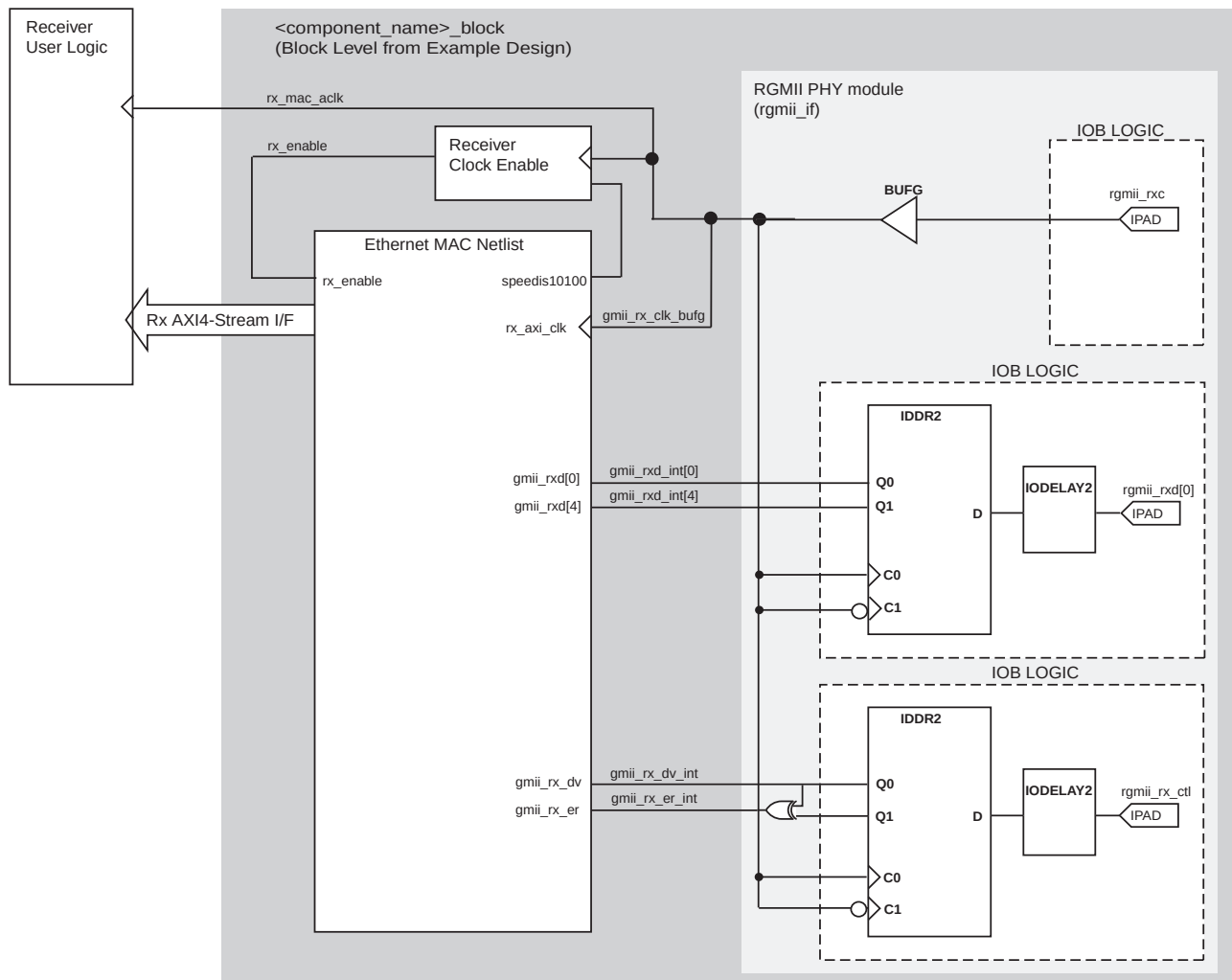


Figure 12-14: RGMII Receiver Logic and Clock Logic for Spartan-6 Devices

Clock Resource Sharing

Figure 12-15 illustrates clock resource sharing across multiple instantiations of the core when using RGMII in Spartan-6 devices. For all instantiations, `gtx_clk` can be shared between multiple cores, resulting in a common clock domain across the device.

The receiver clocks cannot be shared. Each core is provided with its own local version of `rgmii_rxc` from the connected external PHY device as illustrated.

Figure 12-15 illustrates only two cores. However, more can be added using the same principal. This is done by instantiating the cores using the block level (from the example design) and sharing `gtx_clk` across all instantiations. The receiver clock, which cannot be shared, is unique for every instance of the core.



Virtex-5 Devices

Transmitter Logic

The logic required to implement the RGMII transmitter logic is illustrated in [Figure 12-16](#). `gtx_clk` is a user-supplied 125 MHz reference clock source which is placed onto global clock routing to provide the clock for all transmitter logic, both within the core and for the client side logic which connects to the transmitter client I/F of the core.

As of version 4.1 of the core, this global 125 MHz is used to clock transmitter logic at all three ethernet speeds. The data rate difference between the three speeds is compensated for by the transmitter clock enable logic (the `enable_gen` module from the example design describes the required logic). The derived `tx_enable` signal must be used as a standard clock enable by all flip-flops in the Tx Client I/F logic. At 1 Gb/s operation the clock enable is high at all times. At 100 Mb/s the `tx_enable` signal is asserted for one 125 MHz clock cycle in every ten; at 10 Mb/s the `tx_enable` signal is asserted for one 125 MHz clock cycle in every hundred. The advantage of this approach is that it allows common transmitter global clocks to be shared across any number of instantiated cores; the operating speed is ensured solely by the clock enable logic specific to each instance of the core. See [Clock Resource Sharing](#).

[Figure 12-16](#) illustrates how to use the physical transmitter interface of the core to create an external RGMII. The signal names and logic shown in this figure exactly match those delivered with the example design. [Figure 12-16](#) shows that the output transmitter signals are registered in device IOBs, using DDR registers, before driving them to the device pads.

The logic required to forward the transmitter clock is also shown. This logic uses an IOB output Double-Data-Rate (DDR) register so that the clock signal produced incurs exactly the same delay as the data and control signals. However, the clock signal is then routed through an output delay element (IODELAY) before connecting to the device pad. The result of this is to create a 2 ns delay, which places the `rgmii_txc` forwarded clock in the centre of the data valid window for forwarded RGMII data and control signals when operating at 1 Gb/s ethernet speed. At 10 Mb/s and 100 Mb/s speeds, the `enable_gen` module will toggle the DDR input signals at the required frequency so that the forwarded `rgmii_txc` clock is always of the correct frequency for the forwarded data.

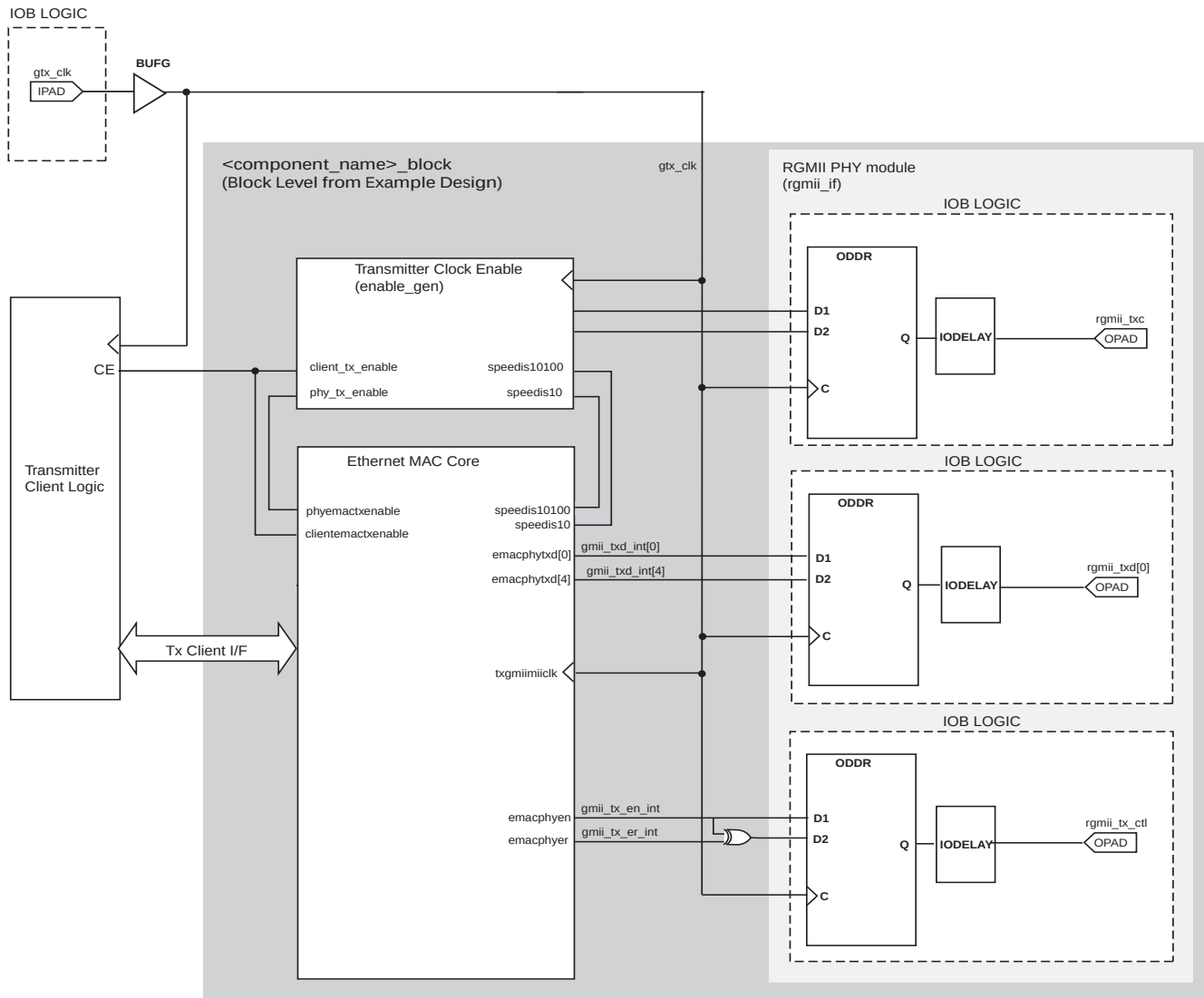


Figure 12-16: RGMI Transmitter Logic and Clock Logic for Virtex-5 Devices

Receiver Logic

Three possible solutions are available for Virtex-5 devices:

1. A DCM can be used on the `rgmii_rxc` clock path, exactly as described for [Spartan-3, Spartan-3A, and Spartan-3A DSP Devices](#).
2. Regional and BUFIO clock routing can be used as described in the section for [Virtex-6 Devices](#).
3. Input Delay Elements can be used on the RGMII data and clock path, using global clock routing and as illustrated in [Figure 12-17](#).

The design for Case 3 provides a simpler solution than Case 1 and fully supports all operating modes, it also allows for a more flexible pinout than for Case 2. It has therefore been chosen as the default example design and is described in this section.

As illustrated in [Figure 12-17](#), IODELAY components can be used on the data and control signals. These can be used to either shift the input clock, `rgmii_rxc`, or the data and control signals to meet the setup and hold requirements at all ethernet speeds and to allow for any bus skew across the data and control inputs. The IODELAY components are used in fixed delay mode, where the attribute `IDELAY_VALUE` determines the tap delay value. An `IDELAYCTRL` primitive must be instantiated for this mode of operation. See the *Virtex-5 FPGA User Guide* for details about using the `IDELAYCTRL` and `IDELAY` components. For details on setting the `IDELAY_VALUE`s, see [Chapter 13, Constraining the Core](#)

The `rgmii_rxc` clock received through the IODELAY component, placed onto global clock routing, is then used to provide the clock for all receiver logic, both within the core and for the client side logic which connects to the receiver client I/F of the core.

Closely linked to the clock logic is the use of the `rx_enable` clock enable derivation. This must be used as a standard clock enable by all flip-flops in the Rx Client I/F logic. At 1 Gb/s operation the clock enable is high at all times. At 10 Mb/s and 100 Mb/s operation this clock enable is high on alternative clock cycles.

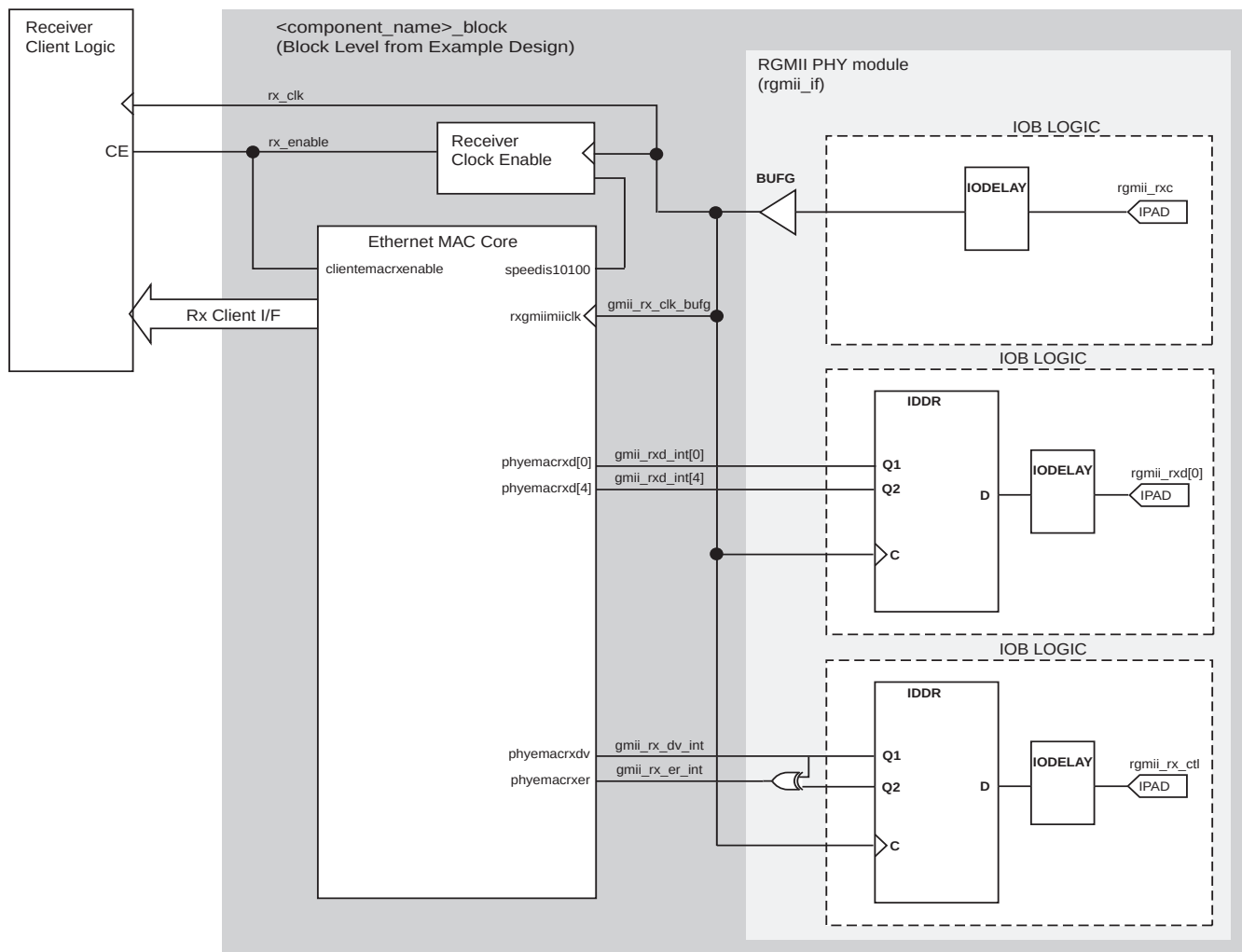


Figure 12-17: RGMII Receiver Logic and Clock Logic for Virtex-5 Devices

Clock Resource Sharing

Figure 12-18 illustrates clock resource sharing across multiple instantiations of the core when using RGMII in Virtex-5 devices. For all instantiations, `gtx_clk` can be shared between multiple cores, resulting in a common clock domain across the device.

The receiver clocks cannot be shared. Each core is provided with its own local version of `rgmii_rxc` from the connected external PHY device as illustrated.

Figure 12-18 illustrates only two cores. However, more can be added using the same principal. This is done by instantiating the cores using the block level (from the example design) and sharing `gtx_clk` across all instantiations. The receiver clock, which cannot be shared, is unique for every instance of the core.



Virtex-4 Devices

Transmitter Logic

The logic required to implement the RGMII transmitter logic is illustrated in [Figure 12-19](#). `gtx_clk` is a user-supplied 125 MHz reference clock source. As illustrated, this is routed to a DCM before being placed onto global clock routing to provide the clock for all transmitter logic, both within the core and for the client side logic which connects to the transmitter client I/F of the core.

As of version 4.1 of the core, this global 125 MHz is used to clock transmitter logic at all three ethernet speeds. The data rate difference between the three speeds is compensated for by the transmitter clock enable logic (the `enable_gen` module from the example design describes the required logic). The derived `tx_enable` signal must be used as a standard clock enable by all flip-flops in the Tx Client I/F logic. At 1 Gb/s operation the clock enable is high at all times. At 100 Mb/s the `tx_enable` signal is asserted for one 125 MHz clock cycle in every ten; at 10 Mb/s the `tx_enable` signal is asserted for one 125 MHz clock cycle in every hundred. The advantage of this approach is that it allows common transmitter global clocks to be shared across any number of instantiated cores; the operating speed is ensured solely by the clock enable logic specific to each instance of the core. See [Clock Resource Sharing](#).

[Figure 12-19](#) illustrates how to use the physical transmitter interface of the core to create an external RGMII. The signal names and logic shown in this figure exactly match those delivered with the example design for a Virtex-4 device when the RGMII is selected. Observe that the output transmitter signals are registered in device IOBs, using DDR registers, before driving them to the device pads.

The logic required to forward the transmitter clock is also shown. This logic uses an IOB output Double-Data-Rate (DDR) register so that the clock signal produced incurs exactly the same delay as the data and control signals. However, the clock source used for this logic is derived from the DCM CLK90 output. The result of this is to create a clock signal which is 90 degrees out of phase with respect to the CLK0 equivalent when operating at 1 Gb/s ethernet speed. This places the `rgmii_txc` forwarded clock in the centre of the data valid window for forwarded RGMII data and control signals. At 10 Mb/s and 100 Mb/s speeds, the `enable_gen` module will toggle the DDR input signals at the required frequency so that the forwarded `rgmii_txc` clock is always of the correct frequency for the forwarded data.

The use of the BUFGMUX shown, with one input connected to the DCM CLK90 output is included so that a reliable 125 MHz clock source is always provided on global routing (when the DCM is held in reset, the DCM input clock is instead selected). This is required to always provide a reliable clock for the receiver logic DCM; see [DCM Reset Circuitry](#).

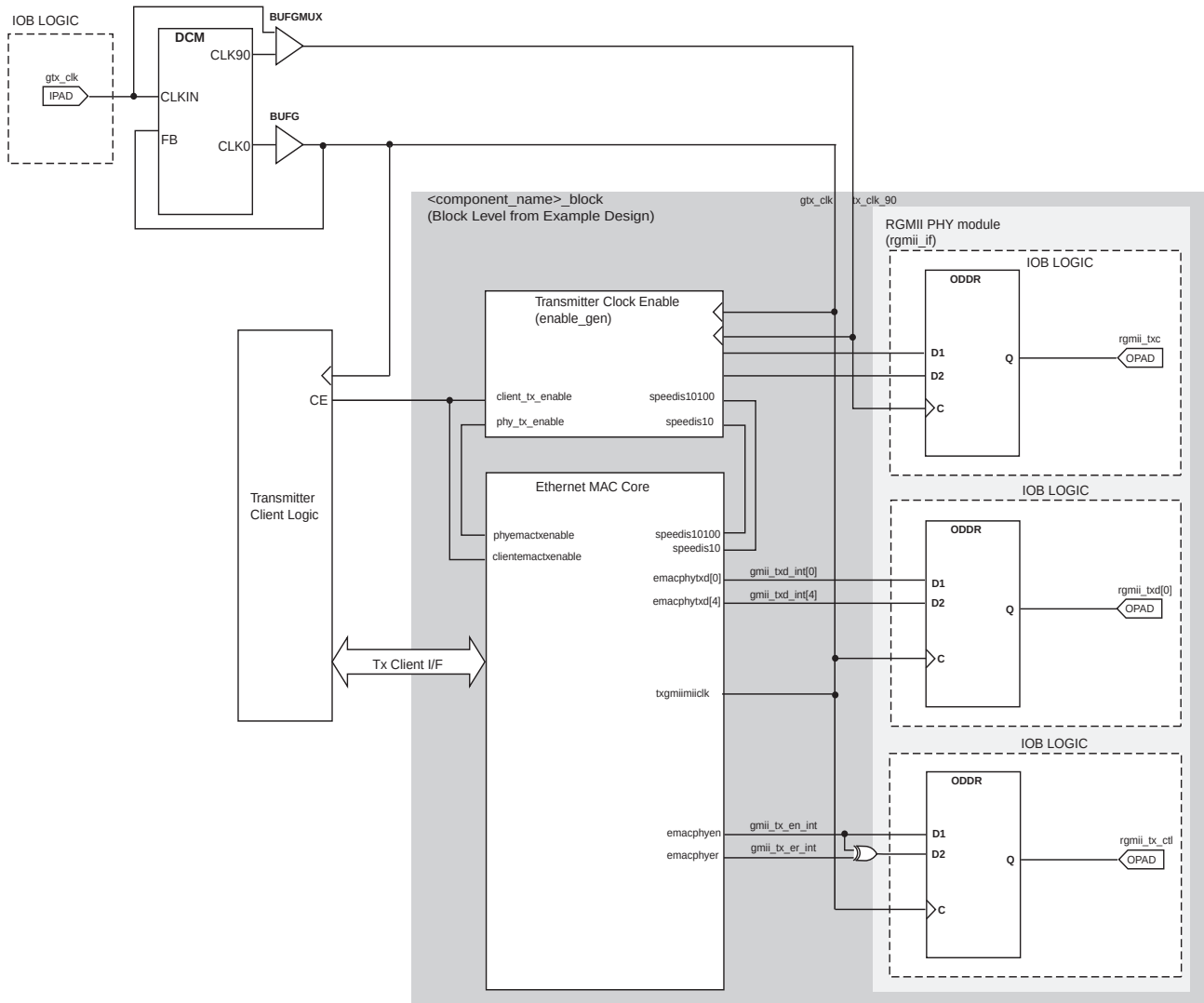


Figure 12-19: RGMII Transmitter Logic and Clock Logic for Virtex-4 devices

Receiver Logic

In Virtex-4 devices, a DCM must be used on the `rgmii_rxc` clock path as illustrated in [Figure 12-20](#) to meet the RGMII input setup and hold requirements when operating at 1 Gb/s. Phase shifting can then be applied to the DCM to fine-tune the setup and hold times of the input RGMII receiver signals which are sampled at the RGMII IOB input flip-flops; fixed-phase shift is applied to the DCM via the example UCF for the example design. See [Chapter 13, Constraining the Core](#).

A limitation of using a DCM on this interface is that, as a DCM is sensitive to a change in input clock, 1 Gb/s Half Duplex is not supported. This is due to the nature of the rx clock in this mode of operation as it is sourced by whichever device has control of the media and a PPM shift is to be expected. This causes the DCM to lose lock, rendering the receive interface inactive.

When operating at 10 Mb/s and 100 Mb/s, the DCM is bypassed and held in reset. This is achieved using the BUFGMUX global clock multiplexer as illustrated. It is a requirement to bypass the DCM since the clock frequency of `rgmii_rxc` is 2.5 MHz when operating at 10 Mb/s; this is below the DCM low frequency threshold for Spartan-3 devices.

The output from the BUFGMUX provides the receiver clock for the core and client logic as illustrated.

Closely linked to the clock logic is the use of the `rx_enable` clock enable derivation. This must be used as a standard clock enable by all flip-flops in the Rx Client I/F logic. At 1 Gb/s operation the clock enable is high at all times. At 10 Mb/s and 100 Mb/s operation this clock enable is high on alternative clock cycles.

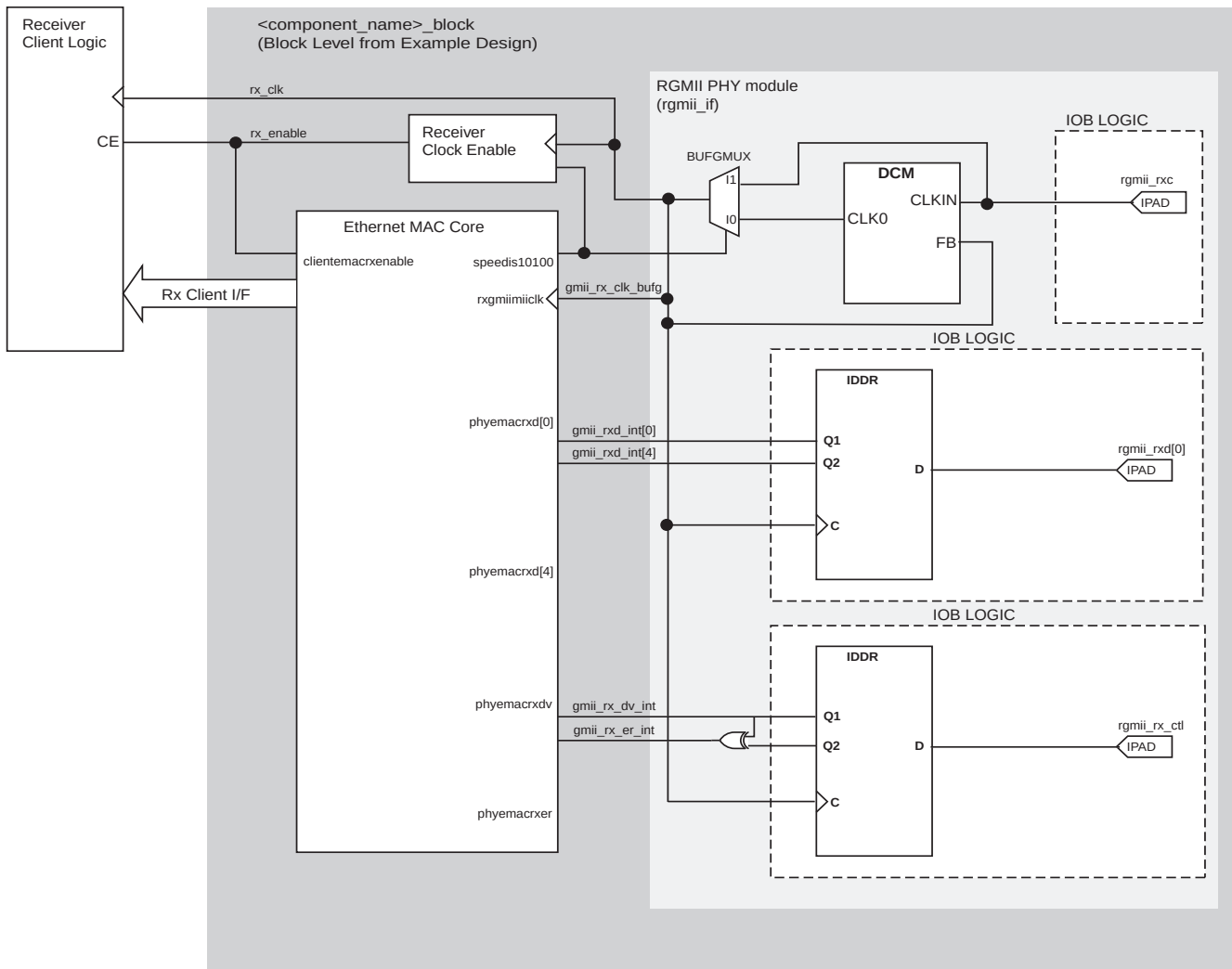


Figure 12-20: RGMII Receiver Logic and Clock Logic for Virtex-4 Devices

DCM Reset Circuitry

A DCM reset module, not illustrated in Figure 12-20, is also present and is instantiated in the example design next to the DCM. Because this logic must be reliable whatever the reset/locked status of the DCM, the module requires a reliable reference clock. In the example design for RGMII, the global transmitter clock is therefore used. This reset circuitry generates an appropriate reset pulse, based on the family, for the receiver DCM under the following conditions:

- The locked signal from the DCM is constantly monitored. Following a high-to-low transition on this signal, indicating that the DCM has lost lock, a reset is issued.
- A timeout counter is enabled when the DCM is in the loss of lock state. If, following the timeout period, the DCM has not obtained lock, another DCM reset is issued. This timeout counter will time a > 1 ms interval. This timeout functionality is required for DCMs connected to Ethernet PHYs because the PHYs may source discontinuous clocks under certain network conditions (for example, when no ethernet cable is connected).

For Virtex-4 devices, the generated reset from within the DCM reset module is further complicated. Here the DCM reset pulse duration must be asserted for a minimum of 200 ms (see the *Virtex-4 Data Sheet*). Consequently, a 200 ms duration timer is also included in the DCM reset module to extend the reset pulse. This reset signal is output from the DCM reset module and **should** be routed to the DCM reset input port.

Caution! In the example designs for Virtex-4 devices, the 200 ms reset pulse from the DCM reset module is NOT connected directly to the DCM; this signal must be connected when implementing the core in real hardware.

For Virtex-4 FPGA example designs, the reason why the 200 ms input is not routed directly to the DCM is so that simulations can occur in a timely manner.

The 200 ms reset pulse takes the signal name `reset_200ms` in the example design HDL files. It is routed from the DCM reset module instantiation up through all levels of hierarchy to the top level. It is then left unconnected in the example design instantiation from within the demonstration test bench. Furthermore, an accompanying signal, `reset_200ms_in`, is routed down from the top level of the example design hierarchy to the DCM instantiation, where it is connected to the DCM reset. From the demonstration test bench, this `reset_200ms_in` signal is driven for a short duration to enable fast simulation start-up. However, to re-iterate, when implementing the design in real hardware, the DCM reset signal must be connected correctly:

- This can be achieved by connecting the `reset_200ms` signal to the `reset_200ms_in` signal at any level of example design HDL hierarchy.

Clock Resource Sharing

Figure 12-21 illustrates clock resource sharing across multiple instantiations of the core when using RGMII in Virtex-4 devices. `gtx_clk` should be routed to a single DCM; the DCM outputs `CLK0` and `CLK90` should both be connected onto global clock routing resources by using `BUFGMUX`/`BUFG` components as illustrated. These two transmitter global clocks can be shared between multiple cores.

The receiver clocks cannot be shared. Each core is provided with its own local version of `rgmii_rxc` from the connected external PHY device as illustrated.

Figure 12-21 illustrates only two cores. However, more can be added using the same principal. This is done by instantiating the cores using the block level (from the example design) and sharing `gtx_clk` across all instantiations. The receiver clock, which cannot be shared, is unique for every instance of the core.

Spartan-3, Spartan-3A, and Spartan-3A DSP Devices

Transmitter Logic

The logic required to implement the RGMII transmitter logic is illustrated in [Figure 12-22](#). `gtx_clk` is a user-supplied 125 MHz reference clock source. As illustrated, this is routed to a DCM before being placed onto global clock routing to provide the clock for all transmitter logic, both within the core and for the client-side logic which connects to the transmitter client I/F of the core.

As of version 4.1 of the core, this global 125 MHz is used to clock transmitter logic at all three ethernet speeds. The data rate difference between the three speeds is compensated for by the transmitter clock enable logic (the `enable_gen` module from the example design describes the required logic). The derived `tx_enable` signal must be used as a standard clock enable by all flip-flops in the Tx Client I/F logic. At 1-Gb/s operation the clock enable is high at all times. At 100 Mb/s the `tx_enable` signal is asserted for one 125 MHz clock cycle in every ten; at 10 Mb/s the `tx_enable` signal is asserted for one 125 MHz clock cycle in every hundred. The advantage of this approach is that it allows common transmitter global clocks to be shared across any number of instantiated cores; the operating speed is ensured solely by the clock enable logic specific to each instance of the core. See [Clock Resource Sharing](#).

[Figure 12-22](#) also illustrates how to use the physical transmitter interface of the core to create an external RGMII. The signal names and logic shown in this figure exactly match those delivered with the example design for a Spartan-3 device when the RGMII is selected. [Figure 12-22](#) shows that the output transmitter signals are registered in device IOBs, using DDR registers, before driving them to the device pads.

The logic required to forward the transmitter clock is also shown. This logic uses an IOB output Double-Data-Rate (DDR) register so that the clock signal produced incurs exactly the same delay as the data and control signals. However, the clock source used for this logic is derived from the DCM CLK90 output. The result of this is to create a clock signal which is 90 degrees out of phase with respect to the CLK0 equivalent when operating at 1-Gb/s ethernet speed. This places the `rgmii_txc` forwarded clock in the centre of the data valid window for forwarded RGMII data and control signals. At 10 Mb/s and 100 Mb/s speeds, the `enable_gen` module toggles the DDR input signals at the required frequency so that the forwarded `rgmii_txc` clock is always of the correct frequency for the forwarded data.

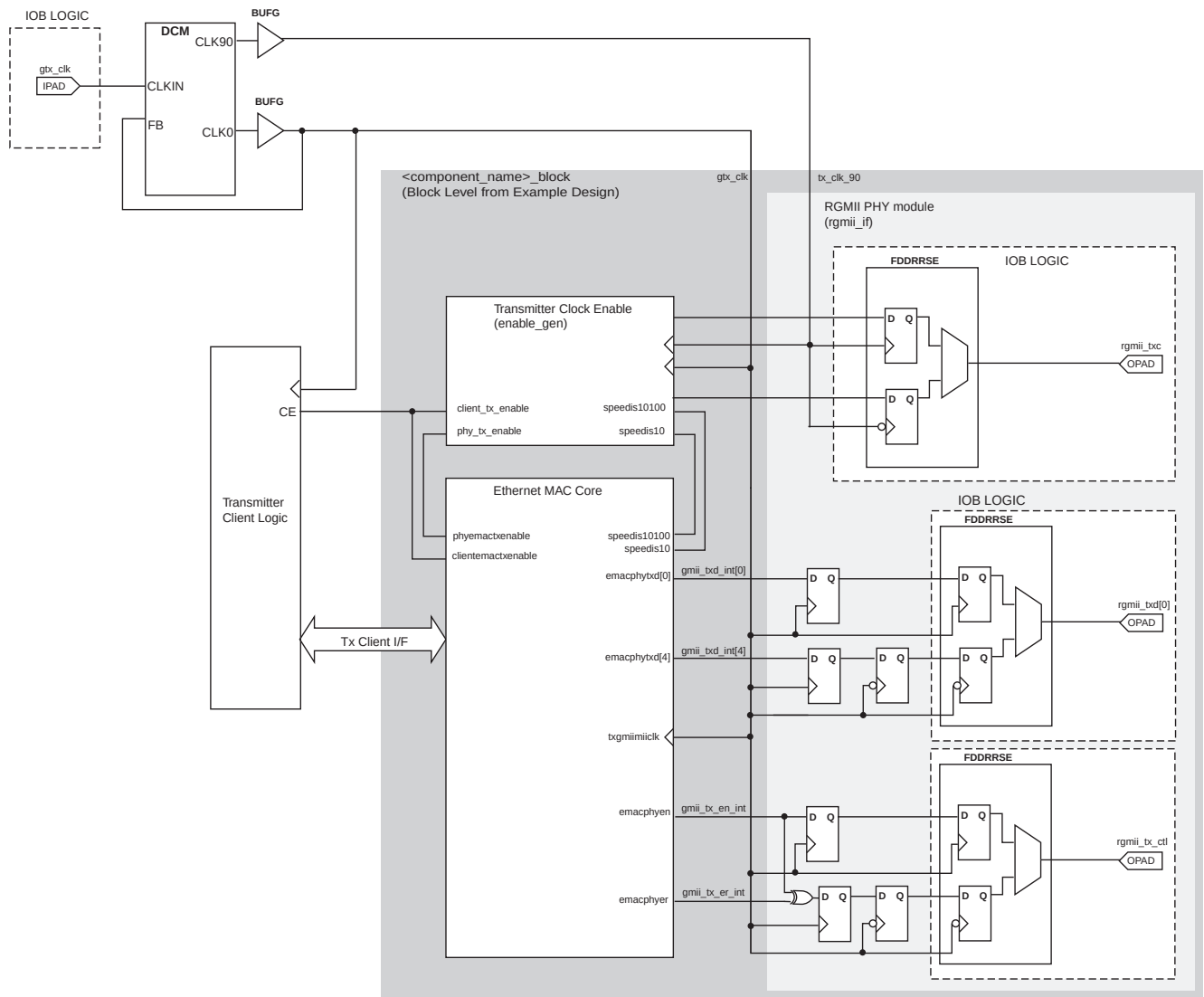


Figure 12-22: RGMII Transmitter Logic and Clock Logic for Spartan-3, Spartan-3A and Spartan-3A DSP Devices

Receiver Logic

In Spartan-3, Spartan-3A and Spartan-3A DSP devices, a DCM must be used on the `rgmii_rxc` clock path as illustrated in [Figure 12-23](#) to meet the RGMII input setup and hold requirements when operating at 1 Gb/s. Phase shifting can then be applied to the DCM to fine-tune the setup and hold times of the input RGMII receiver signals which are sampled at the RGMII IOB input flip-flops; fixed-phase shift is applied to the DCM via the example UCF for the example design. See [Chapter 13, Constraining the Core](#).

A limitation of using a DCM on this interface is that, as a DCM is sensitive to a change in input clock, 1 Gb/s Half Duplex is not supported. This is due to the nature of the rx clock in this mode of operation as it is sourced by whichever device has control of the media and a PPM shift is to be expected. This causes the DCM to lose lock, rendering the receive interface inactive.

When operating at 10 Mb/s and 100 Mb/s, the DCM is bypassed and held in reset. This is achieved using the BUFGMUX global clock multiplexer as illustrated. It is a requirement to bypass the DCM since the clock frequency of `rgmii_rxc` is 2.5 MHz when operating at 10 Mb/s: this is below the DCM low frequency threshold for Spartan-3 devices.

The output from the BUFGMUX provides the receiver clock for the core and client logic as illustrated.

Closely linked to the clock logic is the use of the `rx_enable` clock enable derivation. This must be used as a standard clock enable by all flip-flops in the Rx Client I/F logic. At 1 Gb/s operation the clock enable is high at all times. At 10 Mb/s and 100 Mb/s operation this clock enable is high on alternative clock cycles.

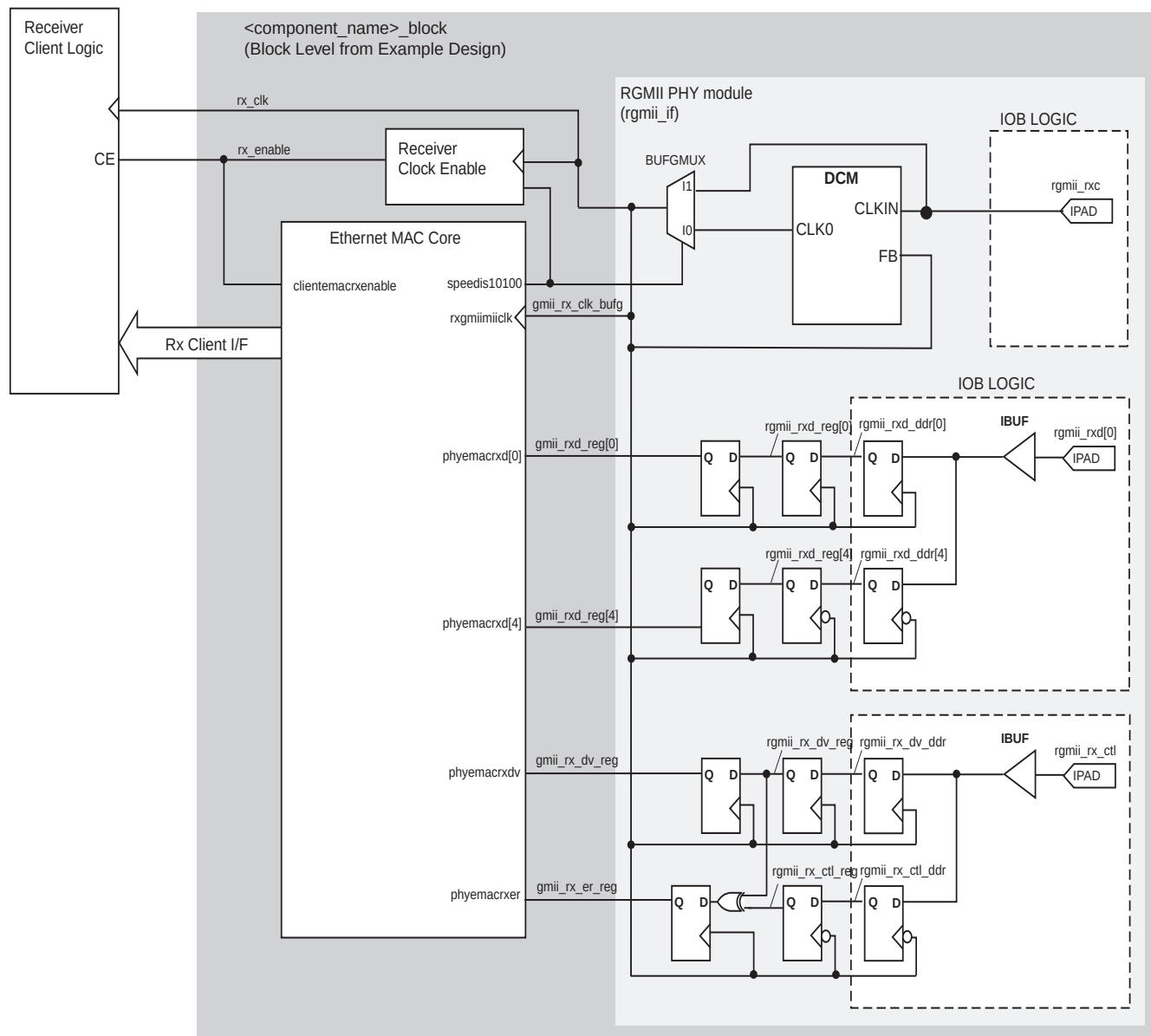


Figure 12-23: RGMII Receiver Logic and Clock Logic for Spartan-3E, Spartan-3A and Spartan-3A DSP Devices

DCM Reset Circuitry

A DCM reset module, not illustrated in [Figure 12-23](#), is also present and is instantiated in the example design next to the DCM. Because this logic must be reliable whatever the reset/locked status of the DCM, the module requires a reliable reference clock. In the example design for RGMII, the global transmitter clock is therefore used. This reset circuitry generates an appropriate reset pulse, based on the family, for the receiver DCM under the following conditions:

- The locked signal from the DCM is constantly monitored. Following a high-to-low transition on this signal, indicating that the DCM has lost lock, a reset is issued.
- A timeout counter is enabled when the DCM is in the loss of lock state. If, following the timeout period, the DCM has not obtained lock, another DCM reset is issued. This timeout counter will time a > 1 ms interval. This timeout functionality is required for DCMs connected to Ethernet PHYs because the PHYs may source discontinuous clocks under certain network conditions (for example, when no ethernet cable is connected).

For Spartan-3 devices, the reset pulse is transferred into the DCM input clock (`gmii_rx_clk` from [Figure 12-23](#)). Here it is extended to three DCM clock periods duration and routed to the reset input of the DCM.

Clock Resource Sharing

[Figure 12-24](#) illustrates clock resource sharing across multiple instantiations of the core when using RGMII in Spartan-3, Spartan-3A and Spartan-3A DSP devices. `gtx_clk` should be routed to a single DCM; the DCM outputs CLK0 and CLK90 should both be connected onto global clock routing resources by using BUFG components. These two transmitter global clocks can be shared between multiple cores as illustrated.

The receiver clocks cannot be shared. Each core is provided with its own local version of `rgmii_rxc` from the connected external PHY device as illustrated.

[Figure 12-24](#) illustrates only two cores. However, more can be added using the same principal. This is done by instantiating the cores using the block level (from the example design) and sharing `gtx_clk` across all instantiations. The receiver clock, which cannot be shared, is unique for every instance of the core.

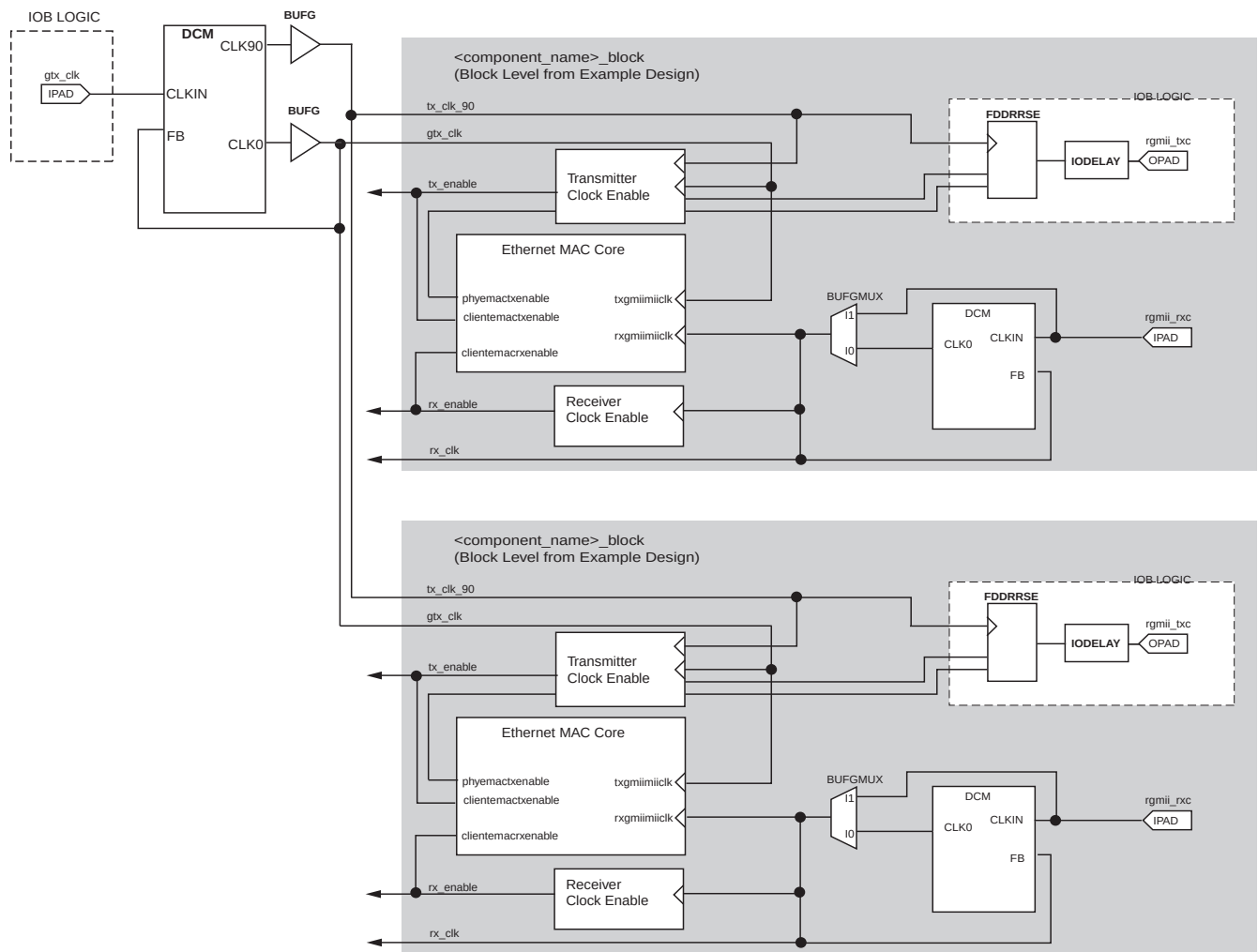


Figure 12-24: Clock Resource Sharing for 1-Gb/s RGMII in Spartan-3, Spartan-3A and Spartan-3A DSP Devices

Constraining the Core

This chapter defines the constraint requirements of the TEMAC solution. An example UCF is provided with the HDL example design to provide samples of constraint requirements for the design. See [Chapter 17, Detailed Example Design](#) for details.

Device, Package, and Speed Grade Selection

The core can be implemented in Virtex®-6, Spartan®-6, Virtex-5, Virtex-4, Spartan-3, Spartan-3E, and Spartan-3A and devices with the following attributes:

- Large enough to accommodate the core
- Contains a sufficient number of IOBs
- Device has a supported speed grade:

Table 13-1: Supported Speed Grades

Device Family	Speed Grade
Virtex-6	-1 or faster
Virtex-6 Lower Power	-1L or faster
Spartan-6	-2 or faster
Virtex-5	-1 or faster
Virtex-4	-10 or faster
Spartan-3/3E	-4 or faster
Spartan-3A/3AN/3A DSP	-4 or faster

I/O Location Constraints

No specific I/O location constraints are required. However, ensure to use the correct type of dedicated clock input pins for clock inputs and to abide by the device bank rules when using SelectIO™ interface logic at different voltage standards.

Additionally, when employing BUFIO and BUFR regional clock routing (provided for the Virtex-6 FPGA example design and available in all Virtex devices), ensure that a BUFIO capable clock input pin is selected for input clock sources, and that all related input synchronous data signals are placed in the respective BUFIO region. The device User Guides should be consulted.

Note: The example designs delivered by the CORE Generator™ software to accompany the core netlist contain I/O placement constraints. These are provided as an example only and should be edited for specific customer placements.

Placement Constraints

The example designs delivered by the CORE Generator software to accompany the core netlist may contain placement information for the global clock buffers and DCMs. These are provided as an example only and can be removed or edited for specific customer placements.

Timing Constraints

Timing Constraints are provided in the UCF delivered with the HDL example design for the core. The remainder of this chapter attempts to describe the constraints that are provided in this file; constraints relating to the core netlist and for the block level of the example design (GMII/MII and RGMII logic) are discussed.

However, because the core can be generated with a large number of clock logic variations, should any inconsistency arise between the UCF and constraints described in this chapter, then the constraints provided in the example design UCF should take precedent.

PERIOD(s) for Clock Nets

Transmitter Clock Constrains

Depending on the selected device family, the selected physical interface, and the supported ethernet speeds, a wide variation of required clock period constraint syntax exists. However, the UCF provided with the example design always provides the correct constraints for the generated example design and so this file should be used for reference. Do not relax these clock period constraints.

Transmitter clock period constraints are always provided until the following comment heading in the UCF:

```
#####
# TX Clock period Constraints                                     #
#####
```

The following syntax provides an example. This is taken from a tri-speed capable Spartan-3 FPGA design using GMII:

```
# Transmitter clock period constraints: please do not relax
NET "gtx_clk*"          TNM_NET = "clk_gtx";
TIMESPEC "TS_gtx_clk"    = PERIOD "clk_gtx" 8000 ps HIGH 50 %;

NET "*tx_gmii_clk*"      TNM_NET = "clk_tx_gmii";
TIMESPEC "TS_tx_clk_gmii" = PERIOD "clk_tx_gmii" 8000 ps HIGH 50 %;
```


Receiver Clock Constraints

Depending on the selected device family, the selected physical interface, and the supported ethernet speeds, a wide variation of required clock period constraint syntax exists. However, the UCF provided with the example design always provides the correct constraints for the generated example design and so this file should be used for reference. Do not relax these clock period constraints.

Receiver clock period constraints are always provided until the following comment heading in the UCF:

```
#####
# RX Clock period Constraints                                     #
#####
```

The following syntax provides an example. This is taken from a tri-speed capable Spartan-3 FPGA design using GMII:

```
# Receiver clock period constraints: please do not relax
NET "rx_gmii_clk_int"          TNM_NET = "clk_rx";
TIMESPEC "TS_rx_clk"          = PERIOD "clk_rx" 8000 ps HIGH 50 %;

# When a dcm is used, the clock declared on its input CANNOT be used in
# a clock group
NET "gmii_rx_clk"              TNM_NET = "clk_rx_local";
TIMESPEC "TS_rx_clk_dcm" = PERIOD "clk_rx_local" 8000 ps HIGH 50 %;
```

IDELAYCTRL Reference Clock Constraints

For Virtex-4, Virtex-5 and Virtex-6 devices, an additional constraint is provided in the UCF for the IDELAYCTRL reference clock. This clock is constrained to run at 200 MHz. See the device *User Guide* for IDELAYCTRL components and the supported reference clock frequency range.

```
NET "*refclk_bufg" TNM_NET = "clk_ref_clk";
TIMESPEC "TS_ref_clk" = PERIOD "clk_ref_clk" 5000 ps HIGH 50 %;
```

Management Clock Constraints

Whenever the optional Management Interface is present in the core, the `host_clk` signal must be constrained to run at the desired frequency. This is set to 100 MHz in the example design. See [hostclk, page 90](#) for the supported frequency range.

```
NET "host_clk" TNM_NET = "host";
TIMESPEC "TS_host_clk" = PERIOD "host" 10000 ps HIGH 50 % PRIORITY 10;
```

MDIO Logic

The MDIO logic (see [MDIO Configuration](#)) is synchronous to `host_clk`, but data only changes at the MDC output rate;; nominally this is set to a frequency of 2.5 MHz. Every flip-flop in the MDIO logic is clocked with `host_clk`, but is sent a clock enable pulse at the MDC frequency. To prevent this logic being over constrained by the `host_clk` period, the relevant flip-flops for the MDIO logic can be grouped together and constrained at a multiple of the `host_clk` period. The Priority setting ensures this constraint takes precedence over the standard `host_clk` period constraint (which is given a priority of 10).

The UCF syntax which follows targets the MDIO logic flip-flops and groups them together. Reduced clock period constraints are then applied.

```
#####
# MDIO Constraints: please do not edit                                     #
#####
# Place the MDIO logic in its own timing groups
INST "*trimac_core*MANIFGEN?MANAGEN?PHY?ENABLE_REG"      TNM =
"mdio_logic";
INST "*trimac_core*MANIFGEN?MANAGEN?PHY?READY_INT"       TNM =
"mdio_logic";
INST "*trimac_core*MANIFGEN?MANAGEN?PHY?STATE_COUNT*"    TNM = FFS
"mdio_logic";
INST "*trimac_core*MANIFGEN?MANAGEN?PHY?MDIO_TRISTATE"    TNM =
"mdio_logic";
INST "*trimac_core*MANIFGEN?MANAGEN?PHY?MDIO_OUT"        TNM =
"mdio_logic";
TIMESPEC "TS_mdio" = PERIOD "mdio_logic" "TS_host_clk" * 40 PRIORITY 0;
```

Timespecs for Critical Logic

Signals must cross clock domains at certain points in the core. These are described in the following section.

Flow Control

Pause requests are received and decoded in the receiver clock domain and must be transferred into the transmitter domain to pause the transmitter. Therefore, the following constraints must always be applied:

```
# Flow Control logic reclocking - control signal is synchronized
INST "*trimac_core?BU2?U0?FLOW?RX_PAUSE?PAUSE_REQ_TO_TX"
TNM="flow_rx_to_tx";
INST "*trimac_core?BU2?U0?FLOW?RX_PAUSE?PAUSE_VALUE_TO_TX*"
TNM="flow_rx_to_tx";
TIMESPEC "TS_flow_rx_to_tx" = FROM "flow_rx_to_tx" TO "clk_tx_gmii"
8000 ps DATAPATHONLY;
```

Configuration Logic

When the optional Management Interface is used with the core (see [Using the Optional Management Interface, page 89](#)), configuration information is written synchronously to `hostclk`. Receiver configuration data must be transferred onto the receiver clock domain for use with the receiver; transmitter configuration data must be transferred onto the transmitter clock domain for use with the transmitter. The following UCF syntax targets this logic and a timing ignore attribute (TIG) is applied. It does not matter when configuration changes take place; the latest configurations is sampled at the beginning of new frames by both the receiver and transmitter.

```
# Configuration Register reclocking
INST "*trimac_core?BU2?U0?MANIFGEN?MANAGEN?CONF?RX0_OUT*"
TNM="async_config";
INST "*trimac_core?BU2?U0?MANIFGEN?MANAGEN?CONF?RX1_OUT*"
TNM="async_config";
INST "*trimac_core?BU2?U0?MANIFGEN?MANAGEN?CONF?FC_OUT_29"
TNM="async_config";

INST "*trimac_core?BU2?U0?MANIFGEN?MANAGEN?CONF?TX_OUT*"
TNM="async_config";
INST "*trimac_core?BU2?U0?MANIFGEN?MANAGEN?CONF?FC_OUT_30"
TNM="async_config";

# speed change config (ONLY PRESENT WHEN GENERATED FOR TRI_SPEED OR
10/100)
NET "*trimac_core?BU2?U0?MANIFGEN?MANAGEN?CONF?CNFG_SPEED*"
TNM="async_config";

TIMESPEC "TS_host_clk_to_rx_clk" = FROM "host" TO "clk_rx" TIG;
TIMESPEC "TS_host_clk_to_tx_clk" = FROM "host" TO "clk_tx_gmii" TIG;

TIMESPEC "TS_config_to_all"      = FROM "async_config" TO
"ffs_except_host" TIG;

# Address filter specific cross clocking (ONLY PRESENT WHEN GENERATED
WITH ADDRESS FILTER)
INST
"*trimac_core?BU2?U0?addr_filter_top/dynamic_af_gen.dynamic_config/uni
cast_addr_*" TNM="addr_config_to_rx";
TIMESPEC "TS_addr_config_to_rx" = FROM "addr_config_to_rx" TO
"ffs_except_host" TIG;
```

Constraints when Implementing an External GMII

The constraints defined in this section are implemented in the UCF for the GMII example design delivered with the core. Sections from this UCF are copied into the following descriptions to act as an example. These should be studied in conjunction with the HDL source code for the example design and with the description given in the Physical Interface chapters within this Guide.

GMII IOB Constraints

The following constraints target the flip-flops that are inferred in the top-level HDL file for the example design; constraints are set to ensure that these are placed in IOBs.

```

INST "*gmii_txd_reg*"      IOB = true;
INST "*gmii_tx_en_reg*"   IOB = true;
INST "*gmii_tx_er_reg*"   IOB = true;

INST "*rx_d_to_mac*"      IOB = true;
INST "*rx_dv_to_mac*"    IOB = true;
INST "*rx_er_to_mac*"     IOB = true;

```

Virtex-6 devices only support GMII/MII at 2.5V and the device default SelectIO interface standard of LVCMOS25 is used. In Virtex-4, Virtex-5, Spartan-3 and Spartan-6 devices, GMII/MII by default is supported at 3.3V and the UCF contains the following syntax. Use this syntax together with the device IO Banking rules.

```

INST "gmii_txd<?>"      IOSTANDARD = LVTTTL;
INST "gmii_tx_en"        IOSTANDARD = LVTTTL;
INST "gmii_tx_er"        IOSTANDARD = LVTTTL;

INST "gmii_rxd<?>"      IOSTANDARD = LVTTTL;
INST "gmii_rx_dv"        IOSTANDARD = LVTTTL;
INST "gmii_rx_er"        IOSTANDARD = LVTTTL;

INST "gmii_tx_clk"        IOSTANDARD = LVTTTL;
INST "gmii_rx_clk"        IOSTANDARD = LVTTTL;

```

In addition, the example design provides pad locking on the GMII I/Os. These are provided as a guideline only; there are no specific I/O location constraints for this core.

GMII Input Setup/Hold Timing

Figure 13-1 and Table 13-2 illustrate the setup and hold time window for the input GMII signals. This is the worst-case data valid window presented to the FPGA device pins.

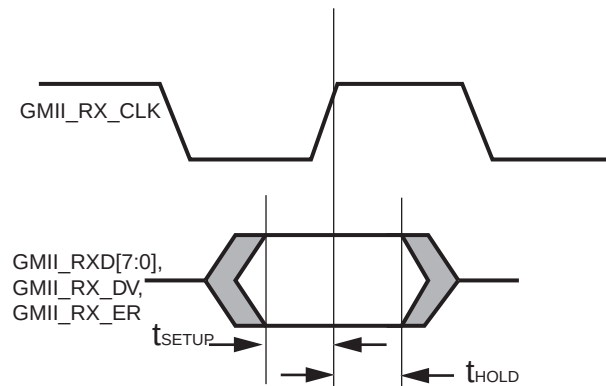


Figure 13-1: Input GMII Timing

Observe that there is a 2 ns data valid window which is presented across the GMII input bus. This must be correctly sampled by the FPGA devices.

Table 13-2: Input GMII Timing

Symbol	Min	Max	Units
t_{SETUP}	2.00	-	ns
t_{HOLD}	0.00	-	ns

Virtex-6 Devices

The GMII example design uses BUFIO/BUFR routing on the clock and IODELAY components on the receiver data and control signals for Virtex-6 devices. A fixed tap delay can be applied to delay the data and control signals so that they are correctly sampled by the `gmii_rx_clk` clock at the IOB flip-flop, thereby meeting GMII setup and hold timing.

The following constraint shows an example of setting the delay value for one of these IODELAY components. All bits can be adjusted individually, if desired, to compensate for any PCB routing skew.

```
INST *gmii_interface/delay_gmii_rx_dv IDELAY_VALUE = 26;
```

The value of IDELAY_VALUE is preconfigured in the example designs to meet the setup and hold constraints for the example GMII pinout in the particular device. The setup/hold timing which is achieved after place-and-route is reported in the data sheet section of the TRCE report (created by the implement script). See [Understanding Timing Reports for GMII Setup/Hold Timing](#).

When IODELAY primitives are instantiated with a fixed delay attribute, an IDELAYCTRL component must be also instantiated to continuously calibrate the individual input delay elements. The IDELAYCTRL module requires a reference clock, which is assumed to be an input to the example design delivered by the CORE Generator software. The most efficient way to use the IDELAYCTRL module is to lock the placement of the instance to the clock region of the device where the IDELAY/IODELAY components are placed. An example LOC constraint for the IDELAYCTRL module is shown commented out in the UCF. See the *Virtex-6 FPGA User Guide* and code comments for details.

In addition, for all Virtex-6 FPGA designs, the following UCF syntax is included:

```
#####
# For Setup and Hold time analysis on GMII inputs      #
#####

# Identify GMII Rx Pads only.
# This prevents setup/hold analysis being performed on false inputs,
# for example, the configuration_vector inputs.
INST "gmii_rxd<?>"          TNM = IN_GMII;
INST "gmii_rx_er"           TNM = IN_GMII;
INST "gmii_rx_dv"           TNM = IN_GMII;

TIMEGRP "IN_GMII" OFFSET = IN 2 ns VALID 2 ns BEFORE "gmii_rx_clk";
```

This syntax causes the Xilinx® implementation tools to analyze the input setup and hold constraints for the input GMII bus. If these constraints are not met, the tools will report timing errors. However, the tools do NOT attempt to automatically correct the timing in the case of failure. These must be corrected manually by changing the IDELAY_VALUES in the UCF.

Spartan-6 Devices

The GMII example design uses BUFIO2/BUFG routing on the clock and IODELAY2 components on the receiver data and control signals for Spartan-6 devices. A fixed tap delay can be applied to delay the data and control signals so that they are correctly sampled by the gmii_rx_clk clock at the IOB flip-flop, thereby meeting GMII setup and hold timing.

The following constraint shows an example of setting the delay value for one of these IODELAY2 components. All bits can be adjusted individually, if desired, to compensate for any PCB routing skew.

```
INST *gmii_interface/delay_gmii_rx_dv IDELAY_VALUE = 26;
```

The value of IDELAY_VALUE is preconfigured in the example designs to meet the setup and hold constraints for the example GMII pinout in the particular device. The setup/hold timing which is achieved after place-and-route is reported in the data sheet section of the TRCE report (created by the implement script). See [Understanding Timing Reports for GMII Setup/Hold Timing](#).

In addition, for all Spartan-6 FPGA designs, the following UCF syntax is included:

```
#####
# For Setup and Hold time analysis on GMII inputs      #
#####

# Identify GMII Rx Pads only.
# This prevents setup/hold analysis being performed on false inputs,
# for example, the configuration_vector inputs.
INST "gmii_rxd<?>"          TNM = IN_GMII;
INST "gmii_rx_er"           TNM = IN_GMII;
INST "gmii_rx_dv"           TNM = IN_GMII;

TIMEGRP "IN_GMII" OFFSET = IN  2 ns VALID 2 ns BEFORE "gmii_rx_clk";
```

This syntax causes the Xilinx implementation tools to analyze the input setup and hold constraints for the input GMII bus. If these constraints are not met, the tools will report timing errors. However, the tools do NOT attempt to automatically correct the timing in the case of failure. These must be corrected manually by changing the IDELAY_VALUES in the UCF.

Virtex-5 Devices

The GMII example design uses IODELAY components on the receiver clock, data and control signals for Virtex-5 devices. A fixed tap delay can be applied to either delay the data and control signals or delay the clock so that the data/control are correctly sampled by the gmii_rx_clk clock at the IOB flip-flop, thereby meeting GMII setup and hold timing.

The choice of delaying data/control or clock is dependent upon a number of factors, not least being the required shift. There are trade-offs to be made with either choice; delaying the clock is clock period specific as we move the clock to line up each edge with data from the following edge. Delaying the data/control introduces more jitter which degrades the overall setup/hold window. The interface timing report in the two cases is also quite different and for this reason this is discussed in [Understanding Timing Reports for GMII Setup/Hold Timing](#).

The following constraint shows an example of setting the delay value for one of these IODELAY components. All bits can be adjusted individually, if desired, to compensate for any PCB routing skew.

```
INST *gmii_interface/delay_gmii_rx_dv IDELAY_VALUE = 33;
```

The value of IDELAY_VALUE is preconfigured in the example designs to meet the setup and hold constraints for the example GMII pinout in the particular device. The setup/hold timing which is achieved after place-and-route is reported in the data sheet section of the TRCE report (created by the implement script).

When IODELAY primitives are instantiated with a fixed delay attribute, an IDELAYCTRL component must be also instantiated to continuously calibrate the individual input delay elements. The IDELAYCTRL module requires a reference clock, which is assumed to be an input to the example design delivered by the CORE Generator software. The most efficient way to use the IDELAYCTRL module is to lock the placement of the instance to the clock region of the device where the IDELAY/IODELAY components are placed. An example LOC constraint for the IDELAYCTRL module is shown commented out in the UCF. See the *Virtex-5 FPGA User Guide* and code comments for details.

In addition, for all Virtex-5 FPGA designs, the following UCF syntax is included:

```
#####
# For Setup and Hold time analysis on GMII inputs      #
#####

# Identify GMII Rx Pads only.
# This prevents setup/hold analysis being performed on false inputs,
# for example, the configuration_vector inputs.
INST "gmii_rxd<?>"          TNM = IN_GMII;
INST "gmii_rx_er"           TNM = IN_GMII;
INST "gmii_rx_dv"           TNM = IN_GMII;

# Use the following syntax if the clock is delayed:
TIMEGRP "IN_GMII" OFFSET = IN -5.8 ns VALID 2.4 ns BEFORE "gmii_rx_clk";

# Alternatively, use the following syntax if the data is delayed:
# TIMEGRP "IN_GMII" OFFSET = IN 2.2 ns VALID 2.4 ns BEFORE
"gmii_rx_clk";
```

This syntax causes the Xilinx implementation tools to analyze the input setup and hold constraints for the input GMII bus. If these constraints are not met, the tools will report timing errors. However, the tools do NOT attempt to automatically correct the timing in the case of failure. These must be corrected manually by changing the IDELAY_VALUE in the UCF.

The constraints defined previously have a built in tolerance of 0.4 ns; this allows all combinations of the example designs to pass timing. However, these should be tightened in a real customer design as per the following syntax. Manual adjustments to IDELAY_VALUES may be required to meet these constraints:

```
# Use the following syntax if the clock is delayed:
TIMEGRP "IN_GMII" OFFSET = IN -6 ns VALID 2 ns BEFORE "gmii_rx_clk";

# Alternatively, use the following syntax if the data is delayed:
# TIMEGRP "IN_GMII" OFFSET = IN 2 ns VALID 2 ns BEFORE "gmii_rx_clk";
```

Virtex-4 Devices

The GMII example design uses a DCM on the receiver clock domain for Virtex-4 devices. Phase-shifting is then applied to the DCM to align the resultant clock so that it correctly samples the 2 ns GMII data valid window at the input flip-flops.

The fixed phase shift is applied to the DCM using the following UCF syntax.

```
INST *gmii_rxc_dcm CLKOUT_PHASE_SHIFT = FIXED;
INST *gmii_rxc_dcm PHASE_SHIFT = -48;
```

The value of PHASE_SHIFT is preconfigured in the example designs to meet the setup and hold constraints for the example GMII pinout in the particular device. The setup/hold timing which is achieved after place-and-route is reported in the data sheet section of the TRCE report (created by the implement script). A further explanation of these numbers is detailed in [Understanding Timing Reports for GMII Setup/Hold Timing](#).

For customers fixing their own pinout, the setup and hold figures reported in the TRCE report can be used to initially setup the approximate DCM phase shift. [Appendix D, Calculating the DCM Phase Shift or IODelay Tap Setting](#) describes a more accurate method for fixing the phase shift by using hardware measurement of a unique PCB design.

In addition, for Virtex-4 FPGA designs, the following UCF syntax is included:


```
#####
# For Setup and Hold time analysis on GMII inputs      #
#####

# Identify GMII Rx Pads only.
# This prevents setup/hold analysis being performed on false inputs,
# for example, the configuration_vector inputs.
INST "gmii_rxd<?>"          TNM = IN_GMII;
INST "gmii_rx_er"           TNM = IN_GMII;
INST "gmii_rx_dv"           TNM = IN_GMII;

TIMEGRP "IN_GMII" OFFSET = IN 2 ns VALID 2 ns BEFORE "gmii_rx_clk";
```

This syntax causes the Xilinx implementation tools to analyze the input setup and hold constraints for the input GMII bus. If these constraints are not met, the tools will report timing errors. However, the tools do NOT attempt to automatically correct the timing in the case of failure. These must be corrected manually by changing the DCM PHASE_SHIFT value in the UCF.

Spartan-3, Spartan-3E, and Spartan-3A Devices

The GMII example design uses a DCM on the receiver clock domain for Spartan-3, Spartan-3E and Spartan-3A devices. Phase-shifting is then applied to the DCM to align the resultant clock so that it correctly samples the 2 ns GMII data valid window at the input flip-flops.

The fixed phase shift is applied to the DCM using the following UCF syntax.

```
INST *gmii_rxc_dcm CLKOUT_PHASE_SHIFT = FIXED;
INST *gmii_rxc_dcm PHASE_SHIFT = -30;
```

The value of PHASE_SHIFT is preconfigured in the example designs to meet the setup and hold constraints for the example GMII pinout in the particular device. The setup/hold timing which is achieved after place-and-route is reported in the data sheet section of the TRCE report (created by the implement script). A further explanation of these numbers is detailed in [Understanding Timing Reports for GMII Setup/Hold Timing](#).

For customers fixing their own pinout, the setup and hold figures reported in the TRCE report can be used to initially setup the approximate DCM phase shift. [Appendix D, Calculating the DCM Phase Shift or IODelay Tap Setting](#) describes a more accurate method for fixing the phase shift by using hardware measurement of a unique PCB design.

In addition, for all Spartan-3 family designs, the following UCF syntax is included:

```
#####
# For Setup and Hold time analysis on GMII inputs      #
#####

# Identify GMII Rx Pads only.
# This prevents setup/hold analysis being performed on false inputs,
# for example, the configuration_vector inputs.
INST "gmii_rxd<?>"          TNM = IN_GMII;
INST "gmii_rx_er"           TNM = IN_GMII;
INST "gmii_rx_dv"           TNM = IN_GMII;

TIMEGRP "IN_GMII" OFFSET = IN 2.2 ns VALID 2.4 ns BEFORE "gmii_rx_clk";
```

This syntax causes the Xilinx implementation tools to analyze the input setup and hold constraints for the input GMII bus. If these constraints are not met, the tools will report timing errors. However, the tools do NOT attempt to automatically correct the timing in the case of failure. These must be corrected manually by changing the DCM PHASE_SHIFT value in the UCF.

The previously defined constraints have a built-in tolerance of 0.4 ns; this allows all combinations of the example designs to pass timing. However, these should be tightened in a real customer design as per the following syntax. Manual adjustments to DCM phase shift values may be required to meet these constraints:

```
TIMEGRP "IN_GMII" OFFSET = IN 2 ns VALID 2 ns BEFORE "gmii_rx_clk"
```

Understanding Timing Reports for GMII Setup/Hold Timing

Virtex-6, Spartan-6, Spartan-3, Virtex-4 and Devices

Setup and Hold results for the GMII input bus can be found in the data sheet section of the Timing Report. The results are self-explanatory and it is easy to see how they relate to [Figure 13-1](#). Here follows an example report. The implementation requires 1.835 ns of setup; this is less than the 2 ns required and so there is slack. The implementation requires -0.226 ns of hold; this is less than the 0 ns required and so there is slack.

Data Sheet report:

All values displayed in nanoseconds (ns)

Setup/Hold to clock gmii_rx_clk

Source	Setup to clk (edge)	Hold to clk (edge)	Internal Clock(s)	Clock Phase
gmii_rx_dv	1.820(R)	-0.281(R)	rx_gmii_mii_clk_int	0.000
gmii_rx_er	1.770(R)	-0.226(R)	rx_gmii_mii_clk_int	0.000
gmii_rxd<0>	1.821(R)	-0.283(R)	rx_gmii_mii_clk_int	0.000
gmii_rxd<1>	1.833(R)	-0.295(R)	rx_gmii_mii_clk_int	0.000
gmii_rxd<2>	1.790(R)	-0.253(R)	rx_gmii_mii_clk_int	0.000
gmii_rxd<3>	1.789(R)	-0.252(R)	rx_gmii_mii_clk_int	0.000
gmii_rxd<4>	1.834(R)	-0.296(R)	rx_gmii_mii_clk_int	0.000
gmii_rxd<5>	1.829(R)	-0.291(R)	rx_gmii_mii_clk_int	0.000
gmii_rxd<6>	1.793(R)	-0.255(R)	rx_gmii_mii_clk_int	0.000
gmii_rxd<7>	1.835(R)	-0.296(R)	rx_gmii_mii_clk_int	0.000

Virtex-5 Devices

Delayed Data/Control

Setup and Hold results for the GMII input bus can be found in the data sheet section of the Timing Report. The results are self-explanatory and it is easy to see how they relate to [Figure 13-1](#). Here follows an example for the GMII report from a Virtex-5 device. The implementation requires 1.962 ns of setup; this is less than the 2 ns required and so there is slack. The implementation requires -0.008 ns of hold: this is less than the 0 ns required and so there is slack.

Data Sheet report:

All values displayed in nanoseconds (ns)

Setup/Hold to clock gmii_rx_clk

Source	Setup to clk (edge)	Hold to clk (edge)	Internal Clock(s)	Clock Phase
gmii_rx_dv	1.955(R)	-0.017(R)	rx_gmii_mii_clk_int	0.000
gmii_rx_er	1.962(R)	-0.031(R)	rx_gmii_mii_clk_int	0.000
gmii_rxd<0>	1.949(R)	-0.013(R)	rx_gmii_mii_clk_int	0.000
gmii_rxd<1>	1.944(R)	-0.009(R)	rx_gmii_mii_clk_int	0.000
gmii_rxd<2>	1.947(R)	-0.012(R)	rx_gmii_mii_clk_int	0.000
gmii_rxd<3>	1.942(R)	-0.008(R)	rx_gmii_mii_clk_int	0.000
gmii_rxd<4>	1.950(R)	-0.015(R)	rx_gmii_mii_clk_int	0.000
gmii_rxd<5>	1.962(R)	-0.026(R)	rx_gmii_mii_clk_int	0.000
gmii_rxd<6>	1.957(R)	-0.022(R)	rx_gmii_mii_clk_int	0.000
gmii_rxd<7>	1.952(R)	-0.020(R)	rx_gmii_mii_clk_int	0.000

Delayed Clock

Setup and Hold results for the GMII input bus can be found in the data sheet section of the Timing Report. However, depending on how the setup/hold requirements have been met, the results can initially look strange and it is not immediately obvious how they relate to [Figure 13-1](#). Here follows an example for the GMII report from a Virtex-4 device where the clock has been delayed to meet the setup/hold requirements.

Data Sheet report:

All values displayed in nanoseconds (ns)

Setup/Hold to clock gmii_rx_clk

Source	Setup to clk (edge)	Hold to clk (edge)	Internal Clock(s)	Clock Phase
gmii_rx_dv	-6.198(R)	7.526(R)	rx_gmii_mii_clk_int	0.000
gmii_rx_er	-6.225(R)	7.554(R)	rx_gmii_mii_clk_int	0.000
gmii_rxd<0>	-6.149(R)	7.484(R)	rx_gmii_mii_clk_int	0.000
gmii_rxd<1>	-6.152(R)	7.486(R)	rx_gmii_mii_clk_int	0.000
gmii_rxd<2>	-6.206(R)	7.532(R)	rx_gmii_mii_clk_int	0.000
gmii_rxd<3>	-6.207(R)	7.533(R)	rx_gmii_mii_clk_int	0.000
gmii_rxd<4>	-6.134(R)	7.476(R)	rx_gmii_mii_clk_int	0.000
gmii_rxd<5>	-6.134(R)	7.476(R)	rx_gmii_mii_clk_int	0.000
gmii_rxd<6>	-6.170(R)	7.506(R)	rx_gmii_mii_clk_int	0.000
gmii_rxd<7>	-6.170(R)	7.506(R)	rx_gmii_mii_clk_int	0.000

The implementation requires -6.134 ns of setup. Figure 13-2 illustrates that this represents a figure of 1.866 ns relative to the following rising edge of the clock (since the IDELAY has acted to delay the clock by an entire period when measured from the input flip-flop). This is less than the 2 ns required and so there is slack.

The implementation requires 7.554 ns of hold. Figure 13-2 illustrates that this represents a figure of -0.446 ns relative to the following rising edge of the clock (since the IDELAY has acted to delay the clock by an entire period when measured from the input flip-flop). This is less than the 0 ns required and so there is slack.

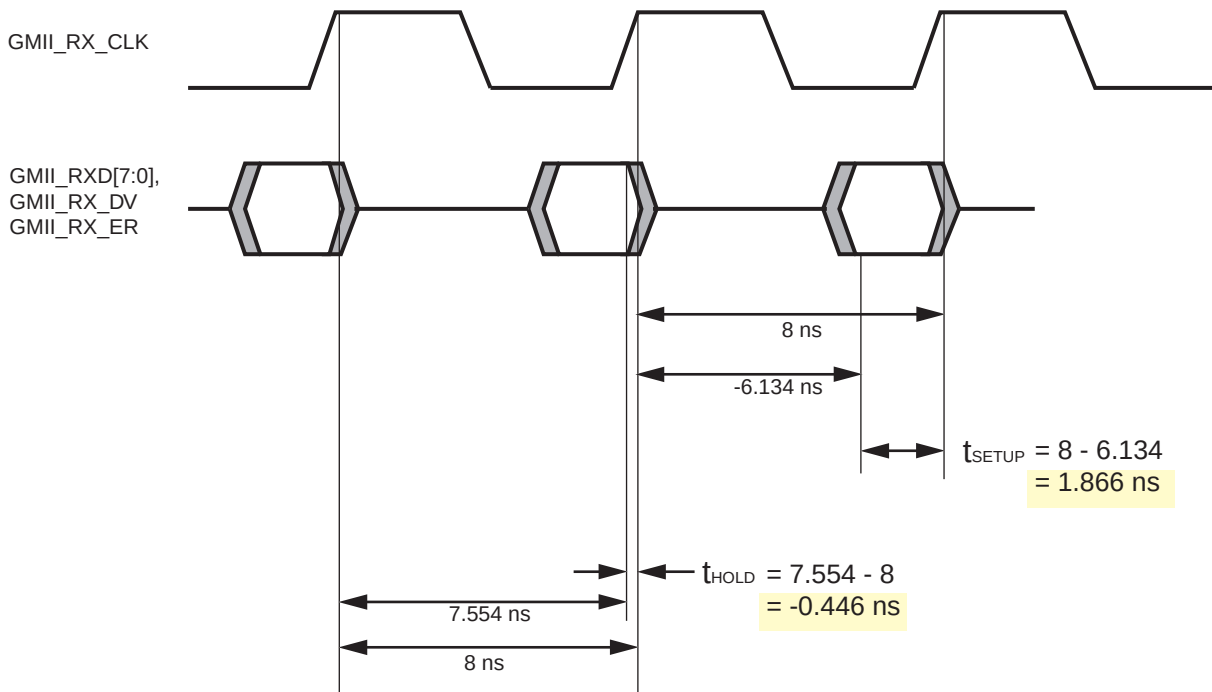


Figure 13-2: Timing Report Setup/Hold

Constraints when Implementing an External RGMII

The constraints defined in this section are implemented in the UCF for the example design delivered with the core. Sections from this UCF are copied into the following descriptions to act as an example. These should be studied in conjunction with the HDL source code for the example design and with the description given in the Physical Interface chapters within this Guide.

RGMII IOB Constraints

All Spartan-3 Devices

The following constraints target the flip-flops that are inferred in the top-level HDL file for the Spartan-3 FPGA example design; constraints are set to ensure that these are placed in IOBs. These constraints are not required for all other families where DDR components are instantiated rather than inferred.

```
INST "rgmii_rxd_ddr*" IOB = true;
INST "rgmii_rx_dv_ddr" IOB = true;
INST "rgmii_rx_ctl_ddr" IOB = true;

INST "inband_link_status" IOB = true;
INST "inband_clock_speed*" IOB = true;
INST "inband_duplex_status" IOB = true;
```

RGMII DDR Constraints

The following constraints are required to constrain the RGMII input registers for 1 Gb/s operation. The RGMII design requires these clock crossing constraints to ensure timing is met when crossing from rising to falling clock edges and vice versa. A stringent time constraint ensures that timing is met with the worst-case timing allowed in the RGMII specification.

```
INST "rgmii_rxd_reg_4" TNM="rgmii_falling";
INST "rgmii_rxd_reg_5" TNM="rgmii_falling";
INST "rgmii_rxd_reg_6" TNM="rgmii_falling";
INST "rgmii_rxd_reg_7" TNM="rgmii_falling";
INST "rgmii_rx_ctl_reg" TNM="rgmii_falling";
INST "rgmii_rxd_reg_4" TNM="rgmii_rising";
INST "rgmii_rxd_reg_5" TNM="rgmii_rising";
INST "rgmii_rxd_reg_6" TNM="rgmii_rising";
INST "rgmii_rxd_reg_7" TNM="rgmii_rising";
INST "rgmii_rx_er_reg" TNM="rgmii_rising";

TIMESPEC "TS_rgmii_falling_to_rising" = FROM "rgmii_falling" TO
"rgmii_rising" 3200 ps;
```

All Devices

The RGMII v2.0 is a 1.5 volt signal-level interface. The 1.5 volt HSTL Class I SelectIO interface standard is used for RGMII interface pins. Use the following constraints with the device IO Banking rules. The IO slew rate is set to fast to ensure that the interface can meet setup and hold times.

```
INST "rgmii_txd<?>" IOSTANDARD = HSTL_I;
INST "rgmii_tx_ctl" IOSTANDARD = HSTL_I;
INST "rgmii_rxd<?>" IOSTANDARD = HSTL_I;
INST "rgmii_rx_ctl" IOSTANDARD = HSTL_I;

INST "rgmii_txc" IOSTANDARD = HSTL_I;
INST "rgmii_rxc" IOSTANDARD = HSTL_I;

INST "rgmii_txd<?>" SLEW = FAST;
INST "rgmii_tx_ctl" SLEW = FAST;
INST "rgmii_txc" SLEW = FAST;
```

In addition, the example design provides pad locking on the RGMII for several families. This is provided as a guideline only; there are no specific I/O location constraints for this core.

RGMII Input Setup/Hold Timing

Figure 13-3 and Table 13-3 illustrate the setup and hold time window for the input RGMII signals. This is the worst-case data valid window presented to the FPGA device pins.

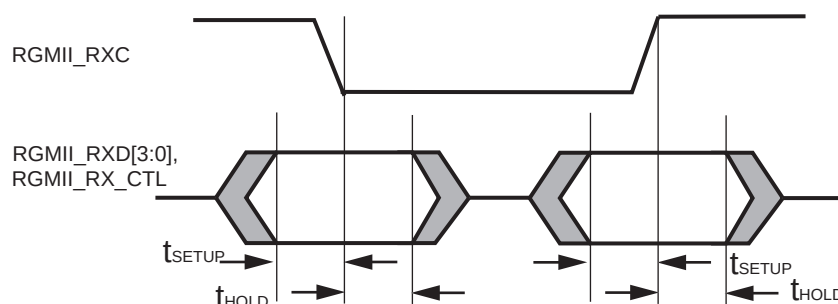


Figure 13-3: Input RGMII Timing

Observe that there is a 2 ns data valid window which is presented across the RGMII input bus. This must be correctly sampled on both clock edges by the FPGA devices.

Table 13-3: Input RGMII Timing

Symbol	Min	Typical	Units
t_{SETUP}	1.0	2.0	ns
t_{HOLD}	1.0	2.0	ns

For RGMII, the lower data bits, `rgmii_rxd[3:0]`, should be sampled internally on the rising edge of `rgmii_rxc`, and the upper data bits, `rgmii_rxd[7:4]`, should be sampled internally on the falling edge of `rgmii_rxc`.

Virtex-6 Devices

The RGMII design uses an IODELAY component on the `rgmii_txc` transmitter output clock. A fixed tap delay is applied to move the rising edge of this clock to the center of the output data window. This is performed with the following UCF syntax:

```
#INST "*delay_rgmii_tx_clk" ODELAY_VALUE = 26;
```

The RGMII receiver design uses BUFIO/BUFR routing on the clock and IODELAY components on the data and control signals. A fixed tap delay can be applied to delay the data and control signals so that they are correctly sampled by the `rgmii_rxc` clock at the IOB IDDR registers, thereby meeting RGMII setup and hold timing.

The following constraint shows an example of setting the delay value for one of these IODELAY components. Data/Control bits can be adjusted individually, if desired, to compensate for any PCB routing skew.

```
INST *gmii_interface/delay_rgmii_rx_ctl IDELAY_VALUE = 20;
```

The value of IDELAY_VALUE is preconfigured in the example designs to meet the setup and hold constraints for the example RGMII pinout in the particular device. The setup/hold timing which is achieved after place-and-route is reported in the data sheet section of the TRCE report (created by the implement script). See [Understanding Timing Reports for RGMII Setup/Hold Timing](#).

When IODELAY primitives are instantiated with a fixed delay attribute, an IDELAYCTRL component must be also instantiated to continuously calibrate the individual input delay elements. The IDELAYCTRL module requires a reference clock, which is assumed to be an input to the example design delivered by the CORE Generator software. The most efficient way to use the IDELAYCTRL module is to lock the placement of the instance to the clock region of the device where the IDELAY/IODELAY components are placed. An example LOC constraint for the IDELAYCTRL module is shown commented out in the UCF. See the *Virtex-6 FPGA User Guide* and code comments for details.

In addition, for all Virtex-6 FPGA designs, the following UCF syntax is included:

```
#####
# For Setup and Hold time analysis on RGMII inputs      #
#####

# Identify RGMII Rx Pads only.
# This prevents setup/hold analysis being performed on false inputs,
# for example, the configuration_vector inputs.
INST "rgmii_rxd<?>"          TNM = IN_RGMII;
INST "rgmii_rx_ctl"          TNM = IN_RGMII;

# Define data valid window with respect to the clock rising edge.
# The spec states that, worst case, the data is valid 1 ns before the
# clock edge.
# The worst case it to provide 1 ns hold time (a 2ns window in total)
TIMEGRP "IN_RGMII" OFFSET = IN 1 ns VALID 2 ns BEFORE "rgmii_rxc"
"RISING";

# Define data valid window with respect to the clock falling edge.
TIMEGRP "IN_RGMII" OFFSET = IN 1 ns VALID 2 ns BEFORE rgmii_rxc
"FALLING";
```

This syntax causes the Xilinx implementation tools to analyze the input setup and hold constraints for the input RGMII bus. If these constraints are not met, the tools will report timing errors. However, the tools do NOT attempt to automatically correct the timing in the case of failure. These must be corrected manually by changing the IDELAY_VALUE in the UCF.

Spartan-6 Devices

The RGMII design uses an IODELAY2 component on the rgmii_txc transmitter output clock. A fixed tap delay is applied to move the rising edge of this clock to the center of the output data window. This is performed with the following UCF syntax:

```
#INST "*delay_rgmii_tx_clk" ODELAY_VALUE = 26;
```

The RGMII receiver design uses BUFIO2/DCM/BUFG routing on the clock. The DCM phase shift can be adjusted to provide the desired setup/hold window at the data/control IOB IDDR2 registers.

The following constraint shows an example of setting the DCM phase shift. The ideal value for this is found through hardware testing.

```
INST *rgmii_rx_dcm          PHASE_SHIFT = 70;
```

The value of PHASE_SHIFT is preconfigured in the example designs to meet the setup and hold constraints for the example RGMII pinout in the particular device. The setup/hold timing which is achieved after place-and-route is reported in the data sheet section of the TRCE report (created by the implement script). See [Understanding Timing Reports for RGMII Setup/Hold Timing](#).

In addition, for all Spartan-6 FPGA designs, the following UCF syntax is included:

```
#####
# For Setup and Hold time analysis on RGMII inputs      #
#####

# Identify RGMII Rx Pads only.
# This prevents setup/hold analysis being performed on false inputs,
# for example, the configuration_vector inputs.
INST "rgmii_rxd<?>"          TNM = IN_RGMII;
INST "rgmii_rx_ctl"          TNM = IN_RGMII;

# Define data valid window with respect to the clock rising edge.
# The spec states that, worst case, the data is valid 1 ns before the
# clock edge.
# The worst case it to provide 1 ns hold time (a 2ns window in total)
TIMEGRP "IN_RGMII" OFFSET = IN 1 ns VALID 2 ns BEFORE "rgmii_rxc"
"RISING";

# Define data valid window with respect to the clock falling edge.
# TIMEGRP "IN_RGMII" OFFSET = IN 1 ns VALID 2 ns BEFORE rgmii_rxc
"FALLING";
```

This syntax causes the Xilinx implementation tools to analyze the input setup and hold constraints for the input RGMII bus. If these constraints are not met, the tools will report timing errors. However, the tools do NOT attempt to automatically correct the timing in the case of failure. These must be corrected manually by changing the PHASE_SHIFT in the UCF.

Virtex-5 Devices

The RGMII design uses an IODELAY component on the rgmii_txc transmitter output clock. A fixed tap delay is applied to move the rising edge of this clock to the center of the output data window. This is performed with the following UCF syntax:

```
#INST "*delay_rgmii_tx_clk" ODELAY_VALUE = 26;
```

For the RGMII receiver clock, data and control signals, a fixed tap delay can be applied to either delay the data and control signals or delay the clock so that the data/control are correctly sampled by the rgmii_rxc clock at the IOB IDDR registers, thereby meeting RGMII setup and hold timing.

The choice of delaying data/control or clock is dependent upon a number of factors, not least being the required shift. There are trade-offs to be made with either choice: Delaying the clock is clock period specific as we move the clock to line up each edge with data from the following edge. Delaying the data/control introduces more jitter which degrades the overall setup/hold window. The interface timing report in the two cases is also quite different and for this reason this is discussed in [Understanding Timing Reports for RGMII Setup/Hold Timing](#).

The following constraint shows an example of setting the delay value for one of these IODELAY components. Data/Control bits can be adjusted individually, if desired, to compensate for any PCB routing skew.


```
INST *gmii_interface/delay_rgmii_rx_ctl IDELAY_VALUE = 20;
```

The value of IDELAY_VALUE is preconfigured in the example designs to meet the setup and hold constraints for the example RGMII pinout in the particular device. The setup/hold timing which is achieved after place-and-route is reported in the data sheet section of the TRCE report (created by the implement script).

When IDELAY primitives are instantiated with a fixed delay attribute, an IDELAYCTRL component must be also instantiated to continuously calibrate the individual input delay elements. The IDELAYCTRL module requires a reference clock, which is assumed to be an input to the example design delivered by the CORE Generator software. The most efficient way to use the IDELAYCTRL module is to lock the placement of the instance to the clock region of the device where the IDELAY/IODELAY components are placed. An example LOC constraint for the IDELAYCTRL module is shown commented out in the UCF. See the *Virtex-5 FPGA User Guide* and code comments for details.

In addition, for all Virtex-5 FPGA designs, the following UCF syntax is included:

```
#####
# For Setup and Hold time analysis on RGMII inputs      #
#####

# Identify RGMII Rx Pads only.
# This prevents setup/hold analysis being performed on false inputs,
# for example, the configuration_vector inputs.
INST "rgmii_rxd<?>"          TNM = IN_RGMII;
INST "rgmii_rx_ctl"          TNM = IN_RGMII;

# Define data valid window with respect to the clock rising edge.
# The spec states that, worst case, the data is valid 1 ns before the
# clock edge.
# The worst case it to provide 1 ns hold time (a 2ns window in total)
TIMEGRP "IN_RGMII" OFFSET = IN 1.1 ns VALID 2.2 ns BEFORE "rgmii_rxc"
"RISING";

# Define data valid window with respect to the clock falling edge.

TIMEGRP "IN_RGMII" OFFSET = IN 1.1 ns VALID 2.2 ns BEFORE rgmii_rxc
"FALLING";
```

This syntax causes the Xilinx implementation tools to analyze the input setup and hold constraints for the input RGMII bus. If these constraints are not met, the tools will report timing errors. However, the tools do NOT attempt to automatically correct the timing in the case of failure. These must be corrected manually by changing the IDELAY_VALUE in the UCF.

The constraints defined previously have a built in tolerance of 0.2 ns; this allows all combinations of the example designs to pass timing. However, these should be tightened in a real customer design as per the following syntax. Manual adjustments to IDELAY_VALUES may be required to meet these constraints:

```
TIMEGRP "IN_RGMII" OFFSET = IN 1 ns VALID 2 ns BEFORE "rgmii_rxc"
"RISING";
TIMEGRP "IN_RGMII" OFFSET = IN 1 ns VALID 2 ns BEFORE "rgmii_rxc"
"FALLING";
```

Virtex-4 Devices

The RGMII design uses a DCM on the receiver clock domain for all devices. Phase-shifting is then applied to the DCM to align the resultant clock so that it correctly samples the 2 ns RGMII data valid window at the input flip-flops.

The fixed phase shift is applied to the DCM using the following UCF syntax.

```
INST *gmii_rxc_dcm CLKOUT_PHASE_SHIFT = FIXED;
INST *gmii_rxc_dcm PHASE_SHIFT = 10;
```

The value of PHASE_SHIFT is preconfigured in the example designs to meet the setup and hold constraints for the example RGMII pinout in the particular device. The setup/hold timing which is achieved after place-and-route is reported in the data sheet section of the TRCE report (created by the implement script).

For customers fixing their own pinout, the setup and hold figures reported in the TRCE report can be used to initially setup the approximate DCM phase shift. [Appendix D, Calculating the DCM Phase Shift or IODelay Tap Setting](#) describes a more accurate method for fixing the phase shift by using hardware measurement of a unique PCB design.

In addition, for all Virtex-4 FPGA designs, the following UCF syntax is included:

```
#####
# For Setup and Hold time analysis on RGMII inputs      #
#####

# Identify RGMII Rx Pads only.
# This prevents setup/hold analysis being performed on false inputs,
# for example, the configuration_vector inputs.
INST "rgmii_rxd<?>"          TNM = IN_RGMII;
INST "rgmii_rx_ctl"          TNM = IN_RGMII;

TIMEGRP "DDR_RISING"          = FFS;
TIMEGRP "DDR_FALLING"        = FALLING FFS;

# Define data valid window with respect to the clock rising edge.
# The spec states that, worst case, the data is valid 1 ns before the
# clock edge.
# The worst case it to provide 1 ns hold time (a 2ns window in total)
TIMEGRP "IN_RGMII" OFFSET = IN 1 ns VALID 2 ns BEFORE "rgmii_rxc"
TIMEGRP "DDR_RISING";

# Define data valid window with respect to the clock falling edge.
# The phase offset is advanced 3 ns after the rising clock edge (IN -3
# ns), therefore putting it 1 ns before the falling edge
TIMEGRP "IN_RGMII" OFFSET = IN -3 ns VALID 2 ns BEFORE "rgmii_rxc"
TIMEGRP "DDR_FALLING";
```

This syntax causes the Xilinx implementation tools to analyze the input setup and hold constraints for the input RGMII bus. If these constraints are not met, the tools will report timing errors. However, the tools do NOT attempt to automatically correct the timing in the case of failure. These must be corrected manually by changing the DCM PHASE_SHIFT value in the UCF.

Spartan-3, Spartan-3E, and Spartan-3A Devices

The RGMII design uses a DCM on the receiver clock domain for all devices. Phase-shifting is then applied to the DCM to align the resultant clock so that it correctly samples the 2 ns RGMII data valid window at the input flip-flops.

The fixed phase shift is applied to the DCM using the following UCF syntax.

```
INST *gmii_rxc_dcm CLKOUT_PHASE_SHIFT = FIXED;
INST *gmii_rxc_dcm PHASE_SHIFT = 10;
```

The value of PHASE_SHIFT is preconfigured in the example designs to meet the setup and hold constraints for the example RGMII pinout in the particular device. The setup/hold timing which is achieved after place-and-route is reported in the data sheet section of the TRCE report (created by the implement script).

For customers fixing their own pinout, the setup and hold figures reported in the TRCE report can be used to initially setup the approximate DCM phase shift. [Appendix D, Calculating the DCM Phase Shift or IODelay Tap Setting](#) describes a more accurate method for fixing the phase shift by using hardware measurement of a unique PCB design.

In addition, for all Spartan-3 FPGA designs, the following UCF syntax is included:

```
#####
# For Setup and Hold time analysis on RGMII inputs      #
#####

# Identify RGMII Rx Pads only.
# This prevents setup/hold analysis being performed on false inputs,
# for example, the configuration_vector inputs.
INST "rgmii_rxd<?>"          TNM = IN_RGMII;
INST "rgmii_rx_ctl"          TNM = IN_RGMII;

TIMEGRP "DDR_RISING"          = FFS;
TIMEGRP "DDR_FALLING"         = FALLING FFS;

# Define data valid window with respect to the clock rising edge.
# The spec states that, worst case, the data is valid 1 ns before the
# clock edge.
# The worst case it to provide 1 ns hold time (a 2ns window in total)
TIMEGRP "IN_RGMII" OFFSET = IN 1.1 ns VALID 2.2 ns BEFORE "rgmii_rxc"
TIMEGRP "DDR_RISING";

# Define data valid window with respect to the clock falling edge.
# The phase offset is advanced 3 ns after the rising clock edge (IN -3
# ns), therefore putting it 1 ns before the falling edge
TIMEGRP "IN_RGMII" OFFSET = IN -2.9 ns VALID 2.2 ns BEFORE rgmii_rxc
TIMEGRP "DDR_FALLING";
```

This syntax causes the Xilinx implementation tools to analyze the input setup and hold constraints for the input RGMII bus. If these constraints are not met then the tools will report timing errors. However, the tools do NOT attempt to automatically correct the timing in the case of failure. These must be corrected manually by changing the DCM PHASE_SHIFT value in the UCF.

The constraints defined previously have a built in tolerance of 0.2 ns; this allows all combinations of the example designs to pass timing. However, these should be tightened in a real customer design as per the following syntax. Manual adjustments to DCM phase shift values may be required to meet these constraints:

```

TIMEGRP "IN_RGMII" OFFSET = IN 1 ns VALID 2 ns BEFORE "rgmii_rxc"
TIMEGRP "DDR_RISING";
TIMEGRP "IN_RGMII" OFFSET = IN -3 ns VALID 2 ns BEFORE "rgmii_rxc"
TIMEGRP "DDR_FALLING";

```

Understanding Timing Reports for RGMII Setup/Hold Timing

Virtex-6 and Virtex-5 Devices

Delayed Data/Control

Setup and Hold results for the RGMII input bus can be found in the data sheet section of the Timing Report. The results are self-explanatory and it is easy to see how they relate to [Figure 13-3](#). Here follows an example for the RGMII report from a Virtex-5 device. Each Input lists two sets of values: one corresponding to the falling edge of the clock and one to the rising edge. The first set listed corresponds to rising edge which occurs at time 0 ns.

Setup: the implementation requires 0.818 ns of setup to the rising edge and 0.794 ns to the falling edge; this is less than the 1 ns required and so there is slack.

Hold: the implementation requires 0.946 ns of hold after the rising edge and 0.972 ns after the falling edge; this is less than the 1ns required and so there is slack.

Data Sheet report:

All values displayed in nanoseconds (ns)

Setup/Hold to clock rgmii_rxc

Source	Setup to clk (edge)	Hold to clk (edge)	Internal Clock(s)	Clock Phase
rgmii_rx_ctl	0.810(R) -3.214(F)	0.933(R) 4.959(F)	rgmii_rx_clk_bufg	0.000 4.000
rgmii_rxd<0>	0.811(R) -3.213(F)	0.940(R) 4.966(F)	rgmii_rx_clk_bufg	0.000 4.000
rgmii_rxd<1>	0.801(R) -3.223(F)	0.946(R) 4.972(F)	rgmii_rx_clk_bufg	0.000 4.000
rgmii_rxd<2>	0.818(R) -3.206(F)	0.929(R) 4.955(F)	rgmii_rx_clk_bufg	0.000 4.000
rgmii_rxd<3>	0.809(R) -3.215(F)	0.936(R) 4.962(F)	rgmii_rx_clk_bufg	0.000 4.000

Delayed Clock

Setup and Hold results for the RGMII input bus can be found in the data sheet section of the Timing Report. However, depending on how the setup/hold requirements have been met, the results can initially look strange and it is not immediately obvious how they relate to [Figure 13-3](#). Here follows an example for the RGMII report from where the clock has been delayed to meet the setup/hold requirements.

Data Sheet report:

All values displayed in nanoseconds (ns)

Setup/Hold to clock rgmii_rxc

Source	Setup to clk (edge)	Hold to clk (edge)	Internal Clock(s)	Clock Phase
rgmii_rx_ctl	-7.178(R) -11.178(F)	8.880(R) 12.880(F)	rgmii_rx_clk_bufg rgmii_rx_clk_bufg	0.000 4.000
rgmii_rxd<0>	-7.192(R) -11.192(F)	8.893(R) 12.893(F)	rgmii_rx_clk_bufg rgmii_rx_clk_bufg	0.000 4.000
rgmii_rxd<1>	-7.182(R) -11.182(F)	8.884(R) 12.884(F)	rgmii_rx_clk_bufg rgmii_rx_clk_bufg	0.000 4.000
rgmii_rxd<2>	-7.180(R) -11.180(F)	8.882(R) 12.882(F)	rgmii_rx_clk_bufg rgmii_rx_clk_bufg	0.000 4.000
rgmii_rxd<3>	-7.179(R) -11.179(F)	8.881(R) 12.881(F)	rgmii_rx_clk_bufg rgmii_rx_clk_bufg	0.000 4.000

Each Input lists two sets of values: one corresponding to the rising edge of the clock and one to the falling edge. The first set listed corresponds to rising edge which occurs at time 8ns as we have delayed the clock to use the following rising edge.

The implementation requires -7.179 ns of setup to the rising edge. Figure 13-4 illustrates that this represents a figure of 0.821 ns relative to the following rising edge of the clock (since the IDELAY has acted to delay the clock by an entire period when measured from the input flip-flop). This is less than the 1 ns required and so there is slack. Equally for the falling edge, we have -11.179 ns of setup: this edge is at time 12 ns and therefore this equates to a setup figure of 0.821 ns.

The implementation requires 8.893 ns of hold to the rising edge. Figure 13-4 illustrates that this represents a figure of 0.893 ns relative to the following rising edge of the clock (since the IDELAY has acted to delay the clock by an entire period when measured from the input flip-flop). This is less than the 1 ns required and so there is slack. Equally for the falling edge, we have 12.893 ns of hold@ this edge is at time 12 ns and therefore this equates to a hold figure of 0.893 ns.

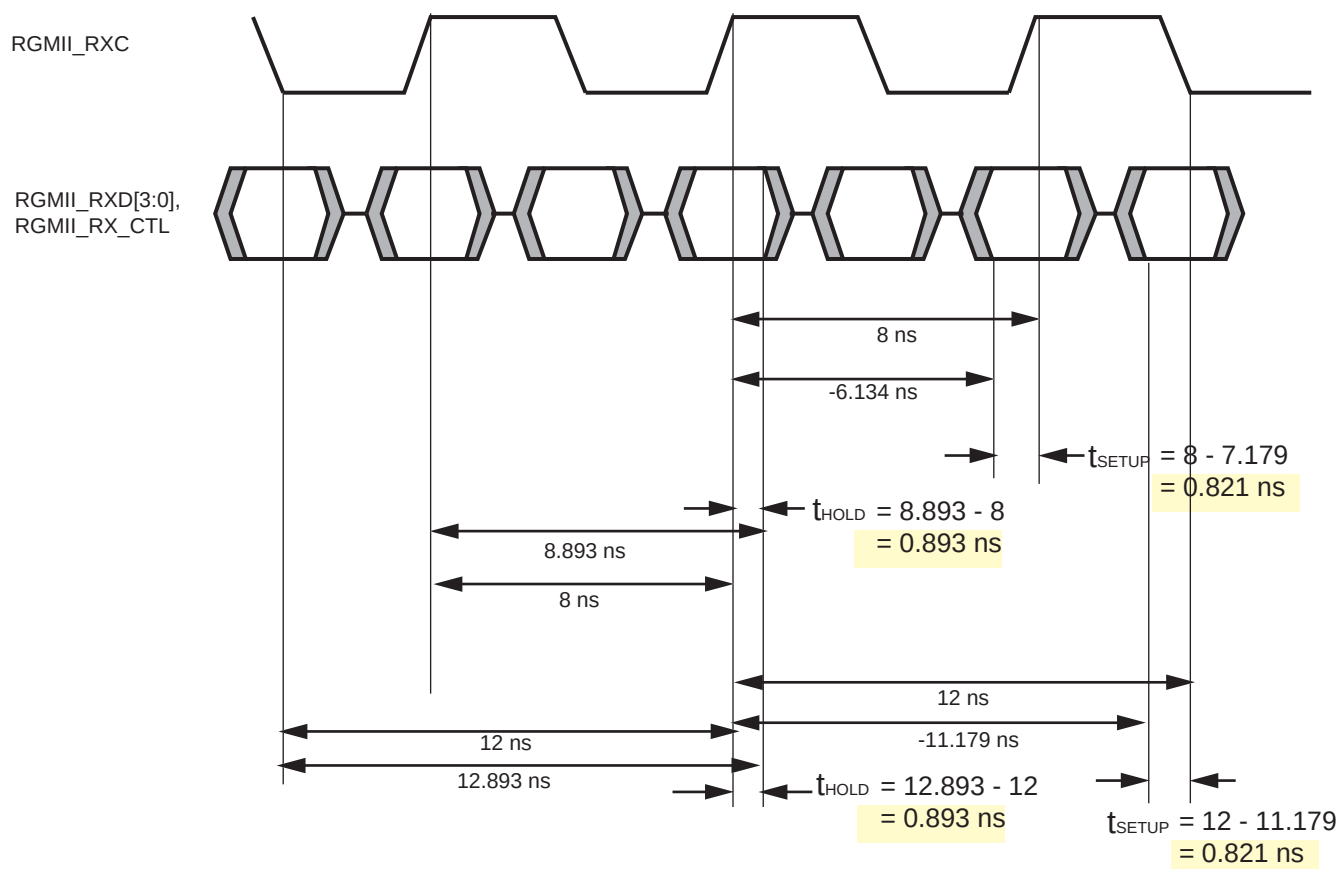


Figure 13-4: Timing Report Setup/Hold

Spartan-6, Virtex-4 and Spartan-3 Devices

Setup and Hold results for the RGMII input bus can be found in the data sheet section of the Timing Report. The results are self-explanatory and it is easy to see how they relate to Figure 13-3. Here follows an example report. Each Input lists two sets of values: one corresponding to the falling edge of the clock and one to the rising edge.

Setup: the first set listed corresponds to falling edge, which occurs at time 4 ns. The implementation requires 0.648 ns of setup to the falling edge and 0.661 ns to the rising edge; this is less than the 1ns required and so there is slack.

Hold: the implementation requires 0.300 ns of hold after the falling edge and 0.316 ns after the falling edge; this is less than the 1ns required and so there is slack.

Data Sheet report:

All values displayed in nanoseconds (ns)

Setup/Hold to clock rgmii_rxc

Source	Setup to clk (edge)	Hold to clk (edge)	Internal Clock(s)	Clock Phase
rgmii_rx_ctl	-3.352(R) 0.661(R)	4.300(R) 0.284(R)	not_rgmii_rx_clk_bufg rgmii_rx_clk_bufg	4.938 0.938
rgmii_rxd<0>	-3.384(R) 0.629(R)	4.332(R) 0.316(R)	not_rgmii_rx_clk_bufg rgmii_rx_clk_bufg	4.938 0.938
rgmii_rxd<1>	-3.348(R) 0.665(R)	4.296(R) 0.280(R)	not_rgmii_rx_clk_bufg rgmii_rx_clk_bufg	4.938 0.938
rgmii_rxd<2>	-3.360(R) 0.653(R)	4.308(R) 0.292(R)	not_rgmii_rx_clk_bufg rgmii_rx_clk_bufg	4.938 0.938
rgmii_rxd<3>	-3.428(R) 0.585(R)	4.382(R) 0.366(R)	not_rgmii_rx_clk_bufg rgmii_rx_clk_bufg	4.938 0.938

Interfacing to Other Xilinx Ethernet Cores

The TEMAC solution functionality has been designed to easily integrate with the following LogiCORE™ IP solutions from Xilinx.

- [Ethernet 1000BASE-X PCS/PMA or SGMII Core](#)
- [Ethernet Statistics Core](#)
- [Ethernet AVB Endpoint](#)

Ethernet 1000BASE-X PCS/PMA or SGMII Core

The Ethernet MAC core can be integrated in a single device with the Ethernet 1000BASE-X PCS/PMA or SGMII core to provide either:

- A MAC with an SGMII interface to an external PHY chip. SGMII can support either tri-speed (10/100/1000 Mb/s) designs or 1 Gb/s only designs.
- A 1 Gb/s Ethernet MAC core with 1000BASE-X PCS/PMA sublayer functionality: this is a 1 Gb/s only PHY standard which is most commonly used for a fibre optic medium.

For more details on the Xilinx no charge Ethernet 1000BASE-X PCS/PMA or SGMII core, see the product page at:

www.xilinx.com/products/ipcenter/DO-DI-GMIIT01GBSXPCS.htm

The *Ethernet 1000BASE-X PCS/PMA or SGMII User Guide* provides the information required to connect the two cores together in any supported configuration.

Ethernet Statistics Core

The Ethernet MAC core can be integrated in a single device with the Ethernet Statistics core. Using the Ethernet Statistics core lets you:

- Count statistics based on the `emaccliintrxstats[27:0]` and `emaccllientxstats[31:0]` outputs from the TEMAC.
- Select 32-bit or 64-bit counters.
- Specify which statistics are counted and have precise control of the conditions under which the counters are incremented.
- Read statistics optionally through the host interface of the TEMAC or independently of the MAC.

For more details on the Xilinx no charge Ethernet Statistics core, see the product page at:

[Ethernet Statistics product page](#)

The *Ethernet Statistics User Guide* provides the information required to connect the two cores together.

Ethernet AVB Endpoint

The TEMAC core can be integrated in a single device with the Ethernet AVB Endpoint core. Ethernet Audio Video Bridging (AVB) is an emerging standard which provides quality of service for live media (audio or/and video) streams. The Ethernet AVB Endpoint core:

- Connects seamlessly to the client interface of any of the TEMAC solutions IP cores, enhancing standard ethernet operation to provide:
 - A dedicated client interface for media streams, including bandwidth reservation and policing across this interface.
 - A separate *legacy* client interface for ethernet data (none time sensitive data).
- A PLB interface, which connects to the Management I/F of the Ethernet MAC and allows easy integration into the EDK environment.
- Dedicated hardware and software drivers required to synchronize to a network wide Real Time Clock.

Ethernet AVB is defined under IEEE802.1.

For more details on the Xilinx[®] Ethernet AVB Endpoint core, see the product page at:

www.xilinx.com/products/ipcenter/DO-DI-EAVB-EPT.htm

The *Ethernet AVB Endpoint User Guide* provides the information required to connect the two cores together.

Implementing Your Design

This chapter describes how to simulate and implement your design containing the Ethernet MAC core.

Pre-implementation Simulation

The CORE Generator™ software generates a functional model of the core netlist to allow simulation of the block in the design phase of the project.

Using the Simulation Model

For information on setting up your simulator to use the functional model, see the Xilinx® *Synthesis and Verification Design Guide*, included in your Xilinx software installation.

The model is provided in the CORE Generator software project directory.

VHDL

`<component_name>.vhd`

Verilog

`<component_name>.v`

This model can be compiled along with your code to simulate the overall system.

Synthesis

XST - VHDL

In the CORE Generator software project directory, there is a `<component_name>.vho` file that is a component and instantiation template for the core. Use this to help instance the core into your VHDL source.

After your entire design is complete, create the following:

- An XST project file `top_level_module_name.prj` listing all your source code files
- An XST script file `top_level_module_name.scr` containing your required synthesis options

To synthesize the design, run:

```
$ xst -ifn top_level_module_name.scr
```

See the *XST User Guide* for details on creating project and synthesis script files, and running the xst program.

XST - Verilog

In the CORE Generator software project directory, locate the module declaration for the core at:

```
<component_name>/implement/<component_name>_mod.v
```

Use this module to help instance the core into your Verilog source.

After your entire design is complete, create

- An XST project file *top_level_module_name.prj* listing all your source code files. Be sure to include

```
%XILINX%/verilog/src/ise/unisim_comp.v
```

and

```
<component_name>/implement/component_name_mod.v
```

as the first two files in the project list.

- An XST script file *top_level_module_name.scr* containing your required synthesis options

To synthesize the design, run

```
$ xst -ifn top_level_module_name.scr
```

See the *XST User Guide* for details on creating project and synthesis script files and running the xst program.

Implementation

Generating the Xilinx Netlist

To generate the Xilinx netlist, the ngdbuild tool is used to translate and merge the individual design netlists into a single design database, the NGD file. Also merged at this stage is the UCF for the design. An example of the ngdbuild command is:

```
$ ngdbuild -sd path_to_core_netlist -sd path_to_user_synth_results \
  -uc top_level_module_name.ucf top_level_module_name
```

Mapping the Design

To map the logic gates of your design netlist into the CLBs and IOBs of the FPGA, run the map command. The map command writes out a physical design to an NCD file. An example of the map command is:

```
$ map -ol high -timing top_level_module_name \
  -o top_level_module_name_map.ncd
```

Placing and Routing the Design

To place and route your design's logic components (mapped physical logic cells) contained within an NCD file in accordance with the layout and timing requirements specified within the PCF file, the `par` command must be executed. The `par` command outputs the placed and routed physical design to an NCD file. An example of the `par` command is:

```
$ par -ol high top_level_module_name_map.ncd \  
top_level_module_name.ncd top_level_module_name.pcf
```

Static Timing Analysis

To evaluate timing closure on a design and create a Timing Report file (TWR) derived from static timing analysis of the Physical Design file (NCD), the `trce` command must be executed. The analysis is typically based on constraints included in the optional PCF file. An example of the `trce` command is:

```
$ trce -o top_level_module_name.twr top_level_module_name.ncd \  
top_level_module_name.pcf
```

Generating a Bitstream

To create the configuration bitstream (BIT) file based on the contents of a physical implementation file (NCD), the `bitgen` command must be executed. The BIT file defines the behavior of the programmed FPGA. An example of the `bitgen` command is:

```
$ bitgen -w top_level_module_name.ncd
```

Post-Implementation Simulation

The purpose of post-implementation simulation is to verify that the design as implemented in the FPGA works as expected.

Generating a Simulation Model

To generate a chip-level simulation netlist for your design, run the `netgen` command.

VHDL

```
$ netgen -sim -ofmt vhdl -ngm top_level_module_name_map.ngm \  
-tm netlist top_level_module_name.ncd \  
top_level_module_name_postimp.vhd
```

Verilog

```
$ netgen -sim -ofmt verilog -ngm top_level_module_name_map.ngm \  
-tm netlist top_level_module_name.ncd \  
top_level_module_name_postimp.v
```

Using the Model

For information on setting up your simulator to use the pre-implemented model, consult the Xilinx *Synthesis and Verification Design Guide*, included in your Xilinx software installation.

Other Implementation Information

For details about using the Xilinx implementation tool flow, including command line switches and options, see the Xilinx ISE[®] software manuals.

Quick Start Example Design

This chapter provides instructions for generating the TEMAC example design using the default parameters.

Overview

The TEMAC example design consists of the following:

- A TEMAC solution netlist
- An HDL example design
- A demonstration test bench to exercise the example design

Figure 16-1 shows the block diagram for the example design and the test bench provided with the TEMAC solution. The example design has been tested with Xilinx® ISE® software v13.1, Cadence Incisive Enterprise Simulator (IES) v10.2, Mentor Graphics ModelSim v6.6d, and Synopsys VCS and VCS MX 2010.06.

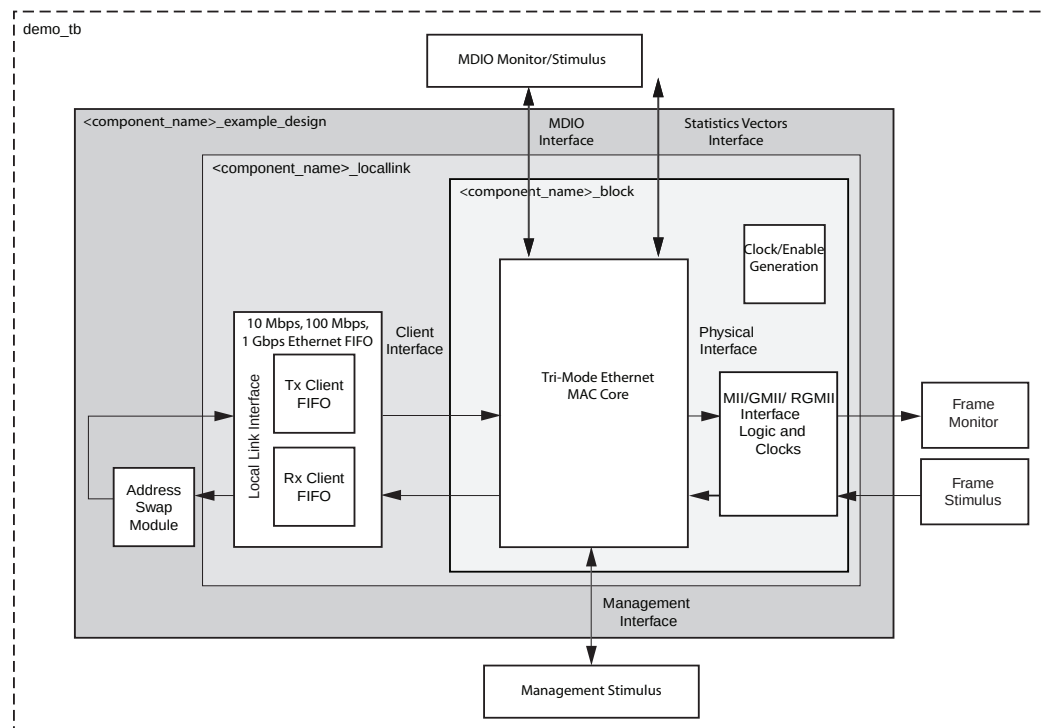


Figure 16-1: Default Example Design and Test Bench

Generating the Core

To begin using the TEMAC example design, start by generating the core.

To generate the core:

1. Start the CORE Generator™ software.

For help starting and using the CORE Generator software, see the documentation supplied with the ISE software, including the *CORE Generator Guide* at www.xilinx.com/support/software_manuals.htm.

2. Choose File > New Project.

3. Do the following to set project options:

- From Target Architecture, select an FPGA family that supports the core, for example Virtex®-5 device.

Note: If an unsupported silicon family is selected, the core does not appear in the taxonomy tree. For a list of supported architectures, see the *Tri-Mode Ethernet MAC Data Sheet*.

- For Design Entry, select either VHDL or Verilog; for Vendor, select Other.

4. After creating the project, locate the directory containing the core in the taxonomy tree. The project appears under one of the following:

- Communications & Networking /Ethernet
- Communications & Networking /Networking
- Communications & Networking /Telecommunications

5. Double-click the core. A warning may appear regarding the limitations of the simulation-only evaluation license.

- Click OK to exit the dialog box. The core customization screen appears.

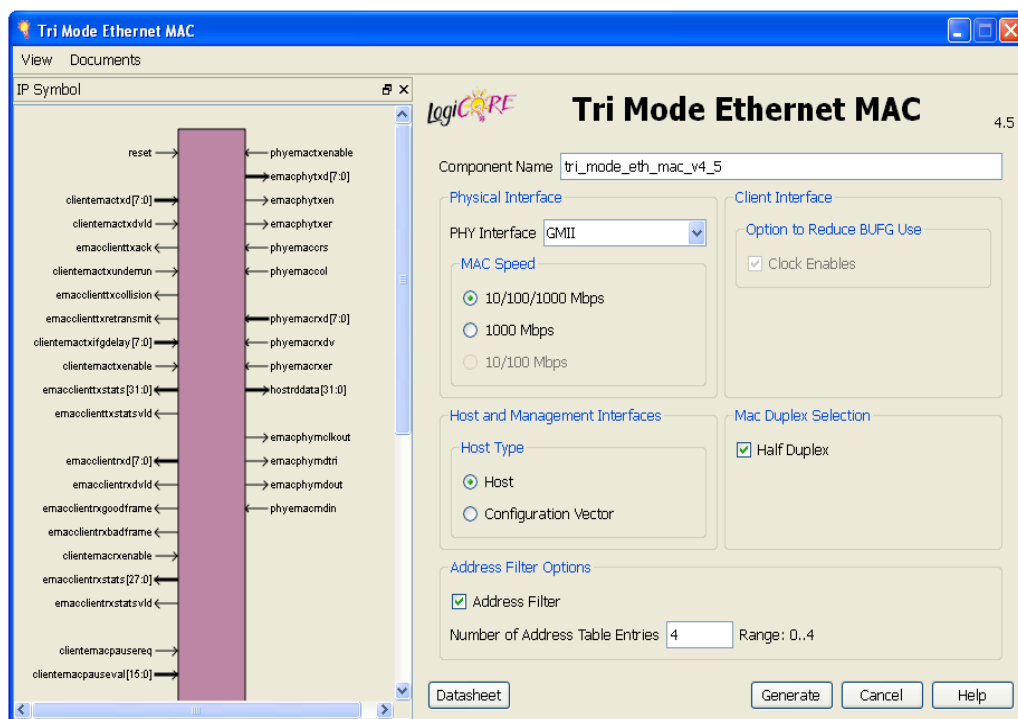


Figure 16-2: Tri-Mode Ethernet MAC Main Screen

- In the Component Name field, enter a name for the core instance.
- Accept the remaining defaults and click Generate to generate the core.
- The core and its supporting files, including the example design, are generated in your project directory. For a detailed description of the design example files and directories, see [Chapter 17, Detailed Example Design](#).

After the core is generated, a functional simulation directory is created, which contains scripts to simulate the core using the structural HDL models. See [Functional Simulation](#), page 234.

Implementing the Example Design

If the core is generated with a Hardware Evaluation or a Full license, the netlist and HDL example design can be processed through the Xilinx implementation toolset. The generated output files include several scripts to assist you in running the Xilinx software.

Note: In the following examples, *<project_dir>* is the CORE Generator software project directory and *<component_name>* is the name entered in the customization dialog box.

Open a command prompt or shell in your project directory, then enter the following commands:

Linux

```
% cd <component_name>/implement
% ./implement.sh
```

Windows

```
ms-dos> cd <component_name>\implement
ms-dos> implement.bat
```

These commands execute a script that synthesizes, builds, maps, and place-and-routes the example design together with the core. The script also generates a gate-level model of the example design and a netlist for use in timing simulation. The resulting files are placed in the results directory, which is created by the implement script at run time. For information on the implementation steps, see [Chapter 15, Implementing Your Design](#).

Running the Simulation

Functional Simulation

To run the functional simulation, you must have the Xilinx Simulation Libraries compiled for your system. See *Compiling Xilinx Simulation Libraries (COMPXLIB)* in the *Xilinx ISE Synthesis and Verification Design Guide*, which can be obtained from www.xilinx.com/support/software_manuals.htm.

Note: In the simulation examples that follow, *<project_dir>* is the CORE Generator software project directory, and *<component_name>* is the component name as entered in the core customization dialog box.

VHDL Simulation

To run a VHDL functional simulation:

1. Open a command prompt or shell and set the current directory to:
<project_dir>/<component_name>/simulation/functional
2. Launch the simulation script:

```
ModelSim: vsim -do simulate_mti.do
IES: ./simulate_ncsim.sh
```

The scripts compile the structural VHDL model, the example design files and the demonstration test bench, add some relevant signals to a wave window, then run the simulation to completion. At this point, you can review the simulation transcript and waveform to observe the operation of the core.

Verilog Simulation

To run a Verilog functional simulation:

1. Open a command prompt or shell and set the current directory to
`<project_dir>/<component_name>/simulation/functional`
2. Launch the simulation script:

```
ModelSim: vsim -do simulate_mti.do
```

```
IES: ./simulate_ncsim.sh
```

```
VCS: ./simulate_vcs.sh
```

The scripts compile the structural Verilog model, the example design files and the demonstration test bench, add some relevant signals to a wave window, then run the simulation to completion. At this point, you can review the simulation transcript and waveform to observe the operation of the core.

Timing Simulation

If the core is generated with a Hardware Evaluation or a Full license, then Timing Simulations are supported. Ensure that the example design has been implemented before attempting a Timing Simulation.

To run the gate-level simulation, you must have the Xilinx Simulation Libraries compiled for your system. See *Compiling Xilinx Simulation Libraries (COMPXLIB)* in the *Xilinx ISE Synthesis and Verification Design Guide*, which can be obtained from www.xilinx.com/support/software_manuals.htm.

Note: In the simulation examples that follow, *<project_dir>* is the CORE Generator software project directory; *<component_name>* is the component name entered in the core customization dialog box.

VHDL Simulation

To run a VHDL timing simulation:

1. Open a command prompt or shell and set the current directory to
`<project_dir>/<component_name>/simulation/timing`
2. Launch the simulation script:

```
ModelSim: vsim -do simulate_mti.do
```

```
IES: ./simulate_ncsim.sh
```

The scripts compile the gate-level model and the demonstration test bench, add some relevant signals to a wave window, then run the simulation to completion. At this point, you can review the simulation transcript and waveform to observe the operation of the core.

Verilog Simulation

To run a Verilog timing simulation:

1. Open a command prompt or shell and set the current directory to
`<project_dir>/<component_name>/simulation/timing`
2. Launch the simulation script:

```
ModelSim: vsim -do simulate_mti.do
```

```
IES: ./implement_ncsim.sh
```

```
VCS: ./implement_vcs.sh
```

The scripts compile the gate-level model and the demonstration test bench, add some relevant signals to a wave window, then run the simulation to completion. At this point, you can review the simulation transcript and waveform to observe the operation of the core.

What's Next?

For detailed information about the example design, including guidelines for modifying the design and extending the test bench, see [Chapter 17, Detailed Example Design](#). To begin using the TEMAC solution in your own design, see the *Tri-Mode Ethernet MAC User Guide*.

Detailed Example Design

This chapter provides detailed information about the example design, including a description of files and the directory structure generated by the Xilinx® CORE Generator™ software, the purpose and contents of the provided scripts, the contents of the example HDL wrappers, and the operation of the demonstration test bench.

-  [`<project directory>`](#)
Top-level project directory; name is user-defined.
 -  [`<project directory>/<component name>`](#)
Core release notes file
 -  [`<component name>/doc`](#)
Product documentation
 -  [`<component name>/example design`](#)
Verilog and VHDL design files
 -  [`example design/fifo`](#)
Files for the FIFO that is instantiated in the LocalLink example
 -  [`example\_design/physical`](#)
Files for the physical interface of the MAC
 -  [`<component name>/implement`](#)
Implementation script files
 -  [`implement/results`](#)
Results directory, created after implementation scripts are run, and contains implement script results
 -  [`<component name>/simulation`](#)
Simulation scripts
 -  [`simulation/functional`](#)
Functional simulation files
 -  [`simulation/timing`](#)
Timing simulation files

Directory and File Contents

The core directories and their associated files are defined in the following sections.

Note: The implement and timing simulation directories are only present when the core is generated with a Full System Hardware Evaluation license or a Full license.

<project directory>

The project directory contains all the CORE Generator software project files.

Table 17-1: Project Directory

Name	Description
<project_dir>	
<component_name>.ngc	Binary Xilinx implementation netlist. Describes how the core is to be implemented. Used as input to the Xilinx Implementation Tools.
<component_name>.v[hd]	VHDL or Verilog structural simulation model. File used to support functional simulation of a core. The model passes customized parameters to the generic core simulation model.
<component_name>.xco	As an output file, the XCO file is a log file which records the settings used to generate a particular core. An XCO file is generated by the CORE Generator software for each core that it creates in the current project directory. An XCO file can also be used as an input to the CORE Generator software.
<component_name>_flist.txt	Text file that defines all the output files produced when a customized core is generated using the CORE Generator software.
<component_name>.{veo vho}	Verilog or VHDL template for the core. This can be copied into your design.

<project directory>/<component name>

The <component name> directory contains the release notes file provided with the core, which can include last-minute changes and updates.

Table 17-2: Component Name Directory

Name	Description
<project_dir>/<component_name>	
tri_mode_eth_mac_readme.txt	Core release notes file

<component name>/doc

The doc directory contains the PDF documentation provided with the core.

Table 17-3: Doc Directory

Name	Description
<project_dir>/<component_name>/doc	
tri_mode_eth_mac_ds297.pdf	Tri-Mode Ethernet MAC Data Sheet
tri_mode_eth_mac_ug138.pdf	Tri-Mode Ethernet MAC User Guide

<component name>/example design

This directory (and subdirectories) contain all of the support files necessary for a VHDL or Verilog implementation of the example design. [Table 17-4](#) defines the HDL files that are always present in this directory. Example designs for certain implementations can contain extra files for clock/clock enable generation logic.

See [Example Design, page 246](#) for details.

Table 17-4: Example Design Directory

Name	Description
<project_dir>/<component_name>/example_design	
<component_name>_example_design.v[hd]	Top-level VHDL or Verilog file for the example design. This instantiates the LocalLink block along with the address swap block, providing a simple loopback function.
<component_name>_example_design.ucf	User constraints file (UCF) for the core and the example design
<component_name>_locallink.v[hd]	Example design with a LocalLink client interface. This instantiates the block level TEMAC wrapper together with a receive and a transmit FIFO.
<component_name>_block.v[hd]	Block-level TEMAC wrapper containing the core and all clocking and physical interface circuitry
<component_name>_mod.v	Verilog module declaration for the core instance in the example design
address_swap_module.v[hd]	Top-level example design instances this to swap the source and destination addresses of the incoming frames
sync_block.v[hd]	Synchronizer module, used for passing signals across a clock domain.
reset_sync.v[hd]	Local reset synchronizer, used to create a synchronous reset output signal from an asynchronous input.

example_design/fifo

This directory contains the files for the FIFO that is instantiated in the LocalLink example design.

Table 17-5: FIFO Directory

Name	Description
<project_dir>/<component_name>/example_design/fifo	
tx_client_fifo.v[hd]	Transmit client FIFO. This takes data from the client in LocalLink format, stores it and sends it to the MAC.
rx_client_fifo.v[hd]	Receive client FIFO. This reads in and stores data from the MAC before outputting it to the client in LocalLink format.
ten_100_1G_eth_fifo.v[hd]	FIFO top level. This instantiates the transmit and receive client FIFOs.

example_design/physical

This directory contains a file for the physical interface of the MAC. A GMII, MII or RGMII version is delivered by the CORE Generator software depending on the selected option.

Table 17-6: Physical Directory

Name	Description
<project_dir>/<component_name>/example_design/physical	
mii_if.v[hd]	For MII only: all clocking and logic required to provide an MII physical interface
gmii_if.v[hd]	For GMII only: all clocking and logic required to provide a GMII physical interface
rgmii_v2_0_if.v[hd]	For RGMII only: all clocking and logic required to provide a RGMII v2.0 physical interface
dcm_reset.v[hd]	Only present when a DCM is used (Virtex®-4 devices and all Spartan®-3 devices). This is a self-contained module for resetting a DCM following Power-on-Reset or loss of lock.

<component_name>/implement

This directory is only available when the core is generated with a Hardware Evaluation or a Full license.

This directory contains the support files necessary for implementation of the example design with the Xilinx® tools. See [Example Design, page 246](#). Execution of an implement script results in creation of the results directory and an xst project directory.

Table 17-7: Implement Directory

Name	Description
<project_dir>/<component_name>/implement	
implement.sh	Linux shell script that processes the example design through the Xilinx tool flow
implement.bat	Windows batch file that processes the example design through the Xilinx tool flow
xst.prj	XST project file for the example design; it enumerates all the HDL files that need to be synthesised.
xst.scr	XST script file for the example design
example_design_xst.xcf	Constraints file automatically used by XST

implement/results

This directory is created by the implement scripts and is used to run the example design files and the <component_name>.ngc file through the Xilinx implementation tools. On completion of an implement script, this directory contains the following files for timing simulation. Output files from the Xilinx implementation tools are also located in this directory.

Table 17-8: Results Directory

Name	Description
<project_dir>/<component_name>/implement/results	
routed.v[hd]	Back-annotated SimPrim based gate-level VHDL or Verilog design. Used for timing simulation.
routed.sdf	Timing information for simulation

<component name>/simulation

The simulation directory and the subdirectories below it provide the files necessary to test a VHDL or Verilog implementation of the example design.

Table 17-9: Simulation Directory

Name	Description
<project_dir>/<component_name>/simulation	
demo_tb.v[hd]	VHDL or Verilog demonstration test bench for the TEMAC solution

simulation/functional

The functional directory contains functional simulation scripts provided with the core.

Table 17-10: Functional Directory

Name	Description
<project_dir>/<component_name>/simulation/functional	
simulate_mti.do	ModelSim macro file that compiles the example design sources and the structural simulation model then runs the functional simulation to completion.
wave_mti.do	ModelSim macro file that opens a wave window and adds interesting signals to it. It is called by the simulate_mti.do macro file.
simulate_ncsim.sh	Cadence IES script file that compiles the example design sources and the structural simulation model and then runs the functional simulation to completion.
wave_ncsim.sv	Cadence IES macro file that opens a wave window and adds interesting signals to it.
simulate_vcs.sh	VCS script file that compiles the Verilog sources and runs the functional simulation to completion.
ucli_commands.key	This file is sourced by VCS at the start of simulation: it configures the simulator.
vcs_session.tcl	VCS macro file that opens a wave window and adds signals of interest to it. It is called by the simulate_vcs.sh script file.

simulation/timing

This directory is only available when the core is generated with a Hardware Evaluation or a Full license.

The timing directory contains timing simulation scripts provided with the core.

Table 17-11: Timing Directory

Name	Description
<project_dir>/<component_name>/simulation/timing	
<code>simulate_mti.do</code>	ModelSim macro file that compiles the VHDL gate-level model and demo test bench then runs the timing simulation to completion.
<code>wave_mti.do</code>	ModelSim macro file that opens a wave window and adds interesting signals to it. It is called used by the <code>simulate_mti.do</code> macro file.
<code>simulate_ncsim.sh</code>	Linux shell script that compiles the VHDL gate-level model and demo test bench and then runs the timing simulation to completion using Cadence IES.
<code>wave_ncsim.sv</code>	IES macro file that opens a wave window and adds interesting signals to it. It is used by the <code>simulate_ncsim.sh</code> script.
<code>simulate_vcs.sh</code>	VCS script file that compiles the Verilog sources and runs the timing simulation to completion.
<code>ucli_commands.key</code>	This file is sourced by VCS at the start of simulation: it configures the simulator.
<code>vcs_session.tcl</code>	VCS macro file that opens a wave window and adds signals of interest to it. It is called by the <code>simulate_vcs.sh</code> script file.

Implementation and Test Scripts

Implementation Scripts for Timing Simulation

When the CORE Generator software is run with a Full System Hardware license or a Full license, an implement script is generated in the `<project_dir>/<component_name>/implement` directory. The implementation script is either a shell script or batch file that processes the example design through the Xilinx tool flow.

Linux

```
<project_dir>/<component_name>/implement/implement.sh
```

Windows

```
<project_dir>/<component_name>/implement/implement.bat
```

The implement script performs the following steps:

- The HDL example design is synthesized using XST.
- NGDBuild is run to consolidate the core netlist and the HDL example netlist into the NGD file containing the entire design.
- The design is mapped to the target technology.
- The design is place-and-routed on the target device.
- Static timing analysis is performed on the routed design using trce.
- A bitstream is generated.
- Netgen runs on the routed design to generate VHDL and Verilog netlists and timing information in the form of SDF files.

The Xilinx tool flow generates several output and report files. These are saved in the following directory which is created by the implement script:

```
<project_dir>/<component_name>/implement/results
```

Test Scripts For Functional Simulation

The functional simulation flow is available with any license type, and the test script that automates the simulation of the test bench is located in one of the following locations:

Mentor ModelSim

```
<project_dir>/<component_name>/simulation/functional/simulate_mti.do
```

Cadence IES

```
<project_dir>/<component_name>/simulation/functional/simulate_ncsim.sh
```

Synopsys VCS

```
<project_dir>/<component_name>/simulation/functional/simulate_vcs.sh
```

The test script performs the following tasks:

- Compiles the structural simulation model of the core
- Compiles the example design files
- Compiles the demonstration test bench
- Starts a simulation of the test bench with no timing information
- Opens a Wave window and adds some signals of interest
- Runs the simulation to completion

Test Scripts For Timing Simulation

When CORE Generator software has been run with a Full System Hardware Evaluation license or Full license, a test script for running timing simulation is generated. The test script that automates the simulation of the test bench is located at the following locations:

Mentor ModelSim

```
<project_dir>/<component_name>/simulation/timing/simulate_mti.do
```

Cadence IES

```
<project_dir>/<component_name>/simulation/timing/simulate_ncsim.sh
```

Synopsys VCS

```
<project_dir>/<component_name>/simulation/timing/simulate_vcs.sh
```

The test script performs the following tasks:

- Compiles the gate-level model of the example design
- Compiles the demonstration test bench
- Starts a simulation of the test bench using timing information
- Opens a Wave window and adds some signals of interest
- Runs the simulation to completion

Example Design

HDL Example Design

Figure 17-1 illustrates the top-level design for the TEMAC solution example design.

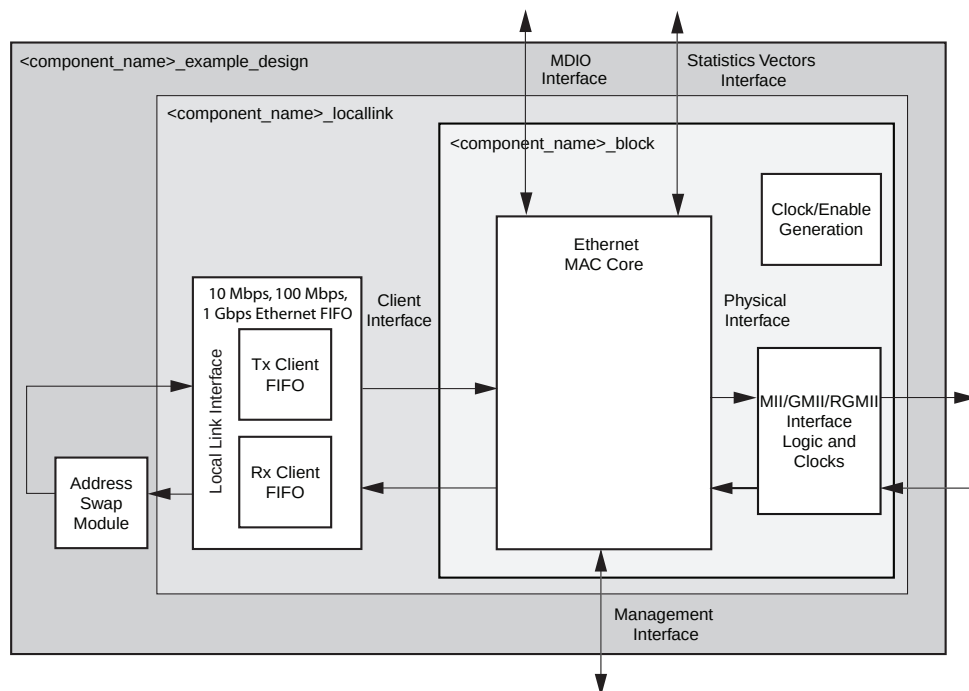


Figure 17-1: HDL Example Design

The top-level example design for the TEMAC solution is defined in the following files:

VHDL

```
<project_dir>/<component_name>/example_design/  
<component_name>_example_design.vhd
```

Verilog

```
<project_dir>/<component_name>/example_design/  
<component_name>_example_design.v
```

The HDL example design contains the following:

- An instance of the TEMAC solution
- Clock management logic, including DCM and Global Clock Buffer instances, where required
- MII, GMII or RGMII interface logic, including IOB and DDR registers instances, where required
- Client Transmit and Receive FIFOs with a LocalLink interface
- Client LocalLink loopback module that performs address swapping

The HDL example design provides client loopback functionality on the client side of the TEMAC solution and connects the GMII/RGMII interface to external IOBs. This allows the functionality of the core to be demonstrated either using a simulation package, as discussed in this guide, or in hardware, if placed on a suitable board.

10 Mb/s /100 Mb/s/1 Gb/s Ethernet FIFO

The 10 Mb/s/100 Mb/s/1 Gb/s Ethernet FIFO is described in the following files:

VHDL

```
<project_dir>/<component_name>/example_design/fifo/ten_100_1g_eth_fifo.vhd
<project_dir>/<component_name>/example_design/fifo/tx_client_fifo.vhd
<project_dir>/<component_name>/example_design/fifo/rx_client_fifo.vhd
```

Verilog

```
<project_dir>/<component_name>/example_design/fifo/ten_100_1g_eth_fifo.v
<project_dir>/<component_name>/example_design/fifo/tx_client_fifo.v
<project_dir>/<component_name>/example_design/fifo/rx_client_fifo.v
```

For a full description of the 10 Mb/s/100 Mb/s/1 Gb/s Ethernet FIFO, see [Appendix A](#). In addition, see the “*Parameterizable LocalLink FIFO Application Note*” (XAPP691) for a detailed description of the LocalLink interface.

The 10 Mb/s/100 Mb/s/1 Gb/s Ethernet FIFO contains an instance of `tx_client_fifo` to connect to the MAC client side transmitter interface, and an instance of the `rx_client_fifo` to connect to the MAC client receiver interface. Both transmit and receive FIFO components implement a LocalLink user interface, through which the frame data can be read/written. [Figure 17-2](#) illustrates a straightforward frame transfer across the LocalLink interface.

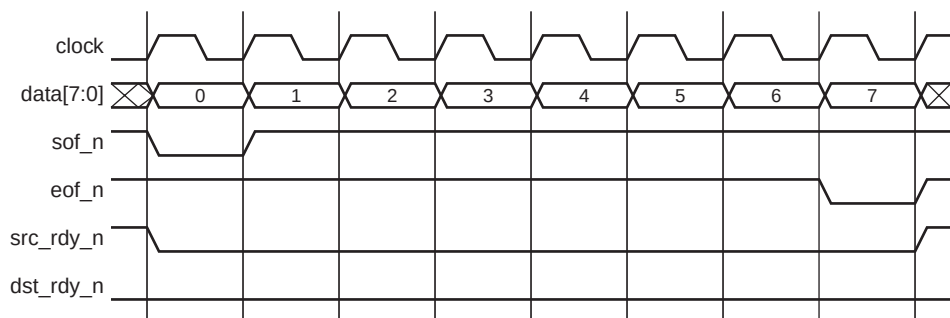


Figure 17-2: Frame Transfer across LocalLink Interface

rx_client_fifo

The rx_client_fifo is built around two Dual Port Block RAMs, giving a total memory capacity of 4096 bytes. The receive FIFO writes in data received through the TEMAC solution. If the frame is marked as good, that frame is presented on the LocalLink interface for reading by you, (in this case the tx_client_fifo module). If the frame is marked as bad, that frame is dropped by the receive FIFO.

If the receive FIFO memory overflows, the frame currently being received is dropped, regardless of whether it is a good or bad frame, and the signal rx_overflow is asserted. Situations in which the memory can overflow are:

- The FIFO can overflow if the receiver clock is running at a faster rate than the transmitter clock or if the interpacket gap between the received frames is smaller than the interpacket gap between the transmitted frames. If this is the case, the tx FIFO will not be able to read data from the rx FIFO as fast as it is being received.
- The FIFO size of 4096 bytes limits the size of the frames that it can store without error. If a frame is larger than 4000 bytes, the FIFO can overflow and data will be lost. It is therefore recommended that the example design is not used with the TEMAC solution in jumbo frame mode for frames of larger than 4000 bytes.

tx_client_fifo

The tx_client_fifo is built around two Dual Port Block RAMs, giving a total memory capacity of 4096 bytes.

When a full frame has been written into the transmit FIFO, the FIFO presents data to the MAC transmitter. On receiving the tx_ack signal from the MAC, the rest of the frame is transmitted.

If the FIFO memory fills up, the dst_rdy_out_n signal is used to halt the LocalLink interface writing in data, until space becomes available in the FIFO. If the FIFO memory fills up but no full frames are available for transmission. For example if a frame larger than 4000 bytes is written into the FIFO, the FIFO asserts the tx_overflow signal and continue to accept the rest of the frame from you. The overflow frame is dropped by the FIFO. This ensures that the LocalLink interface does not lock up.

Address Swap Module

VHDL

```
<project_dir>/<component_name>/example_design/address_swap_module.vhd
```

Verilog

```
<project_dir>/<component_name>/example_design/address_swap_module.v
```

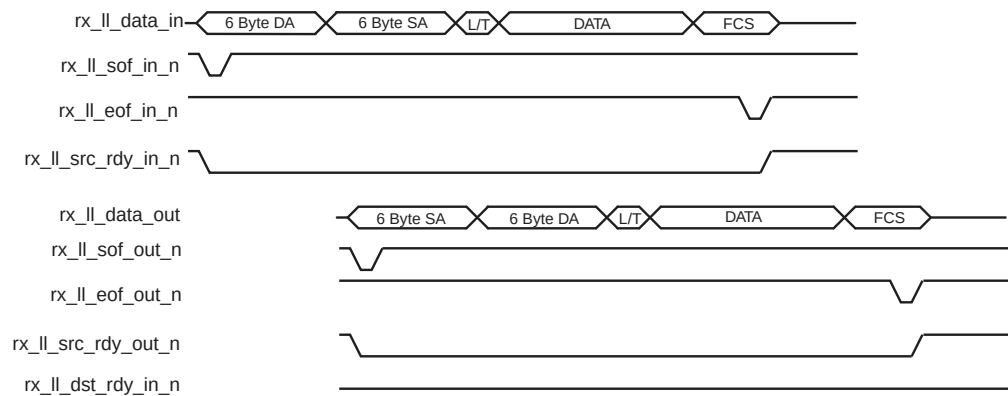


Figure 17-3: Modification of Frame Data by Address Swap Module

The address swap module takes frame data from the MAC receiver client LocalLink interface. The module swaps the destination and source addresses of each frame as shown in Figure 17-3 to ensure that the outgoing frame destination address matches the source address of the link partner. The module transmits the frame control signals with an equal latency to the frame data.

Demonstration Test Bench

Test Bench Functionality

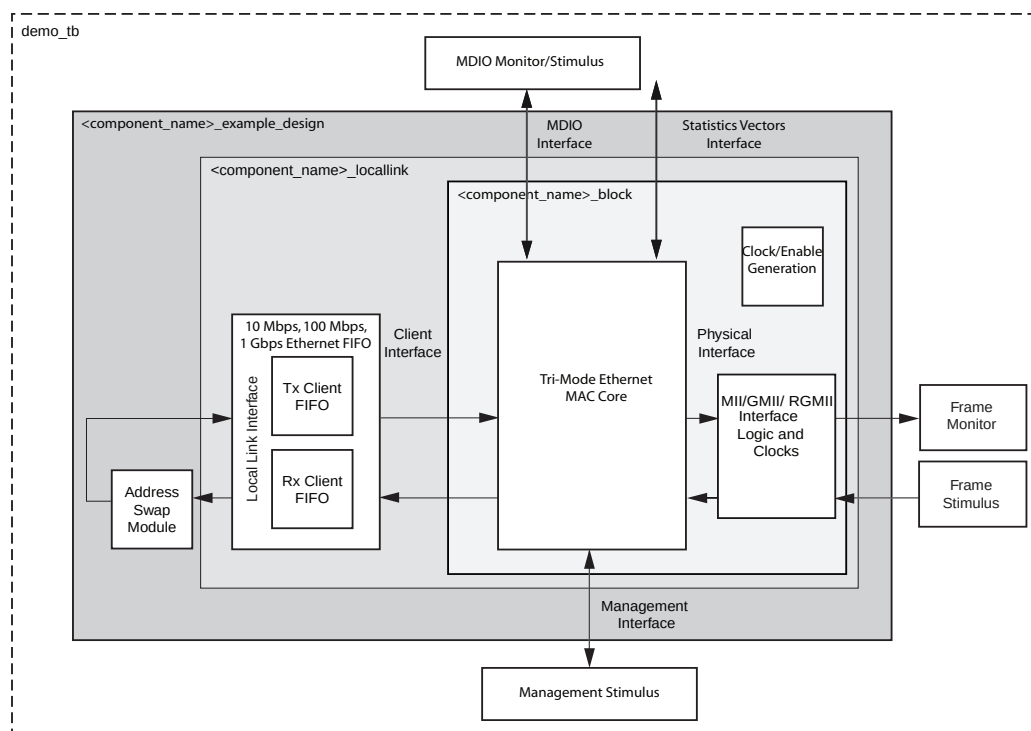


Figure 17-4: Demonstration Test Bench

The demonstration test bench is defined in the following files:

VHDL

```
<project_dir>/<component_name>/simulation/demo_tb.vhd
```

Verilog

```
<project_dir>/<component_name>/simulation/demo_tb.v
```

The demonstration test bench is a simple VHDL or Verilog program to exercise the example design and the core itself.

The test bench consists of the following:

- Clock generators
- A stimulus block that connects to the GMII/MII or RGMII receiver interface of the example design
- A monitor block to check data returned through the GMII/MII or RGMII transmitter interface
- A management block to exercise the management interface, if selected
- An MDIO monitor/stimulus to check and respond to MDIO accesses, if a management interface is selected.
- A control mechanism to manage the interaction of management, stimulus and monitor blocks

Core with Management Interface

The demonstration test bench performs the following tasks:

- Input clock signals are generated.
- A reset is applied to the example design.
- The TEMAC solution is configured through the management interface, setting up the MDC clock frequency and disabling flow control. If the Address Filter is selected, the Unicast Address registers are configured and the Address Filter is enabled. The MAC speed is configured to the fastest supported in the configuration selected.
- Four frames are pushed into the GMII/MII or RGMII receiver interface at the fastest MAC speed supported:
 - The first frame is a minimum length frame
 - The second frame is a type frame
 - The third frame is an errored frame
 - The fourth frame is a padded frame
- The frames received at the GMII/MII or RGMII transmitter interface are checked against the stimulus frames to ensure data is the same. The monitor process takes into account the source/destination address field and FCS modifications resulting from the address swap module.
- If either the Tri-speed or MII configurations have been selected, the TEMAC solution is configured through the management interface to run at the next fastest available speed. This is 100 Mb/s or 10 Mb/s respectively. The same four frames are then sent to the MII or RGMII interface and checked against the stimulus frames.
- If the Tri-speed configuration has been selected the TEMAC solution is configured through the management interface to run at 10 Mb/s. The same four frames are then sent to the MII or RGMII interface and checked against the stimulus frames. If the core is generated with the RGMII option, or if the core has been generated for a Spartan-3 device or Virtex-4 device, warnings about the input clock period of the `gmi_rxc_dcm` component *may* appear in timing simulations. These can be safely ignored, because the `gmi_rxc_dcm` component is bypassed at 10 and 100 Mb/s.
- For the Tri-speed configuration, the speed is then changed back to 1 Gb/s and the same four frames are sent and checked for a final time. This tests the speed switching between 1 Gb/s and 10/100 Mb/s in both directions
- If the TEMAC solution is configured to support 10 Mb/s then an MDIO read and write access are performed immediately prior to the four frames being sent. The MDIO Monitor responds to the read with all '1's. The write is expected to have bit 10 set to '0' and all other bits '1'.

Core with No Management Interface

The demonstration test bench performs the following tasks:

- Input clock signals are generated
- A reset is applied to the example design
- The TEMAC solution is configured through the configuration vector, disabling flow control and selecting the fastest supported MAC speed
- The stimulus block pushes four frames into the GMII/MII or RGMII receiver interface at the fastest speed supported by the selected configuration:
 - The first frame is a minimum-length frame
 - The second frame is a type frame
 - The third frame is an errored frame
 - The fourth frame is a padded frame
- The frames received at the GMII/MII or RGMII transmitter interface are checked against the stimulus frames to ensure data is the same. The monitor process takes into account the source/destination address field and FCS modifications resulting from the address swap module.
- If either the Tri-speed or MII configurations have been selected, the TEMAC solution is configured through the configuration vector to run at the next available speed. This is 100 Mb/s or 10 Mb/s respectively. The same four frames are then sent to the MII or RGMII interface and checked against the stimulus frames.
- If the Tri-speed configuration has been selected the TEMAC solution is configured through the configuration vector to run at 10 Mb/s. The same four frames are then sent to the MII or RGMII interface and checked against the stimulus frames. If the core is generated with the RGMII option, or if the core has been generated for a Spartan-3 device or Virtex-4 device, warnings about the input clock period of the `gmii_rxc_dcm` component *may* appear in timing simulations. These can be safely ignored, because the `gmii_rxc_dcm` component is bypassed at 10 and 100 Mb/s.
- For the Tri-speed configuration, the speed is then changed back to 1 Gb/s and the same four frames are sent and checked for a final time. This tests the speed switching between 1 Gb/s and 10/100 Mb/s in both directions.

Changing the Test Bench

Changing Frame Data

The contents of the frame data passed into the TEMAC receiver can be changed by editing the DATA fields for each frame defined in the test bench. The test bench automatically calculates the new FCS field to pass into the GEMAC, as well as calculating the new expected FCS value. Further frames can be added by defining a new frame of data.

Changing Frame Error Status

Errors can be inserted into any of the pre-defined frames by changing the error field to '1' in any column of that frame.

When an error is introduced into a frame, the `bad_frame` field for that frame must be set to disable the monitor checking for that frame.

The error currently written into the third frame can be removed by setting all error fields for the frame to '0' and unsetting the bad_frame field.

Changing the Tri-Mode Ethernet MAC Configuration

The configuration of the TEMAC used in the demonstration test bench can be altered.

Caution! Certain configurations of the TEMAC cause the test bench either to result in failure or cause processes to run indefinitely. You must determine which configurations can safely be used with the test bench.

If the Management Interface option has been selected, the TEMAC can be reconfigured by adding further steps in the test bench management process, to write new configurations to the TEMAC. See [Using the Management Interface in Chapter 9](#) for details on using the management interface.

If the Management Interface option has not been selected, the TEMAC can be reconfigured by modifying the configuration vector directly. See [Accessing Configuration without the Management Interface in Chapter 9](#) for information about using the configuration vector.

Performing MDIO accesses if 10 Mb/s is not supported

If the management interface is selected then the test bench is provided with the required MDIO monitor/stimulus logic. To perform an MDIO access at any other speed simply identify the p_management process and add the following commands after the relative call to the configure_mac task/procedure:

VHDL

```
p_mdio_read(<phy_addr>, <reg_addr>, rd_data_value);  
p_mdio_write(<phy_addr>, <reg_addr>, wr_data_value);
```

Verilog

```
mdio_read_task(<phy_addr>, <reg_addr>, rd_data_value);  
mdio_write_task(<phy_addr>, <reg_addr>, wr_data_value);
```

The MDIO monitor only responds to a phy_addr value of 7 with the reg_addr set to 0. Upon an MDIO read the returned data will be all '1's and the monitor will only provide a pass if bit 10 of wr_data_value is set to '0' (all other bits have to be '1').

Using the Client Side FIFO

The example design provided with the core contains a LocalLink FIFO used to interface to the client side of the core. The source code for the FIFO is provided and can be used and edited for user applications.

The 10 Mb/s/100 Mb/s/1 Gb/s Ethernet FIFO consists of independent transmit and receive FIFOs integrated in a top-level wrapper. [Figure A-1](#) shows how the FIFO fits into a typical implementation. Each FIFO is built around two Dual Port Block RAMs giving a memory capacity of 4096 bytes in each FIFO. This chapter describes the operation of the FIFO.

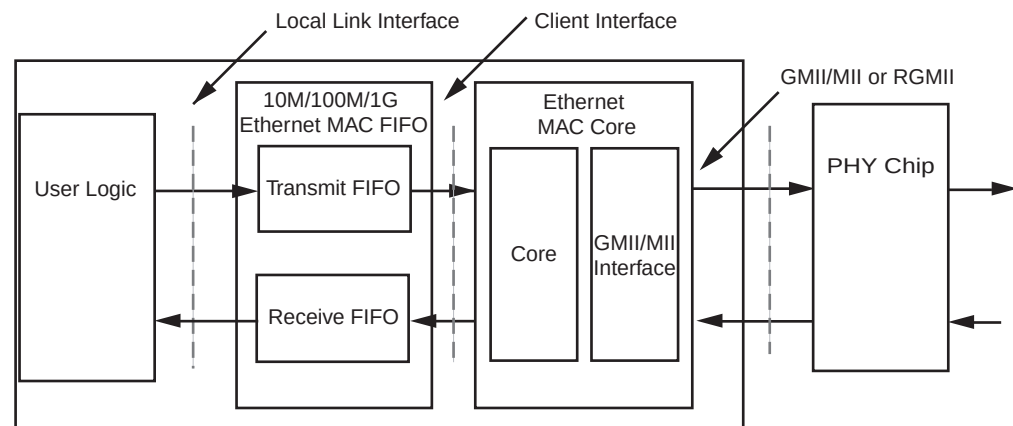


Figure A-1: Typical 10 M/100 M/1 G Ethernet FIFO Implementation

Overview of LocalLink Interface

Data is transferred on the LocalLink interface from source to destination, with the flow governed by the four active low control signals `sof_n`, `eof_n`, `src_rdy_n` and `dst_rdy_n`. The flow of data is controlled by the `src_rdy_n` and `dst_rdy_n` signals. Only when these signals are asserted simultaneously is data transferred from source to destination. The individual packet boundaries are marked by the `sof_n` and `eof_n` signals. For details on the LocalLink interface, see Xilinx® Application Note XAPP691, "Parameterizable LocalLink FIFO." [Figure A-2](#) shows the transfer of an 8-byte frame.

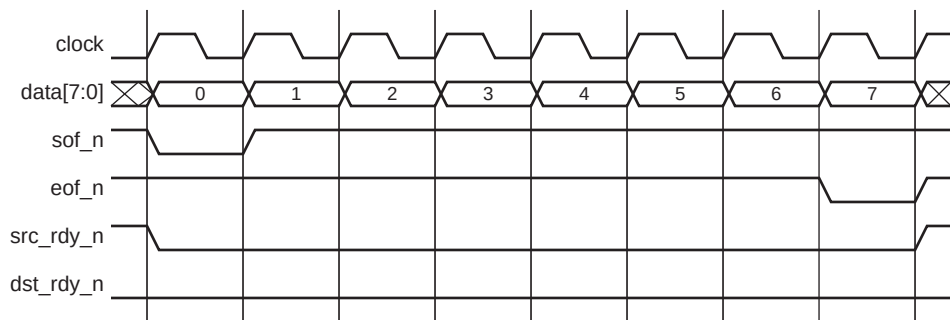


Figure A-2: Frame Transfer across LocalLink Interface

Figure A-3 illustrates frame transfer of a 5-byte frame, where both the `src_rdy_n` and `dst_rdy_n` signals are used to control the flow of data across the interface.

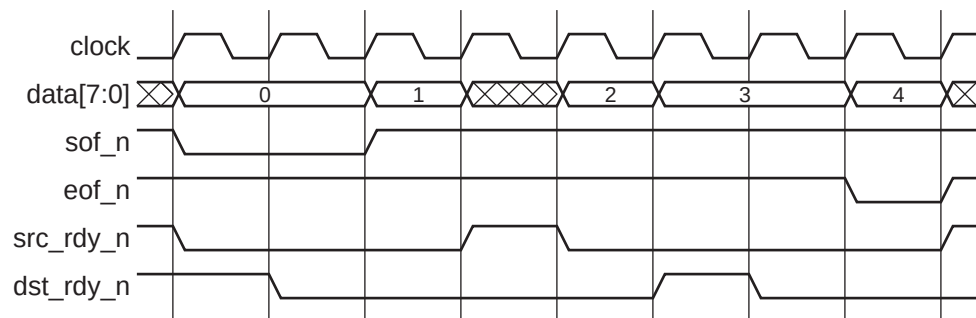


Figure A-3: Frame Transfer with Flow Control

Receive FIFO Operation

The receive FIFO takes data from the client interface of the core and converts it into LocalLink format. See [Receiving Inbound Frames, page 66](#) for a description of the receive client interface. If the frame is marked as good by the core, that frame is then presented on the LocalLink interface for reading by you. If the frame is marked as bad, that frame is dropped by the FIFO.

LocalLink Interface

[Table A-1](#) describes the receive FIFO LocalLink interface.

Table A-1: Receive FIFO LocalLink Interface

Signal	Direction	Clock Domain	Description
rx_ll_clock	Input	N/A	Read clock for LocalLink interface
rx_ll_reset	Input	rx_ll_clock	Synchronous reset
rx_ll_data_out[7:0]	Output	rx_ll_clock	Data read from FIFO
rx_ll_sof_out_n	Output	rx_ll_clock	Start of frame indicator
rx_ll_eof_out_n	Output	rx_ll_clock	End of frame indicator
rx_ll_src_rdy_out_n	Output	rx_ll_clock	Source ready indicator
rx_ll_dst_rdy_in_n	Input	rx_ll_clock	Destination ready indicator
rx_fifo_status[3:0]	Output	rx_ll_clock	FIFO memory status

If the receive FIFO memory overflows, the frame currently being received is dropped, regardless of whether it is a good or bad frame, and the signal `rx_overflow` is asserted. Frames continue to be dropped until space is made available in the FIFO by reading data out. The FIFO status signal indicates the occupancy of the FIFO.

Transmit FIFO Operation

The transmit FIFO accepts frames in LocalLink format and stores them in block RAM for transmission via the core. When a full frame is written into the transmit FIFO, the FIFO presents the data to the transmitter client interface. On receiving the `tx_ack` signal from the core, the rest of the frame is transmitted. For a description of the transmit client interface, see [Transmitting Outbound Frames](#), page 73.

LocalLink Interface

[Table A-2](#) shows the transmit FIFO LocalLink interface signals.

Table A-2: Transmit FIFO LocalLink Interface

Signal	Direction	Clock Domain	Description
<code>tx_ll_clock</code>	Input	N/A	Write clock for LocalLink interface
<code>tx_ll_reset</code>	Input	<code>tx_ll_clock</code>	Synchronous reset
<code>tx_ll_data_in[7:0]</code>	Input	<code>tx_ll_clock</code>	Write data to be sent to transmitter
<code>tx_ll_sof_in_n</code>	Input	<code>tx_ll_clock</code>	Start of frame indicator
<code>tx_ll_eof_in_n</code>	Input	<code>tx_ll_clock</code>	End of frame indicator
<code>tx_ll_src_rdy_in_n</code>	Input	<code>tx_ll_clock</code>	Source ready indicator
<code>tx_ll_dst_rdy_out_n</code>	Output	<code>tx_ll_clock</code>	Destination ready indicator
<code>tx_fifo_status[3:0]</code>	Output	<code>tx_ll_clock</code>	FIFO memory status

In half-duplex operation, if the client interface `emacclienttxcollision` signal is asserted by the core, the current frame transmission is terminated. If the `emacclienttxretransmit` signal is also asserted, the FIFO re-queues the frame for transmission.

If the FIFO memory fills up, the `dst_rdy_out_n` signal is used to halt the LocalLink interface writing in data, until space becomes available in the FIFO. If the FIFO memory fills up but no frames are available for transmission, that is, if a frame larger than 4000 bytes is written into the FIFO, the FIFO asserts the `tx_overflow` signal and continue to accept the rest of the frame from you. The overflow frame is dropped by the FIFO. This ensures that the LocalLink interface does not lock up.

Clock Requirements

The FIFO has been designed to work with transmitter/receiver clocks running at speeds in the range of 125 MHz to 1.25 GHz. The `rx_ll_clock` should be no slower than the clock on the receiver client interface. The `tx_ll_clock` should be no slower than the clock on the transmitter client interface divided by 2. It is therefore suggested that the `rx_ll_clock` and `tx_ll_clock` are always 125 MHz or faster.

User Interface Data Width Conversion

Conversion of the user interface 8-bit datapath to a 16, 32, 64 or 128 bit datapath can be made by connecting the LocalLink interface directly to the Parameterizable LocalLink FIFO (XAPP691).

Core Verification, Compliance and Interoperability

The TEMAC solution has been verified with extensive simulation and hardware verification.

Verification by Simulation

A highly parameterizable transaction-based test bench (not part of the primary core deliverables) was used to test the core. Tests include:

- Register access
- MDIO access
- Frame transmission and error handling
- Frame reception and error handling
- Speed switching
- Address filter operation

Hardware Verification

The core has been tested in a variety of hardware test platforms at Xilinx to cover a variety of parameterizations, including the following:

- Testing with the Ethernet 1000BASE-X PCS/PMA or SGMII cores from Xilinx. A test platform was built around these cores, including a back-end FIFO capable of performing a simple ping function and a test pattern generator. Software running on the embedded PowerPC® processor was used to provide access to all configuration, status and statistical counter registers. Testing has been performed in Virtex®-5 and Virtex-4 devices.
- Testing with an external PHY device. The MAC has been connected to external BASE-T PHY devices using both GMII and RGMII interfaces. A test platform was built around the core, including a back-end FIFO capable of performing a simple ping function and a test pattern generator. Software running on an embedded processor (PowerPC or MicroBlaze™) was used to provide access to all configuration, status and statistical counter registers. Testing has been performed in Virtex-5, Virtex-4 and Spartan®-3A DSP devices.

Core Latency

The latency figures given in the following sections apply to all permutations of the core with the exception of the legacy mode of operation described in [Appendix G, 10/100/1000 Mb/s Operation Without Using Clock Enables](#).

Transmit Path Latency

The transmit path latency is measured by counting the number of clock-enabled cycles between a data byte being placed on the client interface (clientemactxd), and it appearing at the GMII/MII output (emacphytxd) of the core netlist. So latency values do not include any GMII/MII or RGMII logic within the example design. Transmitter path latency has been measured as:

- 7 clock-enabled cycles at 1 Gb/s ethernet speed.
- 7 or 7.5 clock-enabled cycles at 10 Mb/s and 100 Mb/s ethernet speeds. This extra half cycle of uncertainty is due to the conversion of 8-bit client data to 4-bit MII width conversion: data is presented to the MII at the earliest possible opportunity.

Receive Path Latency

The receive path latency is measured as the number of clock-enabled cycles between a byte being driven onto the GMII/MII receive interface (phyemacrx), and it appearing at the client (emacclientrx) of the core netlist. So latency values do not include any GMII/MII or RGMII logic within the example design. Receiver path latency has been measured as:

- 11 clock-enabled cycles at 1 Gb/s ethernet speed.
- 11 or 11.5 clock-enabled cycles at 10 Mb/s and 100 Mb/s ethernet speeds. This extra half cycle of uncertainty is due to the conversion of 4-bit MII data width to 8-bit client data conversion.

Calculating the DCM Phase Shift or IODelay Tap Setting

Two differing methods are used by the core to meet input bus (GMII/MII or RGMII) setup and hold timing specifications. There are:

- [DCM Usage](#)

A DCM is used in the receiver clock path to meet the input setup and hold requirements when implementing GMII/MII and RGMII using the core in Virtex®-4, Spartan®-3, Spartan-3E, Spartan-3A and Spartan-3A DSP devices. See the Physical Interface chapters in this Guide).

- [IODelay Usage](#)

IODelays are used in the receiver clock path to meet the input setup and hold requirements when implementing GMII/MII and RGMII using the core in Virtex-6, Spartan-6 and Virtex-5 devices (see the Physical Interface chapters in this Guide).

DCM Usage

DCM Phase Shifting Requirements

When using a DCM, a fixed-phase shift offset is applied to the receiver clock DCM to skew the clock; this performs static alignment by using the receiver clock DCM to shift the internal version of the receiver clock such that the input data is sampled at the optimum time. The ability to shift the internal clock in small increments is critical for sampling high-speed source synchronous signals. For statically aligned systems, the DCM output clock phase offset (as set by the phase shift value) is a critical part of the system, as is the requirement that the PCB is designed with precise delay and impedance-matching for all the GMII/MII or RGMII receiver data bus and control signals.

You must determine the best DCM setting (phase shift) to ensure that the target system has the maximum system margin to perform across voltage, temperature, and process (multiple chips) variations. Testing the system to determine the best DCM phase shift setting has the added advantage of providing a benchmark of the system margin based on the UI (unit interval or bit time). System margin is defined as the following:

System Margin (ps) = UI(ps) * (working phase shift range/128)

Finding the Ideal Phase Shift Value

Xilinx cannot recommend a singular phase shift value that is effective across all hardware families. Xilinx does not recommend attempting to determine the phase shift setting empirically. In addition to the clock-to-data phase relationship, other factors such as package flight time (package skew) and clock routing delays (internal to the device) affect the clock to data relationship at the sample point (in the IOB) and are difficult to characterize.

Xilinx recommends extensive investigation of the phase shift setting during hardware integration and debugging. The phase shift settings provided in the example design constraint file are placeholders, and work successfully in back-annotated simulation of the example design.

Perform a complete sweep of phase-shift settings during your initial system test. Use a test range which covers at least half of the clock period or 128 taps. This does not imply that 128 phase-shift values must be tested; increments of 4 (52, 56, 60, and so forth) correspond to roughly one DCM tap at 125 MHz, and consequently provide an appropriate step size. Additionally, it is not necessary to characterize areas outside the working phase-shift range.

At the edge of the operating phase shift range, system behavior changes dramatically. In eight phase shift settings or less, the system can transition from no errors to exhibiting errors. Checking the operational edge at a step size of two (on more than one board) refines the typical operational phase shift range. After the range is determined, choose the average of the high and low working phase shift values as the default. During the production test, Xilinx recommends that you re-examine the working range at corner case operating conditions to determine whether any final adjustments to the final phase shift setting are needed.

You can use the FPGA Editor to generate the required test file set instead of resorting to multiple PAR runs. Performing the test on design files that differ only in phase shift setting prevents other variables from affecting the test results. FPGA Editor operations can even be scripted further, reducing the effort needed to perform this characterization.

IODelay Usage

IODelay Tap Setting Requirements

With this method, an IODelay is used on either the clock or Data (or both) to adjust the Clock/Data relationship such that the input data is sampled at the optimum time. The ability to adjust this relationship in small increments is critical for sampling high-speed source synchronous signals. For statically aligned systems, the IODelay Tap setting is a critical part of the system, as is the requirement that the PCB is designed with precise delay and impedance-matching for all the GMII/MII or RGMII receiver data bus and control signals.

You must determine the best IODelay Tap setting to ensure that the target system has the maximum system margin to perform across voltage, temperature, and process (multiple chips) variations.

Finding the Ideal Tap Setting Value

Xilinx cannot recommend a singular tap value that is effective across all hardware families. Xilinx does not recommend attempting to determine the tap setting empirically. In addition to the clock-to-data phase relationship, other factors such as package flight time (package skew) and clock routing delays (internal to the device) affect the clock to data relationship at the sample point (in the IOB) and are difficult to characterize.

Xilinx recommends extensive investigation of the tap setting during hardware integration and debugging. The tap settings provided in the example design constraint file are placeholders, and work successfully in back-annotated simulation of the example design.

Perform a complete sweep of tap settings during your initial system test. If possible use a test range which covers at least half of the clock period. This does not imply that all values must be tested as it may be simpler to use a large step size initially to identify a tighter range for a subsequent run. Additionally, it is not necessary to characterize areas outside the working range. If an IODelay is used on both Clock and Data then ensure this test range covers both clock only and data only adjustments.

At the edge of the operating range, system behavior changes dramatically. In four tap settings or less, the system can transition from no errors to exhibiting errors. Checking the operational edge at a step size of two (on more than one board) refines the typical operational range. After the range is determined, choose the average of the high and low working values as the default. During the production test, Xilinx recommends that you re-examine the working range at corner case operating conditions to determine whether any final adjustments to the final setting are needed. Where IODelays are used on the data it may be necessary or beneficial to use slightly different values for each bit.

You can use the FPGA Editor to generate the required test file set instead of resorting to multiple PAR runs. Performing the test on design files that differ only in tap setting prevents other variables from affecting the test results. FPGA Editor operations can even be scripted further, reducing the effort needed to perform this characterization.

Differences between the Embedded Tri-Mode Ethernet MACs and the Soft TEMAC Solution IP Core

This appendix describes the differences between the Embedded Tri-Mode Ethernet MAC blocks, available in selected Virtex®-6, Virtex-5 and Virtex-4 devices, and the soft IP cores, Tri-Mode Ethernet MAC (TEMAC), solutions provided by the CORE Generator™ tool.

The functionality provided by the Embedded Tri-Mode Ethernet MACs can be provided by linking together the Tri-Mode Ethernet MAC soft IP core and the Ethernet 1000BASE-X PCS/PMA or SGMII core. More details are available at:

www.xilinx.com/products/design_resources/conn_central/protocols/gigabit_ethernet.htm

There are, however, some differences in the operation of the Embedded Tri-Mode Ethernet MACs themselves, which have evolved over three generations, and between the embedded MACs and the soft IP cores. These differences are detailed in the following sections. See the relevant FPGA family:

- [Virtex-6 Device](#)
- [Virtex-5 and Virtex-4 Devices](#)

Virtex-6 Device

Features Exclusive to the Embedded Tri-Mode Ethernet MAC

These features are exclusive to the Embedded Tri-Mode Ethernet MAC prior to v2_1:

- Includes integrated SGMII and 1000BASE-X PCS/PMA functionality.
- Supports up to 2.5 Gb/s overclocked operation in 1000BASE-X mode, when configured with a 16-bit client interface.
- Includes an RGMII/SGMII status register.
- Can be configured using the DCR bus for embedded processor support.

Note: The V6 Embedded Tri-Mode Ethernet MAC has been updated to support AXI4-Lite and AXI4-Stream from v2_1 and on. This document considers only versions prior to this.

Features Exclusive to Soft 10/100/1000 Mb/s, 1000 Mb/s and 10/100 Mb/s IP Cores

These features are exclusive to soft IP cores:

- The soft Tri-Mode Ethernet MAC solution:
 - Supports 1 GB half-duplex mode for parallel physical interfaces.
 - Has parallel statistics outputs.
 - Supports IFG adjustment down to 8 bytes if half-duplex support is added or 4 bytes if full-duplex only.
- The soft PCS/PMA core:
 - Supports the ten bit interface (TBI).
 - Outputs a status vector with bits that indicate:
 - The status of the link.
 - The status of the link synchronization state machine.
 - When the core is receiving /C/ ordered sets.
 - When the core is receiving /I/ ordered sets.
 - When the core is receiving invalid data.
- Soft PCS/PMA and Tri-Mode Ethernet MAC solution IP cores:
 - Can be connected together to provide a single Ethernet interface, similar to the solution provided by the Embedded Tri-Mode Ethernet MAC.
 - Can be configured by a vector when the management interface is not required. The vector signals equate to attributes in the Embedded Tri-Mode Ethernet MAC.
 - Can be targeted to a wide variety of Xilinx devices. The Embedded Tri-Mode Ethernet MAC is available in the Virtex-6 FPGA, the Virtex-5 FPGA (LXT/SXT/FXT/TXT families), and the Virtex-4 FPGA (FX family).

Virtex-5 and Virtex-4 Devices

Features Exclusive to the Embedded Tri-Mode Ethernet MAC

The Tri-Mode Ethernet MAC tile features two Tri-Mode Ethernet MACs, each with optional PCS/PMA functionality. In addition, each Embedded Tri-Mode Ethernet MAC:

- Includes integrated SGMII and 1000BASE-X PCS/PMA functionality.
- Supports up to 2 Gb/s overclocked operation in 1000BASE-X mode, when configured with a 16-bit client interface.
- Includes an RGMII/SGMII status register.
- Can be configured using the DCR bus for embedded processor support.

Features Exclusive to Soft 10/100/1000 Mb/s, 1000 Mb/s and 10/100 Mb/s IP Cores

These features are exclusive to soft IP cores:

- The soft Tri-Mode Ethernet MAC solution:
 - Supports 1 GB half-duplex mode for parallel physical interfaces.
 - Supports control frames larger than 64 bytes.
 - Included a statistics bit to indicate an address filter match.
 - Has parallel statistics outputs.
 - Supports IFG adjustment down to 8 bytes if half-duplex support is added or 4 bytes if full-duplex only.
- The soft PCS/PMA core:
 - Supports the ten bit interface (TBI).
 - Can dynamically switch between SGMII and 1000BASE-X modes of operation.
 - Outputs a status vector with bits that indicate:
 - The status of the link.
 - The status of the link synchronization state machine.
 - When the core is receiving /C/ ordered sets.
 - When the core is receiving /I/ ordered sets.
 - When the core is receiving invalid data.
- The Soft PCS/PMA and Tri-Mode Ethernet MAC cores:
 - Can be connected together to provide a single Ethernet interface, similar to the solution provided by the Embedded Tri-Mode Ethernet MAC.
 - Can be configured by a vector when the management interface is not required. The vector signals equate to attributes in the Embedded Tri-Mode Ethernet MAC.
 - Can be targeted to a wide variety of Xilinx® devices. The Embedded Tri-Mode Ethernet MAC is available in the Virtex-6 FPGA, the Virtex-5 FPGA (LXT/SXT/FXT/TXT families), and the Virtex-4 FPGA (FX family).

Upgrading from the 1-Gigabit Ethernet MAC LogiCORE

The 1-Gigabit Ethernet MAC LogiCORE™ IP core (GEMAC) has been deprecated as of 24 April 2009; version 8.5 will be the final release of the GEMAC core. All existing GEMAC users will automatically be upgraded to the TEMAC.

This core, the Tri-Mode Ethernet MAC (TEMAC) from version 4.1 onwards, now contains an option to be generated with only 1 Gb/s ethernet speed support. This is the replacement for the GEMAC and has been designed to be backwards compatible wherever possible.

This Appendix describes how to upgrade from the GEMAC to an equivalent core using the TEMAC. Throughout this appendix, except where explicitly stated, TEMAC version 4.5 is compared directly against GEMAC version 8.5 (the latest and final version of GEMAC).

To upgrade from GEMAC to TEMAC, follow these steps:

1. [Generate the TEMAC Core](#) with the required options.
2. Replace the block level instantiation of the GEMAC with the block level instantiation of the TEMAC version of the GEMAC in your design. These block level files *may* contain minor port differences: see [Block Level Port Differences](#).
3. The GEMAC and TEMAC cores contain minor differences in their Management Registers. Appropriate amendments should be made to any software driver files which are assigned to this interface. See [Management Register Differences](#).

Generate the TEMAC Core

Generate the TEMAC version of the GEMAC core as described in [Chapter 4, Generating the Core](#) For compatibility with the GEMAC core, use the following options:

- Component Name: as per GEMAC
- Physical Interface: select GMII or RGMII as per GEMAC
- MAC Speed: set to 1 Gb/s only operation
- Clock Enables: this option is not available for selection when the MAC Speed is set to 1 Gb/s
- Half Duplex: uncheck this box (set to FALSE)
- Management Interface: set as per GEMAC
- Address Filter: set as per GEMAC
- Number of Address Table Entries: set as per GEMAC

Block Level Port Differences

When the TEMAC solution is generated with the 1 Gb/s only speed support option, the block level wrapper, created as part of the cores example design, will contain identical ports to the equivalent block level wrapper delivered by the GEMAC version 8.5 design.

However, GEMAC version 8.5 has been enhanced to have equivalent statistic vector outputs to the TEMAC solution. GEMAC versions 8.4 and earlier contain differing statistic vector definitions: these are summarized in [Table F-1](#) and [Table F-2](#).

Table F-1: Rx Statistics Vector Differences

TEMAC solution and GEMAC version 8.5 rx_statistics_vector bit(s)	GEMAC version 8.4 (and earlier) rx_statistics_vector bit(s)	Notes
27	26	Bit 27 is equivalent to bit 26 of all previous GEMAC core versions.
26	N/A	Bit 26 (reserved) has been inserted into GEMAC version 8.5 for statistic vector compatibility with the TEMAC solution
25:0	25:0	No differences

Table F-2: Tx Statistics Vector Differences

TEMAC solution and GEMAC version 8.5 tx_statistics_vector bit(s)	GEMAC version 8.4 (and earlier) tx_statistics_vector bit(s)	Notes
31	21	Bit 31 is equivalent to bit 21 of all previous GEMAC core versions.
30	20	Bit 30 is equivalent to bit 20 of all previous GEMAC core versions.
29:20	N/A	Reserved: have been inserted into GEMAC version 8.5 for statistic vector compatibility with the TEMAC solution.
19:0	19:0	No differences

Management Register Differences

The TEMAC solution and GEMAC contain identical logic for accessing MDIO, Address Filter and other configuration logic. Additionally, all Configuration registers between the TEMAC solution and GEMAC have an equal address map and have identical definitions with only the following two exceptions:

1. The [MAC Speed Configuration Word](#) register is only present in the TEMAC (read-only when the TEMAC is generated for 1 Gb/s only operation). In the GEMAC this register is undefined and returns only 0s.
2. The [MDIO Configuration Word](#) register, defined in [Table 9-7](#), contains bit map differences which require edits to an existing GEMAC customer design (most probably in the form of software driver updates). The bit map differences are summarized in [Table F-3](#).

Table F-3: MDIO Configuration Word Differences between the GEMAC and TEMAC

Description	GEMAC Bit(s)	TEMAC Bit(s)
Clock Divide	4-0	5-0
MDIO Enable	5	6
Reserved	31-6	31-7

Feature Differences

The GEMAC provided support for full-duplex mode only. The TEMAC solution enhances this in that it can optionally provide full and half-duplex operation.

Core Netlist Port Differences

It is expected that the block level wrapper, delivered as part of the example design by the CORE Generator™ software, is instantiated in customer designs rather than the core netlist directly. If this is the case, then see [Block Level Port Differences](#).

However, this section is provided for completeness. Despite the port names being different between the GEMAC and TEMAC solution netlists, there is a 1-1 correspondence when the TEMAC has been generated with the appropriate options. These are defined in [Table F-4](#).

Table F-4: GEMAC to TEMAC Netlist Port Comparison

GEMAC version 8.5 Ports	Equivalent TEMAC version 4.5 Ports
gtx_clk	txgmiimiiclk
tx_data[7:0]	clientemactxd[7:0]
tx_data_valid	clientemactxdvld
tx_ifg_delay[7:0]	clientemactxifgdelay[7:0]
tx_ack	emacclienttxack
tx_underrun	clientemactxunderrun
tx_statistics_vector[31:0]	emacclienttxstats[31:0]
tx_statistics_valid	emacclienttxstatsvld
rx_data[7:0]	emacclientrx[7:0]

Table F-4: GEMAC to TEMAC Netlist Port Comparison

GEMAC version 8.5 Ports	Equivalent TEMAC version 4.5 Ports
rx_data_valid	emacclientrxdvld
rx_good_frame	emacclientrxgoodframe
rx_bad_frame	emacclientrxbadframe
rx_statistics_vector[27:0]	emacclientrxstats[27:0]
rx_statistics_valid	emacclientrxstatsvld
pause_req	clientemacpausereq
pause_val[15:0]	clientemacpauseval[15:0]
host_clk	hostclk
host_opcode[1:0]	hostopcode[1:0]
host_addr[9:0]	hostaddr[9:0]
host_wr_data[31:0]	hostwrdata[31:0]
host_rd_data[31:0]	hostrddata[31:0]
host_miim_sel	hostmiimsel
host_req	hostreq
host_miim_rdy	hostmiimrdy
mac_unicast_address[47:0]	tieemacunicastaddr[47:0]
configuration_vector[67:0]	tieemacconfigvec[67:0]
reset	reset
gmii_txd[7:0]	emacphytxd[7:0]
gmii_tx_en	emacphytxen
gmii_tx_er	emacphytxer
gmii_rx_clk	rxgmiiimclk
gmii_rxd[7:0]	phyemacrx[7:0]
gmii_rx_dv	phyemacrxdv
gmii_rx_er	phyemacr[er]
mdc	emacphymclkout
mdio_in ¹	emacphymdin
mdio_out	emacphymdout
mdio_tri	emacphymdtri

Additionally, the TEMAC solution netlist will contain these extra ports:

corehassgmii: an input which should be tied high
 speedis100 and speedis10100; outputs which can be ignored

10/100/1000 Mb/s Operation Without Using Clock Enables

Background

When version 1.1 of the TEMAC core was released, it was designed to be as similar as possible to the Virtex®-4 FPGA Embedded Tri-Mode Ethernet MAC, and utilized a similar clocking scheme.

Since then, this TEMAC solution and the embedded Ethernet MACs of more recent families have had been provided with enhanced clocking schemes, using clock enables to reduce the amount of device clock resources required when implementing Ethernet MAC designs.

The use of clock enables, for 10/100/1000 Mb/s operation, is the now the recommended clocking scheme and only this approach has been considered thus far in this guide.

However, 10/100/1000 Mb/s operation without the use of clock enables has been maintained for backwards compatibility in older device families. This mode of operation is not be supported in the Virtex-6 and Spartan®-6 device families.

Caution! This configuration is inefficient with respect to the device clock resources required. Xilinx recommends using the optional clock enables for tri-speed ethernet designs.

Caution! When in Half duplex mode, the min IFG is limited to at least 17 bytes and therefore max throughput cannot be achieved. This is NOT an issue with clock enables.

Port Map Changes

The following clock enable ports, described in [Core Interfaces in Chapter 5](#), will not be present on the core:

- clientemacrxcenable
- clientemactxcenable
- phyemactxcenable

Instead, two additional clock ports are provided as defined in [Table G-1](#).

Table G-1: Additional Clock Ports

Signal	Direction	Description
txcoreclk	Input	The clock for data transmission on the client side of the core. This is 125 MHz at 1 Gb/s, 12.5 MHz at 100 Mb/s and 1.25 MHz at 10 Mb/s. This clock should be used to clock the client transmit I/F.
rxcoreclk	Input	The clock for data reception on the client side of the core. This is 125 MHz at 1 Gb/s, 12.5 MHz at 100 Mb/s and 1.25 MHz at 10 Mb/s. This clock should be used to clock the client receive I/F.

Client Interface

[Chapter 7, Client Interface](#), provides descriptions and timing diagrams for using the client interface; the descriptions for [Receiving Inbound Frames](#) and [Transmitting Outbound Frames](#) remain valid. However, in all timing diagrams, the vertical dotted lines represent a rising edge of txcoreclk (for the transmitter figures) or a rising edge of rxcoreclk (for the receiver figures).

Physical Interfaces for 10/100/1000 Mb/s Operation (No Clock Enables)

Example designs can be provided for the following interfaces when used in this configuration:

- [GMII/MII](#)
- [RGMII](#)

GMII/MII

Transmitter Logic

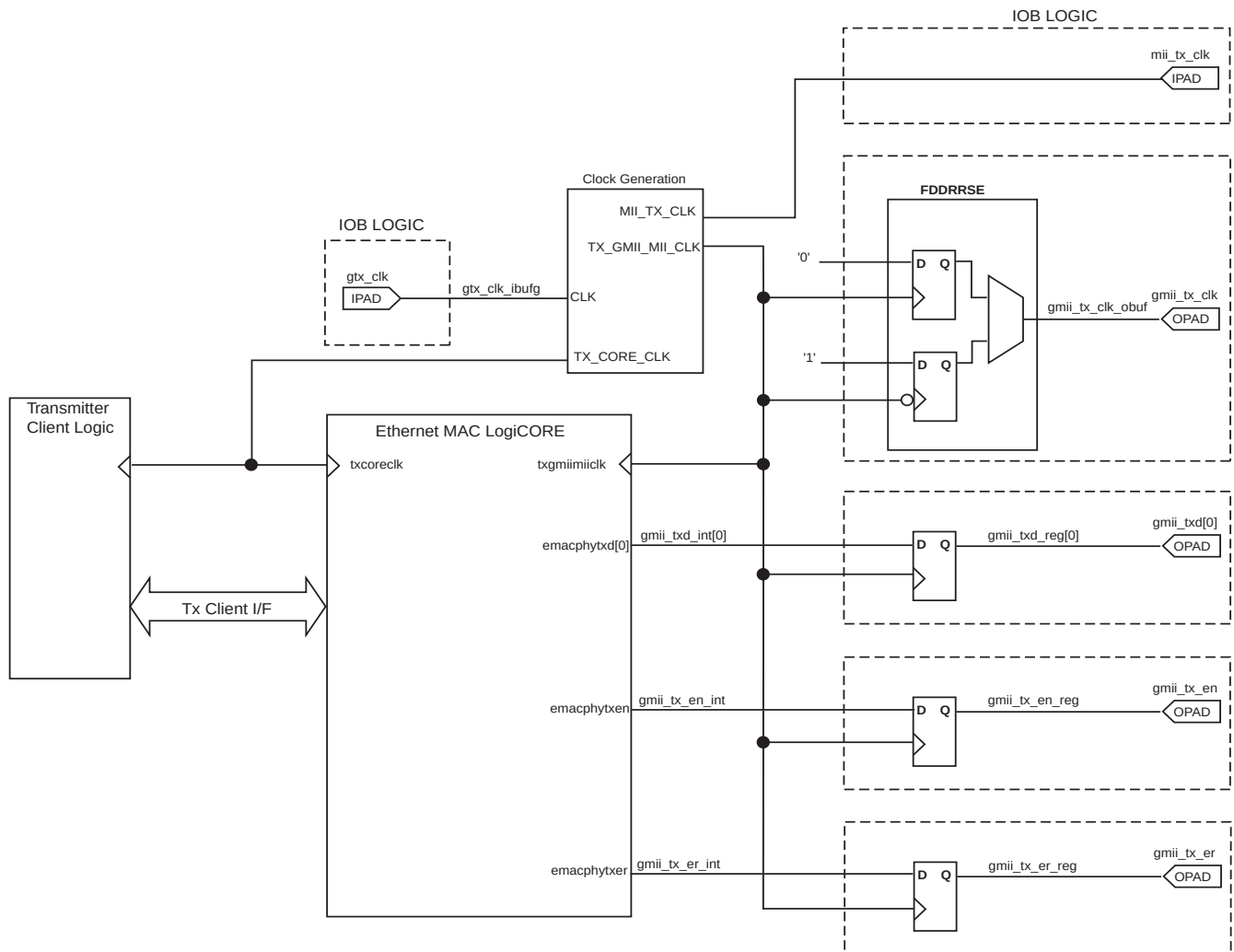


Figure G-1: GMII/MII Transmitter Logic

Figure G-1 illustrates the logic provided by the example when GMII is selected. The clock logic itself is created by the Clock Generation block (tx_clk_gen.v(hd) in the example design). Observe that:

- The clock generation module has two input clocks, CLK and TX_GMII_MII_CLK, connected to gtx_clk and mii_tx_clk respectively.
- The clock generation module has two output clocks:
 - TX_CORE_CLK is connected to the txcoreclk port of the core; it is used to clock the transmitter client logic.
 - TX_GMII_MII_CLK is connected to the txgmiiiclk port of the core; it is used to clock the transmitter GMII logic.

The internal logic of the Clock generation block of [Figure G-1](#) is illustrated in [Figure G-2](#).

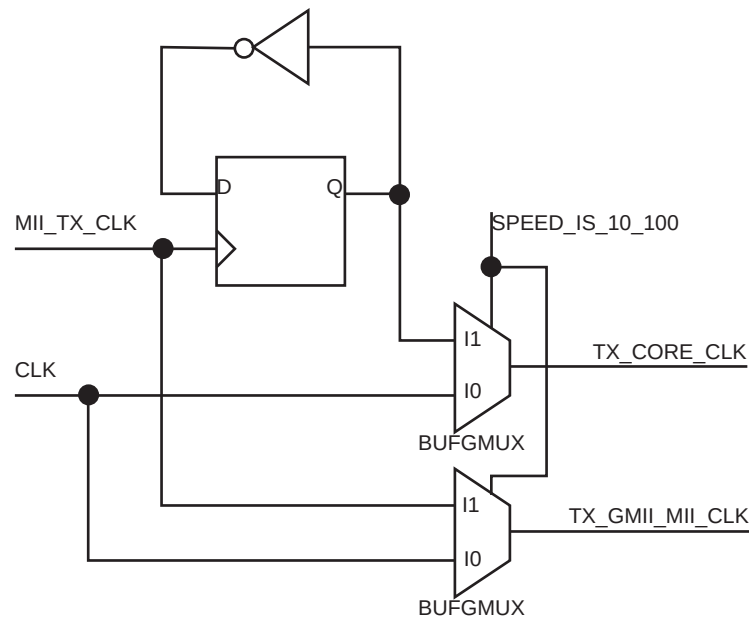


Figure G-2: GMII/MII Transmit Clock Generation

Observe that:

- TX_GMII_MII_CLK is sourced from the CLK input at 1 Gb/s speeds (gtx_clk, the local 125 MHz oscillator source of [Figure G-1](#)).
At 10/100 Mb/s speeds, TX_GMII_MII_CLK is sourced from the MII_TX_CLK input (sourced by the PHY).
- TX_CORE_CLK is sourced from the CLK input at 1 Gb/s speeds (gtx_clk, the local 125MHz oscillator source of [Figure G-1](#)).
At 10/100 Mb/s speeds, TX_CORE_CLK is sourced from a divided by 2 version of the MII_TX_CLK input (source by the PHY).

The use of the BUFGMUXs ensure that the TX_GMII_MII_CLK and TX_CORE_CLK outputs remain glitch less despite clock switching. However, on the transmitter path, these can optionally be exchanged for FPGA logic MUXs followed by a BUFG element; this is only beneficial to preserve global clock resources in Spartan-3 devices (see [Spartan-3 Device BUFGMUX Usage](#)).

Receiver Logic

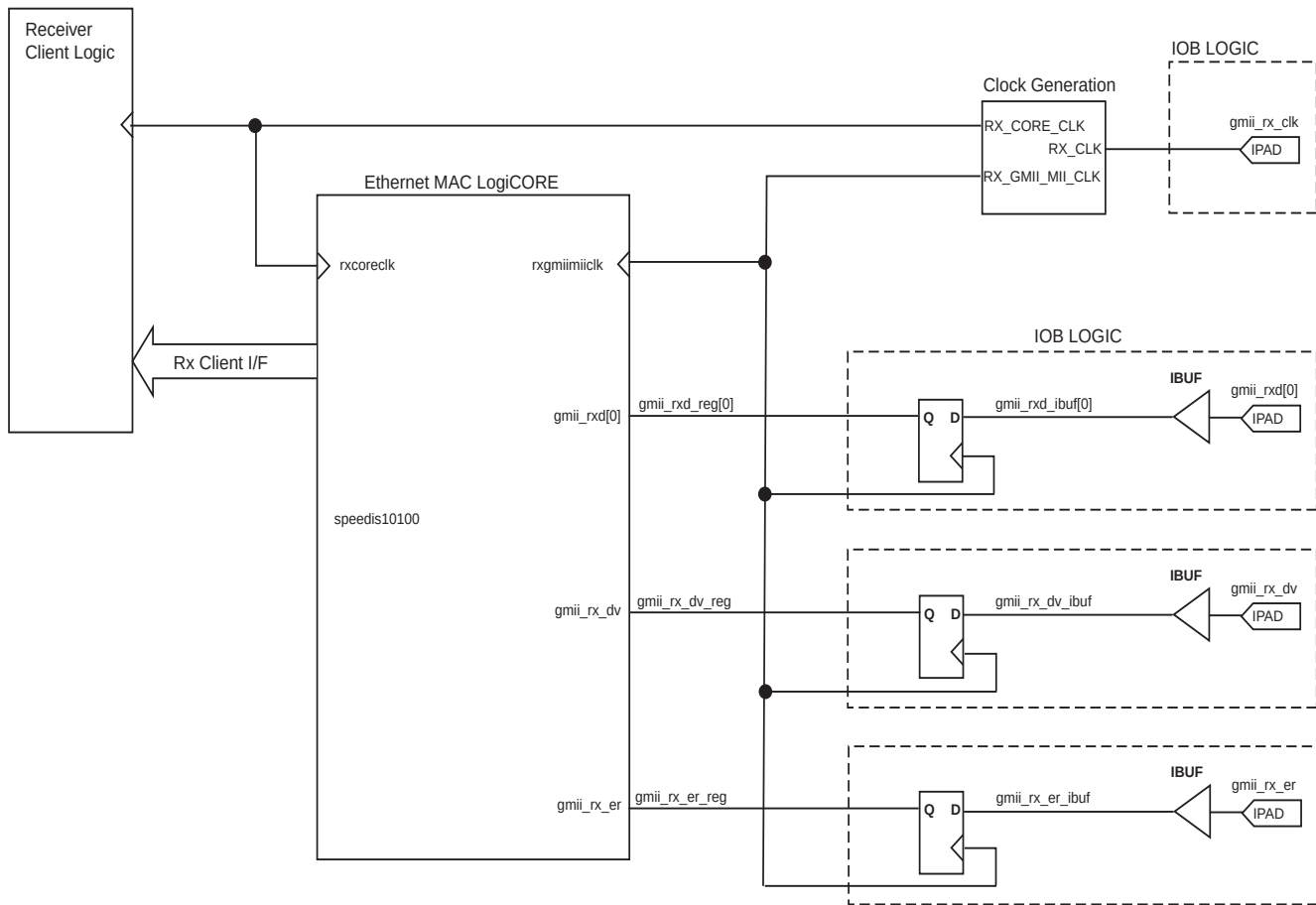


Figure G-3: GMII Receiver Logic

Figure G-3 illustrates the logic provided by the example when GMII is selected in a Spartan-3 device. Other device families contain similar logic (see [Chapter 12, Physical Interfaces for Tri-speed \(10 Mb/s, 100 Mb/s and 1 Gb/s\) Ethernet MAC IP Core](#) for further information).

The clock logic itself is illustrated in a separate Clock Generation block (this is coded in RTL in the `gmii_if.v(hd)` module of the example design). Observe that:

- The clock generation module has only one input clock, obtained from the connected PHY: `gmii_rx_clk`.
- The clock generation module has two output clocks:
 - `RX_CORE_CLK` is connected to the `rxcoreclk` port of the core; it is used to clock the receiver client logic.
 - `RX_GMII_MII_CLK` is connected to the `rxgmiiiclk` port of the core; it is used to clock the receiver GMII logic.

The internal logic of the Clock generation block is family specific and is detailed in the following sections.

Spartan-3, Spartan-3E, Spartan-3A, Spartan-3A DSP and Virtex-4 Devices

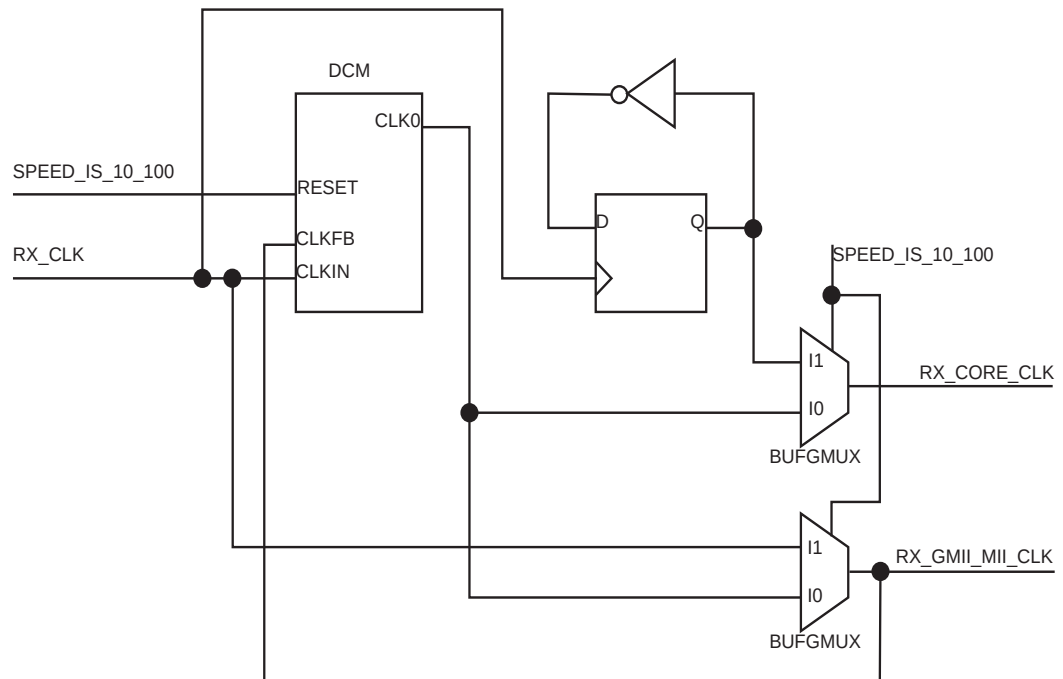


Figure G-4: GMII/MII Receive Clock Generation for Spartan-3 and Virtex-4 Devices

Figure G-4 shows the GMII/MII receiver clock generation circuit.

RX_CLK is connected to `gmi_rx_clk` of Figure G-3 and is therefore provided by the PHY chip. This clock is connected to a DCM. The use of a DCM on this interface implies that the input clock is continuous. Therefore, this limits the functionality of the core as 1 Gb/s Half Duplex cannot be supported.

Observe that:

- RX_GMII_MII_CLK is sourced from the CLK0 DCM output at 1 Gb/s speeds. This DCM is used in fixed phase shift mode, providing skew on the clock to meet the tight input setup and hold constraints at 1 Gb/s.
At 10/100 Mb/s speeds, RX_GMII_MII_CLK is sourced directly from the RX_CLK input since the frequency of this clock can be lower than the DCM can accept.
- RX_CORE_CLK is sourced from the CLK0 DCM output at 1 Gb/s speeds.
At 10/100 Mb/s speeds, RX_CORE_CLK is sourced from a divided by 2 version of the RX_CLK input.

The use of the BUFGMUXs ensure that the RX_GMII_MII_CLK and RX_CORE_CLK outputs remain glitch less despite clock switching. These BUFGMUX elements should not be exchanged for FPGA logic clock MUXes.

Virtex-5 Devices

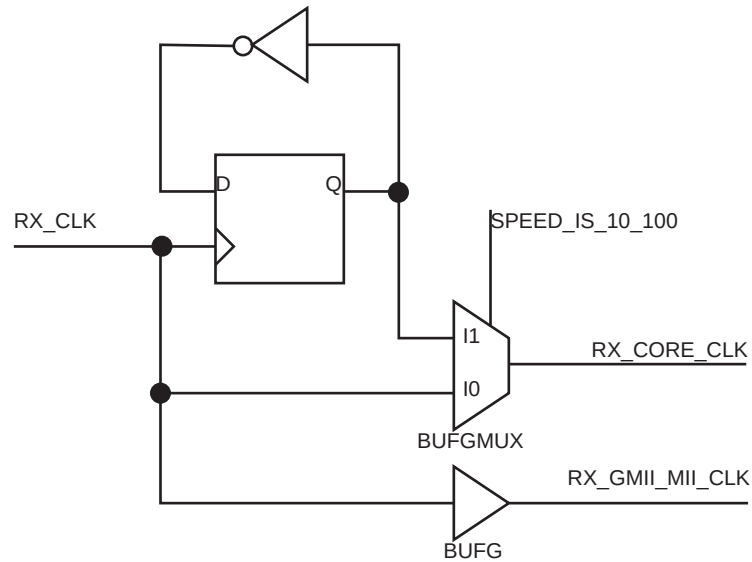


Figure G-5: GMII/MII Receive Clock Generation for Virtex-5 Devices

Figure G-5 shows the GMII/MII receiver clock generation circuit.

RX_CLK is connected to `gmii_rx_clk` of Figure G-3 and is therefore provided by the PHY chip. Observe that:

- RX_GMII_MII_CLK is sourced from the RX_CLK clock input at all speeds.
- RX_CORE_CLK is sourced from RX_CLK at 1 Gb/s speed.

At 10/100 Mb/s speeds, RX_CORE_CLK is sourced from a divided by 2 version of the RX_CLK input.

The use of the BUFGMUXs ensure that the RX_GMII_MII_CLK and RX_CORE_CLK outputs remain glitch less despite clock switching. These BUFGMUX elements should not be exchanged for FPGA logic clock MUXes.

RGMII

Transmitter Logic

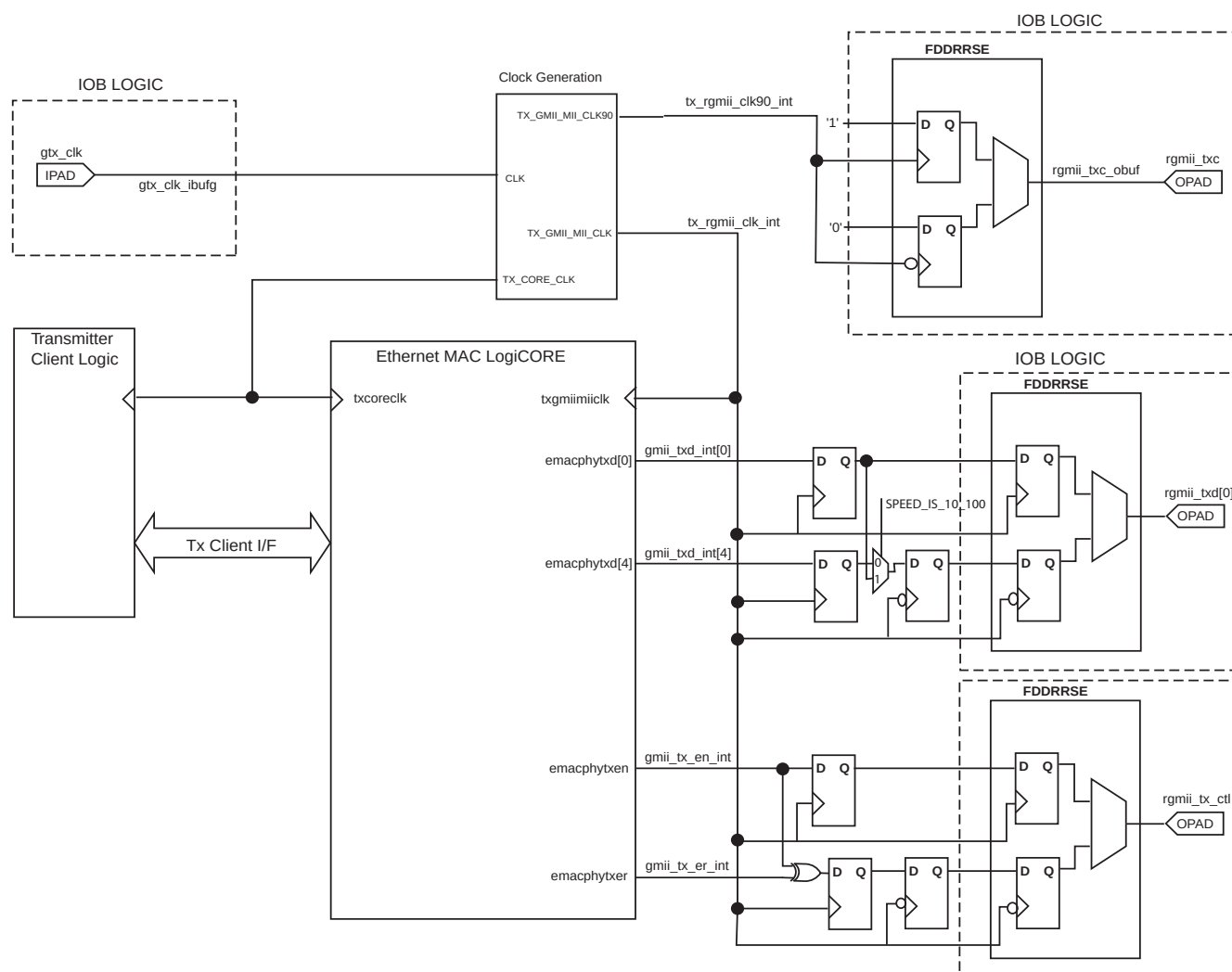


Figure G-6: RGMII Transmitter Logic

Figure G-6 illustrates the logic provided by the example when RGMII is selected in a Spartan-3 device. Other device families contain similar logic (see [Chapter 12, Physical Interfaces for Tri-speed \(10 Mb/s, 100 Mb/s and 1 Gb/s\) Ethernet MAC IP Core](#), for further information).

The clock logic itself is created by the Clock Generation block (tx_clk_gen.v(hd) in the example design). Observe that:

- The clock generation module has only one input clock; CLK is connected to gtx_clk.
- The clock generation module has three output clocks:
 - TX_CORE_CLK is connected to the txcoreclk port of the core; it is used to clock the transmitter client logic.
 - TX_GMII_MII_CLK is connected to the txgmiiimiclk port of the core; it is used to clock the transmitter RGMII data logic.
 - TX_GMII_MII_CLK90 is used to clock the transmitter RGMII data logic. This output is not required or present when a Virtex-5 device is used.

The internal logic of the Clock generation block is family-specific and is detailed in the following sections.

Spartan-3, Spartan-3A and Spartan-3A DSP Devices

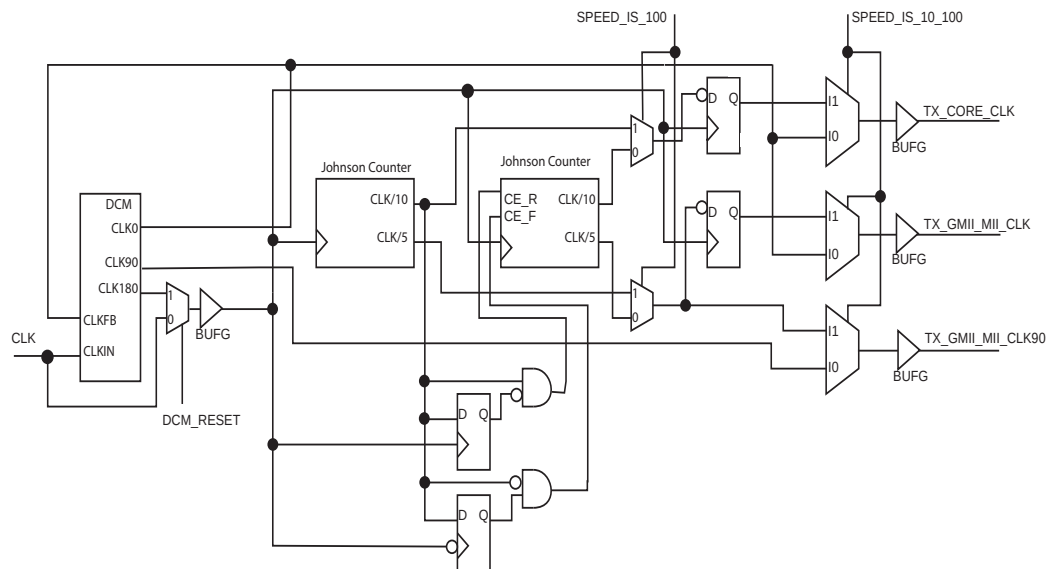


Figure G-7: RGMII Transmit Clock Generation for Spartan-3 Devices

Figure G-7 illustrates the RGMII transmitter clock logic for all Spartan-3 family devices.

A high-quality 125 MHz clock (CLK, connected to gtx_clk of Figure G-6) is supplied to the clock circuit. This is then input to a DCM. The CLK0 output from the DCM is used to generate the TX_CORE_CLK and TX_GMII_MII_CLK outputs.

A Johnson counter is used to divide the CLK input down by 5 and 10 to provide clocks of 25 MHz and 12.5 MHz. The 12.5 MHz clock is then used to generate clock enable inputs to a second Johnson counter to provide clocks of 2.5 MHz and 1.25 MHz. These are then routed to the TX_CORE_CLK and TX_GMII_MII_CLK outputs depending on the state of the SPEED_IS_10_100 and SPEED_IS_100 outputs from the core.

The Johnson counter logic is clocked with DCM output CLK180 (or directly from the DCM input clock if the DCM is held in reset). These alternative clock sources are multiplexed using an FPGA logic multiplexer, followed by routing onto a BUFG (global clock routing). This ensures that there is always a 125 MHz clock present for the Johnson counter logic for any ethernet speed setting.

The logic ensures that there is always a clock present whenever the DCM is held in reset because it is also this clock source which is used to clock the DCM reset circuitry (not shown)!

When the core is running at 1 Gb/s, the `rgmii_txc` clock output must toggle at the center of the valid data. To do this, the CLK90 output from the DCM is routed to the TX_GMII_MII_CLK90 output where it is used to generate `rgmii_txc` (see Figure G-6). At lower speeds, the clock for `rgmii_txc` is generated from the CLK180 output of the DCM via the two Johnson counters.

All clock multiplexing on the transmit path in Spartan-3 devices is performed in the FPGA logic rather than by using the glitch less BUFGMUX elements. This is beneficial to preserve global clock resources in Spartan-3 devices (see [Spartan-3 Device BUFGMUX Usage](#)).

Virtex-4 Devices

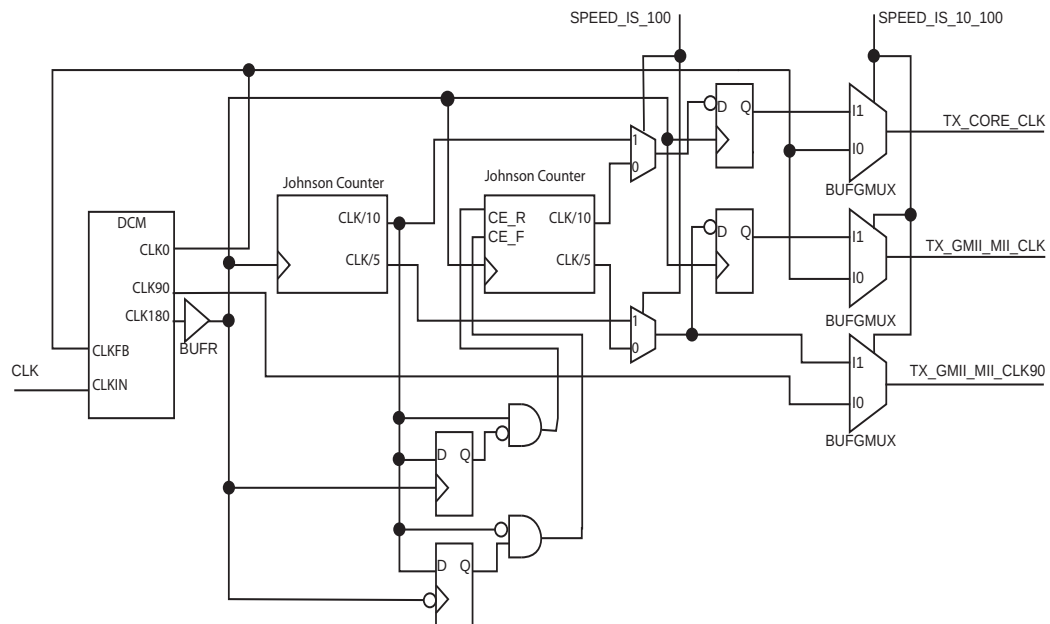


Figure G-8: RGMII Transmit Clock Generation for Virtex-4 Devices

Figure G-7 illustrates the RGMII transmitter clock logic for Virtex-4 devices.

A high-quality 125 MHz clock (CLK, connected to `gtx_clk` of Figure G-6) is supplied to the clock circuit. This is then input to a DCM. The CLK0 output from the DCM is used to generate the TX_CORE_CLK and TX_GMII_MII_CLK outputs.

A Johnson counter is used to divide the CLK input down by 5 and 10 to provide clocks of 25 MHz and 12.5 MHz. The 12.5 MHz clock is then used to generate clock enable inputs to a second Johnson counter to provide clocks of 2.5 MHz and 1.25 MHz. These are then routed to the TX_CORE_CLK and TX_GMII_MII_CLK outputs depending on the state of the SPEED_IS_10_100 and SPEED_IS_100 outputs from the core.

The Johnson counter logic is clocked with DCM output CLK180 via a BUFR (regional clock routing). This ensures that there is always a 125 MHz clock present for the Johnson counter logic for any ethernet speed setting.

When the core is running at 1 Gb/s, the `rgmii_txc` clock output must toggle at the center of the valid data. To do this, the CLK90 output from the DCM is routed to the

TX_GMII_MII_CLK90 output where it is used to generate rgmii_txc (see Figure G-6). At lower speeds, the clock for rgmii_txc is generated from the CLK180 output of the DCM via the two Johnson counters.

The use of the BUFGMUXs ensure that the TX_GMII_MII_CLK, TX_CORE_CLK and TX_GMII_MII_CLK90 outputs remain glitch less despite clock switching.

Virtex-5 Devices

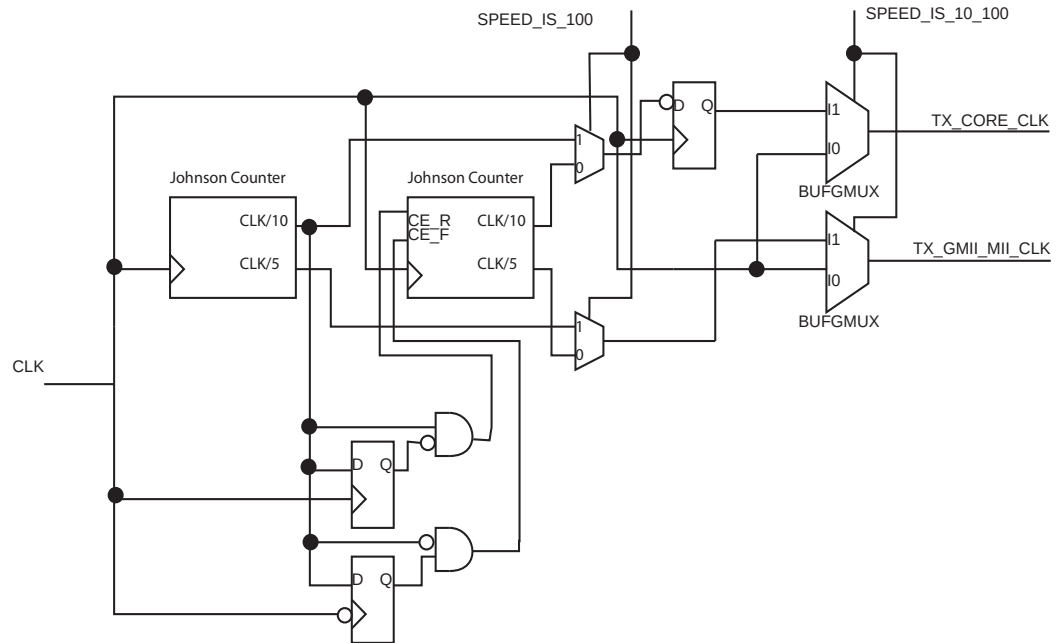


Figure G-9: RGMII Transmit Clock Generation for Virtex-5 Devices

Figure G-9 illustrates the RGMII transmitter clock logic for Virtex-5 devices.

A high-quality 125 MHz clock (CLK, connected to gtx_clk of Figure G-6) is supplied to the clock circuit. For 1 Gb/s operation, this clock is routed through separate BUFGMUXs to create the TX_GMII_MII_CLK output and the TX_CORE_CLK output.

For operation at other speeds, a Johnson counter is used to divide the CLK input down by 5 and 10 to provide clocks of 25 MHz and 12.5 MHz. The 12.5 MHz clock is then used to generate clock enable inputs to a second Johnson counter to provide clocks of 2.5 MHz and 1.25 MHz. These are then routed to the TX_CORE_CLK and TX_GMII_MII_CLK outputs depending on the state of the SPEED_IS_10_100 and SPEED_IS_100 outputs from the core.

The use of the BUFGMUXs ensure that the TX_GMII_MII_CLK and TX_CORE_CLK outputs remain glitch less despite clock switching.

Receiver Logic

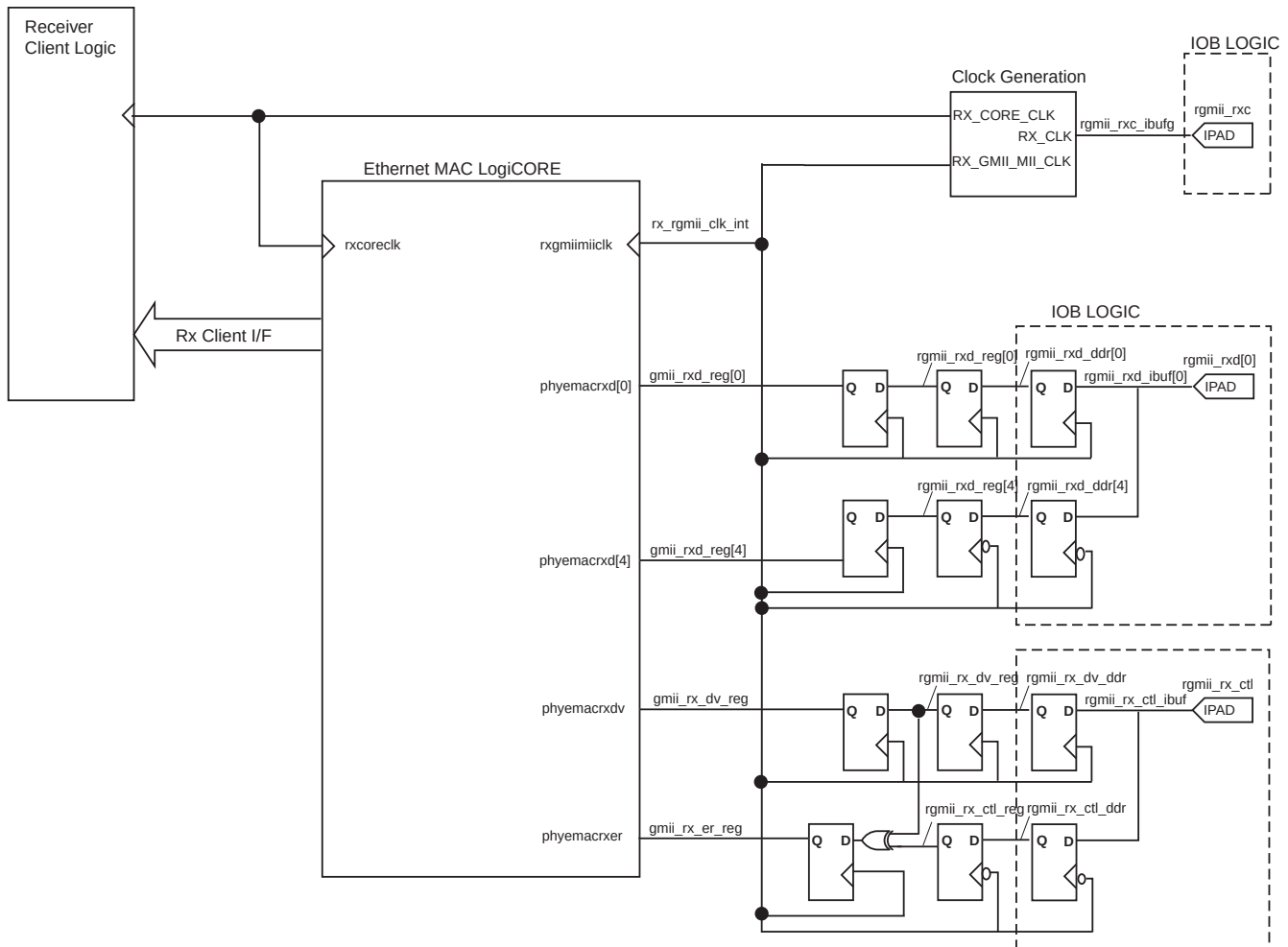


Figure G-10: RGMII Receiver Logic

Figure G-10 illustrates the logic provided by the example when RGMII is selected in a Spartan-3 device. Other device families contain similar logic (see [Chapter 12, Physical Interfaces for Tri-speed \(10 Mb/s, 100 Mb/s and 1 Gb/s\) Ethernet MAC IP Core](#) for further information). The clock logic itself is illustrated in a separate Clock Generation block (this is coded in RTL in the `rgmii_v2_0_if.v(hd)` module of the example design). Observe that:

- The clock generation module has only one input clock, obtained from the connected PHY: `rgmii_rxc`.
- The clock generation module has two output clocks:
 - `RX_CORE_CLK` is connected to the `rxcoreclk` port of the core; it is used to clock the receiver client logic.
 - `RX_GMII_MII_CLK` is connected to the `rxgmiiimiclk` port of the core; it is used to clock the receiver RGMII logic.

The internal logic of the Clock generation block is family-specific and is detailed in the following sections.

Spartan-3, Spartan-3A, Spartan-3A DSP and Virtex-4 Devices

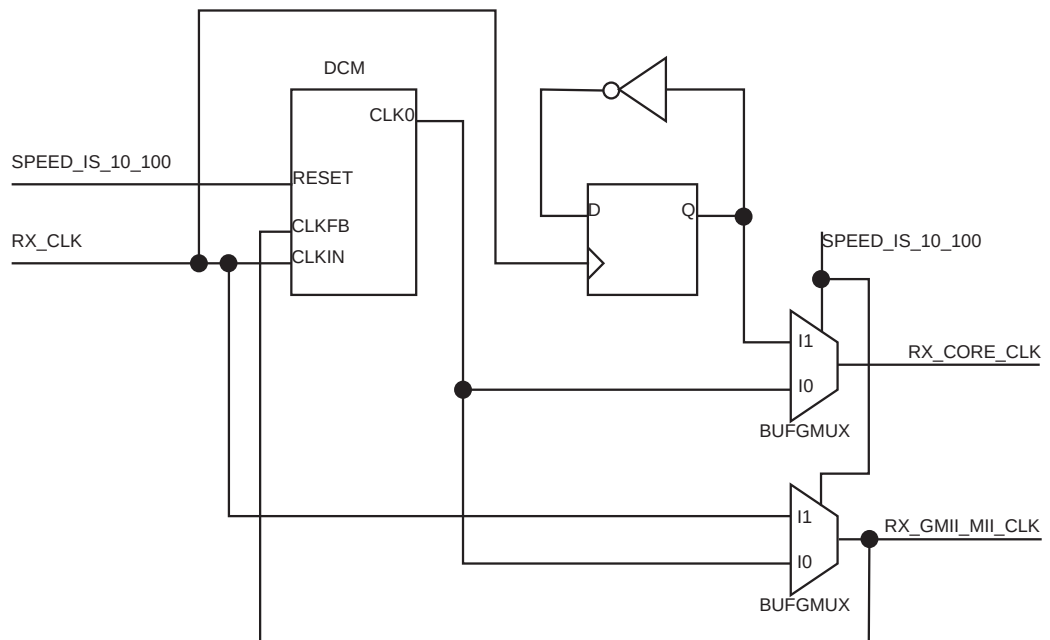


Figure G-11: RGMII Receive Clock Generation for Spartan-3 and Virtex-4 Devices

Figure G-11 shows the clock generation circuitry for the RGMII receiver.

RX_CLK is connected to `rgmii_rxc` of Figure G-10 and is therefore provided by the PHY chip. This clock is connected to a DCM.

Note: The use of a DCM on this interface implies that the input clock is continuous. Therefore, this limits the functionality of the core as 1 Gb/s Half Duplex cannot be supported.

Observe that:

- RX_GMII_MII_CLK is sourced from the CLK0 DCM output at 1 Gb/s speeds. This DCM is used in fixed phase shift mode, providing skew on the clock to meet the tight input setup and hold constraints at 1 Gb/s.

At 10/100 Mb/s speeds, RX_GMII_MII_CLK is sourced directly from the RX_CLK input since the frequency of this clock can be lower than the DCM can accept.

- RX_CORE_CLK is sourced from the CLK0 DCM output at 1 Gb/s speeds. At 10/100 Mb/s speeds, RX_CORE_CLK is sourced from a divided by 2 version of the RX_CLK input.

The use of the BUFGMUXs ensure that the RX_GMII_MII_CLK and RX_CORE_CLK outputs remain glitch less despite clock switching. These BUFGMUX elements should not be exchanged for FPGA logic clock MUXes.

Virtex-5 Devices

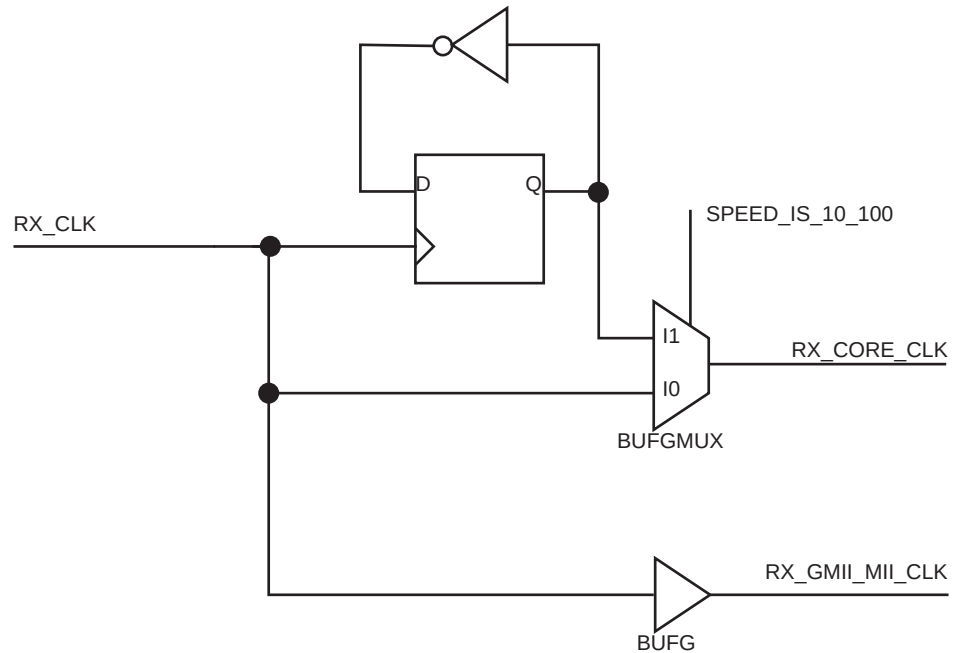


Figure G-12: RGMII Receive Clock Generation for Virtex-5 Devices

Figure G-12 shows the RGMII receiver clock generation circuit.

RX_CLK is connected to `rgmii_rxc` of Figure G-10 and is therefore provided by the PHY chip. Observe that:

- RX_GMII_MII_CLK is sourced from the RX_CLK clock input at all speeds.
- RX_CORE_CLK is sourced from RX_CLK at 1 Gb/s speed.

At 10/100 Mb/s speeds, RX_CORE_CLK is sourced from a divided by 2 version of the RX_CLK input.

The use of the BUFGMUXs ensure that the RX_GMII_MII_CLK and RX_CORE_CLK outputs remain glitch less despite clock switching. These BUFGMUX elements should not be exchanged for FPGA logic clock MUXes.

Multiple Cores

It is not possible, whether using GMII/MII or RGMII, to share transmitter or receiver clock resources across multiple instantiations of the core. Each instance of the core requires its own independent clocking resources. Therefore all logic described in this Appendix must be duplicated for each instance of the core.

Spartan-3 Device BUFGMUX Usage

The BUFGMUX components in Spartan-3, Spartan-3E, Spartan-3A, and Spartan-3A DSP devices are arranged in pairs. Both the inputs to each pair must be the same.

Unfortunately, for both GMII/MII and RGMII example designs, none of the required clock multiplexers share common clock inputs. If BUFGMUX elements were used for the clock multiplexing, this would result in the situation whereby the partner of each BUFGMUX could not be used by another clock. This is illustrated in Figure G-13 for the generation of txgmiimiiclk in GMII example designs.

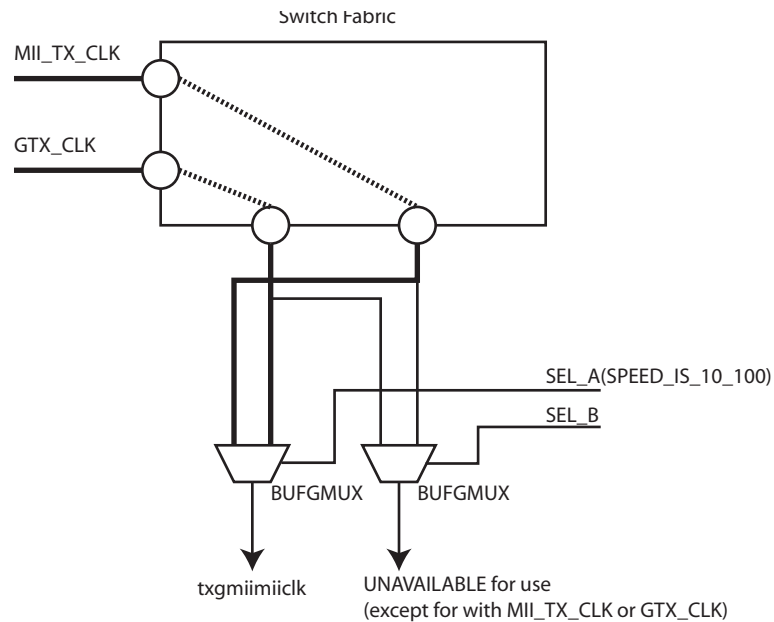


Figure G-13: Suggested BUFGMUX Scheme

Therefore, where possible, it is beneficial to use a clock multiplexer in the FPGA logic. The output clock from this MUX can then be routed to a BUFGMUX component (essentially a BUFGMUX with a constant select line) to place the clock onto global clock routing. This frees up the partner BUFGMUX for use by a different clock. This is illustrated in Figure G-14.

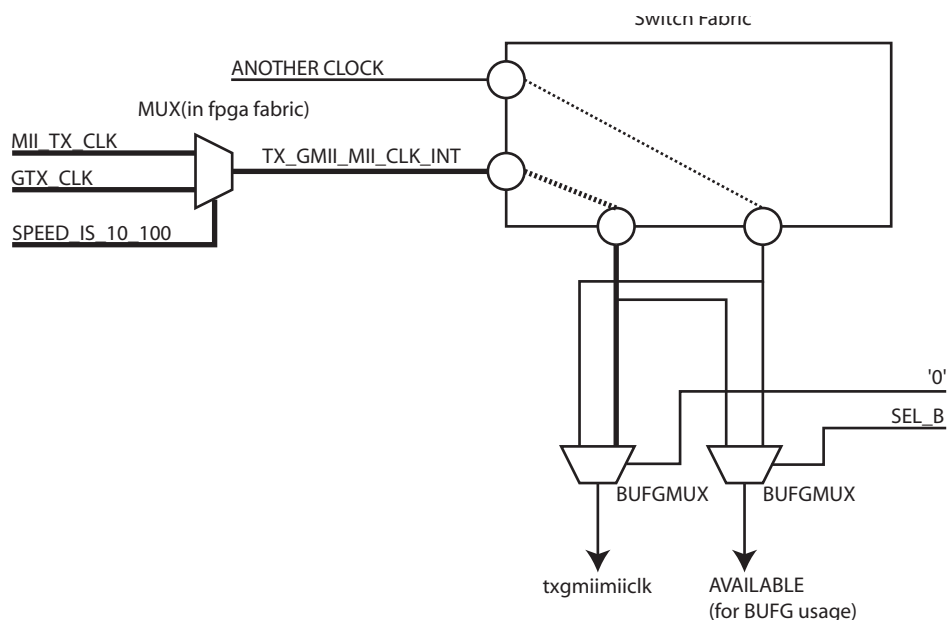


Figure G-14: Alternative BUFGMUX Scheme

However, when using a BUFGMUX component, no glitches or short pulses appear on the output when the select line is toggled. Using the logic clock Multiplexer method described previously removes this safeguard; glitches and short pulses are likely to appear. Therefore, a reset should **always** be performed after a speed change.

Required Constraints

Use the UCF, delivered as part of the example design by the CORE Generator™ software, as the main reference for required constraints when using this configuration. However, to summarize the required constraints:

- Extra clock period descriptions are required for the additional clocks. When used with the example design provided and described in [Physical Interfaces for 10/100/1000 Mb/s Operation \(No Clock Enables\)](#), these constraints are:

```
NET "*rx_clk_int" TNM_NET = "clk_rx_core";
TIMESPEC "TS_rx_clk_core" = PERIOD "clk_rx_core" 8000 ps HIGH 50 %;
```

```
NET "*tx_clk_int" TNM_NET = "clk_tx_core";
TIMESPEC "TS_tx_clk_core" = PERIOD "clk_tx_core" 8000 ps HIGH 50%;
```

- Internal Netlist clock domain crossing constraint edits are required to the description given in [Chapter 13, Constraining the Core](#). Flow Control and Configuration Register reclocking should refer to the clock nets connected to the txcoreclk and rxcoreclk ports rather than the clock ports connected to the txgmiimiiclk and rxgmiimiiclk ports.

When the TEMAC solution is generated in this mode, core transmitter and receiver data paths contain small FIFOs to account for the phase difference between the frequency related clocks at the client and physical side interfaces. These FIFOs require the following constraints. These FIFOs are NOT present in any other TEMAC solution configuration.

```
#####
# Elastic Buffer Clock Crossing Constraints                                     #
#####
# no clock enable clock crossing - need to define clock relationship
TIMESPEC "TS_tx_elastic_buffer" = FROM "clk_tx_core" TO "tx_clock_mii"
8000 ps DATAPATHONLY;
TIMESPEC "TS_rx_elastic_buffer" = FROM "clk_rx_core" TO "rx_clock"
8000 ps DATAPATHONLY;
```

